Weak Steinberg relations imply the full graded A3, B3-small and B3-large relation sets

Eric Wang, Arohee Bhoja, Cayden Codel, Noah G. Singer

10.4230/LIPIcs.ITP.2025.9; CC BY 4.0 (paper); Apache-2.0 (author code)

Editorial note. The collected proof is fixed author Lean source, including all 26 local imported modules. Lean, Lake and kernel checking were not run by this collection. Typesetting success, when obtained by the parent, will not establish formal verification.

Editorial note. The exact counted scope is relation implication for the three formal target declarations, including their native ring/field assumptions. It is collected as one main result. The paper's HDX consequences, matrix realizations and isomorphism commentary are not independently counted or supplied as new formal claims.

Editorial note. The conference paper is licensed CC BY 4.0; companion author code is separately Apache-2.0. Original copyright notices and the complete Apache license are retained.

Editorial note. All Lean code is editable text in proof_spans and may be stored verbatim in database proof_text. Raw selected .lean files and fixed toolchain/package metadata are preserved as assets.

Editorial note. External Lean/mathlib prerequisites are pinned by the author manifest. Current local construction/proof/macro modules are included and are not substituted by standard-background citations.

Editorial scope. One collected author-code main result. The preparation records give the time-bounded identity and semantic-neighbor checks. Typesetting receipts do not certify the mathematics or formal kernel verification. No formal production problem ID or admission is assigned here.

Allowed standard prerequisites (editorial).

- Lean 4 language and ordinary mathematical library support. Fixed Lean 4.15.0-rc1; exact mathlib revision and all package dependencies are retained in the author lake-manifest.json. Standard groups, rings, fields, free groups, presented groups and tactics. External package code is not recursively reproved or executed. All current-paper local modules are retained.
- Basic group and field identities. Mathlib standard imports listed in source-map.json. Ordinary external algebraic facts; the current-paper graded transport, lifted-relation construction, degree reflection, missing-root relations and macro implementations remain in proof_spans

Author main-result context

Theorem 1 (Revisited; references [35], [25]). Let G be one of three specific graded unipotent Chevalley groups (A_3, B_3-small, or B_3-large), and let Φ be a positive root system for G. Then the full set of Steinberg relations R_{full} for G can be derived from the weak relations R_{weak}. In other words, the presentations $\langle S \mid R_{\text{full}} \rangle$ and $\langle S \mid R_{\text{weak}} \rangle$ are isomorphic.

Formal scope (editorial). The counted material below is the exact formal relation-implication result in the fixed companion code. The paper uses finite-field language; the code states the A3 part over an arbitrary ring, the B3-small part over a field with nonzero 2, and the B3-large part additionally assumes that every element is a sum of two squares. The three parts form one collected result; no group-matrix identification or HDX corollary is added.

Original supporting references. [25] Tali Kaufman and Izhar Oppenheim. Coboundary and Cosystolic Expansion from Strong Symmetry. ICALP 2021, LIPIcs 198, 84:1–84:16. DOI: 10.4230/LIPIcs.ICALP.2021.84.

[35] Ryan O'Donnell and Noah G. Singer. Coboundary expansion inside Chevalley coset complex HDXs. In submission, November 2024. DOI: 10.48550/arXiv.2411.05916.

Formal proof status (editorial). Complete author source text is collected as editable TeX. Typesetting receipts are recorded separately and do not certify the formal proof. Lean, Lake, and kernel checking have not been run by this collector.

A3 exact author formal target

Source: Steinberg/A3/Thm.lean; original target and proof lines 384–444.

```
namespace Steinberg.A3
variable {R : Type TR} [Ring R]

theorem full_relations_implied_by_weak_relations :
  ∀ r ∈ (fullA3Graded R).allRelations, (weakA3Graded R).project r = 1 := by
```

B3Small exact author formal target

Source: Steinberg/B3Small/Thm.lean; original target and proof lines 518–588.

```
namespace Steinberg.B3Small
variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)
include Fchar

theorem full_relations_implied_by_weak_relations :
  ∀ r ∈ (fullB3SmallGraded F).allRelations, (weakB3SmallGraded F).project r = 1 := by
```

B3Large exact author formal target

Source: Steinberg/B3Large/Thm.lean; original target and proof lines 572–691.

```
namespace Steinberg.B3Large
variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)
variable (F_sum_of_squares : ∀ (a : F), ∃ (x y : F), a = x^2 + y^2)
include Fchar
include F_sum_of_squares

theorem full_relations_implied_by_weak_relations :
  ∀ r ∈ (fullB3LargeGraded F).allRelations, (weakB3LargeGraded F).project r = 1 := by
```

Complete author formal proof code

The following 26 sections reproduce the complete fixed author Lean modules in local import order, including the definitions, helper proofs, root-relation constructions and proof macros required by the three target declarations. They contain 8,276 original source lines. The paper context above is not a replacement for this proof code.

Each code section is headed by its exact module name; its line numbers refer to the original complete module. The final declarations and full proof bodies are in:

- Steinberg.A3.Thm, original lines 384–444.
- Steinberg.B3Small.Thm, original lines 518–588.
- Steinberg.B3Large.Thm, original lines 572–691.

All three declarations are named `full_relations_implied_by_weak_relations` in their respective namespaces. The raw editable `.lean` files, `source-map.json`, author license, toolchain and fixed package manifest are retained as companion assets. External Lean/mathlib library support is referenced at the author-pinned versions; local author proof dependencies are reproduced in full.

Code copyright (c) 2025 The Steinberg Group, Apache License 2.0. The article is by Eric Wang, Arohee Bhoja, Cayden Codel and Noah G. Singer, under CC BY 4.0. Only the source selection and presentation wrapper were prepared here. No Lean, Lake or kernel check was run.

Author source module Steinberg.Macro.Attr

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.Tactic.Attr.Register

/-!

Declares class and simp attributes for the Steinberg project.

These attributes are used to tag certain theorem for use in
custom tactics (see `Macro/Group`).

-/


namespace Steinberg

register_simp_attr chev_simps
register_simp_attr height_simps

end Steinberg
```

Author source module Steinberg.Macro.Group

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

/- Tactic and metaprogramming imports -/
import Lean.Elab.Declaration
import Lean.Elab.Tactic.Rewrite
import Mathlib.Tactic.SimpRw
import Mathlib.Util.AddRelatedDecl
import Mathlib.Tactic.CategoryTheory.Reassoc

/- Mathlib mathematics imports -/
import Mathlib.Algebra.Group.Defs
import Mathlib.Algebra.Group.Basic
import Mathlib.Algebra.Ring.Defs
import Mathlib.Algebra.Group.Hom.Defs
```

```
import Mathlib.GroupTheory.FreeGroup.Basic
import Mathlib.GroupTheory.Commutator.Basic

import Steinberg.Macro.Attr

/-! Macros and other convenience keywords when doing proofs on groups.

See:
  - Associating: `mul_assoc_l`, `mal`, `mul_assoc_r`, `mar`
  - Chevalley simp automation: `chev_simp`
  - Group rewrite: `grw`

-/

namespace Steinberg

open Mathlib.Tactic
open Lean Meta Elab Term Tactic Parser.Tactic PrettyPrinter

attribute [chev_simps] neg_add_cancel add_neg_cancel
  one_mul mul_one zero_mul mul_zero
  mul_neg neg_mul neg_neg inv_inv mul_inv_rev
  div_neg neg_div
  pow_two
  zero_add add_zero neg_zero
  map_one map_zero map_mul map_add map_neg map_commutatorElement
  commutatorElement_inv commutatorElement_one_left commutatorElement_one_right
  and_true true_and and_self or_self
  Int.cast_ofNat Int.cast_one Int.cast_two Int.cast_three Int.cast_neg Int.reduceNeg
  FreeGroup.map.of FreeGroup.lift.of

/-- A macro for a common simplification when rewriting with ghost component equations. -/
syntax (name := chevSimp) "chev_simp" (simpArgs)? (location)? : tactic
macro_rules
  | `(tactic| chev_simp $[[$simpArgs,*]]? $[at $loc]?) => do
    let args := simpArgs.map (·.getElems) |>.getD #[]
    `(tactic| simp only [chev_simps, $args,*] $[at $loc]?)

syntax (name := chevSimp?) "chev_simp?" (simpArgs)? (location)? : tactic
macro_rules
  | `(tactic| chev_simp? $[[$simpArgs,*]]? $[at $loc]?) => do
    let args := simpArgs.map (·.getElems) |>.getD #[]
    `(tactic| simp? [chev_simps, $args,*] $[at $loc]?)

syntax (name := heightSimp) "height_simp" (simpArgs)? (location)? : tactic
macro_rules
  | `(tactic| height_simp $[[$simpArgs,*]]? $[at $loc]?) => do
    let args := simpArgs.map (·.getElems) |>.getD #[]
    `(tactic| simp only [height_simps, $args,*] $[at $loc]?)

/-- A simple tactic to solve `PosRootSys` height equations. Uses `omega`. -/
macro "ht" : tactic =>
  `(tactic| (
   first
    | trivial
    | assumption
    | omega
    | (try height_simp at *; omega)
  ))

namespace Macro

/-- Shorthand for `simp_rw [← mul_assoc]`, which applies the `mul_assoc` tactic to the left. -/
macro (name := mul_assoc_l) "mul_assoc_l" l:(location)? : tactic => `(tactic|
  simp_rw [← mul_assoc] $l ?
)

@[inherit_doc mul_assoc_l]
macro (name := mal) "mal" l:(location)? : tactic => `(tactic|
  mul_assoc_l $l ?
)

/-- Shorthand for `simp_rw [mul_assoc]`, which applies the `mul_assoc` tactic to the right. -/
macro (name := mul_assoc_r) "mul_assoc_r" l:(location)? : tactic => `(tactic|
  simp_rw [mul_assoc] $l ?
)

@[inherit_doc mul_assoc_r]
macro (name := mar) "mar" l:(location)? : tactic => `(tactic|
  mul_assoc_r $l ?
```

```
)

/- TODO-B: Use these two theorems to generate an automatic new theorem from a commutative theorem. -/

private theorem mul_assoc_left {G : Type u} [Semigroup G] {b c d : G} (h : b * c = d) (a : G)
    : a * b * c = a * d := by
  rw [mul_assoc, h]

private theorem mul_assoc_right {G : Type u} [Semigroup G] {b c d : G} (h : b * c = d) (a : G)
    : b * (c * a) = d * a := by
  rw [← mul_assoc, h]

private theorem add_assoc_left {G : Type u} [AddSemigroup G] {b c d : G} (h : b + c = d) (a : G)
    : a + b + c = a + d := by
  rw [add_assoc, h]

private theorem add_assoc_right {G : Type u} [AddSemigroup G] {b c d : G} (h : b + c = d) (a : G)
    : b + (c + a) = d + a := by
  rw [← add_assoc, h]

private theorem mul_assoc_symm {G : Type u} [Semigroup G] (a b c : G)
    : a * (b * c) = a * b * c := by
  exact Eq.symm (mul_assoc _ _ _)

private theorem mul_left_inj_assoc {G : Type u} [Semigroup G] [IsRightCancelMul G] (a : G) {b c d e : G}
    : d * (b * a) = e * (c * a) ↔ d * b = e * c := by
  mal; rw [mul_left_inj]

private theorem mul_left_inj_assoc_l {G : Type u} [Semigroup G] [IsRightCancelMul G] (a : G) {b c d : G}
    : d * (b * a) = c * a ↔ d * b = c := by
  mal; rw [mul_left_inj]

private theorem mul_left_inj_assoc_r {G : Type u} [Semigroup G] [IsRightCancelMul G] (a : G) {b c e : G}
    : b * a = e * (c * a) ↔ b = e * c := by
  mal; rw [mul_left_inj]

private theorem mul_right_inj_assoc {G : Type u} [Semigroup G] [IsLeftCancelMul G] (a : G) {b c d e : G}
    : (a * b) * d = (a * c) * e ↔ b * d = c * e := by
  mar; rw [mul_right_inj]

private theorem mul_right_inj_assoc_l {G : Type u} [Semigroup G] [IsLeftCancelMul G] (a : G) {b c d : G}
    : (a * b) * d = a * c ↔ b * d = c := by
  mar; rw [mul_right_inj]

private theorem mul_right_inj_assoc_r {G : Type u} [Semigroup G] [IsLeftCancelMul G] (a : G) {b c e : G}
    : a * b = (a * c) * e ↔ b = c * e := by
  mar; rw [mul_right_inj]

/--
  Apply `mul_left_inj` and `mul_right_inj` after reassociating.
  End with the term reassociated to the left (i.e., `(a * b) * c`).
-/
macro (name := mul_inj) "mul_inj" l:(location)? : tactic => `(tactic| (
  (try simp only [mul_assoc, mul_right_inj,
    mul_inv_cancel, inv_mul_cancel, one_mul, mul_one] $l ?);
  (try simp only [← mul_assoc, mul_left_inj,
    mul_inv_cancel, inv_mul_cancel, one_mul, mul_one] $l ?)
))

/--
  An empty list of `simp` lemmas and terms.

  TODO-B: See `Mathlib.Tactic.CategoryTheory.Reassoc` for a list of potential
      relevant lemmas, but for category theory. The problem of using a lot
      of standard group theory lemmas is that they will simplify away
      `a * 1`, which we often don't want to do in our proofs. If we find
      that we *do* need some lemmas, add them here.
-/
def groupSimp_left (e : Expr) : MetaM Simp.Result :=
  simpOnlyNames [``mul_assoc_symm] e

def groupSimp_right (e : Expr) : MetaM Simp.Result :=
  simpOnlyNames [``mul_assoc] e

/--
  An attribute to automatically generate a new theorem of the same name,
  plus a suffix of `_assoc`.

  Note: This is a copy of `reassocExpr` in `Mathlib.Tactic.CategoryTheory.Reassoc`.

```

```
  Essentially, theorems with this attribute automatically generate a proof
  of that same theorem, modulo associativity. See the `private theorems` above.

  Only theorems of the form `∀ ..., a * b = c` are allowed,
  where `*` is the group-theoretic binary operation.
-/
def reassocExpr_left (e : Expr) : MetaM Expr := do
  mapForallTelescope (fun e => do simpType groupSimp_left (← mkAppM ``mul_assoc_left #[e])) e

/--
  An attribute to automatically generate a new theorem of the same name,
  plus a suffix of `_assoc'`.

  Note: This is a copy of `reassocExpr` in `Mathlib.Tactic.CategoryTheory.Reassoc`.

  Essentially, theorems with this attribute automatically generate a proof
  of that same theorem, modulo associativity. See the `private theorems` above.

  Only theorems of the form `∀ ..., a * b = c` are allowed,
  where `*` is the group-theoretic binary operation.
-/
def reassocExpr_right (e : Expr) : MetaM Expr := do
  mapForallTelescope (fun e => do simpType groupSimp_right (← mkAppM ``mul_assoc_right #[e])) e

syntax (name := group_reassoc) "group_reassoc" (" (" &"attr" ":=" Parser.Term.attrInstance,* ")")? : attr

initialize registerBuiltinAttribute {
  name := `group_reassoc
  descr := "Attribute for generating reassociated versions of commutator lemmas"
  applicationTime := .afterCompilation
  add := fun src ref kind => match ref with
  | `(attr| group_reassoc $[(attr := $stx?,*)]?) => MetaM.run' do
    if (kind != AttributeKind.global) then
      throwError "`reassoc` can only be used as a global attribute"
    addRelatedDecl src "_assoc_left" ref stx? fun type value levels => do
      pure (← reassocExpr_left (← mkExpectedTypeHint value type), levels)
    -- TODO-B: (3/18) Removing this still allows `B3Large/Thm.lean` to compile.
    -- TODO-B: I've removed this to speed up compilation.
    -- addRelatedDecl src "_assoc_right" ref stx? fun type value levels => do
    --  pure (← reassocExpr_right (← mkExpectedTypeHint value type), levels)
  | _ => throwUnsupportedSyntax
}

open Term in
/--
  A term elaborator `greassoc_of% t`, where `t` is
  an equation `f = g` on `Semigroup`s (possibly under a `∀` binder),
  used to reassociate the group equation with a fresh third term to the right.

  To use, place before a proof term or hypothesis name to create a new term
  that has been reassociated in the other way. For example,

  `rw [greassoc_of% h]` will rewrite with `h` after being reassociated.

  Note that the elaborator creates only one term. Using this with `rw` will
  NOT
-/
elab "greassoc_of% " t:term : term => do
  reassocExpr_left (← elabTerm t none)

open Term in
/--
  A term elaborator `greassoc_of% t`, where `t` is
  an equation `f = g` on `Semigroup`s (possibly under a `∀` binder),
  used to reassociate the group equation with a fresh third term to the right.

  To use, place before a proof term or hypothesis name to create a new term
  that has been reassociated in the other way. For example,

  `rw [greassoc_of% h]` will rewrite with `h` after being reassociated.

  Note that the elaborator creates only one term. Using this with `rw` will
  NOT
-/
elab "greassoc_of_r% " t:term : term => do
  reassocExpr_right (← elabTerm t none)

/-
  TODO-B: It's very, very hard to get `(rwRule| )` to type check, since it expects
      a `term`, while most outputs of this function are `Term.app`, which for
```

```
      whatever reason is not a `term`. So we have to open the `.Compat`
      environment to extract coercions between the `TSyntax` kinds.
      Ideally, we wouldn't need to do this.
-/
open TSyntax.Compat in
/--
  Peers into a `Syntax` object and tries to add `_assoc` to the base theorem
  used in the term.

  For example, if the term is `add_comm h₁ h₂ a b`, then this function
  returns `add_comm_assoc h₁ h₂ a b`.
-/
partial def replaceWithAssocName (stx : Syntax) : Option (TSyntax `term × TSyntax `term) :=
  let k := stx.getKind
  if k == ``Lean.Parser.Term.app then
    let ⟨assocName_left, assocName_right⟩ :=
      match stx[0].getId with
      | Name.str n s =>
        let str_left := s ++ "_assoc_left"
        let str_right := s ++ "_assoc_right"
        let s_left := Name.mkStr n <| str_left
        let s_right := Name.mkStr n <| str_right
        Prod.mk s_left s_right
      | x => Prod.mk x x
    let appNode_left := mkNode ``Lean.Parser.Term.app #[mkIdent assocName_left, stx[1]]
    let appNode_right := mkNode ``Lean.Parser.Term.app #[mkIdent assocName_right, stx[1]]
    some (appNode_left, appNode_right)
  else if k == ``Lean.Parser.Term.proj then
    replaceWithAssocName stx[0]
  else if k == ``Lean.Parser.Term.paren then
    replaceWithAssocName stx[1]
  else
    none

/--
  A special group `rw` tactic that works well with the `group_reassoc`
  attribute for theorems.

  Like `rw`, `grw` takes a list of rewrite terms and applies them in order.
  If the rewrite fails, it reassociates the identified goal/expression
  to the left and tries again. If *that* fails, it tries to extract the
  base theorem name of the rewrite rule and add `_assoc` to it.

  Basically, if a rewrite rule has the form `a * b = c`, but the identified
  goal/expression has the form `(d * a) * b`, then `grw` will use the
  `group_reassoc`-generated theorem to replace `a * b` with `c` by using
  the assoc version of the theorem instead.

  TODO-B: This tactic is a hack. There's no guarantee that the rewrite rule
      has the right form, nor that there exists a reassoc theorem. Another
      attempt of the tactic tried to extract the sub-expression to reassociate
      instead, but constructing the right `mul_assoc` proof term was hard.
-/
elab s:"grw " cfg:optConfig rws:rwRuleSeq l:(location)? : tactic => Elab.Tactic.focus do
  /-
    For each `rwRule` in the `rwRuleSeq`, let `rewrite` process the
    rewrite term to give us a term's `Syntax`. On that syntax,
    examine its type to get out the right expression.
  -/
  withRWRulesSeq s rws fun symm term => do
    match term with
    | `(term| $e:term) => do
      let cont : TacticM Unit := (do
        match replaceWithAssocName e with
        | none => evalTactic <| ← `(tactic| skip)
        | some ⟨assoc_l, _⟩ =>
          let rule_l := ← do if symm then `(rwRule| ← $assoc_l:term) else `(rwRule| $assoc_l:term)
          -- let rule_r := ← do if symm then `(rwRule| ← $assoc_r:term) else `(rwRule| $assoc_r:term)
          evalTactic <| ← `(tactic|
            (rw $cfg [$rule_l] $l ?)
            --first
            -- | (rw $cfg [$rule_l] $l ?)
            -- | (rw $cfg [$rule_r] $l ?)
          )
      )

      let rwTerm ← do if symm then `(rwRule| ← $e:term) else `(rwRule| $e:term)

      -- Now try to reassociate the term `e`
      -- This might fail, but it's an okay transformation to make here
```

```
      -- because all we're extracting at this point is syntax
      let reassocTerm_l ← do
        if symm then `(rwRule | ← greassoc_of% $e:term) else `(rwRule | greassoc_of% $e:term)

      let reassocTerm_r ← do
        if symm then `(rwRule | ← greassoc_of_r% $e:term) else `(rwRule | greassoc_o_rf% $e:term)

      ((evalTactic <| ← `(tactic|
        first
        | rw $cfg [$rwTerm] $l ?
        | (mal $l ?; rw $cfg [$rwTerm] $l ?)
        | rw $cfg [$reassocTerm_l] $l ?
        | rw $cfg [$reassocTerm_r] $l ?
      ))
      <|>
      cont);
      (evalTactic <| ← `(tactic| mul_inj $l ?));
      (evalTactic <| ← `(tactic| try chev_simp [] $l ?))

/-- `rwa` is short-hand for `rw; assumption`. -/
macro "rwa " rws:rwRuleSeq loc:(location)? : tactic =>
  `(tactic| (rw $rws:rwRuleSeq $[$loc:location]?; assumption))

macro "grwa " rws:rwRuleSeq l:(location)? : tactic =>
  `(tactic| (grw $rws:rwRuleSeq $[$l:location]?; assumption))

end Macro

end Steinberg
```

Author source module Steinberg.Defs.PositiveRootSystem

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Macro.Group
import Mathlib.Data.Finset.Defs

namespace Steinberg

/-- A typeclass representing the roots in a positive root system.

For now, we do not encode anything about the additive structure of the roots
(except for requiring that a `height` function be supplied).
-/
class PositiveRootSystem (Root : semiOutParam (Type TΦ))
  extends
    ToString Root
  where
  height : Root → Nat

attribute [height_simps] PositiveRootSystem.height

open PositiveRootSystem

/-! ## Encoding Steinberg relations -/

/-- Dependent product type, where each element encodes a Steinberg relation between two roots which span
  ↪  *one* additional root.

Elements of this type have the form `⟨ ζ, η, θ, C, h ⟩`. This represents the following Steinberg relation:
* `ζ` and `η` are the original pair of roots,
* `θ = ζ + η`.
* `C` is the Chevalley constant.
(`h` is a well-definedness condition on the heights of the roots).

In the ungraded case, this tuple creates relations of the form
  `⁅ {ζ, t}, {η, u} ⁆ = {θ, C * t * u}`.
In the graded case, this tuple creates relations of the form
  `⁅ {ζ, i, t}, {η, j, u} ⁆ = {θ, i + j, C * t * u}`.
 -/
abbrev SingleSpanRootPair (Φ : Type TΦ) [PositiveRootSystem Φ] :=
  (ζ : Φ) × (η : Φ) × (θ : Φ) × ℤ ×' (height θ = height ζ + height η)

```

```
/-- Dependent product type, where each element encodes a Steinberg relation between two roots which span
  ↪ *two* additional roots.

Elements of this type have the form `⟨ ζ, η, θ₁, θ₂, C₁, C₂, h₁, h₂ ⟩`. This represents the following
  ↪ Steinberg relation:
* `ζ` and `η` are the original pair of roots,
* `θ₁ = ζ + η` and `θ₂ = ζ + 2η`.
* `C₁` and `C₂` are the respectively Chevalley constants.
(`h₁` and `h₂` are well-definedness conditions on the heights of the roots).

In the ungraded case, this tuple creates relations of the form
  `⁅ {ζ, t}, {η, u} ⁆ = {θ₁, C₁ * t * u} * {θ₂, C₂ * t * u^2}`.
In the graded case, this tuple creates relations of the form
  `⁅ {ζ, i, t}, {η, j, u} ⁆ = {θ₁, i + j, C₁ * t * u} * {θ₂, i + 2 * j, C₂ * t * u^2}`.
 -/
abbrev DoubleSpanRootPair (Φ : Type TΦ) [PositiveRootSystem Φ] := (
  (ζ : Φ) × (η : Φ) × (θ₁ : Φ) × (θ₂ : Φ)
  × ℤ × ℤ ×' (height θ₁ = height ζ + height η) ×' (height θ₂ = height ζ + 2 * height η)
)

/-- A structure encoding a (possibly strict) subset of Steinberg relations for the (graded or ungraded)
unipotent Chevalley groups on `Φ`. This structure is used to build `PartialChevalleyGroup`s and
`GradedPartialChevalleyGroup`s.
-/
structure PartialChevalleySystem (Φ : Type TΦ) [PositiveRootSystem Φ] where
  mk ::
  presentRoots : Set Φ
  trivialSpanRootPairs : Set (Φ × Φ)
  singleSpanRootPairs : Set (SingleSpanRootPair Φ)
  doubleCommutatorRootPairs : Set (DoubleSpanRootPair Φ)

  -- validity hypotheses (needed for degree reflection stuff)
  h_trivial_valid : ∀ p ∈ trivialSpanRootPairs,
    p.1 ∈ presentRoots ∧ p.2 ∈ presentRoots
  h_single_valid : ∀ p ∈ singleSpanRootPairs,
    p.1 ∈ presentRoots ∧ p.2.1 ∈ presentRoots ∧ p.2.2.1 ∈ presentRoots
  h_double_valid : ∀ p ∈ doubleCommutatorRootPairs,
    p.1 ∈ presentRoots ∧ p.2.1 ∈ presentRoots ∧ p.2.2.1 ∈ presentRoots ∧ p.2.2.2.1 ∈ presentRoots

  -- TODO: hypothesis stating that if a pair appears in one of these sets, neither it nor its swap appears
  ↪ again

namespace PartialChevalleySystem

/-! ### 'Full' systems -/

/-- Extract the set of all ordered pairs of roots for which there exists a commutator relation,
either trivial-span, single-span, or double-span. -/
def toRootPairs (Φ : Type TΦ) [PositiveRootSystem Φ] (trivialSpanRootPairs : Set (Φ × Φ))
  ↪ (singleSpanRootPairs : Set (SingleSpanRootPair Φ))
  (doubleCommutatorRootPairs : Set (DoubleSpanRootPair Φ)) : Set (Φ × Φ) :=
  (fun p => (p.1, p.2)) '' trivialSpanRootPairs ∪ (fun p => (p.1, p.2.1)) '' singleSpanRootPairs
    ∪ (fun p => (p.1, p.2.1)) '' doubleCommutatorRootPairs

/-- Proposition that every pair of (distinct) roots in `Φ` appears in `toRootPairs` (or its swap does).
Ensures that a "full" `PartialChevalleySystem` is indeed full, for sanity checking. -/
def everyRootPairInRootPairs (Φ : Type TΦ) [PositiveRootSystem Φ]
  (trivialSpanRootPairs : Set (Φ × Φ)) (singleSpanRootPairs : Set (SingleSpanRootPair Φ))
  (doubleCommutatorRootPairs : Set (DoubleSpanRootPair Φ)) : Prop :=
  ∀ (ζ η : Φ), ζ ≠ η →
   (ζ, η) ∈ toRootPairs Φ trivialSpanRootPairs singleSpanRootPairs doubleCommutatorRootPairs
    ∪ Prod.swap '' toRootPairs Φ trivialSpanRootPairs singleSpanRootPairs doubleCommutatorRootPairs

/-- Build a "full" `PartialChevalleySystem`. This allows you to omit the validity hypotheses,
instead passing a hypothesis that every root in `Φ` is contained in `presentRoots`.
For sanity checking, we also require a hypothesis that `everyRootPairInRootPairs` holds.
 -/
def mkFull (Φ : Type TΦ) [PositiveRootSystem Φ]
  (presentRoots : Set Φ) (trivialSpanRootPairs : Set (Φ × Φ)) (singleSpanRootPairs : Set
  ↪ (SingleSpanRootPair Φ))
  (doubleCommutatorRootPairs : Set (DoubleSpanRootPair Φ))
  (h_full : ∀ (ζ : Φ), ζ ∈ presentRoots)
  (h_full' : everyRootPairInRootPairs Φ trivialSpanRootPairs singleSpanRootPairs
  ↪ doubleCommutatorRootPairs)
  : PartialChevalleySystem Φ :=
  PartialChevalleySystem.mk presentRoots trivialSpanRootPairs singleSpanRootPairs
  ↪ doubleCommutatorRootPairs (by tauto) (by tauto) (by tauto)

end PartialChevalleySystem
```

```
end Steinberg
```

Author source module Steinberg.Defs.Commutator

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.Tactic.Group
import Steinberg.Macro.Group

namespace Steinberg

variable {G H : Type TG} [Group G] [Group H]
         {w x y z : G}

-- Star operator
def starCommutator_def (x y : G) := y⁻¹ * x * y^2 * x⁻¹ * y⁻¹
infix:68 " ⋆ " => starCommutator_def

theorem star_commutator (x y : G) : (x ⋆ y) * ⁅(x ⋆ y)⁻¹, y⁆ = ⁅x, y^2⁆ := by
  unfold starCommutator_def
  group

theorem commutator_star (x y : G) : ⁅x ⋆ y, y⁆⁻¹ * (x ⋆ y) = ⁅x, y^2⁆ := by
  unfold starCommutator_def
  group

theorem starCommutator_helper {x y z : G} (w : G) (h1 : ⁅ x, y^2 ⁆ = z * w) (h2 : ⁅ z⁻¹, y ⁆ = w) :
  z = x ⋆ y := by
  unfold starCommutator_def
  subst w
  simp only [commutatorElement_def] at h1
  group at h1
  apply @mul_left_cancel _ _ _ y
  apply @mul_right_cancel _ _ _ _ y⁻¹
  group
  rw [←h1]

theorem map_starCommutator {x y : G} (f : G →* H) : f (x ⋆ y) = (f x) ⋆ (f y) := by
  unfold starCommutator_def
  simp only [map_pow, map_inv, map_mul]

end Steinberg
```

Author source module Steinberg.Upstream.FreeGroup

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.GroupTheory.FreeGroup.Basic

theorem lift.hom (α β G : Type u) [Group G] (f : α → FreeGroup β) (g : FreeGroup β →* G) :
  FreeGroup.lift (g ∘ f) = g.comp (FreeGroup.lift f) := by
  ext a
  simp only [FreeGroup.lift.of, Function.comp_apply, MonoidHom.coe_comp]
```

Author source module Steinberg.Upstream.PresentedGroup

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.GroupTheory.PresentedGroup

import Steinberg.Upstream.FreeGroup
```

```

/-- A free group element is in the normal closure of the set of relations iff its image
vanishes in the presented group. -/
theorem eq_one_iff_mem_closure {α : Type u} {rels : Set (FreeGroup α)} {x : FreeGroup α} :
  PresentedGroup.mk rels x = 1 ↔ x ∈ Subgroup.normalClosure rels := by
  nth_rewrite 2 [← one_mul x, ← inv_one]
  apply Iff.trans _ (@QuotientGroup.eq _ _ (Subgroup.normalClosure rels) 1 x)
  simp only [QuotientGroup.mk_one, PresentedGroup.mk, MonoidHom.coe_mk, OneHom.coe_mk]
  use Eq.symm, Eq.symm

/-- If a free group element is included in the set of relations, then its image
vanishes in the presented group. -/
theorem eq_one_of_mem_rels {α : Type u} {rels : Set (FreeGroup α)} {x : FreeGroup α} :
  x ∈ rels → PresentedGroup.mk rels x = 1 := by
  intro h
  apply eq_one_iff_mem_closure.mpr
  exact Subgroup.subset_normalClosure h

private theorem helper {α β : Type u} {S : Set (FreeGroup α)} {T : Set (FreeGroup β)} {f : α → FreeGroup
↪  β}
  (h : ∀ r ∈ S, (PresentedGroup.mk T) (FreeGroup.lift f r) = 1) :
    ∀ r ∈ S, (FreeGroup.lift ((PresentedGroup.mk T) ∘ f) r) = 1 := by
  intro r h_r
  rw [lift.hom, MonoidHom.coe_comp, Function.comp_apply]
  exact h r h_r

/-- Build a homomorphism between `PresentedGroup`s by specifying the image of
every generator in the source as a product of generators in the target. Requires
a well-definedness hypothesis, namely, that the image (suitably defined)
of every assumed relation in the source vanishes in the target. -/
def toPresentedGroup {α β : Type u} {S : Set (FreeGroup α)} {T : Set (FreeGroup β)} {f : α → FreeGroup β}
  (h : ∀ r ∈ S, (PresentedGroup.mk T) (FreeGroup.lift f r) = 1) : PresentedGroup S →* PresentedGroup T :=
  PresentedGroup.toGroup (helper h)

/-- Apply `toPresentedGroup` to a generator. -/
theorem toPresentedGroup.of {α β : Type u} {S : Set (FreeGroup α)} {T : Set (FreeGroup β)} {f : α →
↪  FreeGroup β}
  (h : ∀ r ∈ S, (PresentedGroup.mk T) (FreeGroup.lift f r) = 1) (x : α) :
  (toPresentedGroup h) (PresentedGroup.of x) = PresentedGroup.mk T (f x) := by
  rw [toPresentedGroup, PresentedGroup.toGroup.of]
  simp only [Function.comp_apply]

private theorem toPresentedGroup.mk' {α β : Type u} {S : Set (FreeGroup α)} {T : Set (FreeGroup β)} {f : α
↪  → FreeGroup β}
  (h : ∀ r ∈ S, (PresentedGroup.mk T) (FreeGroup.lift f r) = 1) :
  (toPresentedGroup h).comp (PresentedGroup.mk S) = (PresentedGroup.mk T).comp (FreeGroup.lift f) := by
  ext a
  simp only [MonoidHom.coe_comp, Function.comp_apply, FreeGroup.lift.of]
  rw [← PresentedGroup.of]
  exact toPresentedGroup.of h a

/-- Apply `toPresentedGroup` to the image of a free group element in a presented group. -/
theorem toPresentedGroup.mk {α β : Type u} {S : Set (FreeGroup α)} {T : Set (FreeGroup β)} {f : α →
↪  FreeGroup β}
  (h : ∀ r ∈ S, (PresentedGroup.mk T) (FreeGroup.lift f r) = 1) (a : FreeGroup α) :
  (toPresentedGroup h) (PresentedGroup.mk S a) = PresentedGroup.mk T (FreeGroup.lift f a) := by
  have aux₁ : (toPresentedGroup h).comp (PresentedGroup.mk S) a = (toPresentedGroup h) (PresentedGroup.mk
  ↪  S a) := by
    simp only [MonoidHom.coe_comp, Function.comp_apply]
  have aux₂ : (PresentedGroup.mk T).comp (FreeGroup.lift f) a = PresentedGroup.mk T (FreeGroup.lift f a)
  ↪  := by
    simp only [MonoidHom.coe_comp, Function.comp_apply]
  rw [←aux₁, ←aux₂, toPresentedGroup.mk']
```

Author source module Steinberg.Upstream.Commutator

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```

/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.Algebra.Group.Aut
import Mathlib.GroupTheory.Commutator.Basic

import Mathlib.Tactic.Group

```

```
/-!

  File dox go here.

-/

-- namespace Mathlib.GroupTheory.Commutator

/-! ### Notations for theorems involving group elements -/

/-
  These notations essentially act like "inline mathematical notation,"
  in the sense that using a `def` is too strong (i.e., it obscures
  the underlying notion behind a name that we would have to `unfold`
  or `rw` or supply `simp` lemmas for). Rather, this is shorthand
  for longer expressions that would otherwise take a while to type.
-/

namespace CommutatorNotation

/-- `triv_comm(x, y)` unfolds to `⁅ x, y ⁆ = 1`.  -/
notation "triv_comm" "( " x ", " y " )" => ⁅ x, y ⁆ = 1

/-- `commutes(x, y)` unfolds to `x * y = y * x`.  -/
notation "commutes" "( " x ", " y " )" => x * y = y * x

notation "comm_unfold" "( " x ", " y " )" => x * y * x⁻¹ * y⁻¹

/-- `reorder_left(x, y, z)` unfolds to `x * y = z * y * x`.  -/
notation "reorder_left" "( " x ", " y ", " z " )" => x * y = z * y * x
notation "reorder_mid" "( " x ", " y ", " z " )" => x * y = y * z * x
notation "reorder_right" "( " x ", " y ", " z " )" => x * y = y * x * z

end CommutatorNotation

open CommutatorNotation

variable {G : Type TG} [Group G]
         {w x y z : G}

/-! ### Theorems about commutators (holding in any group) -/

theorem comm_left      (x y : G) : x * y = ⁅x, y⁆ * y * x       := by group
theorem comm_mid       (x y : G) : x * y = y * ⁅x, y⁻¹⁆⁻¹ * x   := by group
theorem comm_right     (x y : G) : x * y = y * x * ⁅x⁻¹, y⁻¹⁆   := by group
theorem comm_swap      (x y : G) : ⁅x, y⁆⁻¹ = ⁅y, x⁆            := commutatorElement_inv _ _

theorem comm_left_rev  (x y : G) : x * y = ⁅y, x⁆⁻¹ * y * x     := commutatorElement_inv y _ ▸ comm_left
↪  ..
theorem comm_mid_rev   (x y : G) : x * y = y * ⁅y⁻¹, x⁆ * x     := (commutatorElement_inv x _).symm ▸
↪  comm_mid ..
theorem comm_right_rev (x y : G) : x * y = y * x * ⁅y⁻¹, x⁻¹⁆⁻¹ := commutatorElement_inv y⁻¹ _ ▸
↪  comm_right ..

theorem triv_comm_iff_commutes : triv_comm(x, y) ↔ commutes(x, y) := by
  constructor
  · intro h
    rw [comm_left, h, one_mul]
  · intro h
    rw [commutatorElement_def, h, mul_inv_cancel_right, mul_inv_cancel]

theorem eq_comm_of_reorder_left : reorder_left(x, y, z) → ⁅x, y⁆ = z := by
  intro h
  simp only [commutatorElement_def, h, mul_inv_cancel_right]

theorem reorder_left_of_eq_comm : ⁅x, y⁆ = z → reorder_left(x, y, z) := by
  rw [commutatorElement_def]
  rintro rfl
  simp_rw [inv_mul_cancel_right]

@[simp]
theorem reorder_left_iff_eq_comm : reorder_left(x, y, z) ↔ ⁅x, y⁆ = z :=
  ⟨eq_comm_of_reorder_left, reorder_left_of_eq_comm⟩

/- TODO-B: For these `triv_comm` theorems, I don't like the approach of the proofs.
   So far, the proofs are essentially just unfolding the trivial commutator
   definition and applying commutativity as needed. But this sidesteps the
   potential usefulness of a `triv_comm` definition.

   While some definition-unfolding is necessary to establish an API/starting
```

```
   set of lemmas, I think things can be reworked here to use more
   "commutator theory."
-/

-- TODO-B: Perhaps we can replace this with `triv_comm()`, but I think the statement is clearer this way.
@[symm]
theorem triv_comm_symm : ⁅ x, y ⁆ = 1 ↔ ⁅ y, x ⁆ = 1 := by
  simp_rw [triv_comm_iff_commutes]
  exact eq_comm

@[simp]
theorem inv_triv_comm_iff_triv_comm : triv_comm(x⁻¹, y) ↔ triv_comm(x, y) := by
  simp_rw [triv_comm_iff_commutes]
  apply Iff.intro
  · intro h
    apply @mul_left_cancel _ _ _ x⁻¹
    rw [inv_mul_cancel_left, ← mul_assoc, h, mul_assoc, inv_mul_cancel, mul_one]
  · intro h
    apply @mul_left_cancel _ _ _ x
    rw [mul_inv_cancel_left, ← mul_assoc, h, mul_assoc, mul_inv_cancel, mul_one]

@[simp]
theorem inv_triv_comm_iff_triv_comm' : triv_comm(x, y⁻¹) ↔ triv_comm(x, y) := by
  simp_rw [@triv_comm_symm _ _ x]
  exact inv_triv_comm_iff_triv_comm

-- TODO-B: Better name? Could be `triv_comm_trans_right` or `triv_comm_trans_mul_right`
theorem triv_comm_mul_right : triv_comm(x, y) → triv_comm(x, z) → triv_comm(x, y * z) := by
  simp_rw [triv_comm_iff_commutes]
  intro hxy hxz
  rw [← mul_assoc, hxy, mul_assoc, hxz, ← mul_assoc]

-- TODO-B: Better name? Could be `triv_comm_trans_left`
theorem triv_comm_mul_left : triv_comm(x, z) → triv_comm(y, z) → triv_comm(x * y, z) := by
  simp_rw [triv_comm_iff_commutes]
  intro hxz hyz
  rw [mul_assoc, hyz, ← mul_assoc, hxz, mul_assoc]

theorem trivial_comm_mul_mul : triv_comm(x, w) → triv_comm(x, z)
    → triv_comm(y, w) → triv_comm(y, z) → triv_comm(x * y, w * z) := by
  simp_rw [triv_comm_iff_commutes]
  intro hxw hxz hyw hyz
  -- TODO-B: Ripe for automation/a macro!
  rw [← mul_assoc, mul_assoc x, hyw, ← mul_assoc, hxw,
    mul_assoc, hyz, ← mul_assoc, mul_assoc w, hxz, ← mul_assoc, mul_assoc]

/- Deduce that two elements commute if one can "embed" them in a larger, trivial commutator where all
↪  other pairs commute. -/
theorem trivial_comm_from_embedded_comm_and_pairs : triv_comm(x * y, w * z) → triv_comm(x, w)
    → triv_comm(y, w) → triv_comm(y, z) → triv_comm(x, z) := by
  simp_rw [triv_comm_iff_commutes]
  intro h_all h_xw h_yw h_yz
  -- Add `y` on the right and `w` on the left to maintain relative orders wrt `x` and `z`
  apply @mul_right_cancel _ _ _ _ y
  apply @mul_left_cancel _ _ _ w
  simp only [←mul_assoc]
  rw [← h_xw, mul_assoc, ← h_yz, ← mul_assoc, mul_assoc x, ← h_yw, ← mul_assoc,
    mul_assoc, h_all, ← mul_assoc]

open MulAut -- to access conj function

-- Commutator identities
theorem CI1 : conj x y = ⁅x, y⁆ * y := by
  rw [commutatorElement_def, conj_apply, inv_mul_cancel_right]

theorem CI2 : ⁅y, x⁆ = ⁅x, y⁆⁻¹ := by
  simp only [commutatorElement_def, mul_inv_rev, inv_inv, mul_assoc]

theorem CI3 : ⁅x, y * z⁆ = ⁅x, y⁆ * conj y ⁅x, z⁆ := by
  simp only [conj_apply, commutatorElement_def, mul_inv_rev, ← mul_assoc,
    inv_mul_cancel_right]

theorem CI4 : ⁅x * y, z⁆ = conj x ⁅y, z⁆ * ⁅x, z⁆ := by
  simp only [conj_apply, commutatorElement_def, mul_inv_rev, ← mul_assoc,
    inv_mul_cancel_right]

theorem CI5 : conj x ⁅x⁻¹, y⁆ = ⁅y, x⁆ := by
  simp only [conj_apply, commutatorElement_def, mul_inv_rev, ← mul_assoc,
    inv_inv, mul_inv_cancel, one_mul]

```

```
theorem CI6 : ⁅y, z⁆ * ⁅x, z⁆ = ⁅x, ⁅y, z⁆⁆⁻¹ * ⁅x * y, z⁆ := by
  simp only [conj_apply, commutatorElement_def, mul_inv_rev, ← mul_assoc,
    inv_inv, mul_inv_cancel_right, inv_mul_cancel_right, CI2]

theorem CI7 : ⁅x, z⁆ = 1 → ⁅x, ⁅y, z⁆⁆ = ⁅⁅x, y⁆, conj y z⁆ := by
  intro h
  have hc : Commute x z := by
    apply (commute_iff_eq x z).mpr
    exact triv_comm_iff_commutes.mp h
  have h1 : Commute x⁻¹ z := Commute.inv_left hc
  rw [commute_iff_eq] at h1
  simp [commutatorElement_def, ← mul_assoc, conj_apply]
  simp [mul_assoc, h1]

theorem CI8 : ⁅y, z⁆ = 1 → ⁅⁅x, y⁆, z⁆ = conj (x * y) ⁅x⁻¹, z⁆ * ⁅x, z⁆ := by
  intro h
  have hc : Commute y z := by
    apply (commute_iff_eq y z).mpr
    exact triv_comm_iff_commutes.mp h
  have h1 : Commute y⁻¹ z := Commute.inv_left hc
  rw [commute_iff_eq] at h1
  simp [commutatorElement_def, ← mul_assoc, conj_apply]
  simp [mul_assoc, h1]

theorem CI9 : ⁅x, z⁆ = 1 → ⁅⁅x, y⁆, z⁆ = conj x (conj (y * x⁻¹) ⁅y⁻¹, z⁆ * ⁅y, z⁆) := by
  intro h
  have hc : Commute x z := by
    apply (commute_iff_eq x z).mpr
    exact triv_comm_iff_commutes.mp h
  have h1 : Commute x⁻¹ z := Commute.inv_left hc
  have h2: Commute x⁻¹ z⁻¹ := Commute.inv_inv_iff.mpr hc
  rw [commute_iff_eq] at h1 h2 hc
  simp [commutatorElement_def, ← mul_assoc, conj_apply]
  simp [mul_assoc, h1, h2, hc]

-- Hall Witt identities
theorem HW1 : ⁅⁅y, x⁆, conj x z⁆ * ⁅⁅x, z⁆, conj z y⁆ * ⁅⁅z, y⁆, conj y x⁆ = 1 := by
  simp only [commutatorElement_def, conj_apply, ← mul_assoc, inv_mul_cancel_right,
    mul_inv_rev, inv_inv, mul_inv_cancel_right, mul_inv_cancel]

theorem HW2 : (conj y ⁅⁅y⁻¹, x⁆, z⁆) * (conj z ⁅⁅z⁻¹, y⁆, x⁆) * (conj x ⁅⁅x⁻¹, z⁆, y⁆) = 1 := by
  simp only [commutatorElement_def, conj_apply, ← mul_assoc, inv_mul_cancel_right,
    mul_inv_rev, inv_inv, mul_inv_cancel_right, mul_inv_cancel, one_mul]

theorem commutatorElement_def' : ⁅ x, y ⁆ = (x * y) * (x⁻¹ * y⁻¹) := by group
```

Author source module Steinberg.Macro.Syntax

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

namespace Lean

def Syntax.mapIdent (syn : Syntax) (f : String → String) : Ident :=
  mkIdent <|
    match syn.getId with
    | .str .anonymous s => Name.mkStr .anonymous (f s)
    | i => i

def Syntax.mapIdent₂ (syn₁ syn₂ : Syntax) (f : String → String → String) : Ident :=
  mkIdent <|
    match syn₁.getId, syn₂.getId with
    | .str .anonymous s₁, .str .anonymous s₂ => Name.mkStr .anonymous (f s₁ s₂)
    | i₁, _ => i₁

def Syntax.mapIdent₃ (syn₁ syn₂ syn₃ : Syntax) (f : String → String → String → String) : Ident :=
  mkIdent <|
    match syn₁.getId, syn₂.getId, syn₃.getId with
    | .str .anonymous s₁, .str .anonymous s₂, .str .anonymous s₃ =>
      Name.mkStr .anonymous (f s₁ s₂ s₃)
    | i₁, _, _ => i₁

def Term.mapIdent (term : Term) (f : String → String) : Term :=
  term.raw.mapIdent f

```

```
def Term.mapIdent₂ (t₁ t₂ : Term) (f : String → String → String) : Term :=
  Syntax.mapIdent₂ t₁.raw t₂.raw f

def TSyntax.asIdent {kinds : SyntaxNodeKinds} (syn : TSyntax kinds) : TSyntax `ident :=
  syn.raw.mapIdent id

def TSyntax.mapIdent {kinds : SyntaxNodeKinds} (syn : TSyntax kinds) (f : String → String) : Ident :=
  syn.raw.mapIdent f

def TSyntax.mapIdent₂ {kinds : SyntaxNodeKinds} (s₁ s₂ : TSyntax kinds) (f : String → String → String) :
↪  Ident :=
  Syntax.mapIdent₂ s₁.raw s₂.raw f

def TSyntax.mapIdent₃ {kinds : SyntaxNodeKinds}
    (s₁ s₂ s₃ : TSyntax kinds) (f : String → String → String → String) : Ident :=
  Syntax.mapIdent₃ s₁.raw s₂.raw s₃.raw f
```

Author source module Steinberg.Defs.GradedPartialChevalleyGroup

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Defs.PositiveRootSystem
import Steinberg.Defs.Commutator

import Steinberg.Upstream.PresentedGroup
import Steinberg.Upstream.Commutator

import Steinberg.Macro.Group
import Steinberg.Macro.Syntax

/-!

  Implementation of graded unipotent Chevalley groups.

-/

namespace Steinberg

variable {Φ : Type TΦ} [PositiveRootSystem Φ]
         {R : Type TR} [Ring R]

open PositiveRootSystem PartialChevalleySystem

namespace GradedPartialChevalley

/-! ## Generators of graded unipotent Chevalley group -/

/--
  Generators of the graded unipotent Chevalley group corresponding to a positive root system
  over a ring with monomial entries.
-/
structure GradedChevalleyGenerator (Φ : Type TΦ) [PositiveRootSystem Φ] (R : Type TR) [Ring R] where
  mk ::
  ζ : Φ -- root
  i : ℕ -- degree
  hi : i ≤ height ζ
  t : R -- coefficient

namespace GradedChevalleyGenerator

instance instCoeProd : Coe ((ζ : Φ) × (i : ℕ) × (i ≤ height ζ) ×' R) (GradedChevalleyGenerator Φ R) :=
  ⟨fun ⟨ζ, i, hi, t⟩ => mk ζ i hi t⟩

/--
  Shorthand `{ζ, i, t}` for building free group elements from a root `ζ`, degree `i`, and coefficient `t`.

  Note: To re-use this notation for specific groups,
  re-define it for that group and set the priority higher.
  Then implement delaboration to use the delab defined below.
-/
scoped notation (priority:=1000) "{" ζ ", " i ", " t "}" =>
  FreeGroup.of (GradedChevalleyGenerator.mk ζ i (by ht) t)

/-- Inject a generator but with an explicit proof term provided. -/
```

```
scoped notation (priority:=1000) "{" ζ ", " i ", " t "}'" h:max =>
  FreeGroup.of (GradedChevalleyGenerator.mk ζ i h t)

/-- Inject an already-created generator, rather than data. -/
scoped notation (priority:=999) "{{" g "}}" =>
  FreeGroup.of g

section DelabBraces

open Lean PrettyPrinter Delaborator SubExpr

/--
  Delaborates `GradedChevalleyGenerator.mk` to use the `{ }` notation.

  Delaboration makes it so that the infoview uses the nice notation.
-/
@[delab app.Steinberg.GradedPartialChevalley.GradedChevalleyGenerator.mk]
def delab_generator_mk : Delab := do
  let e ← getExpr

  -- Only delab a full application of `.mk`
  guard $ e.isAppOfArity' ``GradedChevalleyGenerator.mk 8

  let ζ ← withNaryArg 4 delab
  let i ← withNaryArg 5 delab
  let t ← withNaryArg 7 delab
  `({ $ζ:term, $i:term, $t:term })


/--
  Delaborates `FreeGroup.of` to use the `{ }` notation.

  Delaboration makes it so that the infoview uses the nice notation.
-/
@[delab app.FreeGroup.of]
def delab_of : Delab := do
  let e ← getExpr

  -- Only delab `FreeGroup.of` if its type and value are given to the app
  guard $ e.isAppOfArity' ``FreeGroup.of 2

  -- Only delab `FreeGroup.of` if the type is a Chevalley generator
  let ty := e.getAppArgs.get! 0
  guard $ ty.isAppOfArity' ``GradedChevalleyGenerator 4

  -- Use the delaboration of the generator
  -- Since this might be a more complicated term or calculation,
  -- we don't call `delab_generator_mk` directly
  let g ← withNaryArg 1 delab
  `($g)

end DelabBraces /- section -/

/-! ### Equality theorems for generators -/

/-- Addition of degrees is commutative inside generators.  -/
theorem deg_add_comm (ζ : Φ) (i j : ℕ) (h : i + j ≤ height ζ) (t : R)
    : {ζ, i + j, t} = {ζ, j + i, t} := by
  congr 2
  exact add_comm i j

/- Addition of degrees is associative inside generators. -/
theorem deg_add_assoc (ζ : Φ) (i j k : ℕ) (h : i + j + k ≤ height ζ) (t : R)
    : {ζ, i + j + k, t} = {ζ, i + (j + k), t} := by
  congr 2
  exact add_assoc i j k

/- For a fixed root, generators are equal if they have the same degree and coefficient. -/
theorem eq_of_deg_coef_eq (ζ : Φ) {i : ℕ} (j : ℕ) (hij : i = j) {t : R} (u : R) (htu : t = u)
    : ∀ {_ : i ≤ height ζ}, {ζ, i, t} = {ζ, j, u} := by
  intros; congr 2

/- For a fixed root and coefficient, generators equal if they have the same degree. -/
theorem eq_of_deg_eq (ζ : Φ) {i : ℕ} (j : ℕ) (hij : i = j)
    : ∀ {_ : i ≤ height ζ} {t : R}, {ζ, i, t} = {ζ, j, t} := by
  intros; congr 2

/- For a fixed root and degree, generators equal if they have the same coefficient. -/
theorem eq_of_coef_eq (ζ : Φ) {t : R} (u : R) (h : t = u)
    : ∀ {i : ℕ} {_ : i ≤ height ζ}, {ζ, i, t} = {ζ, i, u} := by
```

```
  intros; congr 2

end GradedChevalleyGenerator

open GradedChevalleyGenerator

/-! ## Propositions expressing Steinberg relations

These functions each take a homomorphism `(f : FreeGroup (GradedChevalleyGenerator Φ R) →* G)`,
which embeds graded generators in a group `G`, as well as various data specifying one or more roots and coefficients,
and produce propositions asserting certain that certain equalities, namely, the Steinberg relations,
hold inside the group `G`. In our application, `f` will be `w.project` for `w` a `GradedPartialChevalleyGroup`,
defined below.
-/

section Props

variable {G : Type TG} [Group G]

/- The Steinberg commutator relation for a pair of roots spanning no additional roots. -/
def trivialSpanPropOfRootPair (f : FreeGroup (GradedChevalleyGenerator Φ R) →* G) (p : Φ × Φ) : Prop :=
  let (ζ, η) := p;
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ height ζ) (hj : j ≤ height η) (t u : R),
    ⁅ f {ζ, i, t}, f {η, j, u} ⁆ = 1

/- The Steinberg commutator relation for a pair of roots spanning one additional root. -/
def singleCommutatorPropOfRootPair (f : FreeGroup (GradedChevalleyGenerator Φ R) →* G) (p : SingleSpanRootPair Φ) : Prop :=
  let ⟨ ζ, η, θ, C, h_height ⟩ := p;
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ height ζ) (hj : j ≤ height η) (t u : R),
    ⁅ f {ζ, i, t}, f {η, j, u} ⁆ = f {θ, i + j, ↑C * t * u}

/- The Steinberg commutator relation for a pair of roots spanning two additional roots. -/
def doubleSpanPropOfRootPair (f : FreeGroup (GradedChevalleyGenerator Φ R) →* G) (p : DoubleSpanRootPair Φ) : Prop :=
  let ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩ := p;
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ height ζ) (hj : j ≤ height η) (t u : R),
    ⁅ f {ζ, i, t}, f {η, j, u} ⁆ = f {θ₁, i + j, ↑C₁ * t * u} * f {θ₂, i + 2 * j, ↑C₂ * t * u * u}

/- Generators for the same root commute. -/
def mixedDegreePropOfRoot (f : FreeGroup (GradedChevalleyGenerator Φ R) →* G) (ζ : Φ) : Prop :=
  @trivialSpanPropOfRootPair _ _ _ _ _ _ f (ζ, ζ)

set_option quotPrecheck false

/--
  Linearity of group elements on a particular root.

  Equivalent to `∀ (i : ℕ) (hi : i ≤ height ζ) (t u), f {ζ, i, t} * f {ζ, i, u} = f {ζ, i, t + u}`.

  `(f : FreeGroup (GradedChevalleyGenerator Φ R) →* G)`
  `(ζ : Φ)`
-/
scoped notation "lin_of_root" "(" f ", " ζ ")" =>
  ∀ ⦃i : ℕ⦄ (hi : i ≤ height ζ) (t u),
    f {ζ, i, t} * f {ζ, i, u} = f {ζ, i, t + u}

/--
  Ring coefficient 0 gives an identity element.

  Equivalent to `∀ (i : ℕ) (hi : i ≤ height ζ), f {ζ, i, 0} = 1`.

  `(f : FreeGroup (GradedChevalleyGenerator Φ R) →* G)`
  `(ζ : Φ)`
-/
scoped notation "id_of_root" "(" f ", " ζ ")" =>
  ∀ ⦃i : ℕ⦄ (hi : i ≤ height ζ),
    f {ζ, i, 0} = 1

/--
  Negating the coefficient inverts the generator.

  Equivalent to `∀ (i : ℕ) (hi : i ≤ height ζ) (t : R), (f {ζ, i, t})⁻¹ = 1`.

  `(f : FreeGroup (GradedChevalleyGenerator Φ R) →* G)`
  `(ζ : Φ)`
-/
scoped notation "inv_of_root" "(" f ", " ζ ")" =>
```

```
  ∀ ⦃i : ℕ⦄ (hi : i ≤ height ζ) (t),
    (f {ζ, i, t})⁻¹ = f {ζ, i, -t}

/- Linearity implies identity (essentially a standard fact about group homomorphisms). -/
-- TODO: Replace proof with map_one (use h_lin to show that t => f {ζ, i, t} is an instance of R →+ G)
theorem id_of_lin_of_root {f : FreeGroup (GradedChevalleyGenerator Φ R) →* G} {ζ : Φ}
    : lin_of_root(f, ζ) → id_of_root(f, ζ) := by
  intro h_lin i hi
  apply @mul_left_cancel _ _ _ (f {ζ, i, 0})
  rw [mul_one, h_lin, add_zero]

-- TODO: Replace proof with map_inv
/- Linearity implies inverse-ness (essentially a standard fact about group homomorphisms). -/
theorem inv_of_lin_of_root {f : FreeGroup (GradedChevalleyGenerator Φ R) →* G} {ζ : Φ}
    : lin_of_root(f, ζ) → inv_of_root(f, ζ) := by
  intro h_lin i hi t
  apply @mul_left_cancel _ _ _ (f {ζ, i, t})
  rw [mul_inv_cancel, h_lin, add_neg_cancel, id_of_lin_of_root h_lin]

end Props

/-! ### Sets of Steinberg relations

We will include these sets of relations when building `PresentedGroup`s below.
-/

/- The set of relations corresponding to `trivialSpanPropOfRootPair`. -/
def trivialSpanRelationsOfRootPair (R : Type TR) [Ring R] (p : Φ × Φ)
    : Set (FreeGroup (GradedChevalleyGenerator Φ R)) :=
  let (ζ, η) := p;
  { ⁅ {ζ, i, t}, {η, j, u} ⁆
    | (i : ℕ) (j : ℕ) (hi : i ≤ height ζ) (hj : j ≤ height η) (t : R) (u : R) }

/- The set of relations corresponding to `singleSpanPropOfRootPair`. -/
def singleSpanRelationsOfRootPair (R : Type TR) [Ring R] (p : SingleSpanRootPair Φ)
    : Set (FreeGroup (GradedChevalleyGenerator Φ R)) :=
  let ⟨ ζ, η, θ, C, h_height ⟩ := p;
  { ⁅ {ζ, i, t}, {η, j, u} ⁆ * {θ, i + j, C * t * u}⁻¹
    | (i : ℕ) (j : ℕ) (hi : i ≤ height ζ) (hj : j ≤ height η) (t : R) (u : R) }

/- The set of relations corresponding to `doubleSpanPropOfRootPair`. -/
def doubleSpanRelationsOfRootPair (R : Type TR) [Ring R] (p : DoubleSpanRootPair Φ)
    : Set (FreeGroup (GradedChevalleyGenerator Φ R)) :=
  let ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩ := p;
  { ⁅ {ζ, i, t}, {η, j, u} ⁆ *
    ({θ₁, i + j, C₁ * t * u} * {θ₂, i + 2 * j, C₂ * t * u * u})⁻¹
    | (i : ℕ) (j : ℕ) (hi : i ≤ height ζ) (hj : j ≤ height η) (t : R) (u : R) }

/- The set of relations corresponding to `mixedDegreePropOfRoot`. -/
def mixedDegreeRelationsOfRoot (R : Type TR) [Ring R] (ζ : Φ) : Set (FreeGroup (GradedChevalleyGenerator Φ
↪  R)) :=
  trivialSpanRelationsOfRootPair R (ζ, ζ)

/- The set of relations corresponding to `linearityPropOfRoot`. -/
def linearityRelationsOfRoot (R : Type TR) [Ring R] (ζ : Φ) : Set (FreeGroup (GradedChevalleyGenerator Φ
↪  R)) :=
  { {ζ, i, t} * {ζ, i, u} * {ζ, i, t + u}⁻¹
    | (i : ℕ) (hi : i ≤ height ζ) (t : R) (u : R) }

/-! ### Graded partial Chevalley groups -/

/- A structure which contains the data needed to define a graded unipotent Chevalley group
via a presentation (either the weak or full presentation).

This structure bundles together several data:
* A (possibly partial) set of Steinberg relations represented by a `PartialChevalleySystem`,
  either all the relations in the full case or the codimension-one relations in the weak case.
* A set of sets of "lifted" relations (no validity hypotheses for now).
* A "definition" map, used to relate generators whose roots may not be present to present generators.
-/
structure GradedPartialChevalleyGroup (Φ : Type TΦ) [PositiveRootSystem Φ] (R : Type TR) [Ring R] where
  mk ::
  sys : PartialChevalleySystem Φ
  liftedRelationsSets : Set (Set (FreeGroup (GradedChevalleyGenerator Φ R)))
  define : GradedChevalleyGenerator Φ R → FreeGroup (GradedChevalleyGenerator Φ R)

  h_define_of_present : ∀ {g : GradedChevalleyGenerator Φ R}, g.ζ ∈ sys.presentRoots → define g =
  ↪  FreeGroup.of g
  h_define_is_projection : ∀ {g : GradedChevalleyGenerator Φ R}, (FreeGroup.lift define) (define g) =
  ↪  define g
```

```
291
292   -- TODO: Ensure that everything in the image of `define` is actually present
293   -- TODO: validity hypothesis for liftedRelationsSets here?
294
295 namespace GradedPartialChevalleyGroup
296
297 open GradedPartialChevalleyGroup
298
299 /-- Construct a `GradedPartialChevalleyGroup` in the "full" case, where `liftedRelationsSets` is empty and
300 `define` does nothing. -/
301 def fullMk (Φ : Type TΦ) [PositiveRootSystem Φ] (R : Type TR) [Ring R] (sys : PartialChevalleySystem Φ)
302   : GradedPartialChevalleyGroup Φ R :=
303   GradedPartialChevalleyGroup.mk sys ∅ FreeGroup.of (by tauto) (by tauto)
304
305 /-! ### Defining the actual group
306
307 We now define the actual `PresentedGroup` corresponding to `GradedPartialChevalleyGroup`.
308 We first give definitions which "pack" together the set of relations, and then give theorems
309 which "unpack"
310 -/
311
312 /-- Inductive type enumerating the different classes of Steinberg relations
313 which appear in a `GradedPartialChevalleyGroup`. We opted for this format as it lets us
314 write the set of all relations defining the group as an indexed union, and therefore
315 makes it easy to unpack the set of all relations into subsets. -/
316 inductive GradedSteinbergRelationClass
317   | TrivialSpan | SingleSpan | DoubleSpan | MixedDegree | Linearity -- defined only by the
    ↪  `PartialChevalleySystem`
318   | Lifted | Definition -- defined by additional data in the `GradedPartialChevalleyGroup`
319
320 open GradedSteinbergRelationClass
321
322 /-! #### Packing the group -/
323
324 /-- The set of "definition" relations. Note: If `ζ` is a present root, then
325 `h_define_of_present` implies that
326   `w.define (GradedChevalleyGenerator.mk ζ i hi t) = {ζ, i, t}`
327 and therefore the corresponding relation is trivial. On the other hand,
328 if `ζ` is a missing root, this relation will typically be the only
329 `GradedPartialChevalleyGroup` relation which directly mentions `ζ`'s generators.
330  -/
331 def definitionRelations (w : GradedPartialChevalleyGroup Φ R) :=
332   ⋃ (ζ : Φ), {
333     {ζ, i, t}⁻¹ * w.define (GradedChevalleyGenerator.mk ζ i hi t) | (i : ℕ) (hi : i ≤ height ζ) (t : R)
334   }
335
336 /-- The set of all relations which define a `GradedPartialChevalleyGroup`. An indexed union
337 over `GradedSteinbergRelationClass`es. -/
338 def allRelations (w : GradedPartialChevalleyGroup Φ R) :=
339   ⋃ (K : GradedSteinbergRelationClass),
340   (
341     match K with
342     | TrivialSpan =>
343         ⋃ (p ∈ w.sys.trivialSpanRootPairs), trivialSpanRelationsOfRootPair R p
344     | SingleSpan =>
345         ⋃ (p ∈ w.sys.singleSpanRootPairs), singleSpanRelationsOfRootPair R p
346     | DoubleSpan =>
347         ⋃ (p ∈ w.sys.doubleCommutatorRootPairs), doubleSpanRelationsOfRootPair R p
348     | MixedDegree =>
349         ⋃ (ζ ∈ w.sys.presentRoots), mixedDegreeRelationsOfRoot R ζ
350     | Linearity =>
351         ⋃ (ζ ∈ w.sys.presentRoots), linearityRelationsOfRoot R ζ
352     | Lifted =>
353         ⋃₀ liftedRelationsSets w
354     | Definition =>
355         w.definitionRelations
356   )
357
358 /-- The presented group on `GradedChevalleyGenerator`s given by modding out all the relations in
    ↪  `allRelations`. -/
359 abbrev group (w : GradedPartialChevalleyGroup Φ R) :=
360   PresentedGroup (GradedPartialChevalleyGroup.allRelations w)
361
362 /-- The projection from the `FreeGroup` on `GradedChevalleyGenerator`s to the `PresentedGroup`
363 given by our subset of relations. -/
364 def project (w : GradedPartialChevalleyGroup Φ R) : FreeGroup (GradedChevalleyGenerator Φ R) →* group w :=
365   PresentedGroup.mk (GradedPartialChevalleyGroup.allRelations w)
366
367 /-! #### Unpacking the group
368
```

```
These are 'helper' theorems which let us deduce certain equalities which hold within the
`PresentedGroup` formed by the data in a `GradedPartialChevalleyGroup`.
-/

/-- If a pair of roots occurs in the trivial-span field for the underlying `PartialChevalleySystem`,
then we get a trivial-span theorem for these roots inside the `PresentedGroup`. -/
theorem trivialSpanProp_of_mem_trivialSpanRoot_pairs {w : GradedPartialChevalleyGroup Φ R} {p : Φ × Φ}
    (h : p ∈ w.sys.trivialSpanRootPairs)
      : trivialSpanPropOfRootPair w.project p := by
  intro i j hi hj t u
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists TrivialSpan
  simp only [Set.mem_iUnion]
  exists p, h
  rw [trivialSpanRelationsOfRootPair]
  exists i, j, hi, hj, t, u

/-- If a pair of roots occurs in the single-span field for the underlying `PartialChevalleySystem`,
then we get a single-span theorem for these roots inside the `PresentedGroup`. -/
theorem singleSpanProp_of_mem_singleSpanRoot_pairs (w : GradedPartialChevalleyGroup Φ R) (p :
↪  SingleSpanRootPair Φ)
  (h : p ∈ w.sys.singleSpanRootPairs)
    : singleCommutatorPropOfRootPair w.project p := by
  intro i j hi hj t u
  apply eq_of_mul_inv_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists SingleSpan
  simp only [Set.mem_iUnion]
  exists p, h
  rw [singleSpanRelationsOfRootPair]
  exists i, j, hi, hj, t, u

/-- If a pair of roots occurs in the single-span field for the underlying `PartialChevalleySystem`,
then we get a single-span theorem for these roots inside the `PresentedGroup`. -/
theorem doubleSpanProp_of_mem_doubleSpanRootPairs (w : GradedPartialChevalleyGroup Φ R) (p :
↪  DoubleSpanRootPair Φ)
  (h : p ∈ w.sys.doubleCommutatorRootPairs)
    : doubleSpanPropOfRootPair w.project p := by
  intro i j hi hj t u
  apply eq_of_mul_inv_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists DoubleSpan
  simp only [Set.mem_iUnion]
  exists p, h
  rw [doubleSpanRelationsOfRootPair]
  exists i, j, hi, hj, t, u

/-- If a root is present for the underlying `PartialChevalleySystem`,
then we get a mixed-degree theorem for this root inside the `PresentedGroup`. -/
theorem mixedDegreeProp_of_mem_presentRoots (w : GradedPartialChevalleyGroup Φ R)
  {ζ : Φ} (h : ζ ∈ w.sys.presentRoots)
    : mixedDegreePropOfRoot w.project ζ := by
  intro i j hi hj t u
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists MixedDegree
  simp only [Set.mem_iUnion]
  exists ζ, h
  rw [mixedDegreeRelationsOfRoot]
  exists i, j, hi, hj, t, u

/-- If a root is present for the underlying `PartialChevalleySystem`,
then we get a linearity theorem for this root inside the `PresentedGroup`. -/
theorem lin_of_root_of_mem_presentRoots (w : GradedPartialChevalleyGroup Φ R)
  {ζ : Φ} (h : ζ ∈ w.sys.presentRoots)
    : lin_of_root(w.project, ζ) := by
  intro i hi t u
  apply eq_of_mul_inv_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists Linearity
  simp only [Set.mem_iUnion]
  exists ζ, h
  rw [linearityRelationsOfRoot]
  exists i, hi, t, u

/-- Every "lifted" relation we assume holds inside the `PresentedGroup`. -/
```

```
theorem liftedProp_of_mem_lifted (w : GradedPartialChevalleyGroup Φ R)
    : ∀ S ∈ w.liftedRelationsSets, ∀ r ∈ S, w.project r = 1 := by
  intro S _ _ _
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists Lifted
  apply Set.mem_sUnion.mpr
  exists S

/-- Every "definition" we assume holds inside the `PresentedGroup`. -/
theorem definitionProp_of_define (w : GradedPartialChevalleyGroup Φ R)
    : ∀ (ζ : Φ) (i : ℕ) (hi : i ≤ height ζ) (t : R),
        w.project {ζ, i, t} = w.project (w.define (GradedChevalleyGenerator.mk ζ i hi t)) := by
  intro ζ i hi t
  apply eq_of_inv_mul_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists Definition
  simp only [definitionRelations, Set.mem_iUnion]
  exists ζ
  simp only [Set.mem_setOf_eq]
  exists i, hi, t


/-- This theorem is used to create a homomorphism between two `GradedPartialChevalleyGroup`s (on
the same underlying positive root system `Φ` and ring `R`). It gives a helpful sufficient condition
under which every relation holding in one group also holds in another group. This condition essentially
states that these relations hold trivially when the groups' underlying `PartialChevalleySystem`
structures overlap. Hence, we do not have to worry about the sets of relations arising in these cases. -/
theorem graded_injection (w₁ w₂ : GradedPartialChevalleyGroup Φ R)
  (h_good :
  ∀ (K : GradedSteinbergRelationClass),
    match K with
    | TrivialSpan =>
      ∀ p ∈ w₁.sys.trivialSpanRootPairs, p ∈ w₂.sys.trivialSpanRootPairs ∨
        (∀ r ∈ (trivialSpanRelationsOfRootPair R p), w₂.project r = 1)
    | SingleSpan =>
      ∀ p ∈ w₁.sys.singleSpanRootPairs, p ∈ w₂.sys.singleSpanRootPairs ∨
        (∀ r ∈ (singleSpanRelationsOfRootPair R p), w₂.project r = 1)
    | DoubleSpan =>
      ∀ p ∈ w₁.sys.doubleCommutatorRootPairs, p ∈ w₂.sys.doubleCommutatorRootPairs ∨
        (∀ r ∈ (doubleSpanRelationsOfRootPair R p), w₂.project r = 1)
    | MixedDegree =>
      ∀ p ∈ w₁.sys.presentRoots, p ∈ w₂.sys.presentRoots ∨
        (∀ r ∈ (mixedDegreeRelationsOfRoot R p), w₂.project r = 1)
    | Linearity =>
      ∀ p ∈ w₁.sys.presentRoots, p ∈ w₂.sys.presentRoots ∨
        (∀ r ∈ (linearityRelationsOfRoot R p), w₂.project r = 1)
    | Lifted =>
      ∀ S ∈ w₁.liftedRelationsSets, ∀ p ∈ S, w₂.project p = 1
    | Definition =>
      ∀ p ∈ w₁.definitionRelations, w₂.project p = 1
  )
  : ∀ r ∈ w₁.allRelations, w₂.project r = 1 := by
  simp only [allRelations]
  intro r h
  simp only [Set.mem_iUnion] at h
  rcases h with ⟨ K, h ⟩
  rcases h_K : K
  all_goals (
    simp only [h_K] at h
    specialize h_good K
    simp only [h_K] at h_good
  )
  any_goals (
    simp only [Set.mem_iUnion, exists_prop] at h
    rcases h with ⟨ p, h_p, h_r_p ⟩
    specialize h_good p h_p
    rcases h_good with h|h
    · apply eq_one_of_mem_rels
      simp only [allRelations, Set.mem_iUnion]
      exists K
      rw [h_K]
      simp only [Set.mem_iUnion, exists_prop]
      exists p
    · tauto
  )
  · simp only [Set.mem_sUnion] at h
    tauto
  · tauto
```

```
527
528 open Lean PrettyPrinter Delaborator SubExpr in
529 /--
530   Delaborator for `project` when it's an application.
531
532   Note that this will obscure the widgets on the infoview, such that
533   hovering over the group elements won't bring you back to `project`.
534 -/
535 @[delab app.DFunLike.coe]
536 def delab_project' : Delab := do
537   withOverApp 6 do
538     let e ← getExpr
539     let mkApp5 (.const ``project _) _ _ _ _ _ := e.appFn!.appArg!' | failure
540     let f_mk_mk ← withNaryArg 5 delab
541     `($f_mk_mk)
542
543 section declareThms
544
545 open Lean Parser.Tactic
546 set_option hygiene false
547
548 def makeCommands (m : MacroM Syntax) : MacroM (TSyntax `command) := do
549   let cmds ← Syntax.getArgs <$> m
550   return ⟨mkNullNode cmds⟩
551
552 macro "declare_trivial_span_expr_thm" w:ident R:term:arg r₁:term:arg r₂:term:arg : command => do
553   let exprAs := TSyntax.mapIdent₂ r₁ r₂
554     (fun s₁ s₂ => "expr_" ++ s₁ ++ "_" ++ s₂ ++ "_as_" ++ s₂ ++ "_" ++ s₁)
555   let commName := TSyntax.mapIdent₂ r₁ r₂
556     (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
557   let commOf ← `(rwRule| $commName:term)
558   makeCommands `(section
559     @[group_reassoc] theorem $exprAs
560       : ∀ ⦃i j : ℕ⦄ (hi : i ≤ height $r₁) (hj : j ≤ height $r₂) (t u : $R),
561         commutes(($w $R).project {$r₁:term, i, t},
562                  ($w $R).project {$r₂:term, j, u}) := by
563       intro i j hi hj t u
564       apply triv_comm_iff_commutes.mp
565       rw [$commOf]
566       <;> try assumption
567   end)
568
569 macro "declare_trivial_span_of_root_pair_thms" w:ident R:term:arg r₁:term:arg r₂:term:arg : command => do
570   let commOf := TSyntax.mapIdent₂ r₁ r₂ (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
571   makeCommands `(section
572     theorem $commOf : trivialSpanPropOfRootPair ($w $R).project ($r₁, $r₂) :=
573       ($w $R).trivialSpanProp_of_mem_trivialSpanRoot_pairs (by unfold $w; simp only; tauto)
574     declare_trivial_span_expr_thm $w $R $r₁ $r₂
575   end)
576
577 macro "declare_single_span_expr_thms" w:ident R:term:arg r₁:term:arg r₂:term:arg r₃:term:arg n:num :
    ↪  command => do
578   let innerTerm ←
579     if n.getNat = 1 then `(t * u)
580     else                 `($n * t * u)
581   let commOf := TSyntax.mapIdent₂ r₁ r₂ (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
582   let exprAs := TSyntax.mapIdent₃ r₁ r₂ r₃
583     (fun s₁ s₂ s₃ => "expr_" ++ s₃ ++ "_as_" ++ s₁ ++ "_" ++ s₂ ++ "_" ++ s₁ ++ "_" ++ s₂)
584   let exprAsRev := TSyntax.mapIdent₃ r₁ r₂ r₃
585     (fun s₁ s₂ s₃ => "expr_" ++ s₁ ++ "_" ++ s₂ ++ "_as_" ++ s₃ ++ "_" ++ s₂ ++ "_" ++ s₁)
586   makeCommands `(section
587     theorem $exprAs
588       : ∀ ⦃i j : ℕ⦄ (hi : i ≤ height $r₁) (hj : j ≤ height $r₂) (t u : $R),
589         (($w $R).project
590           {$r₃:term, i + j, $innerTerm})
591           = ($w $R).project {$r₁:term, i, t}
592             * ($w $R).project {$r₂:term, j, u}
593             * ($w $R).project {$r₁:term, i, -t}
594             * ($w $R).project {$r₂:term, j, -u} := by
595       intro i j hi hj t u
596       have := $commOf hi hj t u
597       chev_simp [commutatorElement_def, one_mul, mul_one] at this
598       symm at this
599       exact this
600
601     @[group_reassoc]
602     theorem $exprAsRev
603       : ∀ ⦃i j : ℕ⦄ (hi : i ≤ height $r₁) (hj : j ≤ height $r₂) (t u : $R),
604         reorder_left(
605           ($w $R).project {$r₁:term, i, t},
```

```
          ($w $R).project {$r₂:term, j, u},
          (($w $R).project {$r₃:term, i + j, $innerTerm})
        ) := by
      intro i j hi hj t u
      have := $commOf hi hj t u
      chev_simp [commutatorElement_def, one_mul, mul_one] at this
      grw [← this]
  end)

macro "declare_single_span_of_root_pair_thms"
    w:ident R:term:arg
    r₁:term:arg r₂:term:arg r₃:term:arg n:num : command => do
  let commOf := TSyntax.mapIdent₂ r₁ r₂ (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
  makeCommands `(section
    theorem $commOf : singleCommutatorPropOfRootPair ($w $R).project ⟨$r₁, $r₂, $r₃, $n, rfl⟩ :=
      ($w $R).singleSpanProp_of_mem_singleSpanRoot_pairs ⟨$r₁, $r₂, $r₃, $n, rfl⟩ (by unfold $w; simp
      ↪  only; tauto)
    declare_single_span_expr_thms $w $R $r₁ $r₂ $r₃ $n
  end)

macro "declare_lin_id_inv_thms" w:ident R:term:arg root:term:arg : command => do
  let linOf := root.mapIdent ("lin_of_" ++ ·)
  let idOf := root.mapIdent ("id_of_" ++ ·)
  let invOf := root.mapIdent ("inv_of_" ++ ·)
  makeCommands `(section
    @[group_reassoc (attr := simp, chev_simps)]
    theorem $linOf : lin_of_root(($w $R).project, $root) :=
      ($w $R).lin_of_root_of_mem_presentRoots (by unfold $w; simp only; tauto)

    @[simp, chev_simps]
    theorem $idOf : id_of_root(($w $R).project, $root) :=
      id_of_lin_of_root $linOf

    @[simp, chev_simps]
    theorem $invOf : inv_of_root(($w $R).project, $root) :=
      inv_of_lin_of_root $linOf
  end)

macro "declare_mixed_degree_expr_thm" w:ident R:term:arg r:term:arg : command => do
  let mixedName := r.mapIdent ("comm_of_" ++ ·)
  let mixedRw ← `(rwRule| $mixedName:term)
  let exprName := r.mapIdent (fun s => "expr_" ++ s ++ "_" ++ s ++ "_as_" ++ s ++ "_" ++ s)
  makeCommands `(section
    @[group_reassoc]
    theorem $exprName :
        ∀ ⦃i j : ℕ⦄ (hi : i ≤ height $r) (hj : j ≤ height $r) (t u : $R),
          commutes(($w $R).project {$r:term, i, t},
                   ($w $R).project {$r:term, j, u}) := by
      intro i j hi hj t u
      apply triv_comm_iff_commutes.mp
      rw [$mixedRw]
      <;> try assumption
  end)

macro "declare_mixed_degree_thms" w:ident R:term:arg r:term:arg : command => do
  let mixedName := r.mapIdent ("comm_of_" ++ ·)
  makeCommands `(section
    theorem $mixedName : mixedDegreePropOfRoot ($w $R).project $r :=
      mixedDegreeProp_of_mem_presentRoots ($w $R)
        (by unfold $w; simp only; tauto)
    declare_mixed_degree_expr_thm $w $R $r
  end)

macro "declare_defineThenReflect_thm" w:ident R:term:arg RS:ident r:term:arg : command => do
  let thmName := r.mapIdent ("defineThenReflect_eq_reflect_of_" ++ ·)
  makeCommands `(section
    theorem $thmName
        (g : GradedChevalleyGenerator $RS $R) (h : g.ζ = $r)
        : ($w $R).project (defineThenReflect ($w $R) g)
            = ($w $R).project (FreeGroup.of (reflect g)) := by
      congr
      apply defineThenReflect_eq_reflect_of_mem_presentRoots
      unfold $w
      simp only [h]
      tauto
  end)

-- r₁ is the larger root, as opposed to the above macros
macro "declare_reflected_thm" w:ident R:term:arg v:term:arg
        r₁:term:arg r₂:term:arg r₃:term:arg isNeg:num C:num
```

```
      n₁:num n₂:num n₃:num n₄:num n₅:num n₆:num : command => do
  let innerTerm ← 
    match isNeg.getNat, C.getNat with
    | 0, 1 => `(t * u)
    | 1, 1 => `(-t * u)
    | 1, _ => `(-$C * t * u)
    | _, _ => `($C * t * u)

  let thmName := TSyntax.mapIdent₃ r₁ r₂ r₃
    (fun s₁ s₂ s₃ => "expr_" ++ s₁ ++ "_as_comm_of_" ++ s₂ ++ "_" ++ s₃ ++ s!"_{n₂.getNat}{n₃.getNat}")

  let defineThenReflect₁ := r₁.mapIdent ("defineThenReflect_eq_reflect_of_" ++ ·)
  let defineThenReflect_thm₁ ← `(simpLemma| $defineThenReflect₁:term)

  let defineThenReflect₂ := r₂.mapIdent ("defineThenReflect_eq_reflect_of_" ++ ·)
  let defineThenReflect_thm₂ ← `(simpLemma| $defineThenReflect₂:term)
  let defineThenReflect_thm₂' ← `(simpLemma| $defineThenReflect₂:term (by assumption))

  let defineThenReflect₃ := r₃.mapIdent ("defineThenReflect_eq_reflect_of_" ++ ·)
  let defineThenReflect_thm₃ ← `(simpLemma| $defineThenReflect₃:term)
  let defineThenReflect_thm₃' ← `(simpLemma| $defineThenReflect₃:term (by assumption))

  let exprLemma := TSyntax.mapIdent₃ r₁ r₂ r₃
    (fun s₁ s₂ s₃ => "expr_" ++ s₁ ++ "_as_comm_of_" ++ s₂ ++ "_" ++ s₃ ++ s!"_{n₅.getNat}{n₆.getNat}")
  let exprLemmaRw ← `(rwRule| $exprLemma:term)

  makeCommands `(section
    lemma $thmName :
        ∀ (t u : $R),
          (($w $R).project {$r₁:term, $n₁, $innerTerm})
            = [($w $R).project {$r₂:term, $n₂, t}, ($w $R).project {$r₃:term, $n₃, u}] := by
      intro t u
      have : ($w $R).project {$r₁:term, $n₁, $innerTerm}
        = defineThenReflectOfPresentedGroup $v (($w $R).project {$r₁:term, $n₄, $innerTerm}) := by
          simp only [defineThenReflectOfPresentedGroup_of_project, FreeGroup.lift.of,
            $defineThenReflect_thm₁, reflect, PositiveRootSystem.height, height]
      rw [this]; clear this
      have : [($w $R).project {$r₂:term, $n₂, t}, ($w $R).project {$r₃:term, $n₃, u}]
          = defineThenReflectOfPresentedGroup $v
              [($w $R).project {$r₂:term, $n₅, t}, ($w $R).project {$r₃:term, $n₆, u}] := by
        -- Sometimes, the `defineThenReflect_` lemmas require the `Fchar` assumption
        first
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₂, $defineThenReflect_thm₃, reflect, PositiveRootSystem.height, height]
          done
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₂', $defineThenReflect_thm₃, reflect, PositiveRootSystem.height, height]
          done
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₂, $defineThenReflect_thm₃', reflect, PositiveRootSystem.height, height]
          done
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₂', $defineThenReflect_thm₃', reflect, PositiveRootSystem.height,
            ↪  height]
      rw [this, $exprLemmaRw]
      <;> try assumption
  end)

macro "declare_triv_comm_reflected_thm" w:ident R:term:arg v:term:arg
        r₁:term:arg r₂:term:arg
        n₁:num n₂:num n₃:num n₄:num : command => do
  let commOf := TSyntax.mapIdent₂ r₁ r₂
    (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂ ++ s!"_{n₁.getNat}{n₂.getNat}")
  let commLemma := TSyntax.mapIdent₂ r₁ r₂
    (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂ ++ s!"_{n₃.getNat}{n₄.getNat}")
  let commLemmaRw ← `(rwRule| $commLemma:term)

  let defineThenReflect₁ := r₁.mapIdent ("defineThenReflect_eq_reflect_of_" ++ ·)
  let defineThenReflect_thm₁ ← `(simpLemma| $defineThenReflect₁:term)
  let defineThenReflect_thm₁' ← `(simpLemma| $defineThenReflect₁:term (by assumption))

  let defineThenReflect₂ := r₂.mapIdent ("defineThenReflect_eq_reflect_of_" ++ ·)
  let defineThenReflect_thm₂ ← `(simpLemma| $defineThenReflect₂:term)
  let defineThenReflect_thm₂' ← `(simpLemma| $defineThenReflect₂:term (by assumption))


```

```
  makeCommands `(section
    lemma $commOf : ∀ (t u : $R),
        ⁅ ($w $R).project {$r₁:term, $n₁, t}, ($w $R).project {$r₂:term, $n₂, u} ⁆ = 1 := by
      intro t u
      have : ⁅ ($w $R).project {$r₁:term, $n₁, t}, ($w $R).project {$r₂:term, $n₂, u} ⁆
        = defineThenReflectOfPresentedGroup $v
            ⁅ ($w $R).project {$r₁:term, $n₃, t}, ($w $R).project {$r₂:term, $n₄, u} ⁆ := by
        -- Sometimes, the `defineThenReflect_` lemmas require the `Fchar` assumption
        first
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₁, $defineThenReflect_thm₂, reflect, PositiveRootSystem.height, height]
          done
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₁', $defineThenReflect_thm₂, reflect, PositiveRootSystem.height, height]
          done
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₁, $defineThenReflect_thm₂', reflect, PositiveRootSystem.height, height]
          done
        | simp only [map_commutatorElement, defineThenReflectOfPresentedGroup_of_project,
        ↪  FreeGroup.lift.of,
            $defineThenReflect_thm₁', $defineThenReflect_thm₂', reflect, PositiveRootSystem.height,
            ↪  height]
      rw [this, $commLemmaRw]
      <;> try assumption
      rfl
  end)

end declareThms /- section -/

end GradedPartialChevalleyGroup
```

Author source module Steinberg.Defs.PartialChevalleyGroup

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Defs.PositiveRootSystem
import Steinberg.Defs.Commutator

import Steinberg.Upstream.PresentedGroup
import Steinberg.Upstream.Commutator

import Steinberg.Macro.Group
import Steinberg.Macro.Syntax


/-!

  Implementation of (ungraded) unipotent Chevalley groups.

  *NOTE:* Cleanup in this file lags behind `GradedPartialChevalleyGroup.lean`.

-/

namespace Steinberg

variable {G : Type TG} [Group G]
         {Φ : Type TΦ} [PositiveRootSystem Φ]
         {R : Type TR} [Ring R]

open PositiveRootSystem PartialChevalleySystem

namespace PartialChevalley

/-! ## Generators of (ungraded) unipotent Chevalley group -/

/--
  Generators of the (ungraded) unipotent Chevalley group corresponding to a positive root system
  over a ring.
-/
structure ChevalleyGenerator (Φ : Type TΦ) [PositiveRootSystem Φ] (R : Type TR) [Ring R] where
```

```
  mk ::
  ζ : Φ -- root
  t : R -- coefficient (ring element)

namespace ChevalleyGenerator

instance instCoeProd : Coe (Φ × R) (ChevalleyGenerator Φ R) :=
  ⟨fun p => ⟨p.1, p.2⟩⟩

/--
  Shorthand `{ζ, t}` for building free group elements from a root `ζ` and coefficient `t`.

  Note: To re-use this notation for specific groups,
  re-define it for that group and set the priority higher.
  Then implement delaboration to use the delab defined below.
-/
scoped notation (priority:=1000) "{" ζ ", " t "}" =>
  FreeGroup.of (ChevalleyGenerator.mk ζ t)

section DelabBraces

open Lean PrettyPrinter Delaborator SubExpr

/--
  Delaborates `ChevalleyGenerator.mk` to use the `{ }` notation.

  Delaboration makes it so that the infoview uses the nice notation.
-/
@[delab app.Steinberg.PartialChevalley.ChevalleyGenerator.mk]
def delab_generator_mk : Delab := do
  let e ← getExpr

  -- Only delab a full application of `.mk`
  guard $ e.isAppOfArity' ``ChevalleyGenerator.mk 6

  let ζ ← withNaryArg 4 delab
  let t ← withNaryArg 5 delab
  `({ $ζ:term, $t:term })


/--
  Delaborates `FreeGroup.of` to use the `{ }` notation.

  Delaboration makes it so that the infoview uses the nice notation.
-/
@[delab app.FreeGroup.of]
def delab_free_mk : Delab := do
  let e ← getExpr

  -- Only delab `FreeGroup.of` if its type and value are given to the app
  guard $ e.isAppOfArity' ``FreeGroup.of 2

  -- Only delab `FreeGroup.of` if the type is a Chevalley generator
  let ty := e.getAppArgs.get! 0
  guard $ ty.isAppOfArity' ``ChevalleyGenerator 4

  -- Use the delaboration of the generator
  -- Since this might be a more complicated term or calculation,
  -- we don't call `delab_generator_mk` directly
  let g ← withNaryArg 1 delab
  `($g)

end DelabBraces /- section -/

theorem eq_of_coef_eq (ζ : Φ) {t : R} (u : R) (h : t = u)
    : {ζ, t} = {ζ, u} := by
  congr

end ChevalleyGenerator

open ChevalleyGenerator

/-! ### Statements about generators which we assume and/or prove -/

section Relations

/-! #### Commutator for generators from two roots which span no additional roots -/

/- Theorem stating that commutator of generators for two roots vanishes. -/
def trivialSpanPropOfRootPair (f : FreeGroup (ChevalleyGenerator Φ R) →* G) (p : Φ × Φ) : Prop :=
```

```
  let (ζ, η) := p;
  ∀ (t u : R),
    ⁅ f {ζ, t}, f {η, u} ⁆ = 1

/-
The set of elements which must vanish according to the theorem that the commutator of generators
for two roots vanishes. (Used to construct a `PresentedGroup`.)
-/
def trivialSpanRelationsOfRootPair (R : Type TR) [Ring R] (p : Φ × Φ)
    : Set (FreeGroup (ChevalleyGenerator Φ R)) :=
  let (ζ, η) := p;
  { ⁅ {ζ, t}, {η, u} ⁆
    | (t : R) (u : R) }

/-! #### Commutator for two generators from two roots which span one additional root -/

def singleCommutatorPropOfRootPair (f : FreeGroup (ChevalleyGenerator Φ R) →* G) (p : SingleSpanRootPair
↪  Φ) : Prop :=
  let ⟨ ζ, η, θ, C, _ ⟩ := p;
  ∀ (t u : R),
    ⁅ f {ζ, t}, f {η, u} ⁆ = f {θ, ↑C * t * u}

def singleSpanRelationsOfRootPair (R : Type TR) [Ring R] (p : SingleSpanRootPair Φ) : Set (FreeGroup
↪  (ChevalleyGenerator Φ R)) :=
  let ⟨ ζ, η, θ, C, _ ⟩ := p;
  { ⁅ {ζ, t}, {η, u} ⁆ * {θ, C * t * u}⁻¹
    | (t : R) (u : R) }

/-! #### Commutator for two generators from two roots which span one additional root -/

def doubleSpanPropOfRootPair (f : FreeGroup (ChevalleyGenerator Φ R) →* G) (p : DoubleSpanRootPair Φ) :
↪  Prop :=
  let ⟨ ζ, η, θ₁, θ₂, C₁, C₂, _, _ ⟩ := p;
  ∀ (t u : R),
    ⁅ f {ζ, t}, f {η, u} ⁆ = f {θ₁, ↑C₁ * t * u} * f {θ₂, ↑C₂ * t * u * u}

def doubleSpanRelationsOfRootPair (R : Type TR) [Ring R] (p : DoubleSpanRootPair Φ) : Set (FreeGroup
↪  (ChevalleyGenerator Φ R)) :=
  let ⟨ ζ, η, θ₁, θ₂, C₁, C₂, _, _ ⟩ := p;
  { ⁅ {ζ, t}, {η, u} ⁆ *
    ({θ₁, C₁ * t * u} * {θ₂, C₂ * t * u * u})⁻¹
    | (t : R) (u : R) }

/-! #### Linearity relation for products of generators from a single root -/

/- Linearity of coefficients for products of generators of a single root (with the same degree). -/
def linearityRelationsOfRoot (R : Type TR) [Ring R] (ζ : Φ) : Set (FreeGroup (ChevalleyGenerator Φ R)) :=
  { {ζ, t} * {ζ, u} * {ζ, t + u}⁻¹
    | (t : R) (u : R) }

end Relations

/-! ### Additional properties implied by linearity and implications therein -/

section ofRoot

set_option quotPrecheck false

/--
  Linearity of group elements on a particular root.

  Equivalent to `∀ (t u), f {ζ, t} * f {ζ, i, u} = f {ζ, i, t + u}`.

  `(f : FreeGroup (ChevalleyGenerator Φ R) →* G)`
  `(ζ : Φ)`
-/
scoped notation "lin_of_root" "(" f ", " ζ ")" =>
  ∀ (t u), f {ζ, t} * f {ζ, u} = f {ζ, t + u}

/--
  Ring coefficient 0 gives an identity element.

  Equivalent to `f {ζ, i, 0} = 1`.

  `(f : FreeGroup (ChevalleyGenerator Φ R) →* G)`
  `(ζ : Φ)`
-/
scoped notation "id_of_root" "(" f ", " ζ ")" =>
  f {ζ, 0} = 1
/--
```

```
  Negating the coefficient inverts the generator.

  Equivalent to `∀ (t : R), (f {ζ, t})⁻¹ = 1`.

  `(f : FreeGroup (ChevalleyGenerator Φ R) →* G)`
  `(ζ : Φ)`
-/
scoped notation "inv_of_root" "(" f ", " ζ ")" =>
  ∀ (t), (f {ζ, t})⁻¹ = f {ζ, -t}

/- Linearity implies identity (essentially a standard fact about group homomorphisms). -/
theorem id_of_lin_of_root {f : FreeGroup (ChevalleyGenerator Φ R) →* G} {ζ : Φ}
    : lin_of_root(f, ζ) → id_of_root(f, ζ) := by
  intro h_lin
  apply @mul_left_cancel _ _ _ (f {ζ, 0})
  rw [mul_one, h_lin, add_zero]

/- Linearity implies inverse-ness (essentially a standard fact about group homomorphisms). -/
theorem inv_of_lin_of_root {f : FreeGroup (ChevalleyGenerator Φ R) →* G} {ζ : Φ}
    : lin_of_root(f, ζ) → inv_of_root(f, ζ) := by
  intro h_lin t
  apply @mul_left_cancel _ _ _ (f {ζ, t})
  rw [mul_inv_cancel, h_lin, add_neg_cancel, id_of_lin_of_root h_lin]

end ofRoot

structure PartialChevalleyGroup (Φ : Type TΦ) [PositiveRootSystem Φ] (R : Type TR) [Ring R] where
  mk ::
  sys : PartialChevalleySystem Φ
  define : ChevalleyGenerator Φ R → FreeGroup (ChevalleyGenerator Φ R)
  h_define_of_present : ∀ {g : ChevalleyGenerator Φ R}, g.ζ ∈ sys.presentRoots → define g = FreeGroup.of g
  h_define_is_projection : ∀ {g : ChevalleyGenerator Φ R}, (FreeGroup.lift define) (define g) = define g

namespace PartialChevalleyGroup

open PartialChevalleyGroup

def fullMk (Φ : Type TΦ) [PositiveRootSystem Φ] (R : Type TR) [Ring R] (sys : PartialChevalleySystem Φ)
  : PartialChevalleyGroup Φ R :=
  PartialChevalleyGroup.mk sys FreeGroup.of (by tauto) (by tauto)

/-! ### Sets of relations -/

inductive SteinbergRelationClass
  | TrivialSpan | SingleSpan | DoubleSpan | Linearity | Definition
open SteinbergRelationClass

def definitionRelations (w : PartialChevalleyGroup Φ R) : Set (FreeGroup (ChevalleyGenerator Φ R)) :=
  ⋃ (ζ : Φ), {
      {ζ, t}⁻¹ * w.define (ChevalleyGenerator.mk ζ t) | (t : R)
  }

def allRelations (w : PartialChevalleyGroup Φ R) :=
  ⋃ (K : SteinbergRelationClass),
  (
    match K with
    | TrivialSpan =>
        ⋃ (p ∈ w.sys.trivialSpanRootPairs), trivialSpanRelationsOfRootPair R p
    | SingleSpan =>
        ⋃ (p ∈ w.sys.singleSpanRootPairs), singleSpanRelationsOfRootPair R p
    | DoubleSpan =>
        ⋃ (p ∈ w.sys.doubleCommutatorRootPairs), doubleSpanRelationsOfRootPair R p
    | Linearity =>
        ⋃ (ζ ∈ w.sys.presentRoots), linearityRelationsOfRoot R ζ
    | Definition =>
        w.definitionRelations
  )

/-! ### The group and the embedding -/

abbrev group (w : PartialChevalleyGroup Φ R) :=
  PresentedGroup (PartialChevalleyGroup.allRelations w)

def project (w : PartialChevalleyGroup Φ R) : FreeGroup (ChevalleyGenerator Φ R) →* group w :=
  PresentedGroup.mk (PartialChevalleyGroup.allRelations w)

/-- Mapping between two PartialChevalleyGroups -/
theorem injection (w₁ w₂ : PartialChevalleyGroup Φ R)
  (h_good :
  ∀ (K : SteinbergRelationClass),
```

```
    match K with
    | TrivialSpan =>
      ∀ p ∈ w₁.sys.trivialSpanRootPairs, p ∈ w₂.sys.trivialSpanRootPairs ∨
        (∀ r ∈ (trivialSpanRelationsOfRootPair R p), w₂.project r = 1)
    | SingleSpan =>
      ∀ p ∈ w₁.sys.singleSpanRootPairs, p ∈ w₂.sys.singleSpanRootPairs ∨
        (∀ r ∈ (singleSpanRelationsOfRootPair R p), w₂.project r = 1)
    | DoubleSpan =>
      ∀ p ∈ w₁.sys.doubleCommutatorRootPairs, p ∈ w₂.sys.doubleCommutatorRootPairs ∨
        (∀ r ∈ (doubleSpanRelationsOfRootPair R p), w₂.project r = 1)
    | Linearity =>
      ∀ p ∈ w₁.sys.presentRoots, p ∈ w₂.sys.presentRoots ∨
        (∀ r ∈ (linearityRelationsOfRoot R p), w₂.project r = 1)
    | Definition =>
      ∀ p ∈ w₁.definitionRelations, w₂.project p = 1
  )
  : ∀ r ∈ w₁.allRelations, w₂.project r = 1 := by
  simp only [allRelations]
  intro r h
  simp only [Set.mem_iUnion] at h
  rcases h with ⟨ K, h ⟩
  rcases h_K : K
  all_goals (
    simp only [h_K] at h
    specialize h_good K
    simp only [h_K] at h_good
  )
  any_goals (
    simp only [Set.mem_iUnion, exists_prop] at h
    rcases h with ⟨ p, h_p, h_r_p ⟩
    specialize h_good p h_p
    rcases h_good with h|h
    · apply eq_one_of_mem_rels
      simp only [allRelations, Set.mem_iUnion]
      exists K
      rw [h_K]
      simp only [Set.mem_iUnion, exists_prop]
      exists p
    · tauto
  )
  · tauto

open Lean PrettyPrinter Delaborator SubExpr in
/--
  Delaborator for `project` when it's an application.

  Note that this will obscure the widgets on the infoview, such that
  hovering over the group elements won't bring you back to `project`.
-/
@[delab app.DFunLike.coe]
def delab_project' : Delab := do
  withOverApp 6 do
    let e ← getExpr
    let mkApp5 (.const ``project _) _ _ _ _ _ := e.appFn!.appArg!' | failure
    let f_mk_mk ← withNaryArg 5 delab
    `($f_mk_mk)

/-! ### Helpers -/

theorem trivialSpanProp_of_mem_trivialSpanRoot_pairs {w : PartialChevalleyGroup Φ R} {p : Φ × Φ}
    (h : p ∈ w.sys.trivialSpanRootPairs)
      : trivialSpanPropOfRootPair w.project p := by
  intro t u
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists TrivialSpan
  simp only [Set.mem_iUnion]
  exists p, h
  rw [trivialSpanRelationsOfRootPair]
  exists t, u

theorem singleSpanProp_of_mem_singleSpanRoot_pairs (w : PartialChevalleyGroup Φ R) (p : SingleSpanRootPair
↪  Φ)
  (h : p ∈ w.sys.singleSpanRootPairs)
    : singleCommutatorPropOfRootPair w.project p := by
  intro t u
  apply eq_of_mul_inv_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists SingleSpan
```

```
  simp only [Set.mem_iUnion]
  exists p, h
  rw [singleSpanRelationsOfRootPair]
  exists t, u

theorem doubleSpanProp_of_mem_doubleSpanRootPairs (w : PartialChevalleyGroup Φ R) (p : DoubleSpanRootPair
↪ Φ)
  (h : p ∈ w.sys.doubleCommutatorRootPairs)
    : doubleSpanPropOfRootPair w.project p := by
  intro t u
  apply eq_of_mul_inv_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists DoubleSpan
  simp only [Set.mem_iUnion]
  exists p, h
  rw [doubleSpanRelationsOfRootPair]
  exists t, u

theorem lin_of_root_of_mem_presentRoots (w : PartialChevalleyGroup Φ R) {ζ : Φ} (h : ζ ∈
↪ w.sys.presentRoots)
    : lin_of_root(w.project, ζ) := by
  intro t u
  apply eq_of_mul_inv_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists Linearity
  simp only [Set.mem_iUnion]
  exists ζ, h
  rw [linearityRelationsOfRoot]
  exists t, u

theorem definitionProp_of_define (w : PartialChevalleyGroup Φ R)
    : ∀ (ζ : Φ) (t : R), w.project {ζ, t} = w.project (w.define (ChevalleyGenerator.mk ζ t))
      := by
  intro ζ t
  apply eq_of_inv_mul_eq_one
  apply eq_one_of_mem_rels
  simp only [allRelations, Set.mem_iUnion]
  exists Definition
  simp only [definitionRelations, Set.mem_iUnion]
  exists ζ
  simp only [Set.mem_setOf_eq]
  exists t

section declareThms

open Lean Parser.Tactic
set_option hygiene false

def makeCommands (m : MacroM Syntax) : MacroM (TSyntax `command) := do
  let cmds ← Syntax.getArgs <$> m
  return ⟨mkNullNode cmds⟩

macro "declare_ungraded_lin_id_inv_thms" w:ident R:term:arg root:term:arg : command => do
  let linOf := root.mapIdent ("lin_of_" ++ ·)
  let idOf := root.mapIdent ("id_of_" ++ ·)
  let invOf := root.mapIdent ("inv_of_" ++ ·)
  makeCommands `(section
    @[group_reassoc (attr := simp, chev_simps)]
    theorem $linOf : lin_of_root(($w $R).project, $root) :=
      ($w $R).lin_of_root_of_mem_presentRoots (by unfold $w; simp only [PartialChevalleyGroup.fullMk];
      ↪ tauto)

    @[simp, chev_simps]
    theorem $idOf : id_of_root(($w $R).project, $root) :=
      id_of_lin_of_root $linOf

    @[simp, chev_simps]
    theorem $invOf : inv_of_root(($w $R).project, $root) :=
      inv_of_lin_of_root $linOf
  end)

macro "declare_ungraded_trivial_span_expr_thm" w:ident R:term:arg r₁:term:arg r₂:term:arg : command => do
  let exprAs := TSyntax.mapIdent₂ r₁ r₂
    (fun s₁ s₂ => "expr_" ++ s₁ ++ "_" ++ s₂ ++ "_as_" ++ s₂ ++ "_" ++ s₁)
  let commName := TSyntax.mapIdent₂ r₁ r₂
    (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
  let commOf ← `(rwRule| $commName:term)
  makeCommands `(section
```

```
    @[group_reassoc] theorem $exprAs
      : ∀ (t u : $R),
        commutes(($w $R).project {$r₁:term, t},
                 ($w $R).project {$r₂:term, u}) := by
      intro t u
      apply triv_comm_iff_commutes.mp
      rw [$commOf]
      <;> try assumption
  end)

macro "declare_ungraded_trivial_span_of_root_pair_thms"
    w:ident R:term:arg
    r₁:term:arg r₂:term:arg : command => do
  let commOf := TSyntax.mapIdent₂ r₁ r₂ (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
  makeCommands `(section
    theorem $commOf : trivialSpanPropOfRootPair ($w $R).project ($r₁, $r₂) :=
      ($w $R).trivialSpanProp_of_mem_trivialSpanRoot_pairs (by unfold $w; simp only
      ↪  [PartialChevalleyGroup.fullMk]; tauto)
    declare_ungraded_trivial_span_expr_thm $w $R $r₁ $r₂
  end)

macro "declare_ungraded_single_span_expr_thms"
    w:ident R:term:arg
    r₁:term:arg r₂:term:arg r₃:term:arg isNeg:num n:num : command => do
  let innerTerm ←
    match isNeg.getNat, n.getNat with
    | 0, 1 => `(t * u)
    | 1, 1 => `(-(t * u))
    | 1, _ => `(-($n * t * u))
    | _, _ => `($n * t * u)

  let commOf := TSyntax.mapIdent₂ r₁ r₂ (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
  let exprAs := TSyntax.mapIdent₃ r₁ r₂ r₃
    (fun s₁ s₂ s₃ => "expr_" ++ s₃ ++ "_as_" ++ s₁ ++ "_" ++ s₂ ++ "_" ++ s₁ ++ "_" ++ s₂)
  let exprAsRev := TSyntax.mapIdent₃ r₁ r₂ r₃
    (fun s₁ s₂ s₃ => "expr_" ++ s₁ ++ "_" ++ s₂ ++ "_as_" ++ s₃ ++ "_" ++ s₂ ++ "_" ++ s₁)
  makeCommands `(section
    theorem $exprAs : ∀ (t u : $R),
        (($w $R).project {$r₃:term, $innerTerm})
          = ($w $R).project {$r₁:term, t}
            * ($w $R).project {$r₂:term, u}
            * ($w $R).project {$r₁:term, -t}
            * ($w $R).project {$r₂:term, -u} := by
      intro t u
      have := $commOf t u
      chev_simp [commutatorElement_def, one_mul, mul_one] at this
      symm at this
      exact this

    @[group_reassoc]
    theorem $exprAsRev : ∀ (t u : $R),
        reorder_left(
          ($w $R).project {$r₁:term, t},
          ($w $R).project {$r₂:term, u},
          (($w $R).project {$r₃:term, $innerTerm})
        ) := by
      intro t u
      have := $commOf t u
      chev_simp [commutatorElement_def, one_mul, mul_one] at this
      grw [← this]
  end)

macro "declare_ungraded_single_span_of_root_pair_thms"
    w:ident R:term:arg
    r₁:term:arg r₂:term:arg r₃:term:arg isNeg:num n:num : command => do
  let innerTerm ←
    match isNeg.getNat with
    | 1 => `(-($n))
    | _ => `($n)

  let commOf := TSyntax.mapIdent₂ r₁ r₂ (fun s₁ s₂ => "comm_of_" ++ s₁ ++ "_" ++ s₂)
  makeCommands `(section
    theorem $commOf : singleCommutatorPropOfRootPair ($w $R).project ⟨$r₁, $r₂, $r₃, $innerTerm, rfl⟩ :=
      ($w $R).singleSpanProp_of_mem_singleSpanRoot_pairs ⟨$r₁, $r₂, $r₃, $innerTerm, rfl⟩ (
        by unfold $w; simp only [PartialChevalleyGroup.fullMk]; tauto)
    declare_ungraded_single_span_expr_thms $w $R $r₁ $r₂ $r₃ $isNeg $n
  end)

end declareThms /- section -/

```

```
end PartialChevalleyGroup
```

Author source module Steinberg.Defs.DecomposeFixed

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.Data.Nat.Notation

/-!

  Some hardcoded decompositions of small natural numbers, needed to state the "definition" relations.

-/

namespace Steinberg

def split_3_into_1_2 (i : ℕ) (hi : i ≤ 3) :=
  match i with
  | 0 => (0, 0)
  | 1 => (0, 1)
  | 2 => (1, 1)
  | 3 => (1, 2)

theorem correct_of_split_3_into_1_2 (i : ℕ) (hi : i ≤ 3) :
  (split_3_into_1_2 i hi).1 ≤ 1 ∧ (split_3_into_1_2 i hi).2 ≤ 2 ∧ (split_3_into_1_2 i hi).1 +
    ↪  (split_3_into_1_2 i hi).2 = i := by
  simp only [split_3_into_1_2]
  split
  all_goals trivial

theorem refl_of_split_3_into_1_2 (i : ℕ) (hi : i ≤ 3) :
  split_3_into_1_2 (3-i) (by omega) = (1-(split_3_into_1_2 i hi).1, 2-(split_3_into_1_2 i hi).2) := by
  simp only [split_3_into_1_2]
  symm
  split
  all_goals trivial

def split_4_into_1_3 (i : ℕ) (hi : i ≤ 4) :=
  match i with
  | 0 => (0, 0)
  | 1 => (0, 1)
  | 2 => (1, 1)
  | 3 => (1, 2)
  | 4 => (1, 3)

/-!
Note that it is not possible to prove a `refl_of_split_4_into_1_3` theorem.
-/

theorem correct_of_split_4_into_1_3 (i : ℕ) (hi : i ≤ 4) :
  (split_4_into_1_3 i hi).1 ≤ 1 ∧ (split_4_into_1_3 i hi).2 ≤ 3 ∧ (split_4_into_1_3 i hi).1 +
    ↪  (split_4_into_1_3 i hi).2 = i := by
  simp only [split_4_into_1_3]
  split
  all_goals trivial

def split_5_into_2_3 (i : ℕ) (hi : i ≤ 5) :=
  match i with
  | 0 => (0, 0)
  | 1 => (0, 1)
  | 2 => (1, 1)
  | 3 => (1, 2)
  | 4 => (2, 2)
  | 5 => (2, 3)

theorem correct_of_split_5_into_2_3 (i : ℕ) (hi : i ≤ 5) :
  (split_5_into_2_3 i hi).1 ≤ 2 ∧ (split_5_into_2_3 i hi).2 ≤ 3 ∧ (split_5_into_2_3 i hi).1 +
    ↪  (split_5_into_2_3 i hi).2 = i := by
  simp only [split_5_into_2_3]
  split
  all_goals trivial

theorem refl_of_split_5_into_2_3 (i : ℕ) (hi : i ≤ 5) :
  split_5_into_2_3 (5-i) (by omega) = (2-(split_5_into_2_3 i hi).1, 3-(split_5_into_2_3 i hi).2) := by
```

```
  simp only [split_5_into_2_3]
  symm
  split
  all_goals trivial
```

Author source module Steinberg.A3.Defs

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Defs.GradedPartialChevalleyGroup
import Steinberg.Defs.PartialChevalleyGroup
import Steinberg.Defs.DecomposeFixed

import Mathlib.Tactic.DeriveFintype

import Steinberg.Macro.Attr

/-!

  File dox.

-/

namespace Steinberg.A3

open Steinberg PartialChevalley GradedPartialChevalley GradedChevalleyGenerator PartialChevalleySystem

/-! # The A3 positive root system -/

inductive A3PosRoot
  | α | β | γ | αβ | βγ | αβγ
deriving Fintype, DecidableEq

namespace A3PosRoot

@[reducible, height_simps]
def height : A3PosRoot → Nat
  | α | β | γ => 1
  | αβ | βγ => 2
  | αβγ => 3

def toString : A3PosRoot → String
  | α => "α"
  | β => "β"
  | γ => "γ"
  | αβ => "α+β"
  | βγ => "β+γ"
  | αβγ => "α+β+γ"

instance instPosRootSys : PositiveRootSystem A3PosRoot where
  height := height
  toString := toString

instance instCoeNat : Coe A3PosRoot Nat where
  coe r := height r

end A3PosRoot

open A3PosRoot

variable {R : Type TR} [Ring R]

/-! # Definition of the 'weak' A3 graded group -/

/-! ## Defining the 'weak' positive root system -/

abbrev weakPresentRoots : Set (A3PosRoot) :=
  {α, β, γ, αβ, βγ}

abbrev weakTrivialSpanPairs : Set (A3PosRoot × A3PosRoot) :=
  {(α, γ), (α, αβ), (β, αβ), (β, βγ), (γ, βγ)}

abbrev weakSingleSpanPairs : Set (SingleSpanRootPair A3PosRoot) :=
  {⟨ α, β, αβ, 1, (by ht)⟩, ⟨β, γ, βγ, 1, (by ht)⟩}
```

```
71
72 abbrev weakA3System := PartialChevalleySystem.mk
73   weakPresentRoots
74   weakTrivialSpanPairs
75   weakSingleSpanPairs
76   ∅
77   (by simp only [weakTrivialSpanPairs, Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp,
     ↪  forall_eq]; tauto)
78   (by simp only [singleSpanRootPairs, Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp,
     ↪  forall_eq]; tauto)
79   (by simp only [doubleCommutatorRootPairs, Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp,
     ↪  forall_eq]; tauto)
80
81 /-! ## Lifted relations -/
82
83 def rels_of_nonhom_lift_of_comm_of_αβ_βγ :=
84   { ⁅ {αβ, 2, t₁ * u₁} * {αβ, 1, t₁ * u₀ + t₀ * u₁} * {αβ, 0, t₀ * u₀},
85       {βγ, 2, u₁ * v₁} * {βγ, 1, u₁ * v₀ + u₀ * v₁} * {βγ, 0, u₀ * v₀} ⁆
86     | (t₁ : R) (t₀ : R) (u₁ : R) (u₀ : R) (v₁ : R) (v₀ : R) }
87
88 def lifted_sets (R : Type TR) [Ring R] : Set (Set (FreeGroup (GradedChevalleyGenerator A3PosRoot R))) :=
89   { rels_of_nonhom_lift_of_comm_of_αβ_βγ }
90
91 /-! ## Definition for missing root (αβγ) -/
92
93 def weak_define (R : Type TR) [Ring R] (g : GradedChevalleyGenerator A3PosRoot R) : FreeGroup
   ↪  (GradedChevalleyGenerator A3PosRoot R) :=
94   let ⟨ ζ, i, hi, t ⟩ := g;
95   match ζ with
96   | αβγ => ⁅ {α,(split_3_into_1_2 i (by ht)).1, g.t}'(correct_of_split_3_into_1_2 i (by ht)).1,
97     {βγ, (split_3_into_1_2 i (by ht)).2, 1}'(correct_of_split_3_into_1_2 i (by ht)).2.1 ⁆
98   | ζ => FreeGroup.of g
99
100 theorem weak_define_of_present (R : Type TR) [Ring R] :
101   ∀ {g : GradedChevalleyGenerator A3PosRoot R}, g.ζ ∈ weakA3System.presentRoots → weak_define R g =
     ↪  FreeGroup.of g := by
102   intro g h_g_in_present
103   rcases g with ⟨ ζ, i, hi, t ⟩
104   cases ζ
105   all_goals simp only [weak_define] -- this will close all present roots
106   all_goals ( -- this will close the remaining (nonpresent) roots
107     simp only [weakPresentRoots] at h_g_in_present
108     contradiction
109   )
110
111 theorem weak_define_is_projection (R : Type TR) [Ring R] :
112   ∀ {g : GradedChevalleyGenerator A3PosRoot R}, (FreeGroup.lift (weak_define R)) (weak_define R g) =
     ↪  weak_define R g := by
113   intro g
114   rcases g with ⟨ ζ, i, hi, t ⟩
115   cases ζ
116   all_goals simp only [weak_define, FreeGroup.lift.of, map_commutatorElement]
117
118 def weakA3Graded (R : Type TR) [Ring R] := GradedPartialChevalleyGroup.mk
119   weakA3System
120   (lifted_sets R)
121   (weak_define R)
122   (weak_define_of_present R)
123   (weak_define_is_projection R)
124
125 /-! # Definition of the 'full' A3 ungraded and graded groups -/
126
127 abbrev fullPresentRoots : Set (A3PosRoot) :=
128   weakPresentRoots ∪ {αβγ}
129
130 abbrev fullTrivialSpanPairs : Set (A3PosRoot × A3PosRoot) :=
131   weakTrivialSpanPairs ∪ {(αβ, βγ), (α, αβγ), (β, αβγ), (γ, αβγ), (αβ, αβγ), (βγ, αβγ)}
132
133 abbrev fullSingleSpanPairs : Set (SingleSpanRootPair A3PosRoot) :=
134   (weakSingleSpanPairs) ∪ {⟨ α, βγ, αβγ, 1, (by ht)⟩, ⟨αβ, γ, αβγ, 1, (by ht)⟩}
135
136 theorem all_root_pairs_have_relation : everyRootPairInRootPairs A3PosRoot fullTrivialSpanPairs
   ↪  fullSingleSpanPairs ∅ := by
137   intro ζ η h_ne
138   unfold toRootPairs fullTrivialSpanPairs weakTrivialSpanPairs fullSingleSpanPairs
139     weakSingleSpanPairs
140   simp only [Set.image_insert_eq, Set.image_singleton, Set.union_insert, Set.union_singleton, Prod.swap,
141     Set.mem_insert_iff, Set.mem_singleton_iff, Set.image_empty, Set.union_empty]
142   cases ζ
143   all_goals (
```

```
    simp only [Prod.mk.injEq, reduceCtorEq, false_and, true_and, or_self, or_false, false_or]
    cases η
    all_goals trivial
  )

abbrev fullA3System := PartialChevalleySystem.mkFull A3PosRoot
  fullPresentRoots
  fullTrivialSpanPairs
  fullSingleSpanPairs
  ∅
  (by decide)
  all_root_pairs_have_relation

def fullA3 (R : Type TR) [Ring R] := PartialChevalleyGroup.fullMk A3PosRoot R fullA3System
def fullA3Graded (R : Type TR) [Ring R] := GradedPartialChevalleyGroup.fullMk A3PosRoot R fullA3System

/-! # Notation and macros -/

/- Instantiate the `declare_thms` macros from `PartialChevalley.lean`. -/

-- TODO-B: TODO: Make this a macro to declare all at once for A3.
--     Something like: `declare_thms A3 weakA3Graded R`

macro "declare_A3_trivial_span_expr_thm" R:term:arg r₁:term:arg r₂:term:arg : command =>
  `(command| declare_trivial_span_expr_thm weakA3Graded $R $r₁ $r₂)

macro "declare_A3_trivial_span_of_root_pair_thms" R:term:arg r₁:term:arg r₂:term:arg : command =>
  `(command| declare_trivial_span_of_root_pair_thms weakA3Graded $R $r₁ $r₂)

macro "declare_A3_single_span_expr_thms" R:term:arg r₁:term:arg r₂:term:arg r₃:term:arg : command =>
  `(command| declare_single_span_expr_thms weakA3Graded $R $r₁ $r₂ $r₃ 1)

macro "declare_A3_single_span_of_root_pair_thms" R:term:arg r₁:term:arg r₂:term:arg r₃:term:arg : command
↪  =>
  `(command| declare_single_span_of_root_pair_thms weakA3Graded $R $r₁ $r₂ $r₃ 1)

macro "declare_A3_lin_id_inv_thms" R:term:arg root:term:arg : command =>
  `(command| declare_lin_id_inv_thms weakA3Graded $R $root)

macro "declare_A3_mixed_expr_thm" R:term:arg r:term:arg : command =>
  `(command| declare_mixed_degree_expr_thm weakA3Graded $R $r)

macro "declare_A3_mixed_degree_thms" R:term:arg r:term:arg : command =>
  `(command| declare_mixed_degree_thms weakA3Graded $R $r)

set_option hygiene false in
/-- Shorthand for building free group elements from a root, degree, and ring element. -/
scoped notation (priority:=high) "(( ζ " ", " i ", " t "))" =>
  (weakA3Graded R).project {ζ, i, t}

set_option hygiene false in
/-- Shorthand for building free group elements from a root, degree, and ring element. -/
scoped notation (priority:=high) "(( ζ " ", " i ", " t "))'" h =>
  (weakA3Graded R).project ({ζ, i, t}'h)

section forallNotation

set_option hygiene false

scoped notation "forall_i_t" h:max "," e =>
  ∀ ⦃i : ℕ⦄ (hi : i ≤ h) (t : R), e

scoped notation "forall_ij_t" h₁:max h₂:max "," e =>
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (t : R), e

scoped notation "forall_ij_tu" h₁:max h₂:max "," e =>
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (t u : R), e

scoped notation "forall_ik_tuv" h₁:max h₂:max "," e =>
  ∀ ⦃i k : ℕ⦄ (hi : i ≤ h₁) (hk : k ≤ h₂) (t u v : R), e

scoped notation "forall_ijk_tu" h₁:max h₂:max h₃:max "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (hk : k ≤ h₃) (t u : R), e

scoped notation "forall_ijk_tuv" h₁:max h₂:max h₃:max "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (hk : k ≤ h₃) (t u v : R), e

scoped notation "forall_ijk_tuv" "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ γ.height) (t u v : R), e

```

```
end forallNotation

end Steinberg.A3
```

Author source module Steinberg.A3.Basic

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.A3.Defs

/-!

  By definition, we can immediately prove many theorems about the
  triviality of commutators, that group elements are linear in their
  ring elements, and so on.

  All the proofs for these theorems are extremely similar, so much so
  that we can declare and prove them from macros agnostic to the root system.

  See `GradedPartialChevalleyGroup.lean` for their definitions.

-/

namespace Steinberg.A3

open A3PosRoot GradedPartialChevalley GradedPartialChevalleyGroup GradedChevalleyGenerator

variable {R : Type TR} [Ring R]

/-! # "Given" relations for specific roots -/

declare_A3_lin_id_inv_thms R α
declare_A3_lin_id_inv_thms R β
declare_A3_lin_id_inv_thms R γ
declare_A3_lin_id_inv_thms R αβ
declare_A3_lin_id_inv_thms R βγ

declare_A3_trivial_span_of_root_pair_thms R α γ
declare_A3_trivial_span_of_root_pair_thms R α αβ
declare_A3_trivial_span_of_root_pair_thms R β αβ
declare_A3_trivial_span_of_root_pair_thms R β βγ
declare_A3_trivial_span_of_root_pair_thms R γ βγ

declare_A3_single_span_of_root_pair_thms R α β αβ
declare_A3_single_span_of_root_pair_thms R β γ βγ

/-! ### Mixed-degree theorem for specific roots -/

declare_A3_mixed_degree_thms R αβ
declare_A3_mixed_degree_thms R βγ

end Steinberg.A3
```

Author source module Steinberg.Defs.Lattice

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Mathlib.Data.Finset.Defs
import Mathlib.Data.Finset.Sigma

import Mathlib.Tactic.Use

namespace Steinberg

/-!

  Some helpful theorems and lemmas about the images of certain linear functions on small natural numbers.
```

```

-/

/-! ## General theorems -/

/-- Decompose a number `0 ≤ i ≤ n + m` into `i₁ + i₂`, where `0 ≤ i₁ ≤ n` and `0 ≤ i₂ ≤ m`. -/
theorem decompose (n m i : ℕ) (h : i ≤ (n+m)) : ∃ (i₁ i₂ : ℕ), i = i₁ + i₂ ∧ i₁ ≤ n ∧ i₂ ≤ m := by
  by_cases i ≤ n
  · use i, 0
    omega
  · use n, i-n
    omega

/-! ## Specific decomposition theorems. -/

def booleans : Finset ℕ := {0, 1}
def boolean_square : Finset (ℕ × ℕ) := booleans ×ˢ booleans
def boolean_cube : Finset (ℕ × ℕ × ℕ) := booleans ×ˢ booleans ×ˢ booleans

@[simp]
lemma le_of_mem_booleans {i : ℕ}
    : i ∈ booleans ↔ i ≤ 1 := by
  simp only [booleans, Finset.mem_insert, Finset.mem_singleton]
  omega

@[simp]
lemma le_of_mem_boolean_square {i j : ℕ}
    : (i, j) ∈ boolean_square ↔ (i ≤ 1 ∧ j ≤ 1) := by
  simp only [boolean_square, Finset.mem_product, le_of_mem_booleans]

@[simp]
lemma le_of_mem_boolean_cube {i j k : ℕ}
    : (i, j, k) ∈ boolean_cube ↔ (i ≤ 1 ∧ j ≤ 1 ∧ k ≤ 1) := by
  simp only [boolean_cube, Finset.mem_product, le_of_mem_booleans]

private lemma decompose_3_into_booleans_1_2_helper :
  Finset.image (fun ⟨i, j⟩ => i + 2 * j) boolean_square = Finset.range 4 := by decide

/-- Over `ℕ`, every `i ≤ 3` can be written as `i = i₁ + 2 * i₂` for `i₁, i₂ ≤ 1`. -/
lemma decompose_3_into_booleans_1_2 (i : ℕ) (hi : i ≤ 3)
    : ∃ (i₁ i₂ : ℕ), i = i₁ + 2 * i₂ ∧ i₁ ≤ 1 ∧ i₂ ≤ 1 := by
  have := Finset.mem_range_succ_iff.mpr hi
  simp only [← decompose_3_into_booleans_1_2_helper, Finset.mem_image,
    Prod.exists, le_of_mem_boolean_square] at this
  tauto

private lemma decompose_5_into_booleans_1_2_2_helper :
  Finset.image (fun ⟨i, j, k⟩ => i + 2 * j + 2 * k) boolean_cube = Finset.range 6 := by decide

/-- Over `ℕ`, every `i ≤ 5` can be written as `i = i₁ + 2 * i₂ + 2 * i₃` for `i₁, i₂, i₃ ≤ 1`. -/
lemma decompose_5_into_booleans_1_2_2 (i : ℕ) (hi : i ≤ 5)
    : ∃ (i₁ i₂ i₃ : ℕ), i = i₁ + 2 * i₂ + 2 * i₃ ∧ i₁ ≤ 1 ∧ i₂ ≤ 1 ∧ i₃ ≤ 1 := by
  have := Finset.mem_range_succ_iff.mpr hi
  simp only [Nat.succ_eq_add_one, Nat.reduceAdd, ← decompose_5_into_booleans_1_2_2_helper,
    Finset.mem_image, Prod.exists, le_of_mem_boolean_cube] at this
  tauto

def f_ij_jk : ℕ × ℕ × ℕ → ℕ × ℕ := (fun ijk' : ℕ × ℕ × ℕ => let ( i', j', k' ) := ijk'; (i' + j', j' +
↪  k'))
def ij_jk_image : Finset (ℕ × ℕ) := {(0, 0), (0, 1), (1, 0), (1, 1), (1, 2), (2, 1), (2, 2)}

/-- Over `ℕ`, every `i, j ≤ 2` can be written as `(i, j) = (a + b, b + c)` for `a, b, c ≤ 1`,
except for `(i, j) = (2, 0)` and `(i, j) = (0, 2)`. -/
lemma correct_of_ij_jk_image : (Finset.image f_ij_jk boolean_cube) = ij_jk_image := by decide

end Steinberg
```

Author source module Steinberg.Defs.DegreeReflection

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Defs.GradedPartialChevalleyGroup
import Steinberg.Upstream.PresentedGroup

```

```
import Mathlib.GroupTheory.Commutator.Basic

/-!

  File dox go here.

-/

namespace Steinberg

open PositiveRootSystem GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup
open Set

variable {G : Type TG} [Group G]
         {Φ : Type TΦ} [PositiveRootSystem Φ]
         {R : Type TR} [Ring R]

/-- "Degree-reflection" map on `GradedGenerator` elements corresponding to replacing
degree `i` with `height ζ - i`. -/
def reflect (g : GradedChevalleyGenerator Φ R) : GradedChevalleyGenerator Φ R :=
  GradedChevalleyGenerator.mk g.ζ (height g.ζ - g.i) (by ht) g.t

/-- Reflection on generators is an involution, though we will not need this fact. -/
example (g : GradedChevalleyGenerator Φ R) : reflect (reflect g) = g := by
  let ⟨ ζ, i, hi, t ⟩ := g;
  unfold reflect
  congr
  exact Nat.sub_sub_self hi

/-- Composition of the reflection map with the definition map in a `GradedPartialChevalleyGroup`. The
↪  effect
of this map is to simply apply `reflect` to present generator and more generally to reflect the definition
↪  of all
generators. -/
def defineThenReflect (w : GradedPartialChevalleyGroup Φ R)
    : GradedChevalleyGenerator Φ R → FreeGroup (GradedChevalleyGenerator Φ R) :=
  (FreeGroup.map reflect) ∘ w.define

/- Defining-then-reflecting a present generator is the same as just reflecting the generator. -/
theorem defineThenReflect_eq_reflect_of_mem_presentRoots (w : GradedPartialChevalleyGroup Φ R)
      (g : GradedChevalleyGenerator Φ R) (h : g.ζ ∈ w.sys.presentRoots)
    : defineThenReflect w g = FreeGroup.of (reflect g) := by
  simp only [defineThenReflect, MonoidHom.coe_comp, Function.comp_apply, FreeGroup.lift.of]
  have : w.define g = FreeGroup.of g := by exact w.h_define_of_present h
  rw [this]
  simp only [FreeGroup.map.of]

/-! ### Generic reflection theorems

These theorems state that the relations which arise in a `GradedPartialChevalleyGroup` from its
`PartialChevalleySystem` are always closed under defining-then-reflecting. (Note that these
relations always only involve present roots.)
-/

/-- Defining-then-reflecting preserves the set of trivial commutator relations for any root pair. -/
private theorem defineThenReflect_eq_reflect_of_trivialSpanRelationsOfRootPair_of_mem_presentRoots (ζ η :
↪  Φ)
  (h_ζ : ζ ∈ w.sys.presentRoots) (h_η : η ∈ w.sys.presentRoots)
    : ∀ r ∈ trivialSpanRelationsOfRootPair R (ζ, η),
      (FreeGroup.lift (defineThenReflect w)) r ∈ trivialSpanRelationsOfRootPair R (ζ, η) := by
  intro r h
  simp only [trivialSpanRelationsOfRootPair, mem_setOf_eq] at h
  rcases h with ⟨ i, j, hi, hj, t, u, rfl ⟩
  simp only [map_commutatorElement, FreeGroup.lift.of]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi t) h_ζ]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk η j hj u) h_η]
  exists (height ζ - i), (height η - j), (by omega), (by omega), t, u

/-- Defining-then-reflecting preserves the set of single commutator relations for any root pair. -/
-- TODO: maybe we can replace with passing the generator directly, it'll be shorter
private theorem defineThenReflect_eq_reflect_of_singleSpanRelationsOfRootPair_of_mem_presentRoots
  (ζ η θ : Φ) (C : ℤ) (h_height : height θ = height ζ + height η)
  (h_ζ : ζ ∈ w.sys.presentRoots) (h_η : η ∈ w.sys.presentRoots) (h_θ : θ ∈ w.sys.presentRoots)
    : ∀ r ∈ singleSpanRelationsOfRootPair R ⟨ ζ, η, θ, C, h_height ⟩,
      (FreeGroup.lift (defineThenReflect w)) r ∈ singleSpanRelationsOfRootPair R ⟨ ζ, η, θ, C, h_height ⟩
      ↪  := by
  intro r h
  simp only [singleSpanRelationsOfRootPair, mem_setOf_eq] at h
  rcases h with ⟨ i, j, hi, hj, t, u, rfl ⟩
  simp only [map_mul, map_commutatorElement, FreeGroup.lift.of, map_inv]
```

```
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi t) h_ζ]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk η j hj u) h_η]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk θ (i + j) (by omega)
    ↪ (C * t * u)) h_θ]
  exists (height ζ - i), (height η - j), (by omega), (by omega), t, u
  congr
  simp only
  omega

/-- Defining-then-reflecting preserves the set of trivial commutator relations for any root pair. -/
private theorem defineThenReflect_eq_reflect_of_doubleSpanRelationsOfRootPair_of_mem_presentRoots
  (ζ η θ₁ θ₂ : Φ) (C₁ C₂ : ℤ) (h_height₁ : height θ₁ = height ζ + height η) (h_height₂ : height θ₂ =
    ↪ height ζ + 2 * height η)
  (h_ζ : ζ ∈ w.sys.presentRoots) (h_η : η ∈ w.sys.presentRoots) (h_θ₁ : θ₁ ∈ w.sys.presentRoots) (h_θ₂ :
    ↪ θ₂ ∈ w.sys.presentRoots)
    : ∀ r ∈ doubleSpanRelationsOfRootPair R ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩,
      (FreeGroup.lift (defineThenReflect w)) r ∈ doubleSpanRelationsOfRootPair R ⟨ ζ, η, θ₁, θ₂, C₁, C₂,
        ↪ h_height₁, h_height₂ ⟩ := by
  intro r h
  simp only [doubleSpanRelationsOfRootPair, mem_setOf_eq] at h
  rcases h with ⟨ i, j, hi, hj, t, u, rfl ⟩
  simp only [mul_inv_rev, map_mul, map_commutatorElement, FreeGroup.lift.of, map_inv]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi t) h_ζ]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk η j hj u) h_η]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk θ₁ (i + j) (by
    ↪ omega) (C₁ * t * u)) h_θ₁]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk θ₂ (i + 2 * j) (by
    ↪ omega) (C₂ * t * u * u)) h_θ₂]
  exists (height ζ - i), (height η - j), (by omega), (by omega), t, u
  simp only [mul_inv_rev]
  congr
  all_goals (ht)

/-- Defining-then-reflecting preserves the set of mixed-degree commutator relations for any root. -/
private theorem defineThenReflect_eq_reflect_of_mixedDegRelationsOfRoot_of_mem_presentRoots (ζ : Φ) (h_ζ :
  ↪ ζ ∈ w.sys.presentRoots) :
  ∀ r ∈ mixedDegreeRelationsOfRoot R ζ,
    (FreeGroup.lift (defineThenReflect w)) r ∈ mixedDegreeRelationsOfRoot R ζ := by
  intro r h
  simp only [mixedDegreeRelationsOfRoot, mem_setOf_eq] at h
  rcases h with ⟨ i, j, hi, hj, t, u, rfl ⟩
  simp only [map_commutatorElement, FreeGroup.lift.of]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi t) h_ζ]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ j hj u) h_ζ]
  exists (height ζ - i), (height ζ - j), (by omega), (by omega), t, u

/-- Defining-then-reflecting preserves the set of linearity relations for any root. -/
private theorem defineThenReflect_eq_reflect_of_linearityRelationsOfRoot_of_mem_presentRoots (ζ : Φ) (h_ζ
  ↪ : ζ ∈ w.sys.presentRoots) :
  ∀ r ∈ linearityRelationsOfRoot R ζ,
    (FreeGroup.lift (defineThenReflect w)) r ∈ linearityRelationsOfRoot R ζ := by
  intro r h
  simp only [linearityRelationsOfRoot, mem_setOf_eq] at h
  rcases h with ⟨ i, hi, t, u, rfl ⟩
  simp only [map_mul, FreeGroup.lift.of, map_inv]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi t) h_ζ]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi u) h_ζ]
  rw [defineThenReflect_eq_reflect_of_mem_presentRoots w (GradedChevalleyGenerator.mk ζ i hi (t + u)) h_ζ]
  exists (height ζ - i), (by omega), t, u

/-! ### Establishing the reflection map -/

/-- The following proposition supposes that the reflected definitions of lifted relations vanish.
It is used as a hypothesis to define the reflection map. -/
def reflectValidProp (w : GradedPartialChevalleyGroup Φ R) :=
  (∀ S ∈ w.liftedRelationsSets, ∀ r ∈ S, w.project ((FreeGroup.lift (defineThenReflect w)) r) = 1)

private theorem lift_of_refl_eq_comp (w : GradedPartialChevalleyGroup Φ R) : FreeGroup.lift
  ↪ (defineThenReflect w) = (FreeGroup.map reflect).comp (FreeGroup.lift w.define)
  := by
  ext a : 1
  simp_all only [FreeGroup.lift.of, MonoidHom.coe_comp, Function.comp_apply]
  rfl

-- TODO: can we handle the first five cases uniformly?
private theorem eq_one_of_defineThenReflect_lift_allRelations_of_reflectValidProp
  {w : GradedPartialChevalleyGroup Φ R} (h' : reflectValidProp w) :
    ∀ r ∈ w.allRelations, w.project (FreeGroup.lift (defineThenReflect w) r) = 1 := by
  nth_rewrite 1 [allRelations]
```

```
  intro r h_r
  simp only [mem_iUnion] at h_r
  rcases h_r with ⟨ K, h_r ⟩
  rcases K
  all_goals simp only at h_r
  · apply eq_one_of_mem_rels
    simp only [mem_iUnion, exists_prop, Prod.exists] at h_r
    rcases h_r with ⟨ ζ, η, h_in_pairs, h_t_in_rels ⟩
    simp only [allRelations, mem_iUnion]
    use GradedSteinbergRelationClass.TrivialSpan
    simp only [mem_iUnion, exists_prop, Prod.exists]
    use ζ, η, h_in_pairs
    exact defineThenReflect_eq_reflect_of_trivialSpanRelationsOfRootPair_of_mem_presentRoots
      ζ η (w.sys.h_trivial_valid (ζ, η) h_in_pairs).1 (w.sys.h_trivial_valid (ζ, η) h_in_pairs).2 r
      ↪ h_t_in_rels
  · apply eq_one_of_mem_rels
    simp only [mem_iUnion, exists_prop, Sigma.exists, PProd.exists] at h_r
    rcases h_r with ⟨ ζ, η, θ, C, h_height, h_in_pairs, h_t_in_rels ⟩
    simp only [allRelations, mem_iUnion]
    use GradedSteinbergRelationClass.SingleSpan
    simp only [mem_iUnion, exists_prop, Prod.exists]
    use ⟨ ζ, η, θ, C, h_height ⟩, h_in_pairs
    exact defineThenReflect_eq_reflect_of_singleSpanRelationsOfRootPair_of_mem_presentRoots
      ζ η θ C h_height
      (w.sys.h_single_valid ⟨ ζ, η, θ, C, h_height ⟩ h_in_pairs).1 (w.sys.h_single_valid ⟨ ζ, η, θ, C,
      ↪ h_height ⟩ h_in_pairs).2.1
      (w.sys.h_single_valid ⟨ ζ, η, θ, C, h_height ⟩ h_in_pairs).2.2
      r h_t_in_rels
  · apply eq_one_of_mem_rels
    simp only [mem_iUnion, exists_prop, Sigma.exists, PProd.exists] at h_r
    rcases h_r with ⟨ ζ, η, θ₁, θ₂, ⟨ C₁, C₂, h_height₁, h_height₂ ⟩ , h_in_pairs, h_t_in_rels ⟩
    simp only [allRelations, mem_iUnion]
    use GradedSteinbergRelationClass.DoubleSpan
    simp only [mem_iUnion, exists_prop, Prod.exists]
    use ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩, h_in_pairs
    exact defineThenReflect_eq_reflect_of_doubleSpanRelationsOfRootPair_of_mem_presentRoots ζ η θ₁ θ₂ C₁
    ↪ C₂ h_height₁ h_height₂
      (w.sys.h_double_valid ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩ h_in_pairs).1
      (w.sys.h_double_valid ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩ h_in_pairs).2.1
      (w.sys.h_double_valid ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩ h_in_pairs).2.2.1
      (w.sys.h_double_valid ⟨ ζ, η, θ₁, θ₂, C₁, C₂, h_height₁, h_height₂ ⟩ h_in_pairs).2.2.2
      r h_t_in_rels
  · apply eq_one_of_mem_rels
    simp only [mem_iUnion, exists_prop, Prod.exists] at h_r
    rcases h_r with ⟨ ζ, h_in_present, h_t_in_rels ⟩
    simp only [allRelations, mem_iUnion]
    use GradedSteinbergRelationClass.MixedDegree
    simp only [mem_iUnion, exists_prop, Prod.exists]
    use ζ, h_in_present
    exact defineThenReflect_eq_reflect_of_mixedDegRelationsOfRoot_of_mem_presentRoots ζ h_in_present r
    ↪ h_t_in_rels
  · apply eq_one_of_mem_rels
    simp only [mem_iUnion, exists_prop, Prod.exists] at h_r
    rcases h_r with ⟨ ζ, h_in_present, h_t_in_rels ⟩
    simp only [allRelations, mem_iUnion]
    use GradedSteinbergRelationClass.Linearity
    simp only [mem_iUnion, exists_prop, Prod.exists]
    use ζ, h_in_present
    exact defineThenReflect_eq_reflect_of_linearityRelationsOfRoot_of_mem_presentRoots ζ h_in_present r
    ↪ h_t_in_rels
  · rcases h_r with ⟨ T, ⟨ h_T, h_t_T ⟩ ⟩
    exact h' T h_T r h_t_T
  · simp only [definitionRelations, mem_setOf_eq] at h_r
    rcases h_r with ⟨ S, h_S, h_r ⟩
    simp only [mem_range, SetLike.mem_coe] at h_S
    rcases h_S with ⟨ ζ, h_ζ ⟩
    subst S
    simp only [mem_setOf_eq] at h_r
    rcases h_r with ⟨ i, hi, t, h ⟩
    subst r
    simp only [lift_of_refl_eq_comp, map_mul, map_inv, MonoidHom.coe_comp, Function.comp_apply,
      FreeGroup.lift.of, w.h_define_is_projection, inv_mul_cancel]
    rfl

/-- A homomorphism from a `GradedPartialChevalleyGroup` to itself, assuming `reflectValidProp` holds. -/
def defineThenReflectOfPresentedGroup {w : GradedPartialChevalleyGroup Φ R} (h : reflectValidProp w) :
↪ group w →* group w :=
  toPresentedGroup (eq_one_of_defineThenReflect_lift_allRelations_of_reflectValidProp h)

/-- Defining-then-reflecting commutes with projection. -/
```

```
theorem defineThenReflectOfPresentedGroup_of_project {w : GradedPartialChevalleyGroup Φ R} {h : reflectValidProp w}
  {g : FreeGroup (GradedChevalleyGenerator Φ R)} :
    defineThenReflectOfPresentedGroup h (w.project g) = w.project (FreeGroup.lift (defineThenReflect w) g) := by
  simp only [defineThenReflectOfPresentedGroup, project, toPresentedGroup.mk]

end Steinberg
```

Author source module Steinberg.A3.Thm

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.A3.Basic

import Mathlib.Algebra.Group.Basic

import Mathlib.Tactic.Group
import Mathlib.Tactic.FinCases

import Steinberg.Defs.Lattice
import Steinberg.Defs.DegreeReflection

import Steinberg.Upstream.FreeGroup

/-!

  File dox.

-/

namespace Steinberg.A3

open Steinberg A3PosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {R : Type TR} [Ring R]

/-! ### Nonhomogeneous lift -/

theorem nonhom_lift_of_comm_of_αβ_βγ :
  ∀ (t₁ t₀ u₁ u₀ v₁ v₀ : R),
    ⁅ ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀))
    , ((βγ, 2, u₁ * v₁)) * ((βγ, 1, u₁ * v₀ + u₀ * v₁)) * ((βγ, 0, u₀ * v₀)) ⁆
    = 1 := by
  intro t₁ t₀ u₁ u₀ v₁ v₀
  apply eq_of_mul_inv_eq_one
  apply (weakA3Graded R).liftedProp_of_mem_lifted rels_of_nonhom_lift_of_comm_of_αβ_βγ
  · simp only [weakA3Graded, lifted_sets, Set.mem_singleton_iff]
  · exists t₁, t₀, u₁, u₀, v₁, v₀

/-! ### Definition of missing root -/
theorem def_of_αβγ :
  forall_i_t αβγ,
    ⁅ ((α, (split_3_into_1_2 i hi).1, t))'(correct_of_split_3_into_1_2 i hi).1
    , ((βγ, (split_3_into_1_2 i hi).2, 1))'(correct_of_split_3_into_1_2 i hi).2.1 ⁆
    = ((αβγ, i, t)) := by
  intro i hi t
  symm
  apply (weakA3Graded R).definitionProp_of_define

theorem a3_valid :
  reflectValidProp (weakA3Graded R) := by
  simp only [reflectValidProp]
  nth_rewrite 1 [weakA3Graded]
  simp only [lifted_sets, Set.mem_singleton_iff, forall_eq, rels_of_nonhom_lift_of_comm_of_αβ_βγ, Set.mem_setOf_eq]
  intro r h
  rcases h with ⟨ t₁, t₀, u₁, u₀, v₁, v₀, rfl ⟩
  simp only [map_mul, map_commutatorElement, FreeGroup.lift.of]
  repeat rw [defineThenReflect_eq_reflect_of_mem_presentRoots _ _ (by tauto)]
  simp only [reflect, PositiveRootSystem.height, height]
  simp_arith
  rw [add_comm (t₁ * u₀), add_comm (u₁ * v₀)]
  grw [expr_αβ_αβ_as_αβ_αβ, expr_αβ_αβ_as_αβ_αβ (i := 0), expr_αβ_αβ_as_αβ_αβ,
```

```
      expr_βγ_βγ_as_βγ_βγ, expr_βγ_βγ_as_βγ_βγ (i := 0), expr_βγ_βγ_as_βγ_βγ]
  exact nonhom_lift_of_comm_of_αβ_βγ t₀ t₁ u₀ u₁ v₀ v₁

/-! ### Derive full commutator for αβ and βγ from nonhomogeneous lift -/

-- TODO-A: this section should probably be abstracted for reuse

/- Commutator relation in the case (i,j) is not (0,2) or (2,0) (via the previous theorem). -/
private lemma hom_lift_of_comm_of_αβ_βγ (i j k : ℕ) (hi : i ≤ 1) (hj : j ≤ 1) (hk : k ≤ 1) :
  ∀ (t u : R), ⁅ ((αβ, i + j, t)), ((βγ, j + k, u)) ⁆ = 1 := by
    intro t u
    let t₁ : R := match i with
      | 1 => t
      | 0 => 0
    let t₀ : R := match i with
      | 1 => 0
      | 0 => t
    let u₁ : R := match j with
      | 1 => 1
      | 0 => 0
    let u₀ : R := match j with
      | 1 => 0
      | 0 => 1
    let v₁ : R := match k with
      | 1 => u
      | 0 => 0
    let v₀ : R := match k with
      | 1 => 0
      | 0 => u
    have hf_i : i ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have hf_j : j ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have hf_k : k ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have id₁ : ((αβ, i + j, t)) = ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀)) := by (
      fin_cases hf_i, hf_j, hf_k
      <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
    )
    have id₂ : ((βγ, j + k, u)) = ((βγ, 2, u₁ * v₁)) * ((βγ, 1, u₁ * v₀ + u₀ * v₁)) * ((βγ, 0, u₀ * v₀)) := by (
      fin_cases hf_i, hf_j, hf_k
      <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
    )
    rw [id₁, id₂, nonhom_lift_of_comm_of_αβ_βγ]

private lemma image_of_hom_lift_of_comm_of_αβ_βγ {i j : ℕ} (hi : i ≤ αβ.height) (hj : j ≤ βγ.height)
    : ((i, j) ∈ ij_jk_image) → ∀ (t u : R), ⁅ ((αβ, i, t)), ((βγ, j, u)) ⁆ = 1 := by
  intro h_in_image t u
  have : ∃ ijk' : ℕ × ℕ × ℕ, ijk' ∈ boolean_cube ∧ f_ij_jk ijk' = (i, j) := by
    rw [← Finset.mem_image, correct_of_ij_jk_image]; exact h_in_image
  simp [f_ij_jk] at this
  rcases this with ⟨ i', j', k', ⟨ hi', hj', hk' ⟩, rfl, rfl ⟩
  rw [← hom_lift_of_comm_of_αβ_βγ i' j' k' hi' hj' hk' t u]

private lemma comm_of_αβ_βγ_20 : ∀ (t u : R), ⁅ ((αβ, 2, t)), ((βγ, 0, u)) ⁆ = 1 := by
  intro t u
  apply @trivial_comm_from_embedded_comm_and_pairs _ _ ((βγ, 1, u)) _ ((αβ, 1, t))
  grw [← nonhom_lift_of_comm_of_αβ_βγ t 0 1 1 0 u]
  rw [← hom_lift_of_comm_of_αβ_βγ 1 1 0 (by trivial) (by trivial) (by trivial) t u]
  rw [← hom_lift_of_comm_of_αβ_βγ 0 1 0 (by trivial) (by trivial) (by trivial) t u]
  rw [← hom_lift_of_comm_of_αβ_βγ 1 0 0 (by trivial) (by trivial) (by trivial) t u]

-- symmetric to proof of `comm_of_αβ_βγ_20`
private lemma comm_of_αβ_βγ_02 : ∀ (t u : R), ⁅ ((αβ, 0, t)), ((βγ, 2, u)) ⁆ = 1 := by
  intro t u
  have : ⁅ ((αβ, 0, t)), ((βγ, 2, u)) ⁆ = defineThenReflectOfPresentedGroup a3_valid ⁅ ((αβ, 2, t)), ((βγ, 0, u))
    ↪  ⁆ := by
    rw [map_commutatorElement]; trivial
  rw [this, comm_of_αβ_βγ_20, map_one]

theorem comm_of_αβ_βγ : trivialSpanPropOfRootPair (weakA3Graded R).project (αβ, βγ) := by
  intro i j hi hj t u
  by_cases hij : (i, j) ∈ ij_jk_image
  · apply image_of_hom_lift_of_comm_of_αβ_βγ hi hj hij
  · have : (i = 0 ∧ j = 2) ∨ (i = 2 ∧ j = 0) := by
      simp [ij_jk_image] at hij
      height_simp at hi hj
      omega
    rcases this with ( ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ )
    · rw [← comm_of_αβ_βγ_02 t u]
    · rw [← comm_of_αβ_βγ_20 t u]
declare_A3_trivial_span_expr_thm R αβ βγ

```

```
/-! ### Further useful identities (roughly GENERIC) -/

/- Rewrite β•γ as γ•βγ•β. -/
@[group_reassoc]
theorem expr_β_γ_as_γ_βγ_β :
  forall_ij_tu β γ,
    reorder_mid(((β, i, t)), ((γ, j, u)), ((βγ, i + j, t * u))) := by
  intro i j hi hj t u
  have := comm_of_β_γ hi hj t u
  chev_simp at this
  rw [← this, comm_mid, inv_of_γ, comm_of_β_γ]
  grw [comm_of_β_γ]

/-! ### Interchange theorems between ⁅α,βγ⁆ and ⁅αβ,γ⁆ forms -/

/- Interchange between ⁅α, βγ⁆ and ⁅αβ, γ⁆, "trading" a single degree j : Deg 1 and scalar u : R. -/
theorem Interchange : forall_ijk_tuv α β γ,
     ⁅ ((α, i, t)), ((βγ, j + k, u * v)) ⁆ = ⁅ ((αβ, i + j, t * u)), ((γ, k, v)) ⁆ := by
  intro i j k hi hj hk t u v
  apply eq_comm_of_reorder_left
  have hij : i + j ≤ αβ.height := by ht
  have hjk : j + k ≤ βγ.height := by ht
  grw [expr_βγ_as_β_γ_β_γ hj hk,
    expr_α_β_as_αβ_β_α hi hj, expr_α_γ_as_γ_α hi hk,
    expr_α_β_as_αβ_β_α hi hj, expr_α_γ_as_γ_α hi hk,
    expr_β_γ_as_βγ_γ_β hj hk,
    expr_β_αβ_as_αβ_β hj hij,
    ← expr_γ_βγ_as_βγ_γ hk hjk,
    ← expr_αβ_βγ_as_βγ_αβ hij hjk,
    commutatorElement_def,
    expr_β_γ_as_βγ_γ_β hj hk,
    ← expr_γ_βγ_as_βγ_γ hk hjk]

/- Pass between ⁅α,βγ⁆ and ⁅αβ,γ⁆ forms (specializes `Interchange` to the case `u=1`). -/
theorem InterchangeTrans : forall_ijk_tu α β γ,
    ⁅ ((α, i, t)), ((βγ, j + k, u)) ⁆ = ⁅ ((αβ, i + j, t)), ((γ, k, u)) ⁆ := by
  intro i j k hi hj hk t u
  have := Interchange hi hj hk t 1 u
  rwa [one_mul, mul_one] at this

/- ⁅α,βγ⁆ forms depend only on product of coefficients. Applies `Interchange` twice. -/
theorem InterchangeRefl : forall_ijk_tu α β γ,
    ⁅ ((α, i, t * u)), ((βγ, j + k, 1)) ⁆ = ⁅ ((α, i, t)), ((βγ, j + k, u)) ⁆ := by
  intro i j k hi hj hk t u
  nth_rewrite 2 [← mul_one u]
  rw [Interchange hi hj hk, InterchangeTrans hi hj hk]

/-! ### Commutator relations for (α,βγ) and (αβ,γ) via interchange relations -/

-- TODO-A: Really need to figure out a more sane way to write this section.

-- height 0
private lemma comm_of_α_βγ_00 (t u : R) :
    ⁅ ((α, 0, t)), ((βγ, 0, u)) ⁆ = ((αβγ, 0, t * u)) := by
  rw [← @InterchangeRefl _ _ 0 0 0 (by trivial) (by trivial) (by trivial)]
  rw [← def_of_αβγ (by trivial) (t * u)]
  simp only [split_3_into_1_2]

private lemma comm_of_αβ_γ_00 (t u : R) :
    ⁅ ((αβ, 0, t)), ((γ, 0, u)) ⁆ = ((αβγ, 0, t * u)) := by
  rw [← @InterchangeTrans _ _ 0 0 0 (by trivial) (by trivial) (by trivial)]
  rw [comm_of_α_βγ_00]

-- height 1
private lemma comm_of_α_βγ_01 (t u : R) :
    ⁅ ((α, 0, t)), ((βγ, 1, u)) ⁆ = ((αβγ, 1, t * u)) := by
  rw [← @InterchangeRefl _ _ 0 0 1 (by trivial) (by trivial) (by trivial)]
  rw [← def_of_αβγ (by trivial) (t * u)]
  simp only [split_3_into_1_2]

private lemma comm_of_αβ_γ_10 (t u : R) : ⁅ ((αβ, 1, t)), ((γ, 0, u)) ⁆ = ((αβγ, 1, t * u)) := by
  rw [← @InterchangeTrans _ _ 0 1 0 (by trivial) (by trivial) (by trivial)]
  simp only [add_zero, comm_of_α_βγ_01, zero_add, one_mul]

private lemma comm_of_α_βγ_10 (t u : R) : ⁅ ((α, 1, t)), ((βγ, 0, u)) ⁆ = ((αβγ, 1, t * u)) := by
  rw [@InterchangeTrans _ _ 1 0 0 (by trivial) (by trivial) (by trivial),
        comm_of_αβ_γ_10]

private lemma comm_of_αβ_γ_01 (t u : R) : ⁅ ((αβ, 0, t)), ((γ, 1, u)) ⁆ = ((αβγ, 1, t * u)) := by
  rw [← @InterchangeTrans _ _ 0 0 1 (by trivial) (by trivial) (by trivial),
```

```
        comm_of_α_βγ_01]

-- height 2
private lemma comm_of_α_βγ_11 (t u : R) : ⁅ ((α, 1, t)), ((βγ, 1, u)) ⁆ = ((αβγ, 2, t * u)) := by
  rw [← @InterchangeRefl _ _ 1 0 1 (by trivial) (by trivial) (by trivial),
        ← def_of_αβγ (by trivial) (t * u)]
  simp only [split_3_into_1_2]

private lemma comm_of_αβ_γ_11 (t u : R) : ⁅ ((αβ, 1, t)), ((γ, 1, u)) ⁆ = ((αβγ, 2, t * u)) := by
  rw [← @InterchangeTrans _ _ 1 0 1 (by trivial) (by trivial) (by trivial),
        comm_of_α_βγ_11]

private lemma comm_of_α_βγ_02 (t u : R) : ⁅ ((α, 0, t)), ((βγ, 2, u)) ⁆ = ((αβγ, 2, t * u)) := by
  rw [@InterchangeTrans _ _ 0 1 1 (by trivial) (by trivial) (by trivial),
        comm_of_αβ_γ_11]

private lemma comm_of_αβ_γ_20 (t u : R) : ⁅ ((αβ, 2, t)), ((γ, 0, u)) ⁆ = ((αβγ, 2, t * u)) := by
  rw [← @InterchangeTrans _ _ 1 1 0 (by trivial) (by trivial) (by trivial),
        comm_of_α_βγ_11]

-- height 3
private lemma comm_of_α_βγ_12 (t u : R) : ⁅ ((α, 1, t)), ((βγ, 2, u)) ⁆ = ((αβγ, 3, t * u)) := by
  rw [← @InterchangeRefl _ _ 1 1 1 (by trivial) (by trivial) (by trivial),
        ← def_of_αβγ (by trivial) (t * u)]
  simp only [split_3_into_1_2]

private lemma comm_of_αβ_γ_21 (t u : R) : ⁅ ((αβ, 2, t)), ((γ, 1, u)) ⁆ = ((αβγ, 3, t * u)) := by
  rw [← @InterchangeTrans _ _ 1 1 1 (by trivial) (by trivial) (by trivial),
        comm_of_α_βγ_12]

/- Commutator relation for α and βγ. -/
theorem comm_of_α_βγ : singleCommutatorPropOfRootPair (weakA3Graded R).project ⟨α, βγ, αβγ, 1, (by ht)⟩ :=
  ↪ by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => chev_simp [comm_of_α_βγ_00 t u]
  | 0, 1 => chev_simp [comm_of_α_βγ_01 t u]
  | 0, 2 => chev_simp [comm_of_α_βγ_02 t u]
  | 1, 0 => chev_simp [comm_of_α_βγ_10 t u]
  | 1, 1 => chev_simp [comm_of_α_βγ_11 t u]
  | 1, 2 => chev_simp [comm_of_α_βγ_12 t u]
declare_A3_single_span_expr_thms R α βγ αβγ

/- Commutator relation for αβ and γ. -/
theorem comm_of_αβ_γ : singleCommutatorPropOfRootPair (weakA3Graded R).project ⟨αβ, γ, αβγ, 1, (by ht)⟩ :=
  ↪ by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => chev_simp [comm_of_αβ_γ_00 t u]
  | 1, 0 => chev_simp [comm_of_αβ_γ_10 t u]
  | 2, 0 => chev_simp [comm_of_αβ_γ_20 t u]
  | 0, 1 => chev_simp [comm_of_αβ_γ_01 t u]
  | 1, 1 => chev_simp [comm_of_αβ_γ_11 t u]
  | 2, 1 => chev_simp [comm_of_αβ_γ_21 t u]
declare_A3_single_span_expr_thms R αβ γ αβγ

/-! ### More rewriting theorems -/

theorem expr_αβγ_as_α_βγ_α_βγ_one_mul : forall_ij_t α βγ,
    ((αβγ, i + j, t)) = ((α, i, 1)) * ((βγ, j, t)) * ((α, i, -1)) * ((βγ, j, -t)) := by
  intro i j hi hj u
  have := expr_αβγ_as_α_βγ_α_βγ hi hj 1 u
  rwa [one_mul] at this

theorem expr_αβγ_as_α_βγ_α_βγ_mul_one : forall_ij_t α βγ,
    ((αβγ, i + j, t)) = ((α, i, t)) * ((βγ, j, 1)) * ((α, i, -t)) * ((βγ, j, -1)) := by
  intro i j hi hj t
  have := expr_αβγ_as_α_βγ_α_βγ hi hj t 1
  rwa [mul_one] at this

theorem expr_αβγ_as_αβ_γ_αβ_γ_one_mul : forall_ij_t αβ γ,
    ((αβγ, i + j, t)) = ((αβ, i, 1)) * ((γ, j, t)) * ((αβ, i, -1)) * ((γ, j, -t)) := by
  intro i j hi hj u
  have := expr_αβγ_as_αβ_γ_αβ_γ hi hj 1 u
  rwa [one_mul] at this

theorem expand_αβγ_as_αβ_γ_αβ_γ_mul_one : forall_ij_t αβ γ,
    ((αβγ, i + j, t)) = ((αβ, i, t)) * ((γ, j, 1)) * ((αβ, i, -t)) * ((γ, j, -1)) := by
  intro i j hi hj t
  have := expr_αβγ_as_αβ_γ_αβ_γ hi hj t 1
```

```
  rwa [mul_one] at this

/-! ### Commutators of αβγ with other roots -/

/- α and αβγ commute. -/
theorem comm_of_α_αβγ : trivialSpanPropOfRootPair (weakA3Graded R).project (α, αβγ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.mpr
  rcases decompose αβ.height γ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  grw [expr_αβγ_as_αβ_γ_αβ_γ_one_mul hj₁ hj₂,
      expr_α_αβ_as_αβ_α hi hj₁, expr_α_γ_as_γ_α hi hj₂,
      expr_α_αβ_as_αβ_α hi hj₁, expr_α_γ_as_γ_α hi hj₂]

/- β and αβγ commute. -/
-- the only commutator proof where we have to do something 'interesting'
theorem comm_of_β_αβγ : trivialSpanPropOfRootPair (weakA3Graded R).project (β, αβγ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.mpr
  rcases decompose αβ.height γ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  grw [expr_αβγ_as_αβ_γ_αβ_γ_one_mul hj₁ hj₂,
      expr_β_αβ_as_αβ_β hi hj₁, expr_β_γ_as_γ_βγ_β hi hj₂,
      expr_β_αβ_as_αβ_β hi hj₁, expr_β_γ_as_βγ_γ_β hi hj₂,
      ← expr_αβ_βγ_as_βγ_αβ hj₁]

/- γ and αβγ commute. -/
theorem comm_of_γ_αβγ : trivialSpanPropOfRootPair (weakA3Graded R).project (γ, αβγ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.mpr
  rcases decompose α.height βγ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  grw [expr_αβγ_as_α_βγ_α_βγ_one_mul hj₁ hj₂,
    ← expr_α_γ_as_γ_α hj₁ hi, expr_γ_βγ_as_βγ_γ hi hj₂,
    ← expr_α_γ_as_γ_α hj₁ hi, expr_γ_βγ_as_βγ_γ hi hj₂]

/- αβ and αβγ commute. -/
theorem comm_of_αβ_αβγ : trivialSpanPropOfRootPair (weakA3Graded R).project (αβ, αβγ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.mpr
  rcases decompose α.height βγ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  grw [expr_αβγ_as_α_βγ_α_βγ_one_mul hj₁ hj₂,
    ← expr_α_αβ_as_αβ_α hj₁ hi, expr_αβ_βγ_as_βγ_αβ hi hj₂,
    ← expr_α_αβ_as_αβ_α hj₁ hi, expr_αβ_βγ_as_βγ_αβ hi hj₂]

/- βγ and αβγ commute. -/
theorem comm_of_βγ_αβγ : trivialSpanPropOfRootPair (weakA3Graded R).project (βγ, αβγ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.mpr
  rcases decompose αβ.height γ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  grw [expr_αβγ_as_αβ_γ_αβ_γ_one_mul hj₁ hj₂,
    ← expr_αβ_βγ_as_βγ_αβ hj₁ hi, ← expr_γ_βγ_as_βγ_γ hj₂ hi,
    ← expr_αβ_βγ_as_βγ_αβ hj₁ hi, ← expr_γ_βγ_as_βγ_γ hj₂ hi]

declare_A3_trivial_span_expr_thm R α αβγ
declare_A3_trivial_span_expr_thm R β αβγ
declare_A3_trivial_span_expr_thm R γ αβγ
declare_A3_trivial_span_expr_thm R αβ αβγ
declare_A3_trivial_span_expr_thm R βγ αβγ

/- αβγ commutes with itself. -/
theorem comm_of_αβγ_αβγ : mixedDegreePropOfRoot (weakA3Graded R).project αβγ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.mpr
  rcases decompose α.height βγ.height j (by trivial) with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  grw [expr_αβγ_as_α_βγ_α_βγ_one_mul hj₁ hj₂,
    ← expr_α_αβγ_as_αβγ_α hj₁ hi, ← expr_βγ_αβγ_as_αβγ_βγ hj₂ hi,
    ← expr_α_αβγ_as_αβγ_α hj₁ hi, ← expr_βγ_αβγ_as_αβγ_βγ hj₂ hi]
declare_A3_trivial_span_expr_thm R αβγ αβγ

/- Linearity for αβγ. -/
@[group_reassoc (attr := simp, chev_simps)]
theorem lin_of_αβγ : lin_of_root((weakA3Graded R).project, αβγ) := by
  intro i hi t u
  rcases decompose α.height βγ.height i (by trivial) with ⟨ i₁, i₂, ⟨ rfl, hi₁, hi₂ ⟩ ⟩
  have h_eq' : i₁ + i₂ ≤ PositiveRootSystem.height αβγ := by omega
  grw [expr_αβγ_as_α_βγ_α_βγ_mul_one hi₁ hi₂,
    expr_βγ_αβγ_as_αβγ_βγ hi₂ h_eq',
    expr_α_αβγ_as_αβγ_α hi₁ h_eq',
    expr_βγ_αβγ_as_αβγ_βγ hi₂ h_eq',
    expr_αβγ_as_α_βγ_α_βγ_mul_one hi₁ hi₂,
    ← neg_add, add_comm u t,
    ← expr_αβγ_as_α_βγ_α_βγ hi₁ hi₂]
```

```

theorem full_relations_implied_by_weak_relations :
  ∀ r ∈ (fullA3Graded R).allRelations, (weakA3Graded R).project r = 1 := by
  simp only [weakA3Graded]
  apply GradedPartialChevalleyGroup.graded_injection
  intro K
  rcases K
  all_goals simp only; intro p h
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [trivialSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      rcases h_new with h_αβ_βγ|h_α_αβγ|h_β_αβγ|h_γ_αβγ|h_αβ_αβγ|h_βγ_αβγ
      all_goals subst p r
      · exact comm_of_αβ_βγ hi hj t u
      · exact comm_of_α_αβγ hi hj t u
      · exact comm_of_β_αβγ hi hj t u
      · exact comm_of_γ_αβγ hi hj t u
      · exact comm_of_αβ_αβγ hi hj t u
      · exact comm_of_βγ_αβγ hi hj t u
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [singleSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      rcases h_new with h_α_βγ|h_αβ_γ
      all_goals (
        subst p r
        simp only [map_mul, map_inv, mul_inv_eq_one]
      )
      · exact comm_of_α_βγ hi hj t u
      · exact comm_of_αβ_γ hi hj t u
  · tauto
  · rcases h with h_old|h_new
    · tauto
    · right
      simp_all only [Set.mem_singleton_iff]
      intro r h_r
      simp only [mixedDegreeRelationsOfRoot] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      subst r
      exact comm_of_αβγ_αβγ hi hj t u
  · rcases h with h_old|h_new
    · tauto
    · right
      simp_all only [Set.mem_singleton_iff]
      intro r h_r
      simp only [linearityRelationsOfRoot] at h_r
      rcases h_r with ⟨ i, hi, t, u, goal ⟩
      subst r
      simp only [map_mul, map_inv, mul_inv_eq_one]
      exact lin_of_αβγ hi t u
  · tauto
  · simp only [definitionRelations, Set.mem_iUnion, Set.mem_setOf_eq] at h
    rcases h with ⟨ζ, i, hi, t, h⟩
    subst p
    simp only [fullA3Graded, fullMk, inv_mul_cancel, map_one]

end Steinberg.A3
```

Author source module Steinberg.B3Small.Defs

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Defs.PartialChevalleyGroup
import Steinberg.Defs.GradedPartialChevalleyGroup
import Steinberg.Defs.DegreeReflection
import Steinberg.Defs.DecomposeFixed

```

```
import Mathlib.Tactic.DeriveFintype

/-!

  File dox.

-/

namespace Steinberg.B3Small

open Steinberg PartialChevalley GradedPartialChevalley GradedChevalleyGenerator
  PartialChevalleySystem

/-! # The B3-small positive root system -/

inductive B3SmallPosRoot
  | β | ψ | ω | βψ | ψω | β2ψ | βψω
deriving Fintype, DecidableEq, Inhabited

namespace B3SmallPosRoot

@[reducible, height_simps]
def height : B3SmallPosRoot → Nat
  | β | ψ | ω => 1
  | βψ | ψω => 2
  | βψω | β2ψ => 3

def toString : B3SmallPosRoot → String
  | β => "β"
  | ψ => "ψ"
  | ω => "ω"
  | βψ => "β+ψ"
  | ψω => "ψ+ω"
  | βψω => "β+ψ+ω"
  | β2ψ => "β+2ψ"

instance : PositiveRootSystem B3SmallPosRoot where
  height := height
  toString := toString

instance instCoeNat : Coe B3SmallPosRoot Nat where
  coe r := height r

end B3SmallPosRoot

open B3SmallPosRoot

variable {F : Type TR} [Field F]

/-! # Definition of the 'weak' B3-small graded group -/

/-! ## Defining the 'weak' positive root system -/

abbrev weakPresentRoots : Set (B3SmallPosRoot) := {β, ψ, ω, βψ, ψω, β2ψ}

abbrev weakTrivialSpanPairs : Set (B3SmallPosRoot × B3SmallPosRoot) :=
  {(β, βψ), (β, β2ψ), (ψ, β2ψ), (βψ, β2ψ), (β, ω), (ψ, ψω), (ω, ψω)}

abbrev weakSingleSpanRootPairs : Set (SingleSpanRootPair B3SmallPosRoot)
   := {⟨ ψ, βψ, β2ψ, 2, (by ht)⟩, ⟨ψ, ω, ψω, 2, (by ht)⟩}

abbrev weakDoubleSpanRootPairs : Set (DoubleSpanRootPair B3SmallPosRoot) :=
    {⟨β, ψ, βψ, β2ψ, 1, 1, (by exact rfl), (by exact rfl)⟩}

abbrev weakB3SmallSystem := PartialChevalleySystem.mk
  weakPresentRoots
  weakTrivialSpanPairs
  weakSingleSpanRootPairs
  weakDoubleSpanRootPairs
  (by simp only [Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp, reduceCtorEq, or_self,
    or_false, or_true, and_self, forall_eq])
  (by simp only [Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp, reduceCtorEq, or_self,
    or_false, or_true, and_self, forall_eq])
  (by simp only [Set.mem_singleton_iff, Set.mem_insert_iff, forall_eq, reduceCtorEq, or_self,
    or_false, or_true, and_self])

/-! ## Lifted relations -/

/-
```

```
The specific relation arises from "nonhomogeneously lifting" the commutator of βψ and ψω elements. (There is no analogue
of this relation for other root-pairs, since all other present pairs lie in a common two-dimensional subspace.)
-/
def rels_of_nonhom_lift_of_comm_of_βψ_ψω :=
   { ⁅ {βψ, 2, t₁ * u₁} * {βψ, 1, t₁ * u₀ + t₀ * u₁} * {βψ, 0, t₀ * u₀},
       {ψω, 2, u₁ * v₁} * {ψω, 1, u₁ * v₀ + u₀ * v₁} * {ψω, 0, u₀ * v₀} ⁆
     | (t₁ : F) (t₀ : F) (u₁ : F) (u₀ : F) (v₁ : F) (v₀ : F) }

-- lifted commutator of βψ and ψω
def lifted_sets (F : Type TR) [Field F] : Set (Set (FreeGroup (GradedChevalleyGenerator B3SmallPosRoot F))) := {
  rels_of_nonhom_lift_of_comm_of_βψ_ψω
}

/-! ## Definition for missing root (βψω) -/

def weak_define (F : Type TR) [Field F] (g : GradedChevalleyGenerator B3SmallPosRoot F) : FreeGroup (GradedChevalleyGenerator B3SmallPosRoot F) :=
  let ⟨ ζ, i, hi, t ⟩ := g;
  match ζ with
  | βψω => ⁅ {β,(split_3_into_1_2 i (by ht)).1, g.t}'(correct_of_split_3_into_1_2 i (by ht)).1,
    {ψω, (split_3_into_1_2 i (by ht)).2, 1}'(correct_of_split_3_into_1_2 i (by ht)).2.1 ⁆
  | ζ => FreeGroup.of g

theorem weak_define_of_present (F : Type TR) [Field F] :
  ∀ {g : GradedChevalleyGenerator B3SmallPosRoot F}, g.ζ ∈ weakB3SmallSystem.presentRoots → weak_define F g = FreeGroup.of g := by
  intro g h_g_in_present
  rcases g with ⟨ ζ, i, hi, t ⟩
  cases ζ
  all_goals simp only [weak_define] -- this will close all present roots
  all_goals ( -- this will close the remaining (nonpresent) roots
    simp only [weakPresentRoots] at h_g_in_present
    contradiction
  )

theorem weak_define_is_projection (F : Type TR) [Field F] :
  ∀ {g : GradedChevalleyGenerator B3SmallPosRoot F}, (FreeGroup.lift (weak_define F)) (weak_define F g) = weak_define F g := by
  intro g
  rcases g with ⟨ ζ, i, hi, t ⟩
  cases ζ
  all_goals simp only [weak_define, FreeGroup.lift.of, map_commutatorElement]

def weakB3SmallGraded (F : Type TR) [Field F] := GradedPartialChevalleyGroup.mk
  weakB3SmallSystem
  (lifted_sets F)
  (weak_define F)
  (weak_define_of_present F)
  (weak_define_is_projection F)

/-! # Definition of the 'full' A3 ungraded and graded groups -/

abbrev fullPresentRoots : Set (B3SmallPosRoot) :=
  weakPresentRoots ∪ {βψω}

abbrev fullTrivialSpanPairs : Set (B3SmallPosRoot × B3SmallPosRoot) :=
  weakTrivialSpanPairs ∪ {(βψ, ψω), (βψω, β), (βψω, ψ), (βψω, ω), (βψω, βψ), (βψω, β2ψ), (βψω, ψω), (ω, β2ψ), (ψω, β2ψ)}

abbrev fullSingleSpanRootPairs : Set (SingleSpanRootPair B3SmallPosRoot) :=
  weakSingleSpanRootPairs ∪ {⟨ β, ψω, βψω, 1, (by ht)⟩, ⟨βψ, ω, βψω, 2, (by ht)⟩}

abbrev fullDoubleSpanRootPairs : Set (DoubleSpanRootPair B3SmallPosRoot) :=
  weakDoubleSpanRootPairs

-- TODO: this should really be via 'decide', but we had issues with declaring PartialChevalleySystem as a Finset
theorem all_root_pairs_have_relation : everyRootPairInRootPairs B3SmallPosRoot fullTrivialSpanPairs fullSingleSpanRootPairs fullDoubleSpanRootPairs := by
  intro ζ η h_ne
  unfold toRootPairs fullTrivialSpanPairs weakTrivialSpanPairs fullSingleSpanRootPairs
    weakSingleSpanRootPairs fullDoubleSpanRootPairs weakDoubleSpanRootPairs
  simp only [Set.image_insert_eq, Set.image_singleton, Set.union_insert, Set.union_singleton, Prod.swap,
    Set.mem_insert_iff, Set.mem_singleton_iff]
  cases ζ
  all_goals (
```

```
    simp only [Prod.mk.injEq, reduceCtorEq, false_and, true_and, or_self, or_false, false_or]
    cases η
    all_goals trivial
  )

abbrev fullB3SmallSystem := PartialChevalleySystem.mkFull B3SmallPosRoot
  fullPresentRoots
  fullTrivialSpanPairs
  fullSingleSpanRootPairs
  fullDoubleSpanRootPairs
  (by decide)
  all_root_pairs_have_relation

def fullB3Small (R : Type TR) [Ring R] := PartialChevalleyGroup.fullMk B3SmallPosRoot R fullB3SmallSystem
def fullB3SmallGraded (R : Type TR) [Ring R] := GradedPartialChevalleyGroup.fullMk B3SmallPosRoot R
  ↪ fullB3SmallSystem

/-! # Notation and macros -/

/- Instantiate the `declare_thms` macros from `PartialChevalley.lean`. -/

-- TODO-B: TODO: Make this a macro to declare all at once for A3.
--     Something like: `declare_thms A3 weakB3SmallGraded F`

macro "declare_B3Small_trivial_span_expr_thm" F:term:arg r₁:term:arg r₂:term:arg : command =>
  `(command| declare_trivial_span_expr_thm weakB3SmallGraded $F $r₁ $r₂)

macro "declare_B3Small_trivial_span_of_root_pair_thms" F:term:arg r₁:term:arg r₂:term:arg : command =>
  `(command| declare_trivial_span_of_root_pair_thms weakB3SmallGraded $F $r₁ $r₂)

macro "declare_B3Small_single_span_expr_thms" F:term:arg r₁:term:arg r₂:term:arg r₃:term:arg n:num :
  ↪ command =>
  `(command| declare_single_span_expr_thms weakB3SmallGraded $F $r₁ $r₂ $r₃ $n)

macro "declare_B3Small_single_span_of_root_pair_thms" F:term:arg r₁:term:arg r₂:term:arg r₃:term:arg n:num
  ↪ : command =>
  `(command| declare_single_span_of_root_pair_thms weakB3SmallGraded $F $r₁ $r₂ $r₃ $n)

macro "declare_B3Small_lin_id_inv_thms" F:term:arg root:term:arg : command =>
  `(command| declare_lin_id_inv_thms weakB3SmallGraded $F $root)

macro "declare_B3Small_mixed_expr_thm" F:term:arg r:term:arg : command =>
  `(command| declare_mixed_degree_expr_thm weakB3SmallGraded $F $r)

macro "declare_B3Small_mixed_degree_thms" F:term:arg r:term:arg : command =>
  `(command| declare_mixed_degree_thms weakB3SmallGraded $F $r)

macro "declare_B3Small_defineThenReflect_thm" F:term:arg r:term:arg : command =>
  `(command| declare_defineThenReflect_thm weakB3SmallGraded $F B3SmallPosRoot $r)

-- r₁ is the larger root, as opposed to the above macros
macro "declare_B3Small_reflected_thm"
    F:term:arg v:term:arg r₁:term:arg r₂:term:arg r₃:term:arg
    "const" C:num
    "heights" n₁:num n₂:num n₃:num
    "to" n₄:num n₅:num n₆:num : command =>
  `(command| declare_reflected_thm weakB3SmallGraded $F $v $r₁ $r₂ $r₃ 0 $C $n₁ $n₂ $n₃ $n₄ $n₅ $n₆)

macro "declare_B3Small_triv_comm_reflected_thm"
    F:term:arg v:term:arg r₁:term:arg r₂:term:arg
    "heights" n₁:num n₂:num
    "to" n₃:num n₄:num : command =>
  `(command| declare_triv_comm_reflected_thm weakB3SmallGraded $F $v $r₁ $r₂ $n₁ $n₂ $n₃ $n₄)

set_option hygiene false in
/-- Shorthand for building free group elements from a root, degree, and ring element. -/
scoped notation (priority:=high) "((" ζ ", " i ", " t "))" =>
  (weakB3SmallGraded F).project {ζ, i, t}

set_option hygiene false in
/-- Shorthand for building free group elements from a root, degree, and ring element. -/
scoped notation (priority:=high) "((" ζ ", " i ", " t "))'" h:max =>
  (weakB3SmallGraded F).project ({ζ, i, t}'h)

section forallNotation

set_option hygiene false

scoped notation "forall_i_t" h:max "," e =>
  ∀ ⦃i : ℕ⦄ (hi : i ≤ h) (t : F), e
```

```

scoped notation "forall_ij_tu" h₁:max h₂:max "," e =>
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ h₁) (hk : j ≤ h₂) (t u : F), e

scoped notation "forall_ik_tuv" h₁:max h₂:max "," e =>
  ∀ ⦃i k : ℕ⦄ (hi : i ≤ h₁) (hk : k ≤ h₂) (t u v : F), e

scoped notation "forall_ijk_tu" h₁:max h₂:max h₃:max "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (hk : k ≤ h₃) (t u : F), e

scoped notation "forall_ijk_tuv" h₁:max h₂:max h₃:max "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (hk : k ≤ h₃) (t u v : F), e

end forallNotation

macro "hom_tac " rel:ident " [" intros:ident,* "]" : tactic => `(tactic|
  ( intros $intros*;
    apply GradedPartialChevalleyGroup.helper;
    apply (weakB3SmallGraded _).liftedProp_of_mem_lifted $rel;
    simp only [weakB3SmallGraded, lifted_sets, Set.mem_insert_iff,
      Set.mem_singleton_iff, true_or, or_true];
    exists $intros,* ))

end Steinberg.B3Small
```

Author source module Steinberg.B3Small.Basic

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Small.Defs

/-!

  File dox.

-/

namespace Steinberg.B3Small

open B3SmallPosRoot GradedPartialChevalley
  GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {F : Type TF} [Field F]

/-! # "Given" relations for specific roots -/

declare_B3Small_lin_id_inv_thms F β
declare_B3Small_lin_id_inv_thms F ψ
declare_B3Small_lin_id_inv_thms F ω
declare_B3Small_lin_id_inv_thms F βψ
declare_B3Small_lin_id_inv_thms F ψω
declare_B3Small_lin_id_inv_thms F β2ψ

declare_B3Small_trivial_span_of_root_pair_thms F β βψ
declare_B3Small_trivial_span_of_root_pair_thms F β β2ψ
declare_B3Small_trivial_span_of_root_pair_thms F ψ β2ψ
declare_B3Small_trivial_span_of_root_pair_thms F βψ β2ψ
declare_B3Small_trivial_span_of_root_pair_thms F β ω
declare_B3Small_trivial_span_of_root_pair_thms F ψ ψω
declare_B3Small_trivial_span_of_root_pair_thms F ω ψω

declare_B3Small_single_span_of_root_pair_thms F ψ ω ψω 2
declare_B3Small_single_span_of_root_pair_thms F ψ βψ β2ψ 2

/-! ### Mixed-degree theorem for specific roots -/

declare_B3Small_mixed_degree_thms F βψ
declare_B3Small_mixed_degree_thms F ψω
declare_B3Small_mixed_degree_thms F β2ψ

declare_B3Small_defineThenReflect_thm F β
declare_B3Small_defineThenReflect_thm F ψ
declare_B3Small_defineThenReflect_thm F ω
declare_B3Small_defineThenReflect_thm F βψ
```

```
declare_B3Small_defineThenReflect_thm F ψω
declare_B3Small_defineThenReflect_thm F β2ψ

end Steinberg.B3Small
```

Author source module Steinberg.B3Small.Thm

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Small.Basic

import Mathlib.Tactic.Group
import Mathlib.Tactic.FinCases
import Mathlib.Tactic.FieldSimp

import Steinberg.Defs.Lattice
import Steinberg.Defs.Commutator
import Steinberg.Defs.DegreeReflection

import Steinberg.Upstream.FreeGroup

namespace Steinberg.B3Small

open Steinberg B3SmallPosRoot GradedPartialChevalley
  GradedPartialChevalleyGroup GradedChevalleyGenerator

variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)

/-! ### Double commutator theorem -/

theorem comm_of_β_ψ : doubleSpanPropOfRootPair (weakB3SmallGraded F).project ⟨β, ψ, βψ, β2ψ, 1, 1, rfl, by rfl⟩ :=
  (weakB3SmallGraded F).doubleSpanProp_of_mem_doubleSpanRootPairs ⟨β, ψ, βψ, β2ψ, 1, 1, by rfl, by rfl⟩
    (by simp only [weakB3SmallGraded, weakTrivialSpanPairs]; tauto)

/-! ### Nonhomogeneous lift -/
theorem nonhom_lift_of_comm_of_βψ_ψω :
  ∀ (t₁ t₀ u₁ u₀ v₁ v₀ : F),
    ⁅ ((βψ, 2, t₁ * u₁)) * ((βψ, 1, t₁ * u₀ + t₀ * u₁)) * ((βψ, 0, t₀ * u₀))
    , ((ψω, 2, u₁ * v₁)) * ((ψω, 1, u₁ * v₀ + u₀ * v₁)) * ((ψω, 0, u₀ * v₀)) ⁆
    = 1 := by
  intro t₁ t₀ u₁ u₀ v₁ v₀
  apply eq_of_mul_inv_eq_one
  apply (weakB3SmallGraded F).liftedProp_of_mem_lifted rels_of_nonhom_lift_of_comm_of_βψ_ψω
  · simp only [weakB3SmallGraded, lifted_sets, Set.mem_singleton_iff]
  · exists t₁, t₀, u₁, u₀, v₁, v₀

/-! ### Definition of missing root -/
theorem def_of_βψω : forall_i_t βψω,
    ⁅ ((β, (split_3_into_1_2 i hi).1, t))'(correct_of_split_3_into_1_2 i hi).1
    , ((ψω, (split_3_into_1_2 i hi).2, 1))'(correct_of_split_3_into_1_2 i hi).2.1 ⁆
      = ((βψω, i, t)) := by
  intro t i hi
  symm
  apply (weakB3SmallGraded F).definitionProp_of_define

theorem defineThenReflect_eq_reflect_of_βψω (g : GradedChevalleyGenerator B3SmallPosRoot F) (h : g.ζ = βψω) :
  (weakB3SmallGraded F).project (defineThenReflect (weakB3SmallGraded F) g) = (weakB3SmallGraded F).project (FreeGroup.of (reflect g)) := by
  rcases g with ⟨ ζ, i, hi, t ⟩
  simp only at h
  subst ζ
  simp only [defineThenReflect, MonoidHom.coe_comp, Function.comp_apply, FreeGroup.lift.of]
  rw [weakB3SmallGraded]
  simp only [weak_define, map_commutatorElement, FreeGroup.map.of, reflect]
  rw [← weakB3SmallGraded, ← def_of_βψω]
  congr
  all_goals (
    simp only [PositiveRootSystem.height, split_3_into_1_2]
    split
    all_goals trivial
  )

```

```
theorem b3small_valid :
  reflectValidProp (weakB3SmallGraded F) := by
  simp only [reflectValidProp]
  nth_rewrite 1 [weakB3SmallGraded]
  simp only [lifted_sets, Set.mem_singleton_iff, forall_eq, rels_of_nonhom_lift_of_comm_of_βψ_ψω,
    ↪  Set.mem_setOf_eq]
  intro r h
  rcases h with ⟨ t₁, t₀, u₁, u₀, v₁, v₀, rfl ⟩
  simp only [map_mul, map_commutatorElement, FreeGroup.lift.of]
  simp only [defineThenReflect_eq_reflect_of_ψω, defineThenReflect_eq_reflect_of_βψ]
  simp only [reflect, PositiveRootSystem.height, height]
  simp_arith
  rw [add_comm (t₁ * u₀), add_comm (u₁ * v₀)]
  grw [expr_βψ_βψ_as_βψ_βψ, expr_βψ_βψ_as_βψ_βψ (i := 0), expr_βψ_βψ_as_βψ_βψ,
       expr_ψω_ψω_as_ψω_ψω, expr_ψω_ψω_as_ψω_ψω (i := 0), expr_ψω_ψω_as_ψω_ψω]
  exact nonhom_lift_of_comm_of_βψ_ψω t₀ t₁ u₀ u₁ v₀ v₁

/-! ### 8.37 -/

theorem expr_βψ_as_ψ_β_ψ_β_ψ : forall_ij_tu β ψ,
    ((βψ, i + j, 2 * t * u))
      = ((ψ, j, -u)) * ((β, i, t)) * ((ψ, j, 2 * u)) * ((β, i, -t)) * ((ψ, j, -u)) := by
  intro i j hi hj t u
  have hij : i + j ≤ βψ.height := by ht
  rw [← mul_inv_eq_iff_eq_mul]
  mar; rw [← inv_mul_eq_iff_eq_mul]; mal
  apply mul_right_cancel (b := ((ψ, j, 2 * u))⁻¹)
  rw [← inv_of_β, ← commutatorElement_def]
  grw [comm_of_β_ψ, expr_βψ_β2ψ_as_β2ψ_βψ]
  rw [eq_of_deg_coef_eq β2ψ (j + (i + j)) (by omega) (2 * u * (2 * t * u)) (by ring)]
  grw [expr_β2ψ_as_ψ_βψ_ψ_βψ hj hij]
  ring_nf
  chev_simp

/-! ### Derive full commutator for βψ and ψω from nonhomogeneous lift -/

-- TODO-A: this section should probably be abstracted for reuse

/- Commutator relation in the case (i,j) is not (0,2) or (2,0) (via the previous theorem). -/
private lemma hom_lift_of_comm_of_βψ_ψω (i j k : ℕ) (hi : i ≤ 1) (hj : j ≤ 1) (hk : k ≤ 1) :
  ∀ (t u : F), ⁅ (( βψ, i + j, t)), ((ψω, j + k, u)) ⁆ = 1 := by
    intro t u
    let t₁ : F := match i with
      | 1 => t
      | 0 => 0
    let t₀ : F := match i with
      | 1 => 0
      | 0 => t
    let u₁ : F := match j with
      | 1 => 1
      | 0 => 0
    let u₀ : F := match j with
      | 1 => 0
      | 0 => 1
    let v₁ : F := match k with
      | 1 => u
      | 0 => 0
    let v₀ : F := match k with
      | 1 => 0
      | 0 => u
    have hf_i : i ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have hf_j : j ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have hf_k : k ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have id₁ : ((βψ, i + j, t)) = ((βψ, 2, t₁ * u₁)) * ((βψ, 1, t₁ * u₀ + t₀ * u₁)) * ((βψ, 0, t₀ * u₀)) := by (
      fin_cases hf_i, hf_j, hf_k
      <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
    )
    have id₂ : ((ψω, j + k, u)) = ((ψω, 2, u₁ * v₁)) * ((ψω, 1, u₁ * v₀ + u₀ * v₁)) * ((ψω, 0, u₀ * v₀)) := by (
      fin_cases hf_i, hf_j, hf_k
      <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
    )
    rw [id₁, id₂, nonhom_lift_of_comm_of_βψ_ψω]

private lemma image_of_hom_lift_of_comm_of_βψ_ψω {i j : ℕ} (hi : i ≤ βψ.height) (hj : j ≤ ψω.height)
    : ((i, j) ∈ ij_jk_image) → ∀ (t u : F), ⁅ ((βψ, i, t)), ((ψω, j, u)) ⁆ = 1 := by
  intro h_in_image t u
  have : ∃ ijk' : ℕ × ℕ × ℕ, ijk' ∈ boolean_cube ∧ f_ij_jk ijk' = (i, j) := by
    rw [← Finset.mem_image, correct_of_ij_jk_image]; exact h_in_image
  simp [f_ij_jk] at this
  rcases this with ⟨ i', j', k', ⟨ hi', hj', hk' ⟩, rfl, rfl ⟩
```

```
  rw [← hom_lift_of_comm_of_βψ_ψω i' j' k' hi' hj' hk' t u]

private lemma comm_of_βψ_ψω_20 : ∀ (t u : F), ⁅ ((βψ, 2, t)), ((ψω, 0, u)) ⁆ = 1 := by
  intro t u
  apply @trivial_comm_from_embedded_comm_and_pairs _ _ ((ψω, 1, u)) _ ((βψ, 1, t))
  grw [← nonhom_lift_of_comm_of_βψ_ψω t 0 1 1 0 u]
  rw [← hom_lift_of_comm_of_βψ_ψω 1 1 0 (by trivial) (by trivial) (by trivial) t u]
  rw [← hom_lift_of_comm_of_βψ_ψω 0 1 0 (by trivial) (by trivial) (by trivial) t u]
  rw [← hom_lift_of_comm_of_βψ_ψω 1 0 0 (by trivial) (by trivial) (by trivial) t u]

-- symmetric to proof of `comm_of_βψ_ψω_20`
private lemma comm_of_βψ_ψω_02 : ∀ (t u : F), ⁅ ((βψ, 0, t)), ((ψω, 2, u)) ⁆ = 1 := by
  intro t u
  have : ⁅((βψ, 0, t)), ((ψω, 2, u))⁆ = defineThenReflectOfPresentedGroup b3small_valid ⁅((βψ, 2, t)), ((ψω, 0,
    ↪  u))⁆ := by
    rw [map_commutatorElement]
    trivial
  rw [this, comm_of_βψ_ψω_20, map_one]

-- 8.42
theorem comm_of_βψ_ψω : trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψ, ψω⟩ := by
  intro i j hi hj t u
  have : (i = 0 ∧ j = 2) ∨ (i = 2 ∧ j = 0) ∨ ((i, j) ∈ ij_jk_image) := by
    rw [ij_jk_image]
    simp only [PositiveRootSystem.height] at *
    simp only [B3SmallPosRoot.height] at *
    simp -- should fix
    omega
  rcases this with ( ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | hij )
  · rw [← comm_of_βψ_ψω_02 t u]
  · rw [← comm_of_βψ_ψω_20 t u]
  · apply image_of_hom_lift_of_comm_of_βψ_ψω hi hj hij

declare_B3Small_trivial_span_expr_thm F βψ ψω

/-! ### Further useful identities (roughly GENERIC) -/

-- 8.39
theorem expr_ψ_ω_as_ω_ψ_ψω : forall_ij_tu ψ ω,
    ((ψ, i, t)) * ((ω, j, u)) = ((ω, j, u)) * ((ψ, i, t)) * ((ψω, i + j, 2 * t * u)) := by
  intro i j hi hj t u
  rw [comm_right]
  mul_inj
  rw [inv_of_ω, inv_of_ψ, comm_of_ψ_ω]
  ring_nf

theorem expr_ψ_ω_as_ω_ψω_ψ : forall_ij_tu ψ ω,
    ((ψ, i, t)) * ((ω, j, u)) = ((ω, j, u)) * ((ψω, i + j, 2 * t * u)) * ((ψ, i, t)) := by
  intro i j hi hj t u
  grw [expr_ψ_ω_as_ω_ψ_ψω hi hj, expr_ψ_ψω_as_ψω_ψ]

-- 8.40
theorem expr_β_ψ_as_ψ_β_β2ψ_βψ : forall_ij_tu β ψ,
    ((β, i, t)) * ((ψ, j, u)) = ((ψ, j, u)) * ((β, i, t)) * ((β2ψ, i + 2 * j, -t * u^2)) * ((βψ, i + j, t * u)) := by
  intro i j hi hj t u
  have := comm_of_β_ψ hi hj (-t) (-u)
  chev_simp at this
  grw [← expr_βψ_β2ψ_as_β2ψ_βψ, ← mul_assoc, ← this]

-- 8.40 a

theorem expr_β_ψ_as_ψ_βψ_β2ψ_β : forall_ij_tu β ψ,
    ((β, i, t)) * ((ψ, j, u)) = ((ψ, j, u)) * ((βψ, i + j, t * u)) * ((β2ψ, i + 2 * j, -(t * u^2))) * ((β, i, t)) :=
    ↪  by
  intro i j hi hj t u
  grw [expr_β_ψ_as_ψ_β_β2ψ_βψ hi hj, ← expr_βψ_β2ψ_as_β2ψ_βψ,
        expr_β_βψ_as_βψ_β, expr_β_β2ψ_as_β2ψ_β]

-- 8.41
theorem expr_ψ_β_as_βψ_β2ψ_β_ψ : forall_ij_tu β ψ,
    ((ψ, j, u)) * ((β, i, t)) = ((βψ, i + j, -t * u)) * ((β2ψ, i + 2 * j, -t * u^2)) * ((β, i, t)) * ((ψ, j, u)) := by
  intro i j hi hj t u
  apply reorder_left_of_eq_comm
  rw [← commutatorElement_inv, comm_of_β_ψ hi hj, expr_βψ_β2ψ_as_β2ψ_βψ,
    neg_mul, ←inv_of_βψ, neg_mul, ←inv_of_β2ψ]
  group

/-! ### Establishing βψω -/

include Fchar
```

```
225
226 private lemma write_t_as_2tu (t : F) : ∃ (x y : F), t = 2 * x * y := ⟨1/2, t, by field_simp⟩
227
228 -- 8.43
229 theorem comm_of_β2ψ_ψω :
230     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨β2ψ, ψω⟩ := by
231   intro i j hi hj t u
232   apply triv_comm_iff_commutes.2
233   rcases write_t_as_2tu Fchar t with ⟨x, y, rfl⟩
234   rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
235   grw [expr_β2ψ_as_ψ_βψ_ψ_βψ hi₁ hi₂, expr_βψ_ψω_as_ψω_βψ,
236     expr_ψ_ψω_as_ψω_ψ, expr_βψ_ψω_as_ψω_βψ, expr_ψ_ψω_as_ψω_ψ]
237 declare_B3Small_trivial_span_expr_thm F β2ψ ψω
238
239 -- 8.44
240 theorem Interchange : forall_ijk_tuv β ψ ω,
241     ⁅ ((βψ, i + j, t * u)), ((ω, k, v)) ⁆ = ⁅ ((β, i , t)), ((ψω, j + k, 2 * u * v)) ⁆ := by
242   intro i j k hi hj hk t u v
243   apply eq_comm_of_reorder_left
244   have write_tu : t * u = 2 * t * (u / 2) := by ring_nf; field_simp
245   have : 2 * 2 * u * v / 2 = 2 * u * v := by field_simp; ring
246   have write_βψ : ((βψ, i + j, t * u)) * ((ω, k, v)) = ((ω, k, v)) * ((ψω, j + k, -u * v)) * ((ψ, j, -u/2)) * ((β, i,
    ↪  t))
247       * ((ψ, j, u)) * ((ψω, j + k, 2 * u * v)) * ((β, i, -t)) * ((ψ, j, -u/2)) *
248       ((ψω, j + k, -u * v)) := by
249     -- express βψ as ψ and β elements
250     rw [write_tu, expr_βψ_as_ψ_β_ψ_β_ψ hi hj]
251     -- move ω all the way to the left
252     grw [expr_ψ_ω_as_ω_ψ_ψω, expr_β_ω_as_ω_β, expr_ψ_ω_as_ω_ψ_ψω, expr_β_ω_as_ω_β, expr_ψ_ω_as_ω_ψω_ψ]
253     field_simp; rw [this]
254   -- cyclically combine the left ψω with the right ψω
255   rw [expr_ω_ψω_as_ψω_ω] at write_βψ
256   have write_βψ_left := (mul_right_inj (((ψω, j + k, -u * v))'(add_le_add hj hk))⁻¹).2 write_βψ
257   rw [neg_mul, inv_of_ψω, neg_neg, ←mul_assoc, ←expr_βψ_ψω_as_ψω_βψ, mul_assoc, ←expr_ω_ψω_as_ψω_ω] at
    ↪  write_βψ_left
258   have write_βψ_right := (mul_left_inj (((ψω, j + k, u * v))'(add_le_add hj hk))⁻¹).2 write_βψ_left
259   grw [rfl] at write_βψ_right
260   -- move ψ elements together across β elements
261   grw [write_βψ_right, expr_ψ_β_as_βψ_β2ψ_β_ψ, expr_β_ψ_as_ψ_β_β2ψ_βψ]
262   -- commute ψ elements together across ψω elements and cancel them
263   grw [expr_ψ_ψω_as_ψω_ψ]
264   have : -(u / 2) + u + -(u / 2) = 0 := by ring_nf; field_simp
265   rw [this, id_of_ψ]
266   -- move β2ψ elements together and cancel them
267   grw [expr_β_β2ψ_as_β2ψ_β, ←expr_β2ψ_ψω_as_ψω_β2ψ Fchar, expr_β_β2ψ_as_β2ψ_β]
268   -- collect βψ elements on the right
269   grw [expr_βψ_ψω_as_ψω_βψ, ← expr_β_βψ_as_βψ_β,
270         expr_βψ_ψω_as_ψω_βψ, ← expr_β_βψ_as_βψ_β,
271         expr_βψ_ψω_as_ψω_βψ]
272   -- bring ω as far right as possible
273   grw [commutatorElement_def, ← expr_β_ω_as_ω_β,
274         expr_ω_ψω_as_ψω_ω, ← expr_β_ω_as_ω_β, expr_ω_ψω_as_ψω_ω]
275   ring_nf
276   field_simp
277
278 private lemma βt_ψω2u_to_βψt_ωu : forall_ijk_tu β ψ ω,
279     ⁅ ((β, i, t)), ((ψω, j + k, 2 * u)) ⁆ = ⁅((βψ, i + j, t)), ((ω, k, u)) ⁆ := by
280   intro i j k hi hj hk t u
281   have := Interchange Fchar hi hj hk t 1 u
282   field_simp at this
283   exact this.symm
284
285 private lemma βtu_ψω1_to_βt_ψωu : forall_ijk_tu β ψ ω,
286     ⁅ ((β, i, t * u)), ((ψω, j + k, 1)) ⁆ = ⁅ ((β, i, t)), ((ψω, j + k, u)) ⁆ := by
287   intro i j k hi hj hk t u
288   have aux₁ := Interchange Fchar hi hj hk (t * u) 1 (1 / 2)
289   have aux₂ := Interchange Fchar hi hj hk t u (1 / 2)
290   field_simp at aux₁
291   field_simp at aux₂
292   rwa [aux₁] at aux₂
293
294 omit Fchar in
295 private lemma rewrite_2tu (t u : F) : 2 * t * u = t * (2 * u) := by ring
296 private lemma rewrite_tu (t u : F) : t * u = 2 * t * (u / 2) := by ring_nf; field_simp
297
298 -- height 0
299 private lemma expr_βψω_as_comm_of_β_ψω_00 :
300   ∀ t u : F, ((βψω, 0, t * u)) = ⁅((β, 0, t)), ((ψω, 0, u))⁆ := by
301   intro t u
302   have := @def_of_βψω _ _ 0 (by trivial) (t * u)
```

```
  unfold split_3_into_1_2 at this
  rw [←this, @βtu_ψω1_to_βt_ψωu _ _ Fchar 0 0 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_βψω_as_comm_of_βψ_ω_00 :
  ∀ t u : F, ((βψω, 0, 2 * t * u)) = ⁅((βψ, 0, t)), ((ω, 0, u))⁆ := by
  intro t u
  rw [rewrite_2tu, expr_βψω_as_comm_of_β_ψω_00 Fchar,
    @βt_ψω2u_to_βψt_ωu _ _ Fchar 0 0 0 (by trivial) (by trivial) (by trivial)]

-- height 1
private lemma expr_βψω_as_comm_of_β_ψω_01 :
    ∀ (t u : F), ((βψω, 1, t * u)) = ⁅((β, 0, t)), ((ψω, 1, u))⁆ := by
  intro t u
  have := @def_of_βψω _ _ 1 (by trivial) (t * u)
  unfold split_3_into_1_2 at this
  rw [←this, @βtu_ψω1_to_βt_ψωu _ _ Fchar 0 1 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_βψω_as_comm_of_βψ_ω_01 :
    ∀ (t u : F), ((βψω, 1, 2 * t * u)) = ⁅((βψ, 0, t)), ((ω, 1, u))⁆ := by
  intro t u
  rw [rewrite_2tu, expr_βψω_as_comm_of_β_ψω_01 Fchar,
    @βt_ψω2u_to_βψt_ωu _ _ Fchar 0 0 1 (by trivial) (by trivial) (by trivial)]

private lemma expr_βψω_as_comm_of_βψ_ω_10 :
    ∀ (t u : F), ((βψω, 1, 2 * t * u)) = ⁅((βψ, 1, t)), ((ω, 0, u))⁆ := by
  intro t u
  rw [rewrite_2tu, expr_βψω_as_comm_of_β_ψω_01 Fchar,
    @βt_ψω2u_to_βψt_ωu _ _ Fchar 0 1 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_βψω_as_comm_of_β_ψω_10 :
    ∀ (t u : F), ((βψω, 1, t * u)) = ⁅((β, 1, t)), ((ψω, 0, u))⁆ := by
  intro t u
  rw [rewrite_tu Fchar, expr_βψω_as_comm_of_βψ_ω_10 Fchar,
    ← @βt_ψω2u_to_βψt_ωu _ _ Fchar 1 0 0 (by trivial) (by trivial) (by trivial)]
  field_simp

-- height 2 (reflection of height 1)

declare_B3Small_reflected_thm F b3small_valid βψω β ψω const 1 heights 2 1 1 to 1 0 1
declare_B3Small_reflected_thm F b3small_valid βψω β ψω const 1 heights 2 0 2 to 1 1 0
declare_B3Small_reflected_thm F b3small_valid βψω βψ ω const 2 heights 2 2 0 to 1 0 1
declare_B3Small_reflected_thm F b3small_valid βψω βψ ω const 2 heights 2 1 1 to 1 1 0
declare_B3Small_reflected_thm F b3small_valid βψω β ψω const 1 heights 3 1 2 to 0 0 0
declare_B3Small_reflected_thm F b3small_valid βψω βψ ω const 2 heights 3 2 1 to 0 0 0

-- 8.45a
theorem expand_βψω_as_commutator_of_βψ_ω :
  forall_ij_tu 2 1, ((βψω, i + j, 2 * t * u)) = ⁅((βψ, i, t)), ((ω, j, u))⁆ := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_βψω_as_comm_of_βψ_ω_00 Fchar]
  | 0, 1 => rw [expr_βψω_as_comm_of_βψ_ω_01 Fchar]
  | 1, 0 => rw [expr_βψω_as_comm_of_βψ_ω_10 Fchar]
  | 1, 1 => rw [expr_βψω_as_comm_of_βψ_ω_11 Fchar]
  | 2, 0 => rw [expr_βψω_as_comm_of_βψ_ω_20 Fchar]
  | 2, 1 => rw [expr_βψω_as_comm_of_βψ_ω_21 Fchar]

-- 8.45b
theorem expand_βψω_as_commutator_of_β_ψω :
  forall_ij_tu 1 2, ((βψω, i + j, t * u)) = ⁅((β, i, t)), ((ψω, j, u))⁆ := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_βψω_as_comm_of_β_ψω_00 Fchar]
  | 0, 1 => rw [expr_βψω_as_comm_of_β_ψω_01 Fchar]
  | 0, 2 => rw [expr_βψω_as_comm_of_β_ψω_02 Fchar]
  | 1, 0 => rw [expr_βψω_as_comm_of_β_ψω_10 Fchar]
  | 1, 1 => rw [expr_βψω_as_comm_of_β_ψω_11 Fchar]
  | 1, 2 => rw [expr_βψω_as_comm_of_β_ψω_12 Fchar]

-- 8.46
theorem expr_βψω_as_βψ_ω_βψ_ω :
  forall_ij_tu 2 1, ((βψω, i + j, 2 * t * u)) = ((βψ, i, t)) * ((ω, j, u)) * ((βψ, i, -t)) * ((ω, j, -u)) := by
  intro i j hi hj t u
  rw [expand_βψω_as_commutator_of_βψ_ω Fchar hi hj, ←inv_of_βψ, ←inv_of_ω, commutatorElement_def]

-- 8.47
theorem expr_βψω_as_β_ψω_β_ψω :
  forall_ij_tu 1 2, ((βψω, i + j, t * u)) = ((β, i, t)) * ((ψω, j, u)) * ((β, i, -t)) * ((ψω, j, -u)) := by
  intro i j hi hj t u
  rw [expand_βψω_as_commutator_of_β_ψω Fchar hi hj, ←inv_of_β, ←inv_of_ψω, commutatorElement_def]
```

```
383
384 /-! ### Remaining commutation relations -/
385
386 -- 8.48
387 theorem comm_of_βψω_ω :
388     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψω, ω⟩ := by
389   intro i j hi hj t u
390   apply triv_comm_iff_commutes.2
391   rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
392   rw [←one_mul t, expr_βψω_as_β_ψω_β_ψω Fchar hi₁ hi₂]
393   grw [←expr_ω_ψω_as_ψω_ω, expr_β_ω_as_ω_β, ←expr_ω_ψω_as_ψω_ω, expr_β_ω_as_ω_β]
394 declare_B3Small_trivial_span_expr_thm F βψω ω
395
396 -- 8.49
397 theorem comm_of_βψω_β :
398     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψω, β⟩ := by
399   intro i j hi hj t u
400   apply triv_comm_iff_commutes.2
401   rcases decompose 2 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
402   rcases write_t_as_2tu Fchar t with ⟨x, y, rfl⟩
403   rw [expr_βψω_as_βψ_ω_βψ_ω Fchar hi₁ hi₂]
404   grw [←expr_β_ω_as_ω_β, expr_β_βψ_as_βψ_β, ←expr_β_βψ_as_βψ_β, expr_β_ω_as_ω_β]
405 declare_B3Small_trivial_span_expr_thm F βψω β
406
407 -- 8.50
408 theorem comm_of_βψω_ψ :
409     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψω, ψ⟩ := by
410   intro i j hi hj t u
411   apply triv_comm_iff_commutes.2
412   rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
413   rw [←mul_one t, expr_βψω_as_β_ψω_β_ψω Fchar hi₁ hi₂]
414   grw [←expr_ψ_ψω_as_ψω_ψ, expr_β_ψ_as_ψ_β_β2ψ_βψ hi₁ hj,
415     ← expr_ψ_ψω_as_ψω_ψ, expr_β_ψ_as_ψ_β_β2ψ_βψ hi₁ hj,
416     ← expr_βψ_β2ψ_as_β2ψ_βψ, expr_β2ψ_ψω_as_ψω_β2ψ Fchar,
417     expr_β_β2ψ_as_β2ψ_β,
418     expr_βψ_ψω_as_ψω_βψ, expr_β_βψ_as_βψ_β]
419 declare_B3Small_trivial_span_expr_thm F βψω ψ
420
421 -- 8.51
422 theorem comm_of_βψω_ψω :
423     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψω, ψω⟩ := by
424   intro i j hi hj t u
425   apply triv_comm_iff_commutes.2
426   rcases decompose 1 1 j hj with ⟨j₁, j₂, rfl, hj₁, hj₂⟩
427   rcases write_t_as_2tu Fchar u with ⟨x, y, rfl⟩
428   grw [expr_ψω_as_ψ_ω_ψ_ω hj₁ hj₂, expr_βψω_ψ_as_ψ_βψω Fchar hi hj₁, expr_βψω_ω_as_ω_βψω Fchar hi hj₂,
429   expr_βψω_ψ_as_ψ_βψω Fchar hi hj₁, expr_βψω_ω_as_ω_βψω Fchar]
430 declare_B3Small_trivial_span_expr_thm F βψω ψω
431
432 -- 8.52
433 theorem comm_of_βψω_βψ :
434     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψω, βψ⟩ := by
435   intro i j hi hj t u
436   apply triv_comm_iff_commutes.2
437   rcases decompose 1 1 j hj with ⟨j₁, j₂, rfl, hj₁, hj₂⟩
438   rcases write_t_as_2tu Fchar u with ⟨x, y, rfl⟩
439   grw [expr_βψ_as_ψ_β_ψ_β_ψ hj₁ hj₂, expr_βψω_ψ_as_ψ_βψω Fchar,
440     expr_βψω_β_as_β_βψω Fchar, expr_βψω_ψ_as_ψ_βψω Fchar,
441     expr_βψω_β_as_β_βψω Fchar, expr_βψω_ψ_as_ψ_βψω Fchar]
442 declare_B3Small_trivial_span_expr_thm F βψω βψ
443
444 -- 8.53
445 theorem comm_of_βψω_β2ψ :
446     trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨βψω, β2ψ⟩ := by
447   intro i j hi hj t u
448   apply triv_comm_iff_commutes.2
449   rcases decompose 1 2 j hj with ⟨j₁, j₂, rfl, hj₁, hj₂⟩
450   rcases write_t_as_2tu Fchar u with ⟨x, y, rfl⟩
451   grw [expr_β2ψ_as_ψ_βψ_ψ_βψ hj₁ hj₂, expr_βψω_ψ_as_ψ_βψω Fchar, expr_βψω_βψ_as_βψ_βψω Fchar,
     ↪  expr_βψω_ψ_as_ψ_βψω Fchar,
452   expr_βψω_βψ_as_βψ_βψω Fchar]
453 declare_B3Small_trivial_span_expr_thm F βψω β2ψ
454
455 -- 8.54
456 theorem comm_of_βψω :
457     mixedDegreePropOfRoot (weakB3SmallGraded F).project βψω := by
458   intro i j hi hj t u
459   apply triv_comm_iff_commutes.2
460   rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
461   rcases decompose 1 2 j hj with ⟨j₁, j₂, rfl, hj₁, hj₂⟩
```

```
  rw [←mul_one t]
  grw [expr_βψω_as_β_ψω_β_ψω Fchar hi₁ hi₂, expr_βψω_β_as_β_βψω Fchar, ←expr_βψω_ψω_as_ψω_βψω Fchar,
  expr_βψω_ψω_as_ψω_βψω Fchar, expr_βψω_β_as_β_βψω Fchar]
declare_B3Small_mixed_expr_thm F βψω

-- 8.55
@[simp, chev_simps]
theorem lin_of_βψω : lin_of_root((weakB3SmallGraded F).project, βψω) := by
  intro i hi t u
  rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [←mul_one t, expr_βψω_as_β_ψω_β_ψω Fchar hi₁ hi₂]
  grw [←expr_βψω_ψω_as_ψω_βψω Fchar, ←expr_βψω_β_as_β_βψω Fchar, ←expr_βψω_ψω_as_ψω_βψω Fchar]
  rw [←mul_one u]
  grw [expr_βψω_as_β_ψω_β_ψω Fchar hi₁ hi₂]
  rw [←mul_one (t + u)]
  grw [expr_βψω_as_β_ψω_β_ψω Fchar hi₁ hi₂]
  ring_nf

theorem id_of_βψω : ∀ {i : ℕ} (hi : i ≤ βψω.height), ((βψω, i, 0)) = 1 :=
  fun i hi ↦ mul_right_eq_self.mp (by rw [lin_of_βψω Fchar hi 0 0, zero_add])

-- 8.56
@[simp, chev_simps]
theorem inv_of_βψω : inv_of_root((weakB3SmallGraded F).project, βψω) := by
  intro i hi t
  apply inv_eq_iff_mul_eq_one.2
  grw [lin_of_βψω Fchar, id_of_βψω Fchar hi]

-- 8.57
theorem expr_βψ_ω_as_ω_βψ_βψω : forall_ij_tu βψ ω,
    ((βψ, i, t)) * ((ω, j, u)) = ((ω, j, u)) * ((βψ, i, t)) * ((βψω, i + j, 2 * t * u)) := by
  intro i j hi hj t u
  have : 2 * t * u = 2 * (-t) * (-u) := by ring
  rw [this, expand_βψω_as_commutator_of_βψ_ω Fchar hi hj, ←inv_of_βψ, ←inv_of_ω]
  group

theorem expr_βψ_ω_as_ω_βψω_βψ : forall_ij_tu βψ ω,
    ((βψ, i, t)) * ((ω, j, u)) = ((ω, j, u)) * ((βψω, i + j, 2 * t * u)) * ((βψ, i, t)) := by
  intro i j hi hj t u
  grw [expr_βψ_ω_as_ω_βψ_βψω Fchar hi hj, expr_βψω_βψ_as_βψ_βψω Fchar]

-- 8.58
theorem comm_of_β2ψ_ω :
    trivialSpanPropOfRootPair (weakB3SmallGraded F).project ⟨β2ψ, ω⟩ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rcases write_t_as_2tu Fchar t with ⟨x, t, rfl⟩
  grw [expr_β2ψ_as_ψ_βψ_ψ_βψ hi₁ hi₂, expr_βψ_ω_as_ω_βψ_βψω Fchar hi₂ hj,
        expr_ψ_ω_as_ω_ψω_ψ hi₁ hj, ← expr_ω_ψω_as_ψω_ω,
        expr_βψ_ω_as_ω_βψ_βψω Fchar hi₂ hj, expr_ψ_ω_as_ω_ψ_ψω hi₁ hj,
        expr_βψω_ψω_as_ψω_βψω Fchar, expr_βψω_ψ_as_ψ_βψω Fchar,
        expr_βψω_βψ_as_βψ_βψω Fchar, lin_of_βψω Fchar,
        id_of_βψω Fchar (add_le_add hi₂ hj), expr_βψ_ψω_as_ψω_βψ]
declare_B3Small_trivial_span_expr_thm F β2ψ ω

theorem full_relations_implied_by_weak_relations :
  ∀ r ∈ (fullB3SmallGraded F).allRelations, (weakB3SmallGraded F).project r = 1 := by
  simp only [fullB3SmallGraded, weakB3SmallGraded]
  apply GradedPartialChevalleyGroup.graded_injection
  intro K
  rcases K
  all_goals simp only; intro p h
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [trivialSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      rcases h_new with h_βψ_ψω|h_β_βψω|h_ψ_βψω|h_ω_βψω|h_βψ_βψω|h_β2ψ_βψω|h_ψω_βψω|h_ω_β2ψ|h_ψω_β2ψ
      all_goals (
        subst p r
        simp only
      )
      · exact comm_of_βψ_ψω hi hj t u
      · exact comm_of_βψω_β Fchar hi hj t u
      · exact comm_of_βψω_ψ Fchar hi hj t u
      · exact comm_of_βψω_ω Fchar hi hj t u
      · exact comm_of_βψω_βψ Fchar hi hj t u
```

```
      · exact comm_of_βψω_β2ψ Fchar hi hj t u
      · exact comm_of_βψω_ψω Fchar hi hj t u
      · have := comm_of_β2ψ_ω Fchar hj hi u t
        rwa [triv_comm_symm] at this
      · have := comm_of_β2ψ_ψω Fchar hj hi u t
        rwa [triv_comm_symm] at this
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [singleSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      rcases h_new with h_β_ψω|h_βψ_ω
      all_goals (
        subst p r
        simp only [map_mul, map_inv, mul_inv_eq_one]
      )
      · chev_simp
        exact (expand_βψω_as_commutator_of_β_ψω Fchar hi hj t u).symm
      · chev_simp
        exact (expand_βψω_as_commutator_of_βψ_ω Fchar hi hj t u).symm
  · tauto
  · rcases h with h_old|h_new
    · tauto
    · right
      simp_all only [Set.mem_singleton_iff]
      intro r h_r
      simp only [mixedDegreeRelationsOfRoot] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      subst r
      exact comm_of_βψω Fchar hi hj t u
  · rcases h with h_old|h_new
    · tauto
    · right
      simp_all only [Set.mem_singleton_iff]
      intro r h_r
      simp only [linearityRelationsOfRoot] at h_r
      rcases h_r with ⟨ i, hi, t, u, goal ⟩
      subst r
      simp only [map_mul, map_inv, mul_inv_eq_one]
      exact lin_of_βψω Fchar hi t u
  · tauto
  · simp only [definitionRelations, Set.mem_iUnion, Set.mem_setOf_eq] at h
    rcases h with ⟨ζ, i, hi, t, h⟩
    subst p
    simp only [fullB3SmallGraded, fullMk, inv_mul_cancel, map_one]

end Steinberg.B3Small
```

Author source module Steinberg.B3Large.Defs

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.Defs.PartialChevalleyGroup
import Steinberg.Defs.GradedPartialChevalleyGroup
import Steinberg.Defs.DegreeReflection
import Steinberg.Defs.DecomposeFixed

import Mathlib.Tactic.DeriveFintype
import Mathlib.Tactic.FieldSimp


/-!

  File dox.

-/

namespace Steinberg.B3Large

open Steinberg PartialChevalley GradedPartialChevalley GradedChevalleyGenerator PartialChevalleySystem

/-! # The B3-large positive root system -/
```

```
26
27 inductive B3LargePosRoot
28   | α | β | ψ | αβ | βψ | β2ψ | αβψ | αβ2ψ | α2β2ψ
29 deriving Fintype, DecidableEq, Inhabited
30
31 namespace B3LargePosRoot
32
33 @[reducible, height_simps]
34 def height : B3LargePosRoot → Nat
35   | α | β | ψ => 1
36   | αβ | βψ => 2
37   | β2ψ | αβψ => 3
38   | αβ2ψ => 4
39   | α2β2ψ => 5
40
41 def toString : B3LargePosRoot → String
42   | α => "α"
43   | β => "β"
44   | ψ => "ψ"
45   | αβ => "α+β"
46   | βψ => "β+ψ"
47   | β2ψ => "β+2ψ"
48   | αβψ => "α+β+ψ"
49   | αβ2ψ => "α+β+2ψ"
50   | α2β2ψ => "α+2β+2ψ"
51
52 instance : PositiveRootSystem B3LargePosRoot where
53   height := height
54   toString := toString
55
56 instance instCoeNat : Coe B3LargePosRoot Nat where
57   coe r := r.height
58
59 end B3LargePosRoot
60
61 open B3LargePosRoot
62
63 variable {F : Type TR} [Field F]
64
65 /-! # Lifting -/
66
67 def hom_lift (i j k : ℕ) (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ ψ.height) (t u v : F) (g :
   ↪  ChevalleyGenerator B3LargePosRoot F)
68   : GradedChevalleyGenerator B3LargePosRoot F :=
69   match g.ζ with
70   | α => GradedChevalleyGenerator.mk α i (by ht) (g.t * t)
71   | β => GradedChevalleyGenerator.mk β j (by ht) (g.t * u)
72   | ψ => GradedChevalleyGenerator.mk ψ k (by ht) (g.t * v)
73   | αβ => GradedChevalleyGenerator.mk αβ (i+j) (by ht) (g.t * t * u)
74   | βψ => GradedChevalleyGenerator.mk βψ (j+k) (by ht) (g.t * u * v)
75   | β2ψ => GradedChevalleyGenerator.mk β2ψ (j+2*k) (by ht) (g.t * u * v^2)
76   | αβψ => GradedChevalleyGenerator.mk αβψ (i+j+k) (by ht) (g.t * t * u * v)
77   | αβ2ψ => GradedChevalleyGenerator.mk αβ2ψ (i+j+2*k) (by ht) (g.t * t * u * v^2)
78   | α2β2ψ => GradedChevalleyGenerator.mk α2β2ψ (i+2*j+2*k) (by ht) (g.t * t * u^2 * v^2)
79
80 def hom_lift_set (r : FreeGroup (ChevalleyGenerator B3LargePosRoot F)) :=
81   { FreeGroup.map (hom_lift i j k hi hj hk t u v) r | (i : ℕ) (j : ℕ) (k : ℕ)
82       (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ ψ.height)
83       (t : F) (u : F) (v : F) }
84
85 theorem eq_of_hom_lift_eq
86   (i j k : ℕ) (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ ψ.height)
87   (i' j' k' : ℕ) (hi' : i' ≤ α.height) (hj' : j' ≤ β.height) (hk' : k' ≤ ψ.height) (t u v : F)
88   (hii' : i = i') (hjj' : j = j') (hkk' : k = k') : hom_lift i j k hi hj hk t u v = hom_lift i' j' k' hi'
   ↪  hj' hk' t u v := by
89   ext
90   simp only [hii', hjj', hkk']
91
92 /-! # Definition of the 'weak' B3-large graded group -/
93
94 /-! ## Defining the 'weak' positive root system -/
95
96 -- relations 8.69, 8.70, 8.71, 8.72, 8.73, 8.74
97 abbrev weakPresentRoots : Set B3LargePosRoot :=
98   {α, β, ψ, αβ, βψ, β2ψ}
99
100 -- relations 8.60, 8.61, 8.62, 8.64, 8.65, 8.67, 8.68
101 abbrev weakTrivialSpanPairs : Set (B3LargePosRoot × B3LargePosRoot) :=
102   {(α, αβ), (β, αβ), (α, ψ), (β, βψ), (β, β2ψ), (ψ, β2ψ), (βψ, β2ψ)}
103
```

```
-- relations 8.59, 8.66
abbrev weakSingleSpanRootPairs : Set (SingleSpanRootPair B3LargePosRoot)
  := {⟨α, β, αβ, 1, (by ht)⟩, ⟨ψ, βψ, β2ψ, 2, (by ht)⟩}

-- relation 8.63
abbrev weakDoubleSpanRootPairs : Set (DoubleSpanRootPair B3LargePosRoot) :=
  {⟨β, ψ, βψ, β2ψ, 1, 1, rfl, rfl⟩}

abbrev weakB3LargeSystem := PartialChevalleySystem.mk
  weakPresentRoots
  weakTrivialSpanPairs
  weakSingleSpanRootPairs
  weakDoubleSpanRootPairs
  (by simp only [Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp, reduceCtorEq, or_self,
    or_false, or_true, and_self, forall_eq])
  (by simp only [Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp, reduceCtorEq, or_self,
    or_false, or_true, and_self, forall_eq])
  (by simp only [Set.mem_insert_iff, Set.mem_singleton_iff, forall_eq_or_imp, reduceCtorEq, or_self,
    or_false, or_true, and_self, forall_eq])

-- Relation 8.81
def rels_of_nonhom_lift_of_comm_of_αβ_βψ :=
  { ⁅ {αβ, 2, t₁ * u₁} * {αβ, 1, t₁ * u₀ + t₀ * u₁} * {αβ, 0, t₀ * u₀},
      {βψ, 2, u₁ * v₁} * {βψ, 1, u₁ * v₀ + u₀ * v₁} * {βψ, 0, u₀ * v₀} ⁆
    | (t₁ : F) (t₀ : F) (u₁ : F) (u₀ : F) (v₁ : F) (v₀ : F) }

-- Relation 8.82
def rels_of_nonhom_lift_of_comm_of_α_α2β2ψ :=
  { ⁅ {α, 1, t₁} * {α, 0, t₀},
      ⁅ {αβ, 2, t₁ * u₁} * {αβ, 1, t₁ * u₀ + t₀ * u₁} * {αβ, 0, t₀ * u₀},
        {β2ψ, 3, u₁ * v₁^2} * {β2ψ, 2, u₀ * v₁^2 + 2 * u₁ * v₀ * v₁}
          * {β2ψ, 1, u₁ * v₀^2 + 2 * u₀ * v₀ * v₁} * {β2ψ, 0, u₀ * v₀^2} ⁆ ⁆
    | (t₁ : F) (t₀ : F) (u₁ : F) (u₀ : F) (v₁ : F) (v₀ : F) }

section hom_rels
open ChevalleyGenerator

-- Relation 8.83
def base_rel_of_hom_lift_of_interchange_of_αβψ :=
  {ψ, ((-1/2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)}
    * ({βψ, ((-1/2):F)} * {α, (1:F)} * ({βψ, (1:F)}) * {α, -(1:F)} * {βψ, -(1/2:F)})⁻¹

-- Relation 8.84
def base_rel_of_hom_lift_of_doub_of_αβψ :=
  {ψ, ((-1 / 2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)}
    * {ψ, ((-1 / 2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)}
    * ({ψ, -(1:F)} * {αβ, (1:F)} * {ψ, (2:F)} * {αβ, -(1:F)} * {ψ, -(1:F)})⁻¹

-- Relation 8.85
def base_rel_of_hom_lift_of_interchange_of_αβ2ψ :=
  ⁅ {ψ, ((-1 / 2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)}, {ψ, (1:F)} ⁆
    * ⁅ {α, (1:F)}, {β2ψ, -(2:F)} ⁆⁻¹

-- Relation 8.86
def base_rel_of_hom_lift_of_comm_of_βψ_α_β2ψ :=
  ⁅ {βψ, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆

-- Relation 8.87a
def base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_a :=
  ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆
    * ⁅ {α, -(1:F)}, {β2ψ, -(1:F)}⁆⁻¹

-- Relation 8.87b
def base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_b :=
  ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆
    * ⁅ {α, (1:F)}, {β2ψ, -(1:F)} ⁆

-- Relation 8.87c
def base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_c :=
  ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ * ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆
    * ⁅ {α, (1:F)}, {β2ψ, (2:F)} ⁆⁻¹

-- Relation 8.88
def base_rel_of_hom_lift_of_comm_of_β2ψ_αβψ :=
  ⁅ {β2ψ, (1:F)},
    {ψ, ((-1 / 2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)} ⁆

-- Relation 8.89a
def base_rel_of_hom_lift_of_interchange_of_α2β2ψ_a :=
  ⁅ {αβ, (1:F)}, {β2ψ, (2:F)} ⁆
```

```
    * ⁅ {ψ, ((-1 / 2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)},
      {βψ, (1:F)} ⁆⁻¹

-- Relation 8.89b
def base_rel_of_hom_lift_of_interchange_of_α2β2ψ_b :=
  ⁅ {ψ, ((-1 / 2):F)} * {αβ, (1:F)} * {ψ, (1:F)} * {αβ, -(1:F)} * {ψ, ((-1 / 2):F)}, {βψ, (1:F)} ⁆
    * ⁅ ⁅ {α, (1:F)}, {β2ψ, (2:F)} ⁆, {β, (1:F)} ⁆⁻¹

-- Relation 8.90
def base_rel_of_hom_lift_of_comm_of_ψ_αβ_β2ψ :=
  ⁅ {ψ, (1:F)}, ⁅ {αβ, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆

-- Relation 8.91a (s = 1)
def base_rel_of_hom_lift_of_comm_of_αβ_αβ_β2ψ_a :=
  ⁅ {αβ, (1:F)}, ⁅ {αβ, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆

-- Relation 8.91b (s = -1)
def base_rel_of_hom_lift_of_comm_of_αβ_αβ_β2ψ_b :=
  ⁅ {αβ, (1:F)}, ⁅ {αβ, -(1:F)}, {β2ψ, (1:F)} ⁆ ⁆

-- Relation 8.92a
def base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_a :=
  ⁅ {αβ, (1:F)}, {β2ψ, (1:F)} ⁆
    * ⁅ {αβ, -(1:F)}, {β2ψ, -(1:F)} ⁆⁻¹

-- Relation 8.92b
def base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_b :=
  ⁅ {αβ, (1:F)}, {β2ψ, (1:F)} ⁆
    * ⁅ {αβ, -(1:F)}, {β2ψ, (1:F)} ⁆

-- Relation 8.92c
def base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_c :=
  ⁅ {αβ, (1:F)}, {β2ψ, (1:F)} ⁆ * ⁅ {αβ, (1:F)}, {β2ψ, (1:F)} ⁆
    * ⁅ {αβ, (2:F)}, {β2ψ, (1:F)} ⁆⁻¹

-- Relation 8.93a
def base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_a :=
  ⁅ {β, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆
    * ⁅ {β, (-1:F)}, ⁅ {α, -(1:F)}, {β2ψ, (1:F)} ⁆ ⁆⁻¹

-- Relation 8.93b
def base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_b :=
  ⁅ {β, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆
    * ⁅ {β, (-1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆

-- Relation 8.93c
def base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_c :=
  ⁅ {β, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆ * ⁅ {β, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆
    * ⁅ {β, (2:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆⁻¹

-- Relation 8.94
def base_rel_of_hom_lift_of_comm_of_βψ_αβ2ψ :=
  ⁅ {βψ, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆

-- Relation 8.95
def base_rel_of_hom_lift_of_comm_of_β2ψ_αβ2ψ :=
  ⁅ {β2ψ, (1:F)}, ⁅ {α, (1:F)}, {β2ψ, (1:F)} ⁆ ⁆

end hom_rels

def nonhom_lifted_sets (F : Type TF) [Field F] : Set (Set (FreeGroup (GradedChevalleyGenerator
↪  B3LargePosRoot F))) := {
  rels_of_nonhom_lift_of_comm_of_αβ_βψ, rels_of_nonhom_lift_of_comm_of_α_α2β2ψ}

def hom_lift_base_set (F : Type TF) [Field F] : Set (FreeGroup (ChevalleyGenerator B3LargePosRoot F)) := {
  base_rel_of_hom_lift_of_interchange_of_αβψ, base_rel_of_hom_lift_of_doub_of_αβψ,
  base_rel_of_hom_lift_of_interchange_of_αβ2ψ, base_rel_of_hom_lift_of_comm_of_βψ_α_β2ψ,
  base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_a, base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_b,
  ↪  base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_c,
  base_rel_of_hom_lift_of_comm_of_β2ψ_αβψ, base_rel_of_hom_lift_of_interchange_of_α2β2ψ_a,
  ↪  base_rel_of_hom_lift_of_interchange_of_α2β2ψ_b,
  base_rel_of_hom_lift_of_comm_of_ψ_αβ_β2ψ, base_rel_of_hom_lift_of_comm_of_αβ_αβ_β2ψ_a,
  ↪  base_rel_of_hom_lift_of_comm_of_αβ_αβ_β2ψ_b,
  base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_a, base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_b,
  ↪  base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_c,
  base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_a, base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_b,
  ↪  base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_c,
  base_rel_of_hom_lift_of_comm_of_βψ_αβ2ψ, base_rel_of_hom_lift_of_comm_of_β2ψ_αβ2ψ
}
```

```
def hom_lifted_sets (F : Type TF) [Field F] : Set (Set (FreeGroup (GradedChevalleyGenerator B3LargePosRoot F))) :=
  hom_lift_set '' (hom_lift_base_set F)

def lifted_sets (F : Type TF) [Field F] := (nonhom_lifted_sets F) ∪ (hom_lifted_sets F)

/-! ## Definition for missing root (αβψ, αβ2ψ, α2β2ψ) -/

def weak_define (F : Type TR) [Field F] (g : GradedChevalleyGenerator B3LargePosRoot F) : FreeGroup (GradedChevalleyGenerator B3LargePosRoot F) :=
  let ⟨ ζ, i, hi, t ⟩ := g;
  match ζ with
  | αβψ =>
    {βψ, (split_3_into_1_2 i hi).2, -1/2}'(correct_of_split_3_into_1_2 i hi).2.1 *
    {α, (split_3_into_1_2 i hi).1, t}'(correct_of_split_3_into_1_2 i hi).1 *
    {βψ, (split_3_into_1_2 i hi).2, 1}'(correct_of_split_3_into_1_2 i hi).2.1 *
    {α, (split_3_into_1_2 i hi).1, -t}'(correct_of_split_3_into_1_2 i hi).1 *
    {βψ, (split_3_into_1_2 i hi).2, -1/2}'(correct_of_split_3_into_1_2 i hi).2.1
  | αβ2ψ => ⁅ {α, (split_4_into_1_3 i hi).1, t}'(correct_of_split_4_into_1_3 i hi).1,
      {β2ψ, (split_4_into_1_3 i hi).2, 1}'(correct_of_split_4_into_1_3 i hi).2.1 ⁆
  | α2β2ψ => ⁅ {αβ, (split_5_into_2_3 i hi).1, -t}'(correct_of_split_5_into_2_3 i hi).1,
      {β2ψ, (split_5_into_2_3 i hi).2, 1}'(correct_of_split_5_into_2_3 i hi).2.1 ⁆
  | _ => FreeGroup.of g

theorem weak_define_of_present (F : Type TR) [Field F] :
  ∀ {g : GradedChevalleyGenerator B3LargePosRoot F}, g.ζ ∈ weakB3LargeSystem.presentRoots → weak_define F g = FreeGroup.of g := by
  intro g h_g_in_present
  rcases g with ⟨ ζ, i, hi, t ⟩
  cases ζ
  all_goals simp only [weak_define] -- this will close all present roots
  all_goals ( -- this will close the remaining (nonpresent) roots
    simp only [weakPresentRoots] at h_g_in_present
    contradiction
  )

theorem weak_define_is_projection (F : Type TR) [Field F] :
  ∀ {g : GradedChevalleyGenerator B3LargePosRoot F}, (FreeGroup.lift (weak_define F)) (weak_define F g) = weak_define F g := by
  intro g
  rcases g with ⟨ ζ, i, hi, t ⟩
  cases ζ
  all_goals simp only [weak_define, FreeGroup.lift.of, map_commutatorElement, map_inv, map_mul]

def weakB3LargeGraded (F : Type TF) [Field F] := GradedPartialChevalleyGroup.mk
  weakB3LargeSystem
  (lifted_sets F)
  (weak_define F)
  (weak_define_of_present F)
  (weak_define_is_projection F)

/-! # Definition of the 'full' A3 ungraded and graded groups -/

abbrev fullPresentRoots : Set (B3LargePosRoot) :=
  weakPresentRoots ∪ {αβψ, αβ2ψ, α2β2ψ}

abbrev fullTrivialSpanPairs : Set (B3LargePosRoot × B3LargePosRoot) :=
  weakTrivialSpanPairs ∪ {(αβ, βψ),
                          (α, αβψ), (β, αβψ), (αβ, αβψ), (β2ψ, αβψ),
                          (α, αβ2ψ), (ψ, αβ2ψ), (αβ, αβ2ψ), (βψ, αβ2ψ), (β2ψ, αβ2ψ), (αβψ, αβ2ψ),
                          (α, α2β2ψ), (β, α2β2ψ), (ψ, α2β2ψ), (αβ, α2β2ψ), (βψ, α2β2ψ), (β2ψ, α2β2ψ),
                          (αβψ, α2β2ψ), (αβ2ψ, α2β2ψ)}

abbrev fullSingleSpanRootPairs : Set (SingleSpanRootPair B3LargePosRoot) :=
  weakSingleSpanRootPairs ∪ {⟨ α, β2ψ, αβ2ψ, 1, (by ht)⟩, ⟨αβψ, ψ, αβ2ψ, -2, (by ht)⟩,
                             ⟨αβ, β2ψ, α2β2ψ, -1, (by ht)⟩, ⟨αβψ, βψ, α2β2ψ, -2, (by ht)⟩, ⟨αβ2ψ, β, α2β2ψ, -1, (by ht)⟩}

abbrev fullDoubleSpanRootPairs : Set (DoubleSpanRootPair B3LargePosRoot) :=
  weakDoubleSpanRootPairs ∪ {⟨ α, βψ, αβψ, α2β2ψ, 1, 1, (by ht), (by ht)⟩, ⟨ αβ, ψ, αβψ, αβ2ψ, 1, 1, (by ht), (by ht)⟩}

theorem all_root_pairs_have_relation : everyRootPairInRootPairs B3LargePosRoot fullTrivialSpanPairs fullSingleSpanRootPairs fullDoubleSpanRootPairs := by
  intro ζ η h_ne
  unfold toRootPairs fullTrivialSpanPairs weakTrivialSpanPairs fullSingleSpanRootPairs
    weakSingleSpanRootPairs fullDoubleSpanRootPairs weakDoubleSpanRootPairs
  simp only [Set.image_insert_eq, Set.image_singleton, Set.union_insert, Set.union_singleton, Prod.swap,
    Set.mem_insert_iff, Set.mem_singleton_iff]
```

```
  cases ζ
  all_goals (
    simp only [Prod.mk.injEq, reduceCtorEq, false_and, true_and, or_self, or_false, false_or]
    cases η
    all_goals trivial
  )

def fullB3LargeSystem := PartialChevalleySystem.mkFull B3LargePosRoot
  fullPresentRoots
  fullTrivialSpanPairs
  fullSingleSpanRootPairs
  fullDoubleSpanRootPairs
  (by decide)
  all_root_pairs_have_relation

def fullB3Large (F : Type TR) [Field F] := PartialChevalleyGroup.fullMk B3LargePosRoot F fullB3LargeSystem
def fullB3LargeGraded (F : Type TR) [Field F] := GradedPartialChevalleyGroup.fullMk B3LargePosRoot F
  ↪ fullB3LargeSystem

/-! # Notation and macros -/

/- Instantiate the `declare_thms` macros from `PartialChevalley.lean`. -/

macro "declare_B3Large_trivial_span_expr_thm" F:term:arg r₁:term:arg r₂:term:arg : command =>
  `(command| declare_trivial_span_expr_thm weakB3LargeGraded $F $r₁ $r₂)

macro "declare_B3Large_trivial_span_of_root_pair_thms" F:term:arg r₁:term:arg r₂:term:arg : command =>
  `(command| declare_trivial_span_of_root_pair_thms weakB3LargeGraded $F $r₁ $r₂)

macro "declare_B3Large_single_span_expr_thms" F:term:arg r₁:term:arg r₂:term:arg r₃:term:arg n:num :
  ↪ command =>
  `(command| declare_single_span_expr_thms weakB3LargeGraded $F $r₁ $r₂ $r₃ $n)

macro "declare_B3Large_single_span_of_root_pair_thms" F:term:arg r₁:term:arg r₂:term:arg r₃:term:arg n:num
  ↪ : command =>
  `(command| declare_single_span_of_root_pair_thms weakB3LargeGraded $F $r₁ $r₂ $r₃ $n)

macro "declare_B3Large_lin_id_inv_thms" F:term:arg root:term:arg : command =>
  `(command| declare_lin_id_inv_thms weakB3LargeGraded $F $root)

macro "declare_B3Large_mixed_expr_thm" F:term:arg r:term:arg : command =>
  `(command| declare_mixed_degree_expr_thm weakB3LargeGraded $F $r)

macro "declare_B3Large_mixed_degree_thms" F:term:arg r:term:arg : command =>
  `(command| declare_mixed_degree_thms weakB3LargeGraded $F $r)

macro "declare_B3Large_defineThenReflect_thm" F:term:arg r:term:arg : command =>
  `(command| declare_defineThenReflect_thm weakB3LargeGraded $F B3LargePosRoot $r)

-- r₁ is the larger root, as opposed to the above macros
macro "declare_B3Large_reflected_thm"
    F:term:arg v:term:arg r₁:term:arg r₂:term:arg r₃:term:arg
    "const" C:num
    "heights" n₁:num n₂:num n₃:num
    "to" n₄:num n₅:num n₆:num : command =>
  `(command| declare_reflected_thm weakB3LargeGraded $F $v $r₁ $r₂ $r₃ 0 $C $n₁ $n₂ $n₃ $n₄ $n₅ $n₆)

-- r₁ is the larger root, as opposed to the above macros
macro "declare_B3Large_reflected_thm"
    F:term:arg v:term:arg r₁:term:arg r₂:term:arg r₃:term:arg
    "const" "neg" C:num
    "heights" n₁:num n₂:num n₃:num
    "to" n₄:num n₅:num n₆:num : command =>
  `(command| declare_reflected_thm weakB3LargeGraded $F $v $r₁ $r₂ $r₃ 1 $C $n₁ $n₂ $n₃ $n₄ $n₅ $n₆)

macro "declare_B3Large_triv_comm_reflected_thm"
    F:term:arg v:term:arg r₁:term:arg r₂:term:arg
    "heights" n₁:num n₂:num
    "to" n₃:num n₄:num : command =>
  `(command| declare_triv_comm_reflected_thm weakB3LargeGraded $F $v $r₁ $r₂ $n₁ $n₂ $n₃ $n₄)

set_option hygiene false in
/-- Shorthand for building free group elements from a root, degree, and ring element. -/
scoped notation (priority:=high) "((" ζ ", " i ", " t "))" =>
  (weakB3LargeGraded F).project {ζ, i, t}

set_option hygiene false in
/-- Shorthand for building free group elements from a root, degree, and ring element. -/
scoped notation (priority:=high) "((" ζ ", " i ", " t "))'" h:max =>
  (weakB3LargeGraded F).project ({ζ, i, t}'h)
```

```

section forallNotation

set_option hygiene false

scoped notation "forall_i_t" h:max "," e =>
  ∀ ⦃i : ℕ⦄ (hi : i ≤ h) (t : F), e

scoped notation "forall_ij_tu" h₁:max h₂:max "," e =>
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ h₁) (hk : j ≤ h₂) (t u : F), e

scoped notation "forall_ik_tuv" h₁:max h₂:max "," e =>
  ∀ ⦃i k : ℕ⦄ (hi : i ≤ h₁) (hk : k ≤ h₂) (t u v : F), e

scoped notation "forall_ijk_tu" h₁:max h₂:max h₃:max "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (hk : k ≤ h₃) (t u : F), e

scoped notation "forall_ijk_tuv" h₁:max h₂:max h₃:max "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ h₁) (hj : j ≤ h₂) (hk : k ≤ h₃) (t u v : F), e

/-- By default, if no roots are provided, we use one each of `α`, `β`, `ψ`. -/
scoped notation "forall_ijk_tuv" "," e =>
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ ψ.height) (t u v : F), e

end forallNotation

macro "nonhom_tac " rel:ident " [" intros:ident,* "]" : tactic => `(tactic|
  ( intros $intros*;
    apply eq_of_mul_inv_eq_one;
    apply (weakB3LargeGraded _).liftedProp_of_mem_lifted $rel;
    simp only [weakB3LargeGraded, lifted_sets];
    left;
    simp only [nonhom_lifted_sets, Set.mem_singleton_iff,
      Set.mem_insert_iff, Set.mem_singleton_iff, true_or, or_true];
    exists $intros,*
     ))

macro "hom_tac " rel:ident " [" intros:ident,* "]" : tactic => `(tactic|
  ( intros $intros*;
    apply eq_of_mul_inv_eq_one;
    apply (weakB3LargeGraded _).liftedProp_of_mem_lifted (hom_lift_set $rel);
    simp only [weakB3LargeGraded, lifted_sets];
    right;
    simp only [hom_lifted_sets, hom_lift_base_set, Set.mem_image, Set.mem_insert_iff,
    ↪  Set.mem_singleton_iff, exists_eq_or_imp,
    exists_eq_left, true_or, or_true];
    exists $intros,*;
    unfold $rel;
    simp only [map_mul, map_inv, map_commutatorElement, commutatorElement_def, FreeGroup.map.of,
    ↪  hom_lift];
    simp only [one_mul, inv_one, mul_one, ←mul_assoc];
    try congr;
    try all_goals field_simp
     ))

end Steinberg.B3Large
```

Author source module Steinberg.B3Large.Basic

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Large.Defs

/-!

  File dox.

-/

namespace Steinberg.B3Large

open B3LargePosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {F : Type TF} [Field F]
```

```

/-! # "Given" relations for specific roots -/

declare_B3Large_lin_id_inv_thms F α
declare_B3Large_lin_id_inv_thms F β
declare_B3Large_lin_id_inv_thms F ψ
declare_B3Large_lin_id_inv_thms F αβ
declare_B3Large_lin_id_inv_thms F βψ
declare_B3Large_lin_id_inv_thms F β2ψ

declare_B3Large_trivial_span_of_root_pair_thms F α αβ
declare_B3Large_trivial_span_of_root_pair_thms F β αβ
declare_B3Large_trivial_span_of_root_pair_thms F α ψ
declare_B3Large_trivial_span_of_root_pair_thms F β βψ
declare_B3Large_trivial_span_of_root_pair_thms F β β2ψ
declare_B3Large_trivial_span_of_root_pair_thms F ψ β2ψ
declare_B3Large_trivial_span_of_root_pair_thms F βψ β2ψ

declare_B3Large_single_span_of_root_pair_thms F α β αβ 1
declare_B3Large_single_span_of_root_pair_thms F ψ βψ β2ψ 2

-- Double: {(β, ψ)}
theorem comm_of_β_ψ : doubleSpanPropOfRootPair (weakB3LargeGraded F).project ⟨β, ψ, βψ, β2ψ, 1, 1, by rfl,
  ↪  by rfl⟩ :=
  (weakB3LargeGraded F).doubleSpanProp_of_mem_doubleSpanRootPairs ⟨β, ψ, βψ, β2ψ, 1, 1, by rfl, by rfl⟩
    (by unfold weakB3LargeGraded; simp)

/-! ### Mixed-degree theorem for specific roots -/

declare_B3Large_mixed_degree_thms F α
declare_B3Large_mixed_degree_thms F β
declare_B3Large_mixed_degree_thms F ψ
declare_B3Large_mixed_degree_thms F αβ
declare_B3Large_mixed_degree_thms F βψ
declare_B3Large_mixed_degree_thms F β2ψ

/- ### Reflected theorems for present roots -/

declare_B3Large_defineThenReflect_thm F α
declare_B3Large_defineThenReflect_thm F β
declare_B3Large_defineThenReflect_thm F ψ
declare_B3Large_defineThenReflect_thm F αβ
declare_B3Large_defineThenReflect_thm F βψ
declare_B3Large_defineThenReflect_thm F β2ψ

end Steinberg.B3Large
```

Author source module Steinberg.B3Large.Setup

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```

/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Large.Basic

import Mathlib.Tactic.Group
import Mathlib.Tactic.FinCases
import Mathlib.Tactic.FieldSimp

import Steinberg.Defs.Lattice
import Steinberg.Defs.Commutator
import Steinberg.Defs.DegreeReflection

import Steinberg.Upstream.FreeGroup


namespace Steinberg.B3Large

open Steinberg B3LargePosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)
variable (F_sum_of_squares : ∀ (a : F), ∃ (x y : F), a = x^2 + y^2)

theorem def_of_αβψ : forall_i_t αβψ,
  ((βψ, (split_3_into_1_2 i hi).2, -1/2))'(correct_of_split_3_into_1_2 i hi).2.1 *
  ((α, (split_3_into_1_2 i hi).1, t))'(correct_of_split_3_into_1_2 i hi).1 *
```

```
  ((βψ, (split_3_into_1_2 i hi).2, 1))'(correct_of_split_3_into_1_2 i hi).2.1 *
  ((α, (split_3_into_1_2 i hi).1, -t))'(correct_of_split_3_into_1_2 i hi).1 *
  ((βψ, (split_3_into_1_2 i hi).2, -1/2))'(correct_of_split_3_into_1_2 i hi).2.1
    = ((αβψ, i, t)) := by
  intro i hi t
  symm
  apply (weakB3LargeGraded F).definitionProp_of_define

theorem def_of_αβ2ψ : forall_i_t αβ2ψ,
    [ (((α, (split_4_into_1_3 i hi).1, t))'(correct_of_split_4_into_1_3 i hi).1),
      (((β2ψ, (split_4_into_1_3 i hi).2, 1))'(correct_of_split_4_into_1_3 i hi).2.1)
    ] = ((αβ2ψ, i, t)) := by
  intro i hi t
  symm
  apply (weakB3LargeGraded F).definitionProp_of_define

theorem def_of_α2β2ψ : forall_i_t α2β2ψ,
    [ (((αβ, (split_5_into_2_3 i hi).1, t))'(correct_of_split_5_into_2_3 i hi).1),
      (((β2ψ, (split_5_into_2_3 i hi).2, 1))'(correct_of_split_5_into_2_3 i hi).2.1)
    ] = ((α2β2ψ, i, -t)) := by
  intro i hi t
  symm
  nth_rewrite 2 [←neg_neg t]
  apply (weakB3LargeGraded F).definitionProp_of_define

/-! ### Nonhomogeneous lift -/

-- 8.81
theorem raw_nonhom_lift_of_comm_of_αβ_βψ : ∀ (t₁ t₀ u₁ u₀ v₁ v₀ : F),
    [ ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀))
    , ((βψ, 2, u₁ * v₁)) * ((βψ, 1, u₁ * v₀ + u₀ * v₁)) * ((βψ, 0, u₀ * v₀)) ]
    = 1 := by
  nonhom_tac rels_of_nonhom_lift_of_comm_of_αβ_βψ [t₁, t₀, u₁, u₀, v₁, v₀]

-- 8.82
theorem raw_nonhom_lift_of_comm_of_α_α2β2ψ : ∀ (t₁ t₀ u₁ u₀ v₁ v₀ : F),
    [ ((α, 1, t₁)) * ((α, 0, t₀)),
      [ ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀)),
        ((β2ψ, 3, u₁ * v₁^2)) * ((β2ψ, 2, u₀ * v₁^2 + 2 * u₁ * v₀ * v₁))
          * ((β2ψ, 1, u₁ * v₀^2 + 2 * u₀ * v₀ * v₁)) * ((β2ψ, 0, u₀ * v₀^2)) ] ] = 1 := by
  nonhom_tac rels_of_nonhom_lift_of_comm_of_α_α2β2ψ [t₁, t₀, u₁, u₀, v₁, v₀]

/-! ### Homogeneous lift -/

-- 8.83
theorem raw_hom_lift_of_interchange_of_αβψ : forall_ijk_tuv,
    ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)) =
    ((βψ, j + k, -u * v / 2)) * ((α, i, t)) * ((βψ, j + k, u * v)) * ((α, i, -t)) * ((βψ, j + k, -u * v / 2)) := by
  hom_tac base_rel_of_hom_lift_of_interchange_of_αβψ [i, j, k, hi, hj, hk, t, u, v]


-- 8.84
theorem raw_hom_lift_of_doub_of_αβψ' : forall_ijk_tuv,
    ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)) *
    ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)) =
    ((ψ, k, -v)) * ((αβ, i + j, t * u)) * ((ψ, k, 2 * v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v)) := by
  hom_tac base_rel_of_hom_lift_of_doub_of_αβψ [i, j, k, hi, hj, hk, t, u, v]

-- this is slightly annoying b/c we want to switch the index on i to (0,1,2)
theorem raw_hom_lift_of_doub_of_αβψ : forall_ik_tuv αβ ψ,
    ((ψ, k, -v / 2)) * ((αβ, i, t * u)) * ((ψ, k, v)) * ((αβ, i, -t * u)) * ((ψ, k, -v / 2)) *
    ((ψ, k, -v / 2)) * ((αβ, i, t * u)) * ((ψ, k, v)) * ((αβ, i, -t * u)) * ((ψ, k, -v / 2)) =
    ((ψ, k, -v)) * ((αβ, i, t * u)) * ((ψ, k, 2 * v)) * ((αβ, i, -t * u)) * ((ψ, k, -v)) := by
  intro i k hi hk
  have : ∃ i₁ i₂ : ℕ, i = i₁ + i₂ ∧ i₁ ≤ 1 ∧ i₂ ≤ 1 := decompose 1 1 i (by tauto)
  rcases this with ⟨ i₁, i₂,h,  hi₁, hi₂ ⟩
  simp only [h]
  exact raw_hom_lift_of_doub_of_αβψ' hi₁ hi₂ hk

-- 8.85
theorem raw_hom_lift_of_interchange_of_αβ2ψ : forall_ijk_tuv,
    [ ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)),
      ((ψ, k, v)) ]
      = [ ((α, i, t)), ((β2ψ, j + 2 * k, -2 * u * v^2)) ] := by
  hom_tac base_rel_of_hom_lift_of_interchange_of_αβ2ψ [i, j, k, hi, hj, hk, t, u, v]

-- 8.86
theorem raw_hom_lift_of_comm_of_βψ_α_β2ψ : forall_ijk_tuv,
    [ ((βψ, j + k, u * v)), [ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ] ] = 1 := by
  hom_tac base_rel_of_hom_lift_of_comm_of_βψ_α_β2ψ [i, j, k, hi, hj, hk, t, u, v]
```

```
109
110 -- 8.87a
111 theorem raw_hom_lift_of_inv_doub_of_α_β2ψ_a : forall_ijk_tuv,
112     ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ = ⁅ ((α, i, -t)), ((β2ψ, j + 2 * k, -u * v^2)) ⁆ := by
113   hom_tac base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_a [i, j, k, hi, hj, hk, t, u, v]
114
115 -- 8.87b
116 theorem raw_hom_lift_of_inv_doub_of_α_β2ψ_b : forall_ijk_tuv,
117     ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ * ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, -u * v^2)) ⁆ = 1 := by
118   hom_tac base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_b [i, j, k, hi, hj, hk, t, u, v]
119
120 -- 8.87c
121 theorem raw_hom_lift_of_inv_doub_of_α_β2ψ_c : forall_ijk_tuv,
122     ⁅ ((α, i, t)) , ((β2ψ, j + 2 * k, u * v^2)) ⁆
123     * ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆
124       = ⁅((α, i, t)), ((β2ψ, j + 2 * k, 2 * u * v^2)) ⁆ := by
125   hom_tac base_rel_of_hom_lift_of_inv_doub_of_α_β2ψ_c [i, j, k, hi, hj, hk, t, u, v]
126
127 -- 8.88
128 theorem raw_hom_lift_of_comm_of_β2ψ_αβψ : forall_ijk_tuv,
129     ⁅ ((β2ψ, j + 2 * k, u * v^2)),
130       ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)) ⁆ = 1 := by
131   hom_tac base_rel_of_hom_lift_of_comm_of_β2ψ_αβψ [i, j, k, hi, hj, hk, t, u, v]
132
133 -- 8.89a
134 theorem raw_hom_lift_of_interchange_of_α2β2ψ_a : forall_ijk_tuv,
135     ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, 2 * u * v^2)) ⁆
136       = ⁅ ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)),
137           ((βψ, j + k, u * v)) ⁆ := by
138   hom_tac base_rel_of_hom_lift_of_interchange_of_α2β2ψ_a [i, j, k, hi, hj, hk, t, u, v]
139
140 -- 8.89b
141 theorem raw_hom_lift_of_interchange_of_α2β2ψ_b : forall_ijk_tuv,
142     ⁅ ((ψ, k, -v / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, v)) * ((αβ, i + j, -t * u)) * ((ψ, k, -v / 2)),
143       ((βψ, j + k, u * v)) ⁆
144       = ⁅ ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, 2 * u * v^2)) ⁆, ((β, j, u)) ⁆ := by
145   hom_tac base_rel_of_hom_lift_of_interchange_of_α2β2ψ_b [i, j, k, hi, hj, hk, t, u, v]
146
147 -- 8.90
148 theorem raw_hom_lift_of_comm_of_ψ_αβ_β2ψ : forall_ijk_tuv,
149     ⁅ ((ψ, k, v)), ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ = 1 := by
150   hom_tac base_rel_of_hom_lift_of_comm_of_ψ_αβ_β2ψ [i, j, k, hi, hj, hk, t, u, v]
151
152 -- 8.91a
153 theorem raw_hom_lift_of_comm_of_αβ_αβ_β2ψ_a : forall_ijk_tuv,
154     ⁅ ((αβ, i + j, t * u)), ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ = 1 := by
155   hom_tac base_rel_of_hom_lift_of_comm_of_αβ_αβ_β2ψ_a [i, j, k, hi, hj, hk, t, u, v]
156
157 -- 8.91b
158 theorem raw_hom_lift_of_comm_of_αβ_αβ_β2ψ_b : forall_ijk_tuv,
159     ⁅ ((αβ, i + j, t * u)), ⁅ ((αβ, i + j, -t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ = 1 := by
160   hom_tac base_rel_of_hom_lift_of_comm_of_αβ_αβ_β2ψ_b [i, j, k, hi, hj, hk, t, u, v]
161
162 -- 8.92a
163 theorem raw_hom_lift_of_inv_doub_of_αβ_β2ψ_a : forall_ijk_tuv,
164     ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆
165       = ⁅ ((αβ, i + j, -t * u)), ((β2ψ, j + 2 * k, -u * v^2)) ⁆ := by
166   hom_tac base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_a [i, j, k, hi, hj, hk, t, u, v]
167
168 -- 8.92b
169 theorem raw_hom_lift_of_inv_doub_of_αβ_β2ψ_b : forall_ijk_tuv,
170     ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆
171     * ⁅ ((αβ, i + j, -t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ = 1 := by
172   hom_tac base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_b [i, j, k, hi, hj, hk, t, u, v]
173
174 -- 8.92c
175 theorem raw_hom_lift_of_inv_doub_of_αβ_β2ψ_c : forall_ijk_tuv,
176     ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆
177     * ⁅ ((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆
178       = ⁅ ((αβ, i + j, 2 * t * u)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ := by
179   hom_tac base_rel_of_hom_lift_of_inv_doub_of_αβ_β2ψ_c [i, j, k, hi, hj, hk, t, u, v]
180
181 -- 8.93a
182 theorem raw_hom_lift_of_inv_doub_of_β_αβ2ψ_a : forall_ijk_tuv,
183     ⁅ ((β, j, u)),
184       ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆
185       = ⁅ ((β, j, -u)), ⁅ ((α, i, -t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ := by
186   hom_tac base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_a [i, j, k, hi, hj, hk, t, u, v]
187
188 -- 8.93b
```

```
theorem raw_hom_lift_of_inv_doub_of_β_αβ2ψ_b : forall_ijk_tuv,
    ⁅ ((β, j, u)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆
    * ⁅ ((β, j, -u)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ = 1 := by
  hom_tac base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_b [i, j, k, hi, hj, hk, t, u, v]

-- 8.93c
theorem raw_hom_lift_of_inv_doub_of_β_αβ2ψ_c : forall_ijk_tuv,
    ⁅ ((β, j, u)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆
    * ⁅ ((β, j, u)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆
      = ⁅ ((β, j, 2 * u)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ := by
  hom_tac base_rel_of_hom_lift_of_inv_doub_of_β_αβ2ψ_c [i, j, k, hi, hj, hk, t, u, v]

-- 8.94
theorem raw_hom_lift_of_comm_of_βψ_αβ2ψ : forall_ijk_tuv,
    ⁅ ((βψ, j + k, u * v)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ = 1 := by
  hom_tac base_rel_of_hom_lift_of_comm_of_βψ_αβ2ψ [i, j, k, hi, hj, hk, t, u, v]

-- 8.95
theorem raw_hom_lift_of_comm_of_β2ψ_αβ2ψ : forall_ijk_tuv,
    ⁅ ((β2ψ, j + 2 * k, u * v^2)), ⁅ ((α, i, t)), ((β2ψ, j + 2 * k, u * v^2)) ⁆ ⁆ = 1 := by
  hom_tac base_rel_of_hom_lift_of_comm_of_β2ψ_αβ2ψ [i, j, k, hi, hj, hk, t, u, v]

/-- Reflecting the lift of a generator is the same as taking the opposite lift. -/
theorem reflect_of_hom (g : PartialChevalley.ChevalleyGenerator B3LargePosRoot F)
  (i j k : ℕ) (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ ψ.height)
  (t u v : F) :
    reflect (hom_lift i j k hi hj hk t u v g) = hom_lift (1-i) (1-j) (1-k) (by ht) (by ht) (by ht) t u v g
    ↪  := by
    rcases g with ⟨ ζ, t ⟩
    simp only [reflect, hom_lift]
    split
    all_goals simp only [PositiveRootSystem.height, B3LargePosRoot.height]
    all_goals congr; ht

/-- Extends previous theorem to arbitrary relations. -/
-- TODO: simplify?
theorem reflect_of_hom' (r : FreeGroup (PartialChevalley.ChevalleyGenerator B3LargePosRoot F))
  (i j k : ℕ) (hi : i ≤ α.height) (hj : j ≤ β.height) (hk : k ≤ ψ.height)
  (t u v : F) :
    (FreeGroup.map reflect) (FreeGroup.map (hom_lift i j k hi hj hk t u v) r)
      = FreeGroup.map (hom_lift (1-i) (1-j) (1-k) (by ht) (by ht) (by ht) t u v) r := by
    suffices (FreeGroup.map reflect).comp (FreeGroup.map (hom_lift i j k hi hj hk t u v))
      = FreeGroup.map (hom_lift (1-i) (1-j) (1-k) (by ht) (by ht) (by ht) t u v) by
      have : (FreeGroup.map reflect).comp (FreeGroup.map (hom_lift i j k hi hj hk t u v)) r
        = FreeGroup.map (hom_lift (1-i) (1-j) (1-k) (by ht) (by ht) (by ht) t u v) r := by
          congr
      rw [←this]
      simp only [MonoidHom.coe_comp, Function.comp_apply]
    ext g
    simp only [MonoidHom.coe_comp, Function.comp_apply, FreeGroup.map.of]
    congr
    exact reflect_of_hom g i j k hi hj hk t u v

theorem map_reflect_of_hom (r : FreeGroup (PartialChevalley.ChevalleyGenerator B3LargePosRoot F)) :
  (FreeGroup.map reflect) '' (hom_lift_set r) = hom_lift_set r := by
  simp only [hom_lift_set]
  ext r'
  simp only [Set.mem_image, Set.mem_setOf]
  constructor
  · intro h
    rcases h with ⟨ x, h, _ ⟩
    subst r'
    rcases h with ⟨ i, j, k, hi, hj, hk, t, u, v, h ⟩
    subst x
    use (1-i), (1-j), (1-k), (by ht), (by ht), (by ht), t, u, v
    exact (reflect_of_hom' r i j k hi hj hk t u v).symm
  · intro h
    rcases h with ⟨ i, j, k, hi, hj, hk, t, u, v, h ⟩
    use (FreeGroup.map (hom_lift (1-i) (1-j) (1-k) (by ht) (by ht) (by ht) t u v)) r
    constructor
    · use (1-i), (1-j), (1-k), (by ht), (by ht), (by ht), t, u, v
    · subst r'
      rw [reflect_of_hom' r (1-i) (1-j) (1-k) (by ht) (by ht) (by ht) t u v]
      congr
      apply eq_of_hom_lift_eq
      all_goals ht

private theorem help1 (r : FreeGroup (GradedChevalleyGenerator B3LargePosRoot F))
  (h : ∀ r' ∈ hom_lift_set b, (FreeGroup.lift (defineThenReflect (weakB3LargeGraded F))) r' =
  ↪  (FreeGroup.map reflect) r') :
```

```
  (∃ x ∈ hom_lift_set b, (FreeGroup.lift (defineThenReflect (weakB3LargeGraded F))) x = r) ↔ ∃ x ∈
    ↪  hom_lift_set b, (FreeGroup.map reflect) x = r := by
  constructor
  · intro h
    rcases h with ⟨ r', h_r', h ⟩
    subst r
    use r'
    constructor
    · exact h_r'
    · exact (h r' h_r').symm
  · intro h
    rcases h with ⟨ r', h_r', h ⟩
    subst r
    use r'
    constructor
    · exact h_r'
    · exact (h r' h_r')

set_option maxHeartbeats 0

theorem b3large_valid :
  reflectValidProp (weakB3LargeGraded F) := by
  simp only [reflectValidProp]
  intro S h_S
  simp only [weakB3LargeGraded, lifted_sets, Set.mem_union] at h_S
  rcases h_S with h_nonhom|h_hom
  · simp only [nonhom_lifted_sets, Set.mem_insert_iff,
    Set.mem_singleton_iff] at h_nonhom
    rcases h_nonhom with h | h
    · subst h
      intro r hr
      simp only [rels_of_nonhom_lift_of_comm_of_αβ_βψ] at hr
      rcases hr with ⟨ t₁, t₀, u₁, u₀, v₁, v₀, rfl ⟩
      simp only [map_commutatorElement, map_mul, FreeGroup.lift.of]
      repeat rw [defineThenReflect_eq_reflect_of_mem_presentRoots _ _ (by tauto)]
      simp only [reflect, PositiveRootSystem.height, height]
      simp_arith
      rw [add_comm, add_comm (u₁ * v₀)]
      grw [expr_αβ_αβ_as_αβ_αβ (i := 1), expr_αβ_αβ_as_αβ_αβ, expr_αβ_αβ_as_αβ_αβ (i := 0)]
      grw [expr_βψ_βψ_as_βψ_βψ (i := 1), expr_βψ_βψ_as_βψ_βψ, expr_βψ_βψ_as_βψ_βψ (i := 0)]
      exact raw_nonhom_lift_of_comm_of_αβ_βψ t₀ t₁ u₀ u₁ v₀ v₁
    · subst h
      intro r hr
      simp only [rels_of_nonhom_lift_of_comm_of_α_α2β2ψ] at hr
      rcases hr with ⟨ t₁, t₀, u₁, u₀, v₁, v₀, rfl ⟩
      simp only [map_commutatorElement, map_mul, FreeGroup.lift.of]
      repeat rw [defineThenReflect_eq_reflect_of_mem_presentRoots _ _ (by tauto)]
      simp only [reflect, PositiveRootSystem.height, height]
      simp_arith
      rw [add_comm]
      have := raw_nonhom_lift_of_comm_of_α_α2β2ψ t₀ t₁ u₀ u₁ v₀ v₁
      norm_num at this
      grw [expr_α_α_as_α_α, expr_αβ_αβ_as_αβ_αβ (i := 1), expr_αβ_αβ_as_αβ_αβ (i := 0),
          expr_αβ_αβ_as_αβ_αβ (i := 0), expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := 2),
          expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := 1), expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := 0), expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i
          ↪  := 1),
          expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := 0), expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := 0)]
      rw [← this]
      grw [pow_two, mul_comm v₁ v₀, mul_comm v₁ v₀]
  · suffices (FreeGroup.lift (defineThenReflect (weakB3LargeGraded F))) '' S = S by
      intro r h_r
      apply eq_one_of_mem_rels
      have : (FreeGroup.lift (defineThenReflect (weakB3LargeGraded F))) r ∈ S := by
        rw [←this]
        simp only [Set.mem_image]
        use r
      simp only [allRelations, lifted_sets, Set.mem_iUnion]
      use GradedSteinbergRelationClass.Lifted
      use S
      constructor
      · simp only [weakB3LargeGraded, liftedRelationsSets, lifted_sets, Set.mem_union]
        right
        exact h_hom
      · exact this
    simp only [hom_lifted_sets, hom_lift_base_set, Set.mem_image] at h_hom
    rcases h_hom with ⟨ b, h, h_hom ⟩
    subst S
    nth_rewrite 2 [←map_reflect_of_hom]
    ext r
    simp only [Set.mem_image]
```

```
    apply help1
    intro r' h_r'
    simp only [hom_lift_set, Set.mem_setOf] at h_r'
    rcases h_r' with ⟨ i, j, k, hi, hj, hk, t, u, v, h_r' ⟩
    subst r'
    rcases h with h|h|h|
```

```
-- 8.112a
@[group_reassoc]
theorem expr_ψ_β_as_β_ψ_βψ_β2ψ : forall_ij_tu β ψ,
    ((ψ, j, u)) * ((β, i, t)) = ((β, i, t)) * ((ψ, j, u)) * ((βψ, i + j, -t * u)) * ((β2ψ, i + 2 * j, t * u^2)) := by
  intro i j hi hj t u
  have hij : i + j ≤ βψ.height := by ht
  have hi2j : i + (2 * j) ≤ β2ψ.height := by ht
  rw [comm_right_rev, inv_of_β, inv_of_ψ]
  grw [comm_of_β_ψ, mul_inv_rev, expr_βψ_β2ψ_as_β2ψ_βψ hij hi2j, pow_two]

-- 8.112b
@[group_reassoc]
theorem expr_ψ_β_as_β2ψ_βψ_β_ψ : forall_ij_tu β ψ,
    ((ψ, j, u)) * ((β, i, t)) = ((β2ψ, i + 2 * j, -t * u^2)) * ((βψ, i + j, -t * u)) * ((β, i, t)) * ((ψ, j, u)) := by
  intro i j hi hj t u
  have hij : i + j ≤ βψ.height := by ht
  have hi2j : i + (2 * j) ≤ β2ψ.height := by ht
  rw [comm_left_rev]
  grw [comm_of_β_ψ, mul_inv_rev, pow_two]

-- 8.112c
omit Fchar
@[group_reassoc]
theorem expr_ψ_β_as_β_β2ψ_βψ_ψ : forall_ij_tu β ψ,
    ((ψ, j, u)) * ((β, i, t)) = ((β, i, t)) * ((β2ψ, i + 2 * j, -t * u^2)) * ((βψ, i + j, -t * u)) * ((ψ, j, u)) := by
  intro i j hi hj t u
  have hij : i + j ≤ βψ.height := by ht
  have hi2j : i + (2 * j) ≤ β2ψ.height := by ht
  rw [comm_mid_rev, inv_of_β]
  grw [comm_of_β_ψ, expr_βψ_β2ψ_as_β2ψ_βψ hij hi2j, pow_two]

-- 8.112d
@[group_reassoc]
theorem expr_ψ_β_as_β_βψ_β2ψ_ψ : forall_ij_tu β ψ,
    ((ψ, j, u)) * ((β, i, t)) = ((β, i, t)) * ((βψ, i + j, -t * u)) * ((β2ψ, i + 2 * j, -t * u^2)) * ((ψ, j, u)) := by
  intro i j hi hj t u
  have hij : i + j ≤ βψ.height := by ht
  have hi2j : i + (2 * j) ≤ β2ψ.height := by ht
  rw [expr_ψ_β_as_β_β2ψ_βψ_ψ hi hj t u,
        greassoc_of% expr_βψ_β2ψ_as_β2ψ_βψ hij hi2j (-t * u) (-t * u^2)]

-- 8.113a
@[group_reassoc]
theorem expr_ψ_βψ_as_βψ_β2ψ_ψ : forall_ij_tu ψ βψ,
    ((ψ, i, t)) * ((βψ, j, u)) = ((βψ, j, u)) * ((β2ψ, i + j, 2 * t * u)) * ((ψ, i, t)) := by
  intro i j hi hj t u
  rw [comm_mid, inv_of_βψ]
  grw [comm_of_ψ_βψ hi hj]

-- 8.113b
@[group_reassoc]
theorem expr_ψ_βψ_as_βψ_ψ_β2ψ :
  forall_ij_tu ψ βψ,
    ((ψ, i, t)) * ((βψ, j, u)) = ((βψ, j, u)) * ((ψ, i, t)) * ((β2ψ, i + j, 2 * t * u)) := by
  intro i j hi hj t u
  have hij : i + j ≤ β2ψ.height := by ht
  grw [expr_ψ_β2ψ_as_β2ψ_ψ hi hij, expr_ψ_βψ_as_βψ_β2ψ_ψ hi hj]

-- 8.114a
@[group_reassoc]
theorem expr_βψ_ψ_as_ψ_β2ψ_βψ :
  forall_ij_tu ψ βψ,
    ((βψ, j, u)) * ((ψ, i, t)) = ((ψ, i, t)) * ((β2ψ, i + j, -2 * t * u)) * ((βψ, j, u)) := by
  intro i j hi hj t u
  have hij : j + i ≤ β2ψ.height := by ht
  rw [deg_add_comm β2ψ i j, ← greassoc_of% expr_βψ_β2ψ_as_β2ψ_βψ hj hij, deg_add_comm β2ψ j i]
  grw [expr_ψ_βψ_as_βψ_ψ_β2ψ hi hj]

-- 8.114b
@[group_reassoc]
theorem expr_βψ_ψ_as_ψ_βψ_β2ψ : forall_ij_tu ψ βψ,
    ((βψ, j, u)) * ((ψ, i, t)) = ((ψ, i, t)) * ((βψ, j, u)) * ((β2ψ, i + j, -2 * t * u)) := by
  intro i j hi hj t u
  have hij : i + j ≤ β2ψ.height := by ht
  rw [expr_βψ_ψ_as_ψ_β2ψ_βψ hi hj, greassoc_of% expr_βψ_β2ψ_as_β2ψ_βψ hj hij]

/- Commutator relation in the case (i,j) is not (0,2) or (2,0) (via the previous theorem). -/
private lemma hom_lift_of_comm_of_αβ_βψ (i j k : ℕ) (hi : i ≤ 1) (hj : j ≤ 1) (hk : k ≤ 1) :
  ∀ (t u : F), ⁅ (( αβ, i + j, t)), ((βψ, j + k, u)) ⁆ = 1 := by
    intro t u
```

```
    let t₁ : F := match i with
      | 1 => t
      | 0 => 0
    let t₀ : F := match i with
      | 1 => 0
      | 0 => t
    let u₁ : F := match j with
      | 1 => 1
      | 0 => 0
    let u₀ : F := match j with
      | 1 => 0
      | 0 => 1
    let v₁ : F := match k with
      | 1 => u
      | 0 => 0
    let v₀ : F := match k with
      | 1 => 0
      | 0 => u
    have hf_i : i ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have hf_j : j ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have hf_k : k ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
    have id₁ : ((αβ, i + j, t)) = ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀)) := by (
      fin_cases hf_i, hf_j, hf_k
      <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
    )
    have id₂ : ((βψ, j + k, u)) = ((βψ, 2, u₁ * v₁)) * ((βψ, 1, u₁ * v₀ + u₀ * v₁)) * ((βψ, 0, u₀ * v₀)) := by (
      fin_cases hf_i, hf_j, hf_k
      <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
    )
    rw [id₁, id₂, raw_nonhom_lift_of_comm_of_αβ_βψ]

private lemma image_of_hom_lift_of_comm_of_αβ_βψ {i j : ℕ} (hi : i ≤ αβ.height) (hj : j ≤ βψ.height)
    : ((i, j) ∈ ij_jk_image) → ∀ (t u : F), ⁅ ((αβ, i, t)), ((βψ, j, u)) ⁆ = 1 := by
  intro h_in_image t u
  have : ∃ ijk' : ℕ × ℕ × ℕ, ijk' ∈ boolean_cube ∧ f_ij_jk ijk' = (i, j) := by
    rw [← Finset.mem_image, correct_of_ij_jk_image]; exact h_in_image
  simp [f_ij_jk] at this
  rcases this with ⟨ i', j', k', ⟨ hi', hj', hk' ⟩, rfl, rfl ⟩
  rw [← hom_lift_of_comm_of_αβ_βψ i' j' k' hi' hj' hk' t u]

private lemma comm_of_αβ_βψ_20 : ∀ (t u : F), ⁅ ((αβ, 2, t)), ((βψ, 0, u)) ⁆ = 1 := by
  intro t u
  apply @trivial_comm_from_embedded_comm_and_pairs _ _ ((βψ, 1, u)) _ ((αβ, 1, t))
  grw [← raw_nonhom_lift_of_comm_of_αβ_βψ t 0 1 1 0 u]
  rw [← hom_lift_of_comm_of_αβ_βψ 1 1 0 (by trivial) (by trivial) (by trivial) t u]
  rw [← hom_lift_of_comm_of_αβ_βψ 0 1 0 (by trivial) (by trivial) (by trivial) t u]
  rw [← hom_lift_of_comm_of_αβ_βψ 1 0 0 (by trivial) (by trivial) (by trivial) t u]

private lemma comm_of_αβ_βψ_02 : ∀ (t u : F), ⁅ ((αβ, 0, t)), ((βψ, 2, u))⁆ = 1 := by
  intro t u
  apply triv_comm_symm.1
  apply @trivial_comm_from_embedded_comm_and_pairs _ _ ((αβ, 1, t)) _ (((βψ, 1, u + 1)) * ((βψ, 0, 1)))
  · mul_assoc_l
    rw [←triv_comm_symm.1 (raw_nonhom_lift_of_comm_of_αβ_βψ 0 t 1 1 u 1)]
    simp only [one_mul, mul_one, mul_zero, add_zero, zero_add]
    rw [add_comm u 1, id_of_αβ, one_mul]
  · rw [←triv_comm_symm.1 (hom_lift_of_comm_of_αβ_βψ 0 1 1 (by trivial) (by trivial) (by trivial) t u)]
  · apply triv_comm_mul_left
    rw [←triv_comm_symm.1 (hom_lift_of_comm_of_αβ_βψ 0 1 0 (by trivial) (by trivial) (by trivial) t (u +
    ↪  1))]
    rw [←triv_comm_symm.1 (hom_lift_of_comm_of_αβ_βψ 1 0 0 (by trivial) (by trivial) (by trivial) t 1)]
  apply triv_comm_mul_left
  rw [←triv_comm_symm.1 (hom_lift_of_comm_of_αβ_βψ 0 0 1 (by trivial) (by trivial) (by trivial) t (u +
  ↪  1))]
  rw [←triv_comm_symm.1 (hom_lift_of_comm_of_αβ_βψ 0 0 0 (by trivial) (by trivial) (by trivial) t 1)]

-- 8.115
theorem comm_of_αβ_βψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨αβ, βψ⟩ := by
  intro i j hi hj t u
  by_cases hij : (i, j) ∈ ij_jk_image
  · apply image_of_hom_lift_of_comm_of_αβ_βψ hi hj hij
  · have : (i = 0 ∧ j = 2) ∨ (i = 2 ∧ j = 0) := by
      simp [ij_jk_image] at hij
      ht
    rcases this with ( ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ )
    · rw [← comm_of_αβ_βψ_02 t u]
    · rw [← comm_of_αβ_βψ_20 t u]
declare_B3Large_trivial_span_expr_thm F αβ βψ

```

```
omit Fchar in
theorem defineThenReflect_eq_reflect_of_αβψ (g : GradedChevalleyGenerator B3LargePosRoot F)
  (h : g.ζ = αβψ) :
  (weakB3LargeGraded F).project (defineThenReflect (weakB3LargeGraded F) g)
    = (weakB3LargeGraded F).project (FreeGroup.of (reflect g)) := by
  rcases g with ⟨ ζ, i, hi, t ⟩
  simp only at h
  subst ζ
  simp only [defineThenReflect, MonoidHom.coe_comp, Function.comp_apply,
    FreeGroup.lift.of, reflect, ←def_of_αβψ]
  rw [weakB3LargeGraded]
  simp only
  rw [weak_define]
  simp only [map_mul, map_inv, FreeGroup.map.of, reflect, PositiveRootSystem.height, height]
  congr
  all_goals rw [refl_of_split_3_into_1_2]

omit Fchar in
theorem defineThenReflect_eq_reflect_of_α2β2ψ (g : GradedChevalleyGenerator B3LargePosRoot F)
  (h : g.ζ = α2β2ψ) :
  (weakB3LargeGraded F).project (defineThenReflect (weakB3LargeGraded F) g)
    = (weakB3LargeGraded F).project (FreeGroup.of (reflect g)) := by
  rcases g with ⟨ ζ, i, hi, t ⟩
  simp only at h
  subst ζ
  nth_rewrite 2 [←neg_neg t]
  simp only [defineThenReflect, MonoidHom.coe_comp, Function.comp_apply,
    FreeGroup.lift.of, reflect, ←def_of_α2β2ψ]
  rw [weakB3LargeGraded]
  simp only
  rw [weak_define]
  simp only [map_commutatorElement, FreeGroup.map.of, reflect, PositiveRootSystem.height, height]
  congr
  all_goals rw [refl_of_split_5_into_2_3]
```

Author source module Steinberg.B3Large.EstabAlphaBetaPsi

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```

import Steinberg.B3Large.Basic
import Steinberg.B3Large.Setup

namespace Steinberg.B3Large

open Steinberg B3LargePosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)

/- ### Establishing α + β + ψ -/

private lemma interchange_αβψ_refl_v : forall_ijk_tu α β ψ,
    ((ψ, k, -1/2)) * ((αβ, i + j, t * u)) * ((ψ, k, 1)) * ((αβ, i + j, -t * u)) * ((ψ, k, -1/2))
      = ((βψ, j + k, -u/2)) * ((α, i, t)) * ((βψ, j + k, u)) * ((α, i, -t)) * ((βψ, j + k, -u/2)) := by
  intro i j k hi hj hk t u
  grw [raw_hom_lift_of_interchange_of_αβψ hi hj hk t u 1]

private lemma interchange_αβψ_refl_u : forall_ijk_tu α β ψ,
    ((ψ, k, -u/2)) * ((αβ, i + j, t)) * ((ψ, k, u)) * ((αβ, i + j, -t)) * ((ψ, k, -u/2))
      = ((βψ, j + k, -u/2)) * ((α, i, t)) * ((βψ, j + k, u)) * ((α, i, -t)) * ((βψ, j + k, -u/2)) := by
  intro i j k hi hj hk t u
  rw [← mul_one t, ← neg_mul]
  grw [raw_hom_lift_of_interchange_of_αβψ hi hj hk t 1 u]

private lemma interchange_αβψ_trans_βψ_α : forall_ijk_tu α β ψ,
    ((βψ, j + k, -1/2)) * ((α, i, t * u)) * ((βψ, j + k, 1)) * ((α, i, -t * u)) * ((βψ, j + k, -1/2))
      = ((βψ, j + k, -u/2)) * ((α, i, t)) * ((βψ, j + k, u)) * ((α, i, -t)) * ((βψ, j + k, -u/2)) := by
  intro i j k hi hj hk t u
  have aux₁ := raw_hom_lift_of_interchange_of_αβψ hi hj hk t u 1
  have aux₂ := raw_hom_lift_of_interchange_of_αβψ hi hj hk (t * u) 1 1
  simp only [one_mul, mul_one] at aux₁
  simp only [one_mul, mul_one, ←neg_mul] at aux₂
  rwa [aux₂] at aux₁

private lemma interchange_αβψ_trans_ψ_αβ : forall_ijk_tu α β ψ,
    ((ψ, k, -1 / 2)) * ((αβ, i + j, t * u)) * ((ψ, k, 1)) * ((αβ, i + j, -t * u)) * ((ψ, k, -1 / 2))
      = ((ψ, k, -u / 2)) * ((αβ, i + j, t)) * ((ψ, k, u)) * ((αβ, i + j, -t)) * ((ψ, k, -u / 2)) := by
  intro i j k hi hj hk t u
  have aux₁ := raw_hom_lift_of_interchange_of_αβψ hi hj hk 1 t u
```

```
  have aux₂ := raw_hom_lift_of_interchange_of_αβψ hi hj hk 1 (t * u) 1
  simp only [one_mul, mul_one, neg_mul] at aux₁
  simp only [one_mul, mul_one, neg_mul] at aux₂
  rwa [←aux₁, ←neg_mul] at aux₂

-- height 0
private lemma expr_αβψ_as_βψ_α_βψ_α_βψ_00 : ∀ (t u : F),
    ((αβψ, 0, t * u)) = ((βψ, 0, -u/2)) * ((α, 0, t)) * ((βψ, 0, u)) * ((α, 0, -t)) * ((βψ, 0, -u/2)) := by
  intro t u
  have := @def_of_αβψ _ _ 0 (by trivial) (t * u)
  unfold split_3_into_1_2 at this
  rw [←this, ←neg_mul, @interchange_αβψ_trans_βψ_α _ _ 0 0 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_00 : ∀ (t u : F),
    ((αβψ, 0, t * u)) = ((ψ, 0, -u/2)) * ((αβ, 0, t)) * ((ψ, 0, u)) * ((αβ, 0, -t)) * ((ψ, 0, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_βψ_α_βψ_α_βψ_00, ←@interchange_αβψ_refl_v _ _ 0 0 0 (by trivial) (by trivial) (by
    ↪  trivial),
  interchange_αβψ_trans_ψ_αβ (by trivial) (by trivial) (by trivial)]

-- height 1
private lemma expr_αβψ_as_βψ_α_βψ_α_βψ_01 : ∀ (t u : F),
    ((αβψ, 1, t * u)) = ((βψ, 1, -u/2)) * ((α, 0, t)) * ((βψ, 1, u)) * ((α, 0, -t)) * ((βψ, 1, -u/2)) := by
  intro t u
  have := @def_of_αβψ _ _ 1 (by trivial) (t * u)
  unfold split_3_into_1_2 at this
  rw [←this, ←neg_mul, @interchange_αβψ_trans_βψ_α _ _ 0 1 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_01 : ∀ (t u : F),
    ((αβψ, 1, t * u)) = ((ψ, 1, -u/2)) * ((αβ, 0, t)) * ((ψ, 1, u)) * ((αβ, 0, -t)) * ((ψ, 1, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_βψ_α_βψ_α_βψ_01, @interchange_αβψ_refl_u _ _ 0 0 1 (by trivial) (by trivial) (by
    ↪  trivial)]

private lemma expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_10 : ∀ (t u : F),
    ((αβψ, 1, t * u)) = ((ψ, 0, -u/2)) * ((αβ, 1, t)) * ((ψ, 0, u)) * ((αβ, 1, -t)) * ((ψ, 0, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_βψ_α_βψ_α_βψ_01, ←@interchange_αβψ_refl_v _ _ 0 1 0 (by trivial) (by trivial) (by
    ↪  trivial),
  interchange_αβψ_trans_ψ_αβ (by trivial) (by trivial) (by trivial)]

private lemma expr_αβψ_as_βψ_α_βψ_α_βψ_10 : ∀ (t u : F),
    ((αβψ, 1, t * u)) = ((βψ, 0, -u/2)) * ((α, 1, t)) * ((βψ, 0, u)) * ((α, 1, -t)) * ((βψ, 0, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_10, @interchange_αβψ_refl_u _ _ 1 0 0 (by trivial) (by trivial) (by
    ↪  trivial)]

-- height 2
private lemma expr_αβψ_as_βψ_α_βψ_α_βψ_11 : ∀ (t u : F),
    ((αβψ, 2, t * u)) = ((βψ, 1, -u/2)) * ((α, 1, t)) * ((βψ, 1, u)) * ((α, 1, -t)) * ((βψ, 1, -u/2)) := by
  intro t u
  have := @def_of_αβψ _ _ 2 (by trivial) (t * u)
  unfold split_3_into_1_2 at this
  rw [←this, ←neg_mul, @interchange_αβψ_trans_βψ_α _ _ 1 1 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_20 : ∀ (t u : F),
    ((αβψ, 2, t * u)) = ((ψ, 0, -u/2)) * ((αβ, 2, t)) * ((ψ, 0, u)) * ((αβ, 2, -t)) * ((ψ, 0, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_βψ_α_βψ_α_βψ_11, @interchange_αβψ_refl_u _ _ 1 1 0 (by trivial) (by trivial) (by
    ↪  trivial)]

private lemma expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_11 : ∀ (t u : F),
    ((αβψ, 2, t * u)) = ((ψ, 1, -u/2)) * ((αβ, 1, t)) * ((ψ, 1, u)) * ((αβ, 1, -t)) * ((ψ, 1, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_βψ_α_βψ_α_βψ_11, @interchange_αβψ_refl_u _ _ 1 0 1 (by trivial) (by trivial) (by
    ↪  trivial)]

private lemma expr_αβψ_as_βψ_α_βψ_α_βψ_02 : ∀ (t u : F),
    ((αβψ, 2, t * u)) = ((βψ, 2, -u/2)) * ((α, 0, t)) * ((βψ, 2, u)) * ((α, 0, -t)) * ((βψ, 2, -u/2)) := by
  intro t u
  rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_11, @interchange_αβψ_refl_u _ _ 0 1 1 (by trivial) (by trivial) (by
    ↪  trivial)]

-- height 3
private lemma expr_αβψ_as_βψ_α_βψ_α_βψ_12 : ∀ (t u : F),
    ((αβψ, 3, t * u)) = ((βψ, 2, -u/2)) * ((α, 1, t)) * ((βψ, 2, u)) * ((α, 1, -t)) * ((βψ, 2, -u/2)) := by
  intro t u
  have := @def_of_αβψ _ _ 3 (by trivial) (t * u)
  unfold split_3_into_1_2 at this
  rw [←this, ←neg_mul, @interchange_αβψ_trans_βψ_α _ _ 1 1 1 (by trivial) (by trivial) (by trivial)]
```

```
114
115 private lemma expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_21 : ∀ (t u : F),
116     ((αβψ, 3, t * u)) = ((ψ, 1, -u/2)) * ((αβ, 2, t)) * ((ψ, 1, u)) * ((αβ, 2, -t)) * ((ψ, 1, -u/2)) := by
117   intro t u
118   rw [expr_αβψ_as_βψ_α_βψ_α_βψ_12, @interchange_αβψ_refl_u _ _ 1 1 1 (by trivial) (by trivial) (by
    ↪  trivial)]
119
120 -- 8.116a
121 theorem expr_αβψ_as_ψ_αβ_ψ_αβ_ψ : forall_ij_tu αβ ψ,
122     ((αβψ, i + j, t * u)) = ((ψ, j, -u/2)) * ((αβ, i, t)) * ((ψ, j, u)) * ((αβ, i, -t)) * ((ψ, j, -u/2)) := by
123   intro i j hi hj t u
124   match i, j with
125   | 0, 0 => rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_00]
126   | 1, 0 => rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_10]
127   | 0, 1 => rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_01]
128   | 2, 0 => rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_20]
129   | 1, 1 => rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_11]
130   | 2, 1 => rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ_21]
131
132 -- 8.116b
133 theorem expr_αβψ_as_βψ_α_βψ_α_βψ : forall_ij_tu α βψ,
134     ((αβψ, i + j, t * u)) = ((βψ, j, -u/2)) * ((α, i, t)) * ((βψ, j, u)) * ((α, i, -t)) * ((βψ, j, -u/2)) := by
135   intro i j hi hj t u
136   match i, j with
137   | 0, 0 => rw [expr_αβψ_as_βψ_α_βψ_α_βψ_00]
138   | 1, 0 => rw [expr_αβψ_as_βψ_α_βψ_α_βψ_10]
139   | 0, 1 => rw [expr_αβψ_as_βψ_α_βψ_α_βψ_01]
140   | 0, 2 => rw [expr_αβψ_as_βψ_α_βψ_α_βψ_02]
141   | 1, 1 => rw [expr_αβψ_as_βψ_α_βψ_α_βψ_11]
142   | 1, 2 => rw [expr_αβψ_as_βψ_α_βψ_α_βψ_12]
143
144 -- 8.117
145 theorem comm_of_α_αβψ :
146     trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨α, αβψ⟩ := by
147   intro i j hi hj t u
148   apply triv_comm_iff_commutes.mpr
149   rcases decompose αβ.height ψ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
150   rw [← one_mul u]
151   grw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hj₁ hj₂, expr_α_ψ_as_ψ_α hi hj₂,
152     expr_α_αβ_as_αβ_α hi hj₁, expr_α_ψ_as_ψ_α hi hj₂,
153     expr_α_αβ_as_αβ_α hi hj₁, expr_α_ψ_as_ψ_α hi hj₂]
154 declare_B3Large_trivial_span_expr_thm F α αβψ
155
156 -- 8.118
157 theorem comm_of_αβ_αβψ :
158     trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨αβ, αβψ⟩ := by
159   intro i j hi hj t u
160   apply triv_comm_iff_commutes.mpr
161   rcases decompose αβ.height ψ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
162   rw [deg_add_comm αβψ j₁ j₂, ← one_mul u]
163   grw [expr_αβψ_as_βψ_α_βψ_α_βψ hj₂ hj₁, expr_αβ_βψ_as_βψ_αβ hi hj₁,
164     ← expr_α_αβ_as_αβ_α hj₂ hi, expr_αβ_βψ_as_βψ_αβ hi hj₁,
165     ← expr_α_αβ_as_αβ_α hj₂ hi, expr_αβ_βψ_as_βψ_αβ hi hj₁]
166 declare_B3Large_trivial_span_expr_thm F αβ αβψ
167
168 -- 8.119
169 theorem comm_of_β_αβψ :
170     trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨β, αβψ⟩ := by
171   intro i j hi hj t u
172   apply triv_comm_iff_commutes.mpr
173   rcases decompose αβ.height ψ.height j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
174   have hj₂i : j₂ + i ≤ βψ.height := by ht
175   rw [deg_add_comm αβψ j₁ j₂, ← one_mul u]
176   grw [expr_αβψ_as_βψ_α_βψ_α_βψ hj₂ hj₁, expr_β_βψ_as_βψ_β hi hj₁,
177     expr_β_α_as_αβ_α_β hj₂ hi, expr_β_βψ_as_βψ_β hi hj₁,
178     expr_β_α_as_αβ_α_β hj₂ hi, expr_β_βψ_as_βψ_β hi hj₁,
179     ← expr_α_αβ_as_αβ_α hj₂ hj₂i, expr_αβ_βψ_as_βψ_αβ hj₂i hj₁]
180 declare_B3Large_trivial_span_expr_thm F β αβψ
181
182 -- 8.120a
183 @[simp, chev_simps]
184 private lemma inv_doub_of_αβψ_a : forall_i_t αβψ, ((αβψ, i, t)) * ((αβψ, i, -t)) = 1 := by
185   intro i hi t
186   rcases decompose αβ.height ψ.height i hi with ⟨ i₁, i₂, ⟨ rfl, hi₁, hi₂ ⟩ ⟩
187   rw [← mul_one t]
188   grw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
189   have expand := expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂ t (-1)
190   chev_simp at expand
191   grw [expand]
192
```

```
-- restatement of 8.120a using our naming conventions
@[simp, chev_simps]
theorem inv_of_αβψ : forall_i_t αβψ, ((αβψ, i, t))⁻¹ = ((αβψ, i, -t)) := by
  intro i hi t
  have := eq_inv_of_mul_eq_one_left (inv_doub_of_αβψ_a hi (-t))
  rw [neg_neg] at this
  exact this.symm

-- 8.120b
include Fchar
theorem doub_of_αβψ : forall_i_t αβψ, ((αβψ, i, t)) * ((αβψ, i, t)) = ((αβψ, i, 2 * t)) := by
  intros i hi t
  rcases decompose αβ.height ψ.height i hi with ⟨ i₁, i₂, ⟨ rfl, hi₁, hi₂ ⟩ ⟩
  rw [← mul_one t, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  have := raw_hom_lift_of_doub_of_αβψ hi₁ hi₂ t 1 1
  rw [mul_one, neg_mul, mul_one, mul_one] at this
  grw [this, mul_comm 2 t, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂, div_self Fchar]

lemma half_add_half (u : F) : (u / 2) + (u / 2) = u := by
  have : ((2 : F) / 2) = 1 := (div_eq_one_iff_eq Fchar).mpr rfl
  rw [← mul_two, div_mul_comm, this, one_mul]

-- 8.121
theorem generic_comm_of_αβ_ψ :
  forall_ij_tu 2 1,
    ⁅((αβ, i, t)), ((ψ, j, u))⁆ = ((αβψ, i + j, t * u)) * ⁅((αβψ, i + j, -t * u)), ((ψ, j, u / 2))⁆
    ∧ ⁅((αβ, i, t)), ((ψ, j, u))⁆ = ⁅((αβψ, i + j, t * u)), ((ψ, j, u / 2))⁆⁻¹ * ((αβψ, i + j, t * u)) := by
  intro i j hi hj t u
  set x := ((αβ, i, t)) with hx
  set y := ((ψ, j, u/2)) with hy
  have xinv : x⁻¹ = ((αβ, i, -t)) := by rw [inv_of_αβ]
  have ysquare : y^2 = ((ψ, j, u)) := by
    rw [pow_two, lin_of_ψ, ←two_mul, mul_div_left_comm, div_self Fchar, mul_one]
  have yinv : y⁻¹ = ((ψ, j, -(u / 2))) := by rw [inv_of_ψ]
  have x_star_y : (x ⋆ y) = ((αβψ, i + j, t * u)) := by
    unfold starCommutator_def x y
    grw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi hj, inv_of_ψ, pow_two, inv_of_αβ,
    lin_of_ψ, half_add_half Fchar u]
  have x_star_y_inv : (x ⋆ y)⁻¹ = ((αβψ, i + j, -t * u)) := by
    rw [x_star_y, eq_inv_of_mul_eq_one_left (inv_doub_of_αβψ_a (add_le_add hi hj) (t * u)), inv_inv,
    ↪  neg_mul]
  rw [←ysquare, ←x_star_y, ←x_star_y_inv]
  exact ⟨(star_commutator x y).symm, (commutator_star x y).symm⟩

-- 8.122
theorem generic_comm_of_α_βψ :
  forall_ij_tu 1 2,
    ⁅((α, i, t)), ((βψ, j, u))⁆ = ((αβψ, i + j, t * u)) * ⁅((αβψ, i + j, -t * u)), ((βψ, j, u / 2))⁆
    ∧ ⁅((α, i, t)), ((βψ, j, u))⁆ = ⁅((αβψ, i + j, t * u)), ((βψ, j, u / 2))⁆⁻¹ * ((αβψ, i + j, t * u)) := by
  intro i j hi hj t u
  set x := ((α, i, t)) with hx
  set y := ((βψ, j, u / 2)) with hy
  have ysquare : y^2 = ((βψ, j, u)) := by
    rw [pow_two, hy, lin_of_βψ, half_add_half Fchar u]
  have x_star_y : (x ⋆ y) = ((αβψ, i + j, t * u)) := by
    unfold starCommutator_def x y
    grw [expr_αβψ_as_βψ_α_βψ_α_βψ hi hj, inv_of_βψ, pow_two, lin_of_βψ, half_add_half Fchar u, inv_of_α]
  have x_star_y_inv : (x ⋆ y)⁻¹ = ((αβψ, i + j, -t * u)) := by
    rw [x_star_y, eq_inv_of_mul_eq_one_left (inv_doub_of_αβψ_a (add_le_add hi hj) (t * u)), inv_inv,
    ↪  neg_mul]
  rw [←ysquare, ←x_star_y, ←x_star_y_inv]
  exact ⟨(star_commutator x y).symm, (commutator_star x y).symm⟩

-- 8.123
omit Fchar
theorem lift_hom_interchange_of_αβ2ψ :
  forall_ijk_tuv,
    ⁅((αβψ, i + j + k, t * u)), ((ψ, k, v))⁆ = ⁅((α, i, t)), ((β2ψ, j + 2 * k, -2 * u * v))⁆ := by
  intro i j k hi hj hk t u v
  rcases eq_or_ne v 0 with hv | hv
  · rw [hv, mul_zero, id_of_ψ, id_of_β2ψ]; group
  have expr_αβ2ψ := raw_hom_lift_of_interchange_of_αβ2ψ hi hj hk t (u / v) v
  have : -2 * (u / v) * v^2 = -2 * u * v := by field_simp; group
  rw [this] at expr_αβ2ψ
  have expr_αβψ := expr_αβψ_as_ψ_αβ_ψ_αβ_ψ (add_le_add hi hj) hk (t * (u / v)) v
  have : t * (u / v) * v = t * u := by field_simp
  rw [this] at expr_αβψ
  grw [←expr_αβ2ψ, expr_αβψ, neg_mul]


```

```
-- 8.124
theorem lift_hom_comm_of_βψ_α_β2ψ : forall_ijk_tuv,
    ⁅((βψ, j + k, t)), ⁅((α, i, u)), ((β2ψ, j + 2 * k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  rcases eq_or_ne v 0 with hv | hv
  · rw [hv, id_of_β2ψ]; group
  rcases eq_or_ne t 0 with ht | ht
  · rw [ht, id_of_βψ]; group
  have hβψ := raw_hom_lift_of_comm_of_βψ_α_β2ψ hi hj hk u (t^2 / v) (v / t)
  have : t^2 / v * (v / t) = t := by rw [pow_two]; field_simp
  rw [this] at hβψ
  have : t^2 / v * (v / t)^2 = v := by field_simp; group
  rw [this] at hβψ
  exact hβψ

-- 8.125a
theorem lift_hom_inv_doub_of_α_β2ψ_a : forall_ij_tu α β2ψ,
    ⁅((α, i, t)), ((β2ψ, j, u))⁆ = ⁅((α, i, -t)), ((β2ψ, j, -u))⁆ := by
  intro i j hi hj t u
  rcases decompose_3_into_booleans_1_2 j hj with ⟨j₁, j₂, ⟨ rfl, hj₁, hj₂⟩ ⟩
  have := raw_hom_lift_of_inv_doub_of_α_β2ψ_a hi hj₁ hj₂ t u 1
  field_simp at this
  exact this

-- 8.125b
theorem lift_hom_inv_doub_of_α_β2ψ_b : forall_ij_tu α β2ψ,
    ⁅((α, i, t)), ((β2ψ, j, u))⁆ * ⁅((α, i, t)), ((β2ψ, j, -u))⁆ = 1 := by
  intro i j hi hj t u
  rcases decompose_3_into_booleans_1_2 j hj with ⟨j₁, j₂, ⟨ rfl, hj₁, hj₂⟩ ⟩
  have := raw_hom_lift_of_inv_doub_of_α_β2ψ_b hi hj₁ hj₂ t u 1
  field_simp at this
  exact this

theorem inv_of_comm_of_α_β2ψ : forall_ij_tu α β2ψ,
    ⁅((α, i, t)), ((β2ψ, j, u))⁆⁻¹ = ⁅((α, i, t)), ((β2ψ, j, -u))⁆ := by
  intro i j hi hj t u
  exact inv_eq_of_mul_eq_one_right (lift_hom_inv_doub_of_α_β2ψ_b hi hj t u)

-- 8.125c
theorem lift_hom_inv_doub_of_α_β2ψ_c : forall_ij_tu α β2ψ,
    ⁅((α, i, t)), ((β2ψ, j, u))⁆ * ⁅((α, i, t)), ((β2ψ, j, u))⁆ = ⁅((α, i, t)), ((β2ψ, j, 2 * u))⁆ := by
  intro i j hi hj t u
  rcases decompose_3_into_booleans_1_2 j hj with ⟨j₁, j₂, ⟨ rfl, hj₁, hj₂⟩ ⟩
  have := raw_hom_lift_of_inv_doub_of_α_β2ψ_c hi hj₁ hj₂ t u 1
  field_simp at this
  exact this

-- 8.126
theorem lift_hom_comm_of_β2ψ_αβψ : forall_ijk_tu α β ψ,
    ⁅((β2ψ, j + 2 * k, t)), ((αβψ, i + j + k, u))⁆ = 1 := by
  intro i j k hi hj hk t u
  rcases eq_or_ne t 0 with (rfl | ht)
  · chev_simp
  · have expr_β2ψ := raw_hom_lift_of_comm_of_β2ψ_αβψ hi hj hk (u / t) t 1
    rw [← mul_one u]
    grw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ (add_le_add hi hj) hk u 1]
    field_simp at expr_β2ψ
    have : -(u * t) / t = -u := by field_simp
    grw [this] at expr_β2ψ
    exact expr_β2ψ

-- 8.127
theorem comm_of_ψ_α_β2ψ : forall_ijk_tuv ψ α β2ψ,
    ⁅((ψ, i, t)), ⁅((α, j, u)), ((β2ψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.mpr
  grw [commutatorElement_def, ← expr_α_ψ_as_ψ_α hj hi, expr_ψ_β2ψ_as_β2ψ_ψ hi hk,
       ← expr_α_ψ_as_ψ_α hj hi, expr_ψ_β2ψ_as_β2ψ_ψ hi hk]

@[group_reassoc]
theorem expr_ψ_comm_α_β2ψ_as_comm_α_β2ψ_ψ : forall_ijk_tuv ψ α β2ψ,
    ((ψ, i, t)) * ⁅((α, j, u)), ((β2ψ, k, v))⁆ = ⁅((α, j, u)), ((β2ψ, k, v))⁆ * ((ψ, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_ψ_α_β2ψ hi hj hk t u v)

-- 8.128
theorem comm_of_α_αβψ_ψ : forall_ijk_tuv α αβψ ψ,
    ⁅((α, i, t)), ⁅((αβψ, j, u)), ((ψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.mpr
```

```
  rw [commutatorElement_def]
  grw [inv_of_αβψ hj, expr_α_αβψ_as_αβψ_α hi hj, expr_α_ψ_as_ψ_α hi hk,
    expr_α_αβψ_as_αβψ_α hi hj, expr_α_ψ_as_ψ_α hi hk]

@[group_reassoc]
theorem expr_α_comm_αβψ_ψ_as_comm_αβψ_ψ_α : forall_ijk_tuv α αβψ ψ,
    ((α, i, t)) * ⁅((αβψ, j, u)), ((ψ, k, v))⁆ = ⁅((αβψ, j, u)), ((ψ, k, v))⁆ * ((α, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_α_αβψ_ψ hi hj hk t u v)

include Fchar
-- 8.129
theorem comm_of_α_α_β2ψ : forall_ijk_tuv α α β2ψ,
    ⁅((α, i, t)), ⁅((α, j, u)), ((β2ψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.2
  rcases decompose_3_into_booleans_1_2 k hk with ⟨ j', k', ⟨ rfl, hj', hk' ⟩ ⟩
  have : v = -2 * v * (-1 / 2) := by field_simp
  rw [this, ←lift_hom_interchange_of_αβ2ψ hj hj' hk', expr_α_comm_αβψ_ψ_as_comm_αβψ_ψ_α hi (by ht) hk']

@[group_reassoc] theorem expr_α_comm_α_β2ψ_as_comm_α_β2ψ_α :
  forall_ijk_tuv α α β2ψ,
    ((α, i, t)) * ⁅((α, j, u)), ((β2ψ, k, v))⁆ = ⁅((α, j, u)), ((β2ψ, k, v))⁆ * ((α, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_α_α_β2ψ Fchar hi hj hk t u v)

-- Proposition 8.130
theorem sufficient_conditions_for_comm_of_αβψ_and_ψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ α.height) (hj : j ≤ βψ.height) (hk : k ≤ ψ.height)
  (hyp : ∀ (t u v : F), ⁅((βψ, j, t)), ⁅((α, i, u)), ((β2ψ, j + k, v))⁆⁆ = 1),
    ∀ (t u v : F), ⁅((αβψ, i + j, t * u))'(by height_simp at hi hj ⊢; omega), ((ψ, k, v))⁆
        = ⁅((α, i, t)), ((β2ψ, j + k, -2 * u * v))'(by height_simp at hj hk ⊢; omega)⁆ := by
  intro i j k hi hj hk hyp t u v
  have hjk : j + k ≤ β2ψ.height := by height_simp at hj hk ⊢; omega
  apply eq_comm_of_reorder_left
  grw [expr_αβψ_as_βψ_α_βψ_α_βψ hi hj, expr_βψ_ψ_as_ψ_β2ψ_βψ hk hj,
    expr_α_ψ_as_ψ_α hi hk, expr_βψ_ψ_as_ψ_β2ψ_βψ hk hj,
    expr_α_ψ_as_ψ_α hi hk, expr_βψ_ψ_as_ψ_β2ψ_βψ hk hj]
  simp_rw [deg_add_comm β2ψ k j]
  have : 2 * v * (u / 2) = u * v := by field_simp; group
  rw [this]
  clear this
  have hyp' := fun t' u' v' => triv_comm_iff_commutes.1 (hyp t' u' v')
  have aux₁ : ((β2ψ, j + k, u * v)) * ((α, i, t)) = ⁅((α, i, t)), ((β2ψ, j + k, -u * v))⁆ * ((α, i, t)) * ((β2ψ, j +
    ↪  k, u * v)) := by
    rw [comm_left_rev, inv_of_comm_of_α_β2ψ hi hjk, neg_mul]
  grw [← expr_βψ_β2ψ_as_β2ψ_βψ hj hjk (-(u / 2)) (u * v), aux₁]
  clear aux₁
  have aux₂ : ((α, i, -t)) * ((β2ψ, j + k, u * v)) = ((β2ψ, j + k, u * v)) * ((α, i, -t)) * ⁅((α, i, t)), ((β2ψ, j +
    ↪  k, -u * v))⁆ := by
    rw [← inv_of_α, neg_mul, ← inv_of_β2ψ]
    group
  have : u * v + -(2 * v * u) + u * v = 0 := by ring
  grw [aux₂, expr_βψ_β2ψ_as_β2ψ_βψ hj hjk u (u * v), this]
  clear aux₂ this
  grw [hyp' (-(u / 2)),
        expr_ψ_comm_α_β2ψ_as_comm_α_β2ψ_ψ hk hi hjk,
        expr_α_comm_α_β2ψ_as_comm_α_β2ψ_α Fchar hi hi hjk,
        hyp' u,
        expr_α_comm_α_β2ψ_as_comm_α_β2ψ_α Fchar hi hi hjk,
        hyp' (-(u / 2)),
        expr_ψ_comm_α_β2ψ_as_comm_α_β2ψ_ψ hk hi hjk,
        lift_hom_inv_doub_of_α_β2ψ_c hi hjk, mul_assoc]
```

Author source module Steinberg.B3Large.EstabAlphaBeta2Psi

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Large.Basic
import Steinberg.B3Large.EstabAlphaBetaPsi
import Mathlib.Tactic.Linarith

namespace Steinberg.B3Large

```

```
open Steinberg B3LargePosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)

include Fchar

-- 8.131
theorem partial_A_interchange_of_αβ2ψ :
  ∀ (t u v : F),
  ⁅((αβψ, 0, t * u)), ((ψ, 1, v))⁆ = ⁅((α, 0, t)), ((β2ψ, 1, -2 * u * v))⁆ := by
  apply @sufficient_conditions_for_comm_of_αβψ_and_ψ _ _ Fchar 0 0 1 (by trivial) (by trivial) (by
    ↪ trivial)
  intro t u v
  have h₁ := @lift_hom_interchange_of_αβ2ψ _ _ 1 0 0 (by trivial) (by trivial) (by trivial) u (-v/2) 1
  have h := @lift_hom_interchange_of_αβ2ψ _ _ 0 1 0 (by trivial) (by trivial) (by trivial) u (-v/2) 1
  norm_num at h₁
  norm_num at h
  rw [h₁] at h
  have : -(2 * (-v / 2)) = v := by field_simp
  rw [this] at h
  have h₁ := @lift_hom_comm_of_βψ_α_β2ψ _ _ 1 0 0 (by trivial) (by trivial) (by trivial) t u v
  rwa [h] at h₁

theorem partial_A_interchange_of_αβ2ψ' :
  ∀ (t u v : F),
  ⁅((αβψ, 3, t * u)), ((ψ, 0, v))⁆ = ⁅((α, 1, t)), ((β2ψ, 2, -2 * u * v))⁆ := by
  apply @sufficient_conditions_for_comm_of_αβψ_and_ψ _ _ Fchar 1 2 0 (by trivial) (by trivial) (by
    ↪ trivial)
  intro t u v
  have h₁ := @lift_hom_interchange_of_αβ2ψ _ _ 0 1 1 (by trivial) (by trivial) (by trivial) u (-v/2) 1
  have h := @lift_hom_interchange_of_αβ2ψ _ _ 1 0 1 (by trivial) (by trivial) (by trivial) u (-v/2) 1
  norm_num at h₁
  norm_num at h
  rw [h₁] at h
  have : -(2 * (-v / 2)) = v := by field_simp
  rw [this] at h
  have h₁ := @lift_hom_comm_of_βψ_α_β2ψ _ _ 0 1 1 (by trivial) (by trivial) (by trivial) t u v
  rwa [h] at h₁

-- Proposition 8.132
theorem sufficient_conditions_for_comm_of_βψ_and_α_β2ψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 2) (hj : j ≤ 1) (hk : k ≤ 3) (hyp : ∀ (t u : F), ⁅((αβψ, i + j, t)), ((β2ψ, k, u))⁆
    ↪ = 1),
  ∀ (t u v : F), ⁅((βψ, i, t)), ⁅((α, j, u)), ((β2ψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk hyp t u v
  apply triv_comm_iff_commutes.2
  have hyp' := fun t' u' ↦ triv_comm_iff_commutes.1 (hyp t' u')
  have aux₁ : ∀ (u' : F), ((βψ, i, t)) * ((α, j, u')) = ((α, j, u')) * ⁅((α, j, -u')), ((βψ, i, t))⁆ * ((βψ, i, t)) :=
    ↪ by
    intro u'; rw [← inv_of_α]; group
  have aux₂ : ∀ (u' : F), ((βψ, i, t)) * ((α, j, u')) = ⁅((α, j, u')), ((βψ, i, t))⁆⁻¹ * ((α, j, u')) * ((βψ, i, t))
    ↪ := by
    intro u'; group
  grw [commutatorElement_def, ←inv_of_α, ←inv_of_β2ψ, aux₁ u, expr_βψ_β2ψ_as_β2ψ_βψ hi hk, aux₂ (-u),
    ↪ expr_βψ_β2ψ_as_β2ψ_βψ hi hk]
  suffices h : ⁅((α, j, -u)), ((βψ, i, t))⁆ * ((β2ψ, k, v)) * ⁅((α, j, -u)), ((βψ, i, t))⁆⁻¹ = ((β2ψ, k, v)) by
    have : ⁅((βψ, i, t)), ((α, j, -u))⁆ = ⁅((α, j, -u)), ((βψ, i, t))⁆⁻¹ := by rw [←inv_of_α]; group
    rw [this]; exact h
  apply mul_inv_eq_iff_eq_mul.2
  have := (generic_comm_of_α_βψ Fchar hj hi (-u) t).1
  field_simp at this
  have := calc
    ⁅((α, j, -u)), ((βψ, i, t))⁆ = ((αβψ, j + i, -(u * t))) * ⁅((αβψ, j + i, u * t)), ((βψ, i, t / 2))⁆ := this
    _ = ((αβψ, i + j, -t * u)) * ⁅((αβψ, i + j, t * u)), ((βψ, i, t / 2))⁆ := by group
  grw [this, commutatorElement_def, expr_βψ_β2ψ_as_β2ψ_βψ hi hk, inv_of_αβψ, ←hyp', ←hyp']
  rw [mul_assoc, hyp']
  grw [expr_βψ_β2ψ_as_β2ψ_βψ]

-- 8.133
theorem partial_comm_of_βψ_α_β2ψ :
  ∀ (t u v : F),
  ⁅((βψ, 2, t)), ⁅((α, 0, u)), ((β2ψ, 2, v))⁆⁆ = 1 := by
  apply @sufficient_conditions_for_comm_of_βψ_and_α_β2ψ _ _ Fchar 2 0 2 (by trivial) (by trivial)
    ↪ Nat.AtLeastTwo.prop
  intro t u
  have := triv_comm_iff_commutes.mp (@lift_hom_comm_of_β2ψ_αβψ _ _ 1 0 1 (by trivial) (by trivial) (by
    ↪ trivial) u t)
  apply triv_comm_iff_commutes.mpr
  rw [←this]
```

```

-- 8.134
theorem partial_B_interchange_of_αβ2ψ :
  ∀ (t u v : F),
  ⁅((αβψ, 2, t * u)), ((ψ, 0, v))⁆ = ⁅((α, 0, t)), ((β2ψ, 2, -2 * u * v))⁆ :=
  @sufficient_conditions_for_comm_of_αβψ_and_ψ _ _ Fchar 0 2 0 (by trivial) (by ht) (by trivial)
    ↪  (@partial_comm_of_βψ_α_β2ψ _ _ Fchar)

/- ### Establishing α + β + 2ψ -/

private lemma interchange_of_αβ2ψ_trans_α_β2ψ : forall_ijk_tu α β ψ,
    ⁅((α, i, t * u)), ((β2ψ, j + 2 * k, 1))⁆ = ⁅((α, i, t)), ((β2ψ, j + 2 * k, u))⁆ := by
  intro i j k hi hj hk t u
  have aux₁ := lift_hom_interchange_of_αβ2ψ hi hj hk (t * u) 1 (-1/2)
  have aux₂ := lift_hom_interchange_of_αβ2ψ hi hj hk t u (-1/2)
  field_simp at aux₁
  field_simp at aux₂
  rwa [aux₁] at aux₂

omit Fchar
private lemma interchange_of_αβ2ψ_refl_u : forall_ijk_tu α β ψ,
    ⁅((αβψ, i + j + k, t)), ((ψ, k, u))⁆ = ⁅((α, i, t)), ((β2ψ, j + 2 * k, -2 * u))⁆ := by
  intro i j k hi hj hk t u
  rw [← mul_one t]
  grw [lift_hom_interchange_of_αβ2ψ hi hj hk]

private lemma interchange_of_αβ2ψ_refl_v : forall_ijk_tu α β ψ,
    ⁅((αβψ, i + j + k, t * u)), ((ψ, k, 1))⁆ = ⁅((α, i, t)), ((β2ψ, j + 2 * k, -2 * u))⁆ := by
  intro i j k hi hj hk t u
  grw [lift_hom_interchange_of_αβ2ψ hi hj hk]

private lemma interchange_of_αβ2ψ_trans_αβψ_ψ : forall_ijk_tu α β ψ,
    ⁅((αβψ, i + j + k, t * u)), ((ψ, k, 1))⁆ = ⁅((αβψ, i + j + k, t)), ((ψ, k, u))⁆ := by
  intro i j k hi hj hk t u
  rw [interchange_of_αβ2ψ_refl_v hi hj hk, interchange_of_αβ2ψ_refl_u hi hj hk]

include Fchar
private lemma interchange_A_of_αβ2ψ_refl_u :
  ∀ t u : F, ⁅((αβψ, 0, t)), ((ψ, 1, u))⁆ = ⁅((α, 0, t)), ((β2ψ, 1, -2 * u))⁆ := by
  intro t u
  rw [←mul_one t, partial_A_interchange_of_αβ2ψ Fchar]
  simp only [mul_one, neg_mul]

private lemma interchange_A_of_αβ2ψ_refl_u' :
  ∀ t u : F, ⁅((αβψ, 3, t)), ((ψ, 0, u))⁆ = ⁅((α, 1, t)), ((β2ψ, 2, -2 * u))⁆ := by
  intro t u
  rw [←mul_one t, partial_A_interchange_of_αβ2ψ' Fchar]
  simp only [mul_one, neg_mul]

private lemma interchange_A_of_αβ2ψ_refl_v :
  ∀ t u : F, ⁅((αβψ, 0, t * u)), ((ψ, 1, 1))⁆ = ⁅((α, 0, t)), ((β2ψ, 1, -2 * u))⁆ := by
  intro t u
  rw [partial_A_interchange_of_αβ2ψ Fchar]
  simp only [neg_mul, mul_one]

private lemma interchange_A_of_αβ2ψ_refl_v' :
  ∀ t u : F, ⁅((αβψ, 3, t * u)), ((ψ, 0, 1))⁆ = ⁅((α, 1, t)), ((β2ψ, 2, -2 * u))⁆ := by
  intro t u
  rw [partial_A_interchange_of_αβ2ψ' Fchar]
  simp only [neg_mul, mul_one]

private lemma interchange_A_of_αβ2ψ_trans_αβψ_ψ :
  ∀ t u : F, ⁅((αβψ, 0, t * u)), ((ψ, 1, 1))⁆ = ⁅((αβψ, 0, t)), ((ψ, 1, u))⁆ := by
  intro t u
  rw [interchange_A_of_αβ2ψ_refl_v Fchar, interchange_A_of_αβ2ψ_refl_u Fchar]

private lemma interchange_A_of_αβ2ψ_trans_αβψ_ψ' :
  ∀ t u : F, ⁅((αβψ, 3, t * u)), ((ψ, 0, 1))⁆ = ⁅((αβψ, 3, t)), ((ψ, 0, u))⁆ := by
  intro t u
  rw [interchange_A_of_αβ2ψ_refl_v' Fchar, interchange_A_of_αβ2ψ_refl_u' Fchar]

private lemma interchange_B_of_αβ2ψ_refl_u :
  ∀ t u : F, ⁅((αβψ, 2, t)), ((ψ, 0, u))⁆ = ⁅((α, 0, t)), ((β2ψ, 2, -2 * u))⁆ := by
  intro t u
  rw [←mul_one t, partial_B_interchange_of_αβ2ψ Fchar]
  simp only [mul_one, neg_mul]

private lemma interchange_B_of_αβ2ψ_refl_v :
  ∀ t u : F, ⁅((αβψ, 2, t * u)), ((ψ, 0, 1))⁆ = ⁅((α, 0, t)), ((β2ψ, 2, -2 * u))⁆ := by
  intro t u
```

```
  rw [partial_B_interchange_of_αβ2ψ Fchar]
  simp only [neg_mul, mul_one]

private lemma interchange_B_of_αβ2ψ_trans_αβψ_ψ :
  ∀ t u : F, ⁅((αβψ, 2, t * u)), ((ψ, 0, 1))⁆ = ⁅((αβψ, 2, t)), ((ψ, 0, u))⁆ := by
  intro t u
  rw [interchange_B_of_αβ2ψ_refl_v Fchar, interchange_B_of_αβ2ψ_refl_u Fchar]

-- height 0
private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_00 :
  ∀ t u : F, ((αβ2ψ, 0, t * u)) = ⁅((α, 0, t)), ((β2ψ, 0, u))⁆ := by
  intro t u
  have := @def_of_αβ2ψ _ _ 0 (by trivial) (t * u)
  unfold split_4_into_1_3 at this
  rw [←this, @interchange_of_αβ2ψ_trans_α_β2ψ _ _ Fchar 0 0 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_00 :
  ∀ t u : F, ((αβ2ψ, 0, -2 * t * u)) = ⁅((αβψ, 0, u)), ((ψ, 0, t))⁆ := by
  intro t u
  have : -2 * t * u = t * (-2 * u) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_00 Fchar, ←@interchange_of_αβ2ψ_refl_v _ _ 0 0 0 (by trivial) (by
    ↪  trivial) (by trivial),
  mul_comm t u, interchange_of_αβ2ψ_trans_αβψ_ψ (by trivial) (by trivial) (by trivial)]

-- height 1
private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_01 :
  ∀ t u : F, ((αβ2ψ, 1, t * u)) = ⁅((α, 0, t)), ((β2ψ, 1, u))⁆ := by
  intro t u
  have := @def_of_αβ2ψ _ _ 1 (by trivial) (t * u)
  unfold split_4_into_1_3 at this
  rw [←this, @interchange_of_αβ2ψ_trans_α_β2ψ _ _ Fchar 0 1 0 (by trivial) (by trivial) (by trivial)]

-- `A` edge
private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_01 :
  ∀ t u : F, ((αβ2ψ, 1, -2 * t * u)) = ⁅((αβψ, 0, u)), ((ψ, 1, t))⁆ := by
  intro t u
  have : -2 * t * u = t * (-2 * u) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_01 Fchar, ←interchange_A_of_αβ2ψ_refl_v Fchar,
  mul_comm t u, interchange_A_of_αβ2ψ_trans_αβψ_ψ Fchar]

private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_10 :
  ∀ t u : F, ((αβ2ψ, 1, -2 * t * u)) = ⁅((αβψ, 1, u)), ((ψ, 0, t))⁆ := by
  intro t u
  have : -2 * t * u = t * (-2 * u) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_01 Fchar, ←@interchange_of_αβ2ψ_refl_v _ _ 0 1 0 (by trivial) (by
    ↪  trivial) (by trivial),
  mul_comm t u, @interchange_of_αβ2ψ_trans_αβψ_ψ _ _ 0 1 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_10 :
  ∀ t u : F, ((αβ2ψ, 1, t * u)) = ⁅((α, 1, t)), ((β2ψ, 0, u))⁆ := by
  intro t u
  have : t * u = -2 * (-u / 2) * t := by ring_nf; field_simp
  rw [this, expr_αβ2ψ_as_comm_of_αβψ_ψ_10 Fchar, @interchange_of_αβ2ψ_refl_u _ _ 1 0 0 (by trivial) (by
    ↪  trivial) (by trivial)]
  field_simp

-- height 2
private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_11 :
  ∀ t u : F, ((αβ2ψ, 2, t * u)) = ⁅((α, 1, t)), ((β2ψ, 1, u))⁆ := by
  intro t u
  have := @def_of_αβ2ψ _ _ 2 (by trivial) (t * u)
  unfold split_4_into_1_3 at this
  rw [←this, @interchange_of_αβ2ψ_trans_α_β2ψ _ _ Fchar 1 1 0 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_20 :
  ∀ t u : F, ((αβ2ψ, 2, -2 * t * u)) = ⁅((αβψ, 2, u)), ((ψ, 0, t))⁆ := by
  intro t u
  have : -2 * t * u = t * (-2 * u) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_11 Fchar, ←@interchange_of_αβ2ψ_refl_v _ _ 1 1 0 (by trivial) (by
    ↪  trivial) (by trivial),
  mul_comm t u, @interchange_of_αβ2ψ_trans_αβψ_ψ _ _ 1 1 0 (by trivial) (by trivial) (by trivial)]

-- `B` edge
private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_02 :
  ∀ t u : F, ((αβ2ψ, 2, t * u)) = ⁅((α, 0, t)), ((β2ψ, 2, u))⁆ := by
  intro t u
  have : t * u = -2 * (-u / 2) * t := by ring_nf; field_simp
  rw [this, expr_αβ2ψ_as_comm_of_αβψ_ψ_20 Fchar, interchange_B_of_αβ2ψ_refl_u Fchar]
  field_simp

```

```
private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_11 :
  ∀ t u : F, ((αβ2ψ, 2, -2 * t * u)) = ⟦((αβψ, 1, u)), ((ψ, 1, t))⟧ := by
  intro t u
  have : -2 * t * u = t * (-2 * u) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_02 Fchar, ←@interchange_of_αβ2ψ_refl_v _ _ 0 0 1 (by trivial) (by
    ↪  trivial) (by trivial),
  mul_comm t u, interchange_of_αβ2ψ_trans_αβψ_ψ (by trivial) (by trivial) (by trivial)]

-- height 3
private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_12 :
  ∀ t u : F, ((αβ2ψ, 3, t * u)) = ⟦((α, 1, t)), ((β2ψ, 2, u))⟧ := by
  intro t u
  have := @def_of_αβ2ψ _ _ 3 (by trivial) (t * u)
  unfold split_4_into_1_3 at this
  rw [←this, @interchange_of_αβ2ψ_trans_α_β2ψ _ _ Fchar 1 0 1 (by trivial) (by trivial) (by trivial)]

-- `A` edge
private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_30 :
  ∀ t u : F, ((αβ2ψ, 3, -2 * t * u)) = ⟦((αβψ, 3, u)), ((ψ, 0, t))⟧ := by
  intro t u
  have : -2 * t * u = t * (-2 * u) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_12 Fchar, ←interchange_A_of_αβ2ψ_refl_v' Fchar,
  mul_comm t u, interchange_A_of_αβ2ψ_trans_αβψ_ψ' Fchar]

private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_21 :
  ∀ t u : F, ((αβ2ψ, 3, -2 * t * u)) = ⟦((αβψ, 2, u)), ((ψ, 1, t))⟧ := by
  intro t u
  have : -2 * t * u = u * (-2 * t) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_12 Fchar, ←@interchange_of_αβ2ψ_refl_v _ _ 1 0 1 (by trivial) (by
    ↪  trivial) (by trivial),
  @interchange_of_αβ2ψ_trans_αβψ_ψ _ _ 1 0 1 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_03 :
  ∀ t u : F, ((αβ2ψ, 3, t * u)) = ⟦((α, 0, t)), ((β2ψ, 3, u))⟧ := by
  intro t u
  have : t * u = -2 * (-u / 2) * t := by ring_nf; field_simp
  rw [this, expr_αβ2ψ_as_comm_of_αβψ_ψ_21 Fchar, @interchange_of_αβ2ψ_refl_u _ _ 0 1 1 (by trivial) (by
    ↪  trivial) (by trivial)]
  field_simp

-- height 4
private lemma expr_αβ2ψ_as_comm_of_α_β2ψ_13 :
  ∀ t u : F, ((αβ2ψ, 4, t * u)) = ⟦((α, 1, t)), ((β2ψ, 3, u))⟧ := by
  intro t u
  have := @def_of_αβ2ψ _ _ 4 (by trivial) (t * u)
  unfold split_4_into_1_3 at this
  rw [←this, @interchange_of_αβ2ψ_trans_α_β2ψ _ _ Fchar 1 1 1 (by trivial) (by trivial) (by trivial)]

private lemma expr_αβ2ψ_as_comm_of_αβψ_ψ_31 :
  ∀ t u : F, ((αβ2ψ, 4, -2 * t * u)) = ⟦((αβψ, 3, u)), ((ψ, 1, t))⟧ := by
  intro t u
  have : -2 * t * u = u * (-2 * t) := by group
  rw [this, expr_αβ2ψ_as_comm_of_α_β2ψ_13 Fchar, ←@interchange_of_αβ2ψ_refl_v _ _ 1 1 1 (by trivial) (by
    ↪  trivial) (by trivial),
  @interchange_of_αβ2ψ_trans_αβψ_ψ _ _ 1 1 1 (by trivial) (by trivial) (by trivial)]

-- 8.135a
theorem expr_αβ2ψ_as_comm_of_α_β2ψ :
  forall_ij_tu 1 3,
    ((αβ2ψ, i + j, t * u)) = ⟦((α, i, t)), ((β2ψ, j, u))⟧ := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_00 Fchar]
  | 0, 1 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_01 Fchar]
  | 1, 0 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_10 Fchar]
  | 0, 2 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_02 Fchar]
  | 1, 1 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_11 Fchar]
  | 0, 3 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_03 Fchar]
  | 1, 2 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_12 Fchar]
  | 1, 3 => rw [expr_αβ2ψ_as_comm_of_α_β2ψ_13 Fchar]

-- 8.135b
theorem expr_αβ2ψ_as_comm_of_αβψ_ψ :
  forall_ij_tu 3 1,
    ((αβ2ψ, i + j, -2 * t * u)) = ⟦((αβψ, i, u)), ((ψ, j, t))⟧ := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_00 Fchar]
  | 1, 0 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_10 Fchar]
  | 0, 1 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_01 Fchar]
```

```
  | 1, 1 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_11 Fchar]
  | 2, 0 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_20 Fchar]
  | 2, 1 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_21 Fchar]
  | 3, 0 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_30 Fchar]
  | 3, 1 => rw [expr_αβ2ψ_as_comm_of_αβψ_ψ_31 Fchar]

-- 8.136
theorem comm_of_α_αβ2ψ :
  trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨α, αβ2ψ⟩ := by
  intro i j hi hj t u
  rcases decompose 3 1 j hj with ⟨ j₁, j₂, ⟨ rfl, hj₁, hj₂ ⟩ ⟩
  have expr_αβ2ψ := @expr_αβ2ψ_as_comm_of_αβψ_ψ _ _ Fchar j₁ j₂ hj₁ hj₂ (-1/2) u
  field_simp at expr_αβ2ψ
  have := @comm_of_α_αβψ_ψ _ _ i j₁ j₂ hi hj₁ hj₂ t u (-1/2)
  rwa [←expr_αβ2ψ] at this
declare_B3Large_trivial_span_expr_thm F α αβ2ψ

-- 8.137
theorem comm_of_ψ_αβ2ψ :
  trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨ψ, αβ2ψ⟩ := by
  intro i j hi hj t u
  rcases decompose 3 1 j hj with ⟨ j₂, j₁, ⟨ rfl, hj₂, hj₁ ⟩ ⟩
  have expr_αβ2ψ := @expr_αβ2ψ_as_comm_of_α_β2ψ _ _ Fchar j₁ j₂ hj₁ hj₂ 1 u
  have := @comm_of_ψ_α_β2ψ _ _ i j₁ j₂ hi hj₁ hj₂ t 1 u
  rw [←expr_αβ2ψ] at this
  rw [←this]
  group
declare_B3Large_trivial_span_expr_thm F ψ αβ2ψ

-- 8.138a
@[simp, chev_simps]
private lemma inv_doub_of_αβ2ψ_a :
  forall_i_t αβ2ψ,
    ((αβ2ψ, i, t)) * ((αβ2ψ, i, -t)) = 1 := by
  intro i hi t
  rcases decompose 3 1 i hi with ⟨ i₂, i₁, ⟨ rfl, hi₂, hi₁ ⟩ ⟩
  have := @expr_αβ2ψ_as_comm_of_α_β2ψ _ _ Fchar i₁ i₂ hi₁ hi₂
  calc
    ((αβ2ψ, i₂ + i₁, t)) * ((αβ2ψ, i₂ + i₁, -t)) = ((αβ2ψ, i₁ + i₂, t * 1)) * ((αβ2ψ, i₁ + i₂, t * (-1))) := by
    ↪  group
    _ = ⁅((α, i₁, t)), ((β2ψ, i₂, 1))⁆ * ⁅((α, i₁, t)), ((β2ψ, i₂, -1))⁆ := by rw [this t 1, this t (-1)]
    _ = 1 := by rw [lift_hom_inv_doub_of_α_β2ψ_b hi₁ hi₂]

@[simp, chev_simps]
theorem inv_of_αβ2ψ :
  forall_i_t αβ2ψ,
    ((αβ2ψ, i, t))⁻¹ = ((αβ2ψ, i, -t)) := by
  intro i hi t
  have := @inv_doub_of_αβ2ψ_a _ _ Fchar i hi t
  rw [eq_inv_of_mul_eq_one_left this, inv_inv]

-- 8.138b
theorem doub_of_αβ2ψ :
  forall_i_t αβ2ψ.height,
    ((αβ2ψ, i, t)) * ((αβ2ψ, i, t)) = ((αβ2ψ, i, 2 * t)) := by
  intro i hi t
  rcases decompose 3 1 i hi with ⟨ i₂, i₁, ⟨ rfl, hi₂, hi₁ ⟩ ⟩
  have := @expr_αβ2ψ_as_comm_of_α_β2ψ _ _ Fchar i₁ i₂ hi₁ hi₂
  calc
    ((αβ2ψ, i₂ + i₁, t)) * ((αβ2ψ, i₂ + i₁, t)) = ((αβ2ψ, i₁ + i₂, 1 * t)) * ((αβ2ψ, i₁ + i₂, 1 * t)) := by group
    _ = ⁅((α, i₁, (1 : F))), ((β2ψ, i₂, t))⁆ * ⁅((α, i₁, (1 : F))), ((β2ψ, i₂, t))⁆ := by rw [this]
    _ = ⁅((α, i₁, (1 : F))), ((β2ψ, i₂, 2 * t))⁆ := by rw [lift_hom_inv_doub_of_α_β2ψ_c hi₁ hi₂]
    _ = ((αβ2ψ, i₂ + i₁, 2 * t)) := by rw [←this]; group

-- 8.139a
theorem commutator_of_αβ_ψ_a :
  forall_ij_tu 2 1,
    ⁅((αβ, i, t)), ((ψ, j, u))⁆ = ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, t * u^2)) := by
  intro i j hi hj t u
  have aux := expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar (add_le_add hi hj) hj (u / 2) (-t * u)
  have : ((αβ2ψ, i + j + j, -2 * (u / 2) * (-t * u))) = ((αβ2ψ, i + 2 * j, t * u^2)) := by ring_nf; field_simp
  rw [this] at aux
  rw [(generic_comm_of_αβ_ψ Fchar hi hj t u).1, aux]

-- 8.139b
theorem commutator_of_αβ_ψ_b :
  forall_ij_tu 2 1,
    ⁅((αβ, i, t)), ((ψ, j, u))⁆ = ((αβ2ψ, i + 2 * j, t * u^2)) * ((αβψ, i + j, t * u)) := by
  intro i j hi hj t u
  have aux := expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar (add_le_add hi hj) hj (u / 2) (t * u)
```

```
  rw [(generic_comm_of_αβ_ψ Fchar hi hj t u).2, ←aux, inv_of_αβ2ψ Fchar (by ht)]
  ring_nf; field_simp

-- 8.140a
theorem expr_αβ2ψ_as_α_β2ψ_α_β2ψ :
  forall_ij_tu α β2ψ,
    ((αβ2ψ, i + j, t * u)) = ((α, i, t)) * ((β2ψ, j, u)) * ((α, i, -t)) * ((β2ψ, j, -u)) := by
  intro i j hi hj t u
  rw [expr_αβ2ψ_as_comm_of_α_β2ψ Fchar hi hj, ← inv_of_α, ← inv_of_β2ψ, commutatorElement_def]

-- 8.140b
theorem expr_αβ2ψ_as_β2ψ_α_β2ψ_α :
  forall_ij_tu α β2ψ,
    ((αβ2ψ, i + j, t * u)) = ((β2ψ, j, -u)) * ((α, i, t)) * ((β2ψ, j, u)) * ((α, i, -t)) := by
  intro i j hi hj t u
  calc
    ((αβ2ψ, i + j, t * u)) = ((αβ2ψ, i + j, t * (-u)))⁻¹ := by rw [inv_of_αβ2ψ Fchar (add_le_add hi hj)];
    ↪ group
    _ = ⁅((α, i, t)), ((β2ψ, j, -u))⁆⁻¹ := by rw [expr_αβ2ψ_as_comm_of_α_β2ψ Fchar hi hj]
    _ = ⁅((α, i, t)), ((β2ψ, j, u))⁻¹⁆⁻¹ := by rw [inv_of_β2ψ]
    _ = ((β2ψ, j, -u)) * ((α, i, t)) * ((β2ψ, j, u)) * ((α, i, -t)) := by rw [← inv_of_β2ψ, ← inv_of_α]; group

-- 8.141a
theorem expr_αβ2ψ_as_αβψ_ψ_αβψ_ψ :
  forall_ij_tu αβψ ψ,
    ((αβ2ψ, i + j, -2 * t * u)) = ((αβψ, i, u)) * ((ψ, j, t)) * ((αβψ, i, -u)) * ((ψ, j, -t)) := by
  intro i j hi hj t u
  rw [expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hi hj, ← inv_of_αβψ hi, ← inv_of_ψ hj, commutatorElement_def];

-- 8.141b
theorem expr_αβ2ψ_as_ψ_αβψ_ψ_αβψ :
  forall_ij_tu αβψ ψ,
    ((αβ2ψ, i + j, -2 * t * u)) = ((ψ, j, -t)) * ((αβψ, i, u)) * ((ψ, j, t)) * ((αβψ, i, -u)) := by
  intro i j hi hj t u
  calc
    ((αβ2ψ, i + j, -2 * t * u)) = ((αβ2ψ, i + j, -2 * (-t) * u))⁻¹ := by rw [inv_of_αβ2ψ Fchar (add_le_add hi
    ↪ hj)]; group
    _ = ⁅((αβψ, i, u)), ((ψ, j, -t))⁆⁻¹ := by rw [expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hi hj]
    _ = ⁅((αβψ, i, u)), ((ψ, j, t))⁻¹⁆⁻¹ := by rw [inv_of_ψ]
    _ = ((ψ, j, -t)) * ((αβψ, i, u)) * ((ψ, j, t)) * ((αβψ, i, -u)) := by rw [← inv_of_ψ hj, ← inv_of_αβψ hi];
    ↪ group

-- 8.142a
@[group_reassoc]
theorem expr_α_β2ψ_as_αβ2ψ_β2ψ_α :
  forall_ij_tu α β2ψ,
    ((α, i, t)) * ((β2ψ, j, u)) = ((αβ2ψ, i + j, t * u)) * ((β2ψ, j, u)) * ((α, i, t)) := by
  intro i j hi hj t u
  rw [expr_αβ2ψ_as_comm_of_α_β2ψ Fchar hi hj]
  group

-- 8.142b
@[group_reassoc]
theorem expr_α_β2ψ_as_β2ψ_αβ2ψ_α :
  forall_ij_tu α β2ψ,
    ((α, i, t)) * ((β2ψ, j, u)) = ((β2ψ, j, u)) * ((αβ2ψ, i + j, t * u)) * ((α, i, t)) := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + j, t * u)) = ((αβ2ψ, i + j, t * (-u)))⁻¹ := by rw [inv_of_αβ2ψ Fchar (add_le_add hi hj)];
    ↪ group
    _ = ⁅((α, i, t)), ((β2ψ, j, -u))⁆⁻¹ := by rw [expr_αβ2ψ_as_comm_of_α_β2ψ Fchar hi hj]
    _ = ⁅((α, i, t)), ((β2ψ, j, u))⁻¹⁆⁻¹ := by rw [inv_of_β2ψ]
  rw [this]
  group

-- 8.142c
@[group_reassoc]
theorem expr_α_β2ψ_as_β2ψ_α_αβ2ψ :
  forall_ij_tu α β2ψ,
    ((α, i, t)) * ((β2ψ, j, u)) = ((β2ψ, j, u)) * ((α, i, t)) * ((αβ2ψ, i + j, t * u)) := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + j, t * u)) = ((αβ2ψ, i + j, (-t) * (-u))) := by group
    _ = ⁅((α, i, -t)), ((β2ψ, j, -u))⁆ := by rw [expr_αβ2ψ_as_comm_of_α_β2ψ Fchar hi hj]
    _ = ⁅((α, i, t))⁻¹, ((β2ψ, j, u))⁻¹⁆ := by rw [inv_of_β2ψ, inv_of_α]
  rw [this]
  group

-- 8.143a
@[group_reassoc]
```

```
theorem expr_ψ_αβψ_as_αβψ_αβ2ψ_ψ :
  forall_ij_tu ψ αβψ,
    ((ψ, i, t)) * ((αβψ, j, u)) = ((αβψ, j, u)) * ((αβ2ψ, i + j, 2 * t * u)) * ((ψ, i, t)) := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + j, 2 * t * u)) = ((αβ2ψ, j + i, -2 * t * (-u))) := by group
    _ = ⁅((αβψ, j, u))⁻¹, ((ψ, i, t))⁆ := by rw [expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hj hi, inv_of_αβψ hj]
  rw [this]
  group

-- 8.143b
@[group_reassoc]
theorem expr_ψ_αβψ_as_αβψ_ψ_αβ2ψ :
  forall_ij_tu ψ αβψ,
    ((ψ, i, t)) * ((αβψ, j, u)) = ((αβψ, j, u)) * ((ψ, i, t)) * ((αβ2ψ, i + j, 2 * t * u)) := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + j, 2 * t * u)) = ((αβ2ψ, j + i, -2 * (-t) * (-u)))⁻¹ := by rw [inv_of_αβ2ψ Fchar (by ht)];
    ↪  group
    _ = ⁅((αβψ, j, u))⁻¹, ((ψ, i, t))⁻¹⁆⁻¹ := by rw [expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hj hi, inv_of_αβψ hj,
    ↪  inv_of_ψ]
  rw [this]
  group

-- 8.144a
@[group_reassoc]
theorem expr_αβψ_ψ_as_αβ2ψ_ψ_αβψ :
  forall_ij_tu ψ αβψ,
    ((αβψ, j, u)) * ((ψ, i, t)) = ((αβ2ψ, i + j, -2 * t * u)) * ((ψ, i, t)) * ((αβψ, j, u)) := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + j, -2 * t * u)) = ((αβ2ψ, j + i, -2 * t * u)) := by group
    _ = ⁅((αβψ, j, u)), ((ψ, i, t))⁆ := by rw [expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hj hi]
  rw [this]
  group

-- 8.144b
@[group_reassoc]
theorem expr_αβψ_ψ_as_ψ_αβ2ψ_αβψ :
  forall_ij_tu ψ αβψ,
    ((αβψ, j, u)) * ((ψ, i, t)) = ((ψ, i, t)) * ((αβ2ψ, i + j, -2 * t * u)) * ((αβψ, j, u)) := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + j, -2 * t * u)) = ((αβ2ψ, j + i, -2 * (-t) * u))⁻¹ := by rw [inv_of_αβ2ψ Fchar (by ht)]; group
    _ = ⁅((αβψ, j, u)), ((ψ, i, t))⁻¹⁆⁻¹ := by rw [expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hj hi, inv_of_ψ]
  rw [this]
  group

-- 8.145a
@[group_reassoc]
theorem expr_αβ_ψ_as_ψ_αβ_αβψ_αβ2ψ :
  forall_ij_tu 2 1,
    ((αβ, i, t)) * ((ψ, j, u)) = ((ψ, j, u)) * ((αβ, i, t)) * ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) :=
    ↪  by
  intro i j hi hj t u
  have aux := calc
    ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) = ((αβψ, i + j, (-t) * (-u))) * ((αβ2ψ, i + 2 * j, -t *
    ↪  (-u)^2)) := by ring_nf
    _ = ⁅((αβ, i, -t)), ((ψ, j, -u))⁆ := by rw [commutator_of_αβ_ψ_a Fchar hi hj]
  have := calc
    ((ψ, j, u)) * ((αβ, i, t)) * ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u ^ 2))
    = ((ψ, j, u)) * ((αβ, i, t)) * (((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2))) := rfl
    _ = ((ψ, j, u)) * ((αβ, i, t)) * ⁅((αβ, i, -t)), ((ψ, j, -u))⁆ := by rw [aux]
    _ = ((ψ, j, u)) * ((αβ, i, t)) * ⁅((αβ, i, t))⁻¹, ((ψ, j, u))⁻¹⁆ := by rw [inv_of_αβ, inv_of_ψ]
    _ = ((αβ, i, t)) * ((ψ, j, u)) := by group
  exact this.symm

-- 8.145b
@[group_reassoc]
theorem expr_αβ_ψ_as_ψ_αβψ_αβ2ψ_αβ :
  forall_ij_tu 2 1,
    ((αβ, i, t)) * ((ψ, j, u)) = ((ψ, j, u)) * ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβ, i, t)) :=
    ↪  by
  intro i j hi hj t u
  have aux : ((ψ, j, u)) * ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβ, i, t)) = ((ψ, j, u)) *
  ↪  (((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2))) * ((αβ, i, t)) := rfl
  have := calc
    ((αβψ, i + j, t * u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) = ((αβψ, i + j, t * (-u)))⁻¹ * ((αβ2ψ, i + 2 * j, t *
    ↪  u^2))⁻¹ := by
      rw [inv_of_αβ2ψ Fchar (by ht), inv_of_αβψ (by ht)]; field_simp
```

```
    _ = (((αβ2ψ, i + 2 * j, t * (-u)^2) * ((αβψ, i + j, t * (-u))))⁻¹ := by ring_nf; group
    _ = ⁅((αβ, i, t)), ((ψ, j, -u))⁆⁻¹ := by rw [commutator_of_αβ_ψ_b Fchar hi hj]
    _ = ⁅((αβ, i, t)), ((ψ, j, u))⁻¹⁆⁻¹ := by rw [inv_of_ψ]
  rw [aux, this]
  group

-- 8.145c
@[group_reassoc]
theorem expr_αβ_ψ_as_ψ_αβ2ψ_αβψ_αβ :
  ∀ ⦃i j : ℕ⦄ (hi : i ≤ 2) (hj : j ≤ 1) (t u : F),
  ((αβ, i, t)) * ((ψ, j, u)) = ((ψ, j, u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβψ, i + j, t * u)) * ((αβ, i, t)) :=
    ↪ by
  intro i j hi hj t u
  have aux : ((ψ, j, u)) * ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβψ, i + j, t * u)) * ((αβ, i, t)) = ((ψ, j, u)) *
    ↪ (((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβψ, i + j, t * u))) * ((αβ, i, t)) := rfl
  have := calc
    ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβψ, i + j, t * u)) = ((αβ2ψ, i + 2 * j, t * (-u)^2))⁻¹ * ((αβψ, i + j, t *
      ↪ (-u)))⁻¹ := by rw [inv_of_αβ2ψ Fchar (by ht), inv_of_αβψ (by ht)]; field_simp
    _ = (((αβψ, i + j, t * (-u)) * ((αβ2ψ, i + 2 * j, t * (-u)^2)))⁻¹ := by group
    _ = ⁅((αβ, i, t)), ((ψ, j, -u))⁆⁻¹ := by rw [commutator_of_αβ_ψ_a Fchar hi hj]
    _ = ⁅((αβ, i, t)), ((ψ, j, u))⁻¹⁆⁻¹ := by rw [inv_of_ψ]
  rw [aux, this]
  group

-- 8.145d
@[group_reassoc]
theorem expr_αβ_ψ_as_αβ2ψ_αβψ_ψ_αβ :
  forall_ij_tu 2 1,
    ((αβ, i, t)) * ((ψ, j, u)) = ((αβ2ψ, i + 2 * j, t * u^2)) * ((αβψ, i + j, t * u)) * ((ψ, j, u)) * ((αβ, i, t)) :=
      ↪ by
  intro i j hi hj t u
  rw [←commutator_of_αβ_ψ_b Fchar hi hj]
  group

-- 8.146
@[group_reassoc]
theorem expr_ψ_αβ_as_αβ_αβ2ψ_αβψ_ψ :
  forall_ij_tu αβ ψ,
    ((ψ, j, u)) * ((αβ, i, t)) = ((αβ, i, t)) * ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβψ, i + j, -t * u)) * ((ψ, j, u))
      ↪ := by
  intro i j hi hj t u
  have := calc
    ((αβ2ψ, i + 2 * j, -t * u^2)) * ((αβψ, i + j, -t * u)) = ⁅((αβ, i, -t)), ((ψ, j, u))⁆ := by rw
      ↪ [commutator_of_αβ_ψ_b Fchar hi hj]
    _ = ⁅((αβ, i, t))⁻¹, ((ψ, j, u))⁆ := by rw [inv_of_αβ]
  grw [this]
  rw [←inv_of_αβ]
  group

@[simp, chev_simps]
theorem id_of_αβψ : id_of_root((weakB3LargeGraded F).project, αβψ) := by
  intro i hi
  have := doub_of_αβψ Fchar hi 0
  rw [mul_zero] at this
  exact mul_right_eq_self.1 this

@[simp, chev_simps]
theorem id_of_αβ2ψ : id_of_root((weakB3LargeGraded F).project, αβ2ψ) := by
  intro i hi
  have := doub_of_αβ2ψ Fchar hi 0
  rw [mul_zero] at this
  exact mul_right_eq_self.1 this

theorem defineThenReflect_of_αβ2ψ (g : GradedChevalleyGenerator B3LargePosRoot F)
  (h : g.ζ = αβ2ψ) :
  (weakB3LargeGraded F).project (defineThenReflect (weakB3LargeGraded F) g)
    = (weakB3LargeGraded F).project (FreeGroup.of (reflect g)) := by
  rcases g with ⟨ ζ, i, hi, t ⟩
  simp only at h
  subst ζ
  simp only [defineThenReflect, MonoidHom.coe_comp, Function.comp_apply, FreeGroup.lift.of]
  rw [weakB3LargeGraded]
  simp only
  rw [weak_define]
  simp only [map_commutatorElement, map_mul, map_inv, FreeGroup.map.of, reflect,
    ↪ PositiveRootSystem.height]
  rw [←weakB3LargeGraded, ←expr_αβ2ψ_as_comm_of_α_β2ψ Fchar (by ht) (by ht), mul_one,
    eq_of_deg_eq αβ2ψ (αβ2ψ.height - i)]
  simp only [height]
  nth_rewrite 3 [←(correct_of_split_4_into_1_3 i hi).2.2]
```

```
  have := (correct_of_split_4_into_1_3 i hi).1
  have := (correct_of_split_4_into_1_3 i hi).2.1
  omega
```

Author source module Steinberg.B3Large.EstabAlpha2Beta2Psi

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Large.Basic
import Steinberg.B3Large.EstabAlphaBeta2Psi

namespace Steinberg.B3Large

open Steinberg B3LargePosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)

-- TODO-B: (3/16) is there a better place/file for these theorems?
include Fchar

theorem defineThenReflect_eq_reflect_of_αβ2ψ (g : GradedChevalleyGenerator B3LargePosRoot F) (h : g.ζ =
↪  αβ2ψ) :
  (weakB3LargeGraded F).project (defineThenReflect (weakB3LargeGraded F) g) = (weakB3LargeGraded
  ↪  F).project (FreeGroup.of (reflect g)) := by
  rcases g with ⟨ ζ, i, hi, t ⟩
  simp only at h
  subst ζ
  simp only [defineThenReflect, MonoidHom.coe_comp, Function.comp_apply, FreeGroup.lift.of]
  rw [weakB3LargeGraded]
  simp only [weak_define, map_commutatorElement, FreeGroup.map.of, reflect]
  rw [← weakB3LargeGraded, ← def_of_αβ2ψ]
  simp only [PositiveRootSystem.height, split_4_into_1_3]
  split
  -- this resolves every goal except height 2
  all_goals (simp only [height])
  simp_arith
  repeat rw [←expr_αβ2ψ_as_comm_of_α_β2ψ]
  all_goals assumption

set_option maxHeartbeats 0

-- 8.147a
theorem hom_lift_of_interchange_of_α2β2ψ_a : forall_ijk_tuv,
    ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, 2 * u * v))⁆ = ⁅((αβψ, i + j + k, t * u)), ((βψ, j + k, v))⁆ := by
  intro i j k hi hj hk t u v
  rcases eq_or_ne u 0 with (rfl| hu)
  · chev_simp [id_of_αβψ Fchar]
  rcases eq_or_ne v 0 with (rfl | hv)
  · chev_simp
  have aux := raw_hom_lift_of_interchange_of_α2β2ψ_a hi hj hk (t * u / v) (v / u) u
  rw [eq_of_coef_eq αβ t (by field_simp),
      eq_of_coef_eq β2ψ (2 * u * v) (by field_simp; group),
      eq_of_coef_eq αβ (t := -(t * u / v) * (v / u)) (-t) (by field_simp; group),
      div_mul_cancel₀ v hu] at aux
  grw [aux, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ (add_le_add hi hj) hk]

-- 8.147b
theorem hom_lift_of_interchange_of_α2β2ψ_b : forall_ijk_tuv,
    ⁅((αβψ, i + j + k, t)), ((βψ, j + k, u * v))⁆ = ⁅((αβ2ψ, i + j + 2 * k, 2 * t * u)), ((β, j, v))⁆ := by
  intro i j k hi hj hk t u v
  rcases eq_or_ne u 0 with hu | hu
  · rw [hu, mul_zero, zero_mul, id_of_βψ, id_of_αβ2ψ Fchar]; group
  rcases eq_or_ne v 0 with hv | hv
  · rw [hv, mul_zero, id_of_βψ, id_of_β]; group
  have aux := raw_hom_lift_of_interchange_of_α2β2ψ_b hi hj hk (t / (u * v)) v u
  have : t / (u * v) * v = t / u := by field_simp; group
  rw [this] at aux
  have : -(t / (u * v)) * v = -(t / u) := by field_simp; group
  rw [this] at aux
  grw [← expr_αβψ_as_ψ_αβ_ψ_αβ_ψ (add_le_add hi hj) hk, mul_comm v u] at aux
  field_simp at aux
  rw [aux]; clear aux
  have hijk : i + j + 2 * k ≤ αβ2ψ.height := by ht
  simp_rw [commutatorElement_def]
```

```
  grw [inv_of_αβ2ψ Fchar hijk]
  have : ∀ (c : F), t / (u * v) * (c * v * u * u) = c * t * u := by
      intro c
      grw [mul_comm]
      field_simp
      group
  have h4 := expr_αβ2ψ_as_α_β2ψ_α_β2ψ Fchar (j := j + 2 * k) hi (by ht) (t / (u * v)) (2 * v * u * u)
  rw [← deg_add_assoc _ _ _ _ hijk, this] at h4
  grw [← h4]; clear h4
  have h3 := expr_αβ2ψ_as_β2ψ_α_β2ψ_α Fchar (j := j + 2 * k) hi (by ht) (t / (u * v)) (-2 * v * u * u)
  rw [← deg_add_assoc _ _ _ _ hijk, this] at h3
  chev_simp at h3
  grw [← h3]

-- 8.148
omit Fchar
theorem hom_lift_of_comm_ψ_αβ_β2ψ : forall_ijk_tuv,
    ⁅((ψ, k, t)), ⁅((αβ, i + j, u)), ((β2ψ, j + 2 * k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  rcases eq_or_ne t 0 with ht | ht
  · rw [ht, id_of_ψ]; group
  rcases eq_or_ne v 0 with hv | hv
  · rw [hv, id_of_β2ψ]; group
  have aux := raw_hom_lift_of_comm_of_ψ_αβ_β2ψ hi hj hk (u * t * t / v) (v / (t * t)) t
  have : u * t * t / v * (v / (t * t)) = u := by ring_nf; field_simp
  rw [this] at aux
  have : v / (t * t) * t ^ 2 = v := by ring_nf; field_simp
  rw [this] at aux
  exact aux

-- 8.149
theorem hom_lift_of_comm_αβ_αβ_β2ψ : forall_ijk_tu α β ψ,
    ⁅((αβ, i + j, t)), ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, u))⁆⁆ = 1 ∧
  ⁅((αβ, i + j, t)), ⁅((αβ, i + j, -t)), ((β2ψ, j + 2 * k, u))⁆⁆ = 1 := by
  intro i j k hi hj hk t u
  rcases eq_or_ne u 0 with hu | hu
  · rw [hu, id_of_β2ψ]
    exact ⟨by group, by group⟩
  have aux₁ := raw_hom_lift_of_comm_of_αβ_αβ_β2ψ_a hi hj hk (t / u) u 1
  have aux₂ := raw_hom_lift_of_comm_of_αβ_αβ_β2ψ_b hi hj hk (t / u) u 1
  have : t / u * u = t := by field_simp
  rw [this, pow_two, one_mul, mul_one] at aux₁
  rw [this, pow_two, one_mul, mul_one] at aux₂
  have : -(t / u) * u = -t := by field_simp
  rw [this] at aux₂
  exact ⟨aux₁, aux₂⟩

-- 8.150
theorem hom_lift_of_inv_doub_of_αβ_β2ψ : forall_ijk_tu α β ψ,
  ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, u))⁆ = ⁅((αβ, i + j, -t)), ((β2ψ, j + 2 * k, -u))⁆ ∧
  ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, u))⁆ * ⁅((αβ, i + j, -t)), ((β2ψ, j + 2 * k, u))⁆ = 1 ∧
  ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, u))⁆ * ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, u))⁆ = ⁅((αβ, i + j, 2 * t)),
    ↪  ((β2ψ, j + 2 * k, u))⁆ := by
  intro i j k hi hj hk t u
  rcases eq_or_ne u 0 with hu | hu
  · rw [hu, neg_zero, id_of_β2ψ]
    exact ⟨by group, by group, by group⟩
  have aux₁ := raw_hom_lift_of_inv_doub_of_αβ_β2ψ_a hi hj hk (t / u) u 1
  have aux₂ := raw_hom_lift_of_inv_doub_of_αβ_β2ψ_b hi hj hk (t / u) u 1
  have aux₃ := raw_hom_lift_of_inv_doub_of_αβ_β2ψ_c hi hj hk (t / u) u 1
  have eq1 : (t / u) * u = t := by field_simp
  have eq2 : u * 1^2 = u := by rw [pow_two, mul_one, mul_one]
  have eq3 : -(t / u) * u = -t := by field_simp
  have eq4 : -u * 1^2 = -u := by rw [pow_two, mul_one, mul_one]
  have eq5 : 2 * (t / u) * u = 2 * t := by field_simp
  rw [eq1, eq2, eq3, eq4] at aux₁
  rw [eq1, eq2, eq3] at aux₂
  rw [eq1, eq2, eq5] at aux₃
  exact ⟨aux₁, aux₂, aux₃⟩

-- 8.151
include Fchar
theorem hom_lift_of_inv_doub_of_β_αβ2ψ : forall_ijk_tu α β ψ,
  ⁅((β, j, t)), ((αβ2ψ, i + j + 2 * k, u))⁆ = ⁅((β, j, -t)), ((αβ2ψ, i + j + 2 * k, -u))⁆ ∧
  ⁅((β, j, t)), ((αβ2ψ, i + j + 2 * k, u))⁆ * ⁅((β, j, -t)), ((αβ2ψ, i + j + 2 * k, u))⁆ = 1 ∧
  ⁅((β, j, t)), ((αβ2ψ, i + j + 2 * k, u))⁆ * ⁅((β, j, t)), ((αβ2ψ, i + j + 2 * k, u))⁆ = ⁅((β, j, 2 * t)), ((αβ2ψ, i
    ↪  + j + 2 * k, u))⁆ := by
  intro i j k hi hj hk t u
  rcases eq_or_ne t 0 with (rfl | ht)
  · chev_simp [id_of_β, commutatorElement_one_left, neg_zero, mul_one, mul_zero, and_self]
```

```
  have hijk : i + j + 2 * k ≤ αβ2ψ.height := by ht
  have hjk : j + 2 * k ≤ β2ψ.height := by ht
  have aux₁ := raw_hom_lift_of_inv_doub_of_β_αβ2ψ_a hi hj hk (u / t) t 1
  have aux₂ := raw_hom_lift_of_inv_doub_of_β_αβ2ψ_b hi hj hk (u / t) t 1
  have aux₃ := raw_hom_lift_of_inv_doub_of_β_αβ2ψ_c hi hj hk (u / t) t 1
  field_simp at aux₁ aux₂ aux₃
  rw [neg_div] at aux₁
  have h1 := @expr_αβ2ψ_as_α_β2ψ_α_β2ψ F _ Fchar i (j + 2 * k) hi hjk (-(u/t)) (t)
  have h2 := @expr_αβ2ψ_as_α_β2ψ_α_β2ψ F _ Fchar i (j + 2 * k) hi hjk (u/t) (t)
  norm_num at h1 h2; field_simp at h1 h2
  have eq1 : -u/t = -(u/t) := by field_simp
  simp only [← inv_of_β2ψ, eq1] at h1 h2
  simp only [← inv_of_α] at h2
  have : ((α, i, u / t)) = ((α, i, -(u / t)))⁻¹ := by chev_simp
  rw [this] at h1
  grw [← commutatorElement_def] at h1 h2
  rw [← deg_add_assoc _ _ _ _ hijk] at h1 h2
  grw [h2, h1, aux₂, aux₃, ← eq1, aux₁]

-- 8.152
theorem hom_lift_of_comm_βψ_αβ2ψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 1) (hk : k ≤ 1) (t u : F),
  ⁅((βψ, j + k, t)), ((αβ2ψ, i + j + 2 * k, u))⁆ = 1 := by
  intro i j k hi hj hk t u
  rcases eq_or_ne t 0 with ht | ht
  · rw [ht, id_of_βψ]; group
  have aux := raw_hom_lift_of_comm_of_βψ_αβ2ψ hi hj hk (u / t) t 1
  rw [pow_two, mul_one, mul_one, mul_one] at aux
  have aux' := @expr_αβ2ψ_as_comm_of_α_β2ψ _ _ Fchar i (j + 2 * k) hi (by omega) (u / t) t
  have : ((αβ2ψ, i + (j + 2 * k), u / t * t)) = ((αβ2ψ, i + j + 2 * k, u)) := by field_simp; group
  rw [this] at aux'
  rw [aux', aux]

-- 8.153
theorem hom_lift_of_comm_of_β2ψ_αβ2ψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 1) (hk : k ≤ 1) (t u : F),
  ⁅((β2ψ, j + 2 * k, t)), ((αβ2ψ, i + j + 2 * k, u))⁆ = 1 := by
  intro i j k hi hj hk t u
  rcases eq_or_ne t 0 with ht | ht
  · rw [ht, id_of_β2ψ]; group
  have aux := raw_hom_lift_of_comm_of_β2ψ_αβ2ψ hi hj hk (u / t) t 1
  rw [pow_two, mul_one, mul_one] at aux
  have aux' := @expr_αβ2ψ_as_comm_of_α_β2ψ _ _ Fchar i (j + 2 * k) hi (by omega) (u / t) t
  have : ((αβ2ψ, i + (j + 2 * k), u / t * t))  = ((αβ2ψ, i + j + 2 * k, u)) := by field_simp; group
  rw [this] at aux'
  rw [aux', aux]

-- 8.154
omit Fchar
theorem comm_of_βψ_αβ_β2ψ : forall_ijk_tuv βψ αβ β2ψ,
    ⁅((βψ, i, t)), ⁅((αβ, j, u)), ((β2ψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.2
  grw [commutatorElement_def,
    ← expr_αβ_βψ_as_βψ_αβ hj hi, expr_βψ_β2ψ_as_β2ψ_βψ hi hk,
    ← expr_αβ_βψ_as_βψ_αβ hj hi, expr_βψ_β2ψ_as_β2ψ_βψ hi hk]

@[group_reassoc]
theorem expr_βψ_comm_αβ_β2ψ_as_comm_αβ_β2ψ_βψ : forall_ijk_tuv βψ αβ β2ψ,
    ((βψ, i, t)) * ⁅((αβ, j, u)), ((β2ψ, k, v))⁆ = ⁅((αβ, j, u)), ((β2ψ, k, v))⁆ * ((βψ, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_βψ_αβ_β2ψ hi hj hk t u v)

-- 8.155
theorem comm_of_αβ_βψ_αβψ : forall_ijk_tuv αβ βψ αβψ,
    ⁅((αβ, i, t)), ⁅((βψ, j, u)), ((αβψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.2
  grw [commutatorElement_def, expr_αβ_βψ_as_βψ_αβ hi hj, expr_αβ_αβψ_as_αβψ_αβ hi hk,
  expr_αβ_βψ_as_βψ_αβ hi hj, expr_αβ_αβψ_as_αβψ_αβ hi hk]

@[group_reassoc]
theorem expr_αβ_comm_βψ_αβψ_as_comm_βψ_αβψ_αβ : forall_ijk_tuv αβ βψ αβψ,
    ((αβ, i, t)) * ⁅((βψ, j, u)), ((αβψ, k, v))⁆ = ⁅((βψ, j, u)), ((αβψ, k, v))⁆ * ((αβ, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_αβ_βψ_αβψ hi hj hk t u v)

-- 8.156
theorem comm_of_β_αβ_β2ψ : forall_ijk_tuv β αβ β2ψ,
    ⁅((β, i, t)), ⁅((αβ, j, u)), ((β2ψ, k, v))⁆⁆ = 1 := by
```

```
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.2
  grw [commutatorElement_def, expr_β_αβ_as_αβ_β hi hj, expr_β_β2ψ_as_β2ψ_β hi hk,
  expr_β_αβ_as_αβ_β hi hj, expr_β_β2ψ_as_β2ψ_β hi hk]

@[group_reassoc]
theorem expr_β_comm_αβ_β2ψ_as_comm_αβ_β2ψ_β : forall_ijk_tuv β αβ β2ψ,
    ((β, i, t)) * ⁅((αβ, j, u)), ((β2ψ, k, v))⁆ = ⁅((αβ, j, u)), ((β2ψ, k, v))⁆ * ((β, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_β_αβ_β2ψ hi hj hk t u v)

-- 8.157
theorem comm_of_β_αβψ_βψ : forall_ijk_tuv β αβψ βψ,
    ⁅((β, i, t)), ⁅((αβψ, j, u)), ((βψ, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk t u v
  apply triv_comm_iff_commutes.2
  grw [commutatorElement_def, expr_β_αβψ_as_αβψ_β hi hj, expr_β_βψ_as_βψ_β hi hk,
  expr_β_αβψ_as_αβψ_β hi hj, expr_β_βψ_as_βψ_β hi hk]

@[group_reassoc]
theorem expr_β_comm_αβψ_βψ_as_comm_αβψ_βψ_β : forall_ijk_tuv β αβψ βψ,
    ((β, i, t)) * ⁅((αβψ, j, u)), ((βψ, k, v))⁆ = ⁅((αβψ, j, u)), ((βψ, k, v))⁆ * ((β, i, t)) := by
  intro i j k hi hj hk t u v
  exact triv_comm_iff_commutes.1 (comm_of_β_αβψ_βψ hi hj hk t u v)

include Fchar
-- 8.158 (revised)
theorem sufficient_conditions_for_comm_of_αβψ_and_βψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 2) (hj : j ≤ 1) (hk : k ≤ 2)
  (h35a : ∀ (t u v : F), ⁅((ψ, j, t)), ⁅((αβ, i, u)), ((β2ψ, j + k, v))⁆⁆ = 1)
  (h35b : ∀ (t u : F), ⁅((αβ, i, t)), ⁅((αβ, i, t)), ((β2ψ, j + k, u))⁆⁆ = 1 ∧ ⁅((αβ, i, t)), ⁅((αβ, i, -t)), ((β2ψ,
    ↪ j + k, u))⁆⁆ = 1)
  (h35c : ∀ (t u : F), ⁅((αβ, i, t)), ((β2ψ, j + k, u))⁆ = ⁅((αβ, i, -t)), ((β2ψ, j + k, -u))⁆)
  (h35d : ∀ (t u : F), ⁅((αβ, i, t)), ((β2ψ, j + k, u))⁆ * ⁅((αβ, i, t)), ((β2ψ, j + k, u))⁆ = ⁅((αβ, i, t)), ((β2ψ,
    ↪ j + k, 2 * u))⁆),
  ∀ (t u v : F), ⁅((αβψ, i + j, t * u)), ((βψ, k, v))⁆ = ⁅((αβ, i, t)), ((β2ψ, j + k, 2 * u * v))⁆ := by
  intro i j k hi hj hk h35a h35b h35c h35d t u v
  have h35a' := fun t' u' v' ↦ triv_comm_iff_commutes.1 (h35a t' u' v')
  have h35b₁ := fun t' u' ↦ triv_comm_iff_commutes.1 (h35b t' u').left
  have h35b₂ := fun t' u' ↦ triv_comm_iff_commutes.1 (h35b t' u').right
  have h35b₃ := fun t' u' ↦ triv_comm_iff_commutes.1 (h35b (-t') u').right
  norm_num at h35b₃
  have aux₀ : -(2 * (u / 2) * v) = -(u * v) := by ring_nf; field_simp
  have eq36 : ((β2ψ, j + k, -(u * v))) * ((αβ, i, t)) = ((αβ, i, t)) * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((β2ψ,
    ↪ j + k, -u * v)) := by calc
    ((β2ψ, j + k, -(u * v))) * ((αβ, i, t)) = ((αβ, i, t)) * ⁅((αβ, i, -t)), ((β2ψ, j + k, -(u * v)))⁆ * ((β2ψ, j +
      ↪ k, -(u * v))) := by
      rw [← inv_of_αβ, ← inv_of_β2ψ]; group
    _ = ((αβ, i, t)) * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((β2ψ, j + k, -u * v)) := by grw [h35c]
  have eq37 : ((αβ, i, -t)) * ((β2ψ, j + k, -(u * v))) = ((β2ψ, j + k, -(u * v))) * ((αβ, i, -t)) * ⁅((αβ, i, t)),
    ↪ ((β2ψ, j + k, u * v))⁆ := by
    rw [← inv_of_αβ, ← inv_of_β2ψ]; group
  have := calc
    ((αβψ, i + j, t * u)) * ((βψ, k, v)) = ((ψ, j, -u/2)) * ((αβ, i, t)) * ((ψ, j, u)) * ((αβ, i, -t)) * ((ψ, j, -u/2))
      ↪ * ((βψ, k, v)) := by rw [expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi hj]
    _ = ((βψ, k, v)) * ((ψ, j, -u/2)) * ((β2ψ, j + k, -u * v)) * ((αβ, i, t)) * ((β2ψ, j + k, 2 * u * v)) * ((ψ, j,
      ↪ u)) * ((αβ, i, -t)) * ((β2ψ, j + k, -u * v)) * ((ψ, j, -u/2)) := by
      grw [expr_ψ_βψ_as_βψ_β2ψ_ψ hj hk, aux₀, expr_αβ_βψ_as_βψ_αβ hi hk, expr_ψ_βψ_as_βψ_β2ψ_ψ hj hk,
        ↪ expr_αβ_βψ_as_βψ_αβ hi hk, expr_ψ_βψ_as_βψ_ψ_β2ψ hj hk, aux₀]
    _ = ((βψ, k, v)) * ((ψ, j, -u/2)) * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((αβ, i, t)) * ((β2ψ, j + k, -u * v))
      ↪ * ((β2ψ, j + k, 2 * u * v)) * ((ψ, j, u)) * ((β2ψ, j + k, -(u * v))) * ((αβ, i, -t)) * ⁅((αβ, i, t)), ((β2ψ,
      ↪ j + k, u * v))⁆ * ((ψ, j, -u/2)) := by
      grw [eq36, eq36, eq37, h35b₁, h35b₂]
    _ = ((βψ, k, v)) * ((ψ, j, -u/2)) * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((αβ, i, t)) * ((ψ, j, u)) * ((αβ, i,
      ↪ -t)) * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((ψ, j, -u/2)) := by
      grw [expr_ψ_β2ψ_as_β2ψ_ψ hj (add_le_add hj hk)]; ring_nf
      rw [id_of_β2ψ, one_mul]
    _ = ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((βψ, k, v)) * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((ψ, j, -u/2))
      ↪ * ((αβ, i, t)) * ((ψ, j, u)) * ((αβ, i, -t)) * ((ψ, j, -u/2)) := by
      grw [h35a', expr_βψ_comm_αβ_β2ψ_as_comm_αβ_β2ψ_βψ hk hi (add_le_add hj hk), ← h35a', ← h35b₁, ←
        ↪ h35a', h35b₃]
    _ = ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ⁅((αβ, i, t)), ((β2ψ, j + k, u * v))⁆ * ((βψ, k, v)) * ((ψ, j, -u/2))
      ↪ * ((αβ, i, t)) * ((ψ, j, u)) * ((αβ, i, -t)) * ((ψ, j, -u/2)) := by
      grw [expr_βψ_comm_αβ_β2ψ_as_comm_αβ_β2ψ_βψ hk hi (add_le_add hj hk)]
    _ = ⁅((αβ, i, t)), ((β2ψ, j + k, 2 * u * v))⁆ * ((βψ, k, v)) * ((αβψ, i + j, t * u)) := by
      grw [h35d, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi hj]
  exact eq_comm_of_reorder_left this

lemma interchange_of_α2β2ψ_aux :
```

```
  ∀ (t u : F), ⁅((αβ, 1, t)), ((β2ψ, 1, u))⁆ = ⁅((αβ, 0, t)), ((β2ψ, 2, u))⁆ := by
    intro t u
    have := @hom_lift_of_interchange_of_α2β2ψ_a F _ Fchar 0 1 0 (by norm_num) (by norm_num) (by norm_num)
    norm_num at this
    have this' := @hom_lift_of_interchange_of_α2β2ψ_a F _ Fchar 0 0 1 (by norm_num) (by norm_num) (by
    ↪  norm_num)
    norm_num at this'
    specialize this t (u/2) 1; norm_num at this; field_simp at this
    specialize this' t (u/2) 1; norm_num at this'; field_simp at this'
    exact Eq.trans this (Eq.symm this')

-- 8.159a
theorem partial_A_interchange_of_α2β2ψ_a :
  ∀ (t u v : F),
  ⁅((αβψ, 0, t * u)), ((βψ, 2, v))⁆ = ⁅((αβ, 0, t)), ((β2ψ, 2, 2 * u * v))⁆ := by
  have h1 := @hom_lift_of_comm_ψ_αβ_β2ψ F _ 0 1 0 (by ht) (by ht) (by ht)
  have h2 := @hom_lift_of_comm_αβ_αβ_β2ψ F _ 0 0 1 (by ht) (by ht) (by ht)
  have h3 := @hom_lift_of_inv_doub_of_αβ_β2ψ F _ 0 0 1 (by ht) (by ht) (by ht)
  have h4 := @hom_lift_of_interchange_of_α2β2ψ_a F _ Fchar 0 0 1 (by ht) (by ht) (by ht)
  have := @sufficient_conditions_for_comm_of_αβψ_and_βψ _ _ Fchar 0 0 2 (by ht) (by ht) (by ht)
  norm_num at this
  apply this
  · intro t u v
    have := @interchange_of_α2β2ψ_aux _ _ Fchar u v
    rw [← this]
    specialize h1 t u v; norm_num at h1; exact h1
  · exact h2
  · exact fun t u ↦ (h3 t u).left
  · intro t u
    have h4a := h4 t 1 u; norm_num at h4a
    have h4b := h4 (2*t) (1/2) u; norm_num at h4b; field_simp at h4b
    specialize h3 t u
    grw [h3.right.right, h4a, h4b]

-- 8.159b
theorem partial_A_interchange_of_α2β2ψ_b :
  ∀ (t u v : F),
  ⁅((αβψ, 2, t * u)), ((βψ, 0, v))⁆ = ⁅((αβ, 1, t)), ((β2ψ, 1, 2 * u * v))⁆ := by
  have := @sufficient_conditions_for_comm_of_αβψ_and_βψ _ _ Fchar 1 1 0 (by norm_num) (by norm_num) (by
  ↪  norm_num)
  have h := @interchange_of_α2β2ψ_aux _ _ Fchar
  have h1 := @hom_lift_of_comm_ψ_αβ_β2ψ F _ 0 0 1 (by norm_num) (by norm_num) (by norm_num)
  have h2 := @hom_lift_of_comm_αβ_αβ_β2ψ F _ 0 1 0 (by norm_num) (by norm_num) (by norm_num)
  have h3 := @hom_lift_of_inv_doub_of_αβ_β2ψ F _ 0 0 1 (by norm_num) (by norm_num) (by norm_num)
  have h4 := @hom_lift_of_interchange_of_α2β2ψ_a F _ Fchar 0 0 1 (by norm_num) (by norm_num) (by norm_num)
  norm_num at this
  apply this
  · intro t u v
    rw [h u v]
    specialize h1 t u v; norm_num at h1; exact h1
  · exact h2
  · intro t u
    have h1a := h t u
    have h1b := h (-t) (-u)
    grw [h1a, h1b]
    exact (h3 t u).left
  · intro t u
    specialize h3 t u; norm_num at h3
    rw [h t u]
    grw [h3.right.right]
    rw [h t (2*u)]
    have h4a := h4 t 1 u; norm_num at h4a
    have h4b := h4 (2*t) (1/2) u; norm_num at h4b; field_simp at h4b
    grw [h4a, h4b]

-- 8.160
include Fchar in
theorem more_sufficient_conditions_for_comm_of_αβψ_and_βψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 3) (hj : j ≤ 1) (hk : k ≤ 1)
  (h38a : ∀ (t u v : F), ⁅((β, k, t)), ⁅((αβ2ψ, i + j, u)), ((β, k, v))⁆⁆ = 1)
  (h38b : ∀ (t u v : F), ⁅((ψ, j, t)), ⁅((αβ2ψ, i + j, u)), ((β, k, v))⁆⁆ = 1)
  (h38c : ∀ (t u : F), ⁅((β, k, u)), ((αβ2ψ, i + j, -t))⁆ = ⁅((αβ2ψ, i + j, t)), ((β, k, u))⁆)
  (h38d : ∀ (t u : F), ⁅((αβ2ψ, i + j, t)), ((β, k, u))⁆ * ⁅((αβ2ψ, i + j, t)), ((β, k, u))⁆ = ⁅((αβ2ψ, i + j, 2 *
  ↪  t)), ((β, k, u))⁆),
    ∀ (t u v : F), ⁅((αβψ, i, t)), ((βψ, j + k, u * v))⁆ = ⁅((αβ2ψ, i + j, 2 * t * u)), ((β, k, v))⁆ := by
  intro i j k hi hj hk h38a h38b h38c h38d t u v
  have hi' : i ≤ αβψ.height := by ht
  have hj' : j ≤ ψ.height := by ht
  have hk' : k ≤ β.height := by ht
```

```
  have h39 : ((αβ2ψ, i + j, t * u)) * ((β, k, v)) = ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ * ((β, k, v)) * ((αβ2ψ, i +
    ↪  j, t * u)) := by group
  have h40 : ((β, k, -v)) * ((αβ2ψ, i + j, t * u))  = ((αβ2ψ, i + j, t * u)) * ((β, k, -v)) * ⁅((αβ2ψ, i + j, t *
    ↪  u)), ((β, k, v))⁆ := by
    simp [← h38c, commutatorElement_def]
    have h1 : ((β, k, v))⁻¹ = ((β, k, -v)) := by chev_simp
    have h2 : ((αβ2ψ, i + j, t * u))⁻¹ = ((αβ2ψ, i + j, -(t * u))) := by
      rw [inv_of_αβ2ψ Fchar (by ht)]
    grw [← h1, ← h2]
  have h : i + j ≤ αβ2ψ.height := by simp [height]; omega
  apply reorder_left_iff_eq_comm.mp
  nth_rw 1 [expr_βψ_as_ψ_β_ψ_β_ψ]
  grw [expr_αβψ_ψ_as_ψ_αβ2ψ_αβψ,
        ← expr_β_αβψ_as_αβψ_β,
        expr_αβψ_ψ_as_αβ2ψ_ψ_αβψ,
        ← expr_β_αβψ_as_αβψ_β]
  grw [expr_αβψ_ψ_as_αβ2ψ_ψ_αβψ]
  field_simp [add_comm, mul_comm, ← mul_assoc]
  grw [h39, h40]
  all_goals (try assumption)
  have : ((ψ, j, u)) * ((αβ2ψ, i + j, t * u)) = ((αβ2ψ, i + j, t * u)) * ((ψ, j, u)) := by
    rw [triv_comm_iff_commutes.1]
    rw [comm_of_ψ_αβ2ψ]
    exact Fchar
  grw [this]; clear this
  have : ((αβ2ψ, i + j, t * u)) * ((αβ2ψ, i + j, -(t * u * 2))) * ((αβ2ψ, i + j, t * u)) = 1 := by
    have : -(t * u * 2) = 2 * (-t * u) := by ring
    rw [this, ← doub_of_αβ2ψ Fchar h]
    group
    rw [← inv_of_αβ2ψ Fchar (by ht)]
    simp
  grw [this]; clear this
  grw [h38a, h38b]
  have h1 : ((ψ, j, -(u / 2))) * ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ = ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ * ((ψ,
    ↪  j, -(u / 2))) := by
    rw [triv_comm_iff_commutes.1]
    exact h38b (-(u / 2)) (t * u) v
  have h2 : ((ψ, j, u)) * ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ = ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ * ((ψ, j, u))
    ↪  := by
    rw [triv_comm_iff_commutes.1]
    exact h38b u (t * u) v
  have h3 : ((β, k, -v)) * ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ = ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ * ((β, k,
    ↪  -v)) := by
    rw [triv_comm_iff_commutes.1]
    exact h38a (-v) (t * u) v
  have h4 : ((β, k, v)) * ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ = ⁅((αβ2ψ, i + j, t * u)), ((β, k, v))⁆ * ((β, k, v))
    ↪  := by
    rw [triv_comm_iff_commutes.1]
    exact h38a v (t * u) v
  grw [h1, h2, h3, h2, h4, h1, h38d]
  have : ((βψ, j + k, u * v)) = ((ψ, j, -(u / 2))) * ((β, k, v)) * ((ψ, j, u)) * ((β, k, -v)) * ((ψ, j, -(u / 2))) :=
    ↪  by
    rw [← neg_div, ← expr_βψ_as_ψ_β_ψ_β_ψ Fchar hj hk u v, mul_comm]
  have h5 : 2 * t * u = t * u * 2 := by
    ring
  grw [h5, ← this]

-- 8.161
theorem sufficient_conditions_for_comm_of_αβ2ψ_and_β2ψ :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 3) (hj : j ≤ 1) (hk : k ≤ 3)
    (hyp : ∀ (t u : F), ⁅((β2ψ, k, t)), ((αβ2ψ, i + j, u))⁆ = 1),
    ∀ (t u : F), ⁅((β2ψ, i, t)), ((αβ2ψ, j + k, u))⁆ = 1 := by
  intro i j k hi hj hk hyp t u
  have hyp' := fun t' u' ↦ triv_comm_iff_commutes.1 (hyp t' u')
  apply triv_comm_iff_commutes.2
  -- expand αβ2ψ element as a product of α and β2ψ elements
  rw [←mul_one u, expr_αβ2ψ_as_α_β2ψ_α_β2ψ Fchar hj hk]
  -- commute the β2ψ element fully to the left (working on RHS)
  grw [expr_β2ψ_β2ψ_as_β2ψ_β2ψ, expr_α_β2ψ_as_β2ψ_αβ2ψ_α (t := -u) Fchar hj hi,
  expr_β2ψ_β2ψ_as_β2ψ_β2ψ, expr_α_β2ψ_as_β2ψ_α_αβ2ψ Fchar hj hi]
  rw [eq_of_deg_eq αβ2ψ (i + j) (add_comm j i), eq_of_deg_eq αβ2ψ (i + j) (add_comm j i)]
  rw [←hyp', ← inv_of_αβ2ψ Fchar (add_le_add hi hj)]
  group

-- 8.162a
include Fchar in
theorem partial_comm_of_β2ψ_αβ2ψ_a :
  ∀ (t u : F), ⁅((β2ψ, 2, t)), ((αβ2ψ, 1, u))⁆ = 1 := by
  have := sufficient_conditions_for_comm_of_αβ2ψ_and_β2ψ (F := F) (i := 2) (j := 0) (k := 1) (by ht)
  norm_num at this
```

```
  apply this
  have := hom_lift_of_comm_of_β2ψ_αβ2ψ (i := 1) (j := 1) (k := 0) (by ht)
  norm_num at this
  exact this

-- 8.162b
include Fchar in
theorem partial_comm_of_β2ψ_αβ2ψ_b :
  ∀ (t u : F), ⁅((β2ψ, 0, t)), ((αβ2ψ, 2, u))⁆ = 1 := by
  have := @sufficient_conditions_for_comm_of_αβ2ψ_and_β2ψ F _ Fchar 0 1 1 (by ht) (by norm_num) (by
    ↪ norm_num)
  norm_num at this
  apply this
  have := @hom_lift_of_comm_of_β2ψ_αβ2ψ F _ Fchar 0 1 0 (by norm_num) (by norm_num) (by norm_num)
  norm_num at this
  exact this

-- 8.163
theorem sufficient_conditions_for_comm_of_ψ_and_αβ2ψ_β :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 4) (hk : k ≤ 1)
    (h41a : ∀ (t u : F), ⁅((β2ψ, 2 * i + k, t)), ((αβ2ψ, j, u))⁆ = 1)
    (h41b : ∀ (t u : F), ⁅((βψ, i + k, t)), ((αβ2ψ, j, u))⁆ = 1),
    ∀ (t u v : F), ⁅((ψ, i, t)), ⁅((αβ2ψ, j, u)), ((β, k, v))⁆⁆ = 1 := by
  intro i j k hi hj hk h41a h41b t u v
  have h₁ := fun t' u' ↦ triv_comm_iff_commutes.1 (h41a t' u')
  have h₂ := fun t' u' ↦ triv_comm_iff_commutes.1 (h41b t' u')
  apply triv_comm_iff_commutes.2
  -- expand the commutator (work on LHS)
  rw [commutatorElement_def, inv_of_αβ2ψ Fchar hj, inv_of_β]
  -- move the ψ element fully to the right
  grw [expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar, expr_ψ_β_as_β_βψ_β2ψ_ψ hk hi, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar,
    ↪ expr_ψ_β_as_β2ψ_βψ_β_ψ hk hi]
  -- use assumptions to cancel stuff
  rw [eq_of_deg_eq β2ψ (2 * i + k) (by ht), mul_assoc ((βψ, k + i, -(v * t))), h₁]
  nth_rewrite 2 [eq_of_deg_eq β2ψ (2 * i + k) (by ht)]
  grw [rfl]
  rw [eq_of_deg_eq βψ (i + k) (by ht), h₂]
  nth_rewrite 2 [eq_of_deg_eq βψ (i + k) (by ht)]
  chev_simp

-- 8.164
include Fchar
theorem partial_comm_of_ψ_αβ2ψ_β :
  ∀ (t u v : F), ⁅((ψ, 1, v)), ⁅((αβ2ψ, 1, t)), ((β, 0, u))⁆⁆ = 1 := by
  intro t u v
  have := @sufficient_conditions_for_comm_of_ψ_and_αβ2ψ_β F _ Fchar 1 1 0 (by norm_num) (by norm_num) (by
    ↪ norm_num)
  norm_num at this
  apply this
  exact @partial_comm_of_β2ψ_αβ2ψ_a F _ Fchar
  have := @hom_lift_of_comm_βψ_αβ2ψ F _ Fchar 0 1 0 (by norm_num) (by norm_num) (by norm_num)
  norm_num at this
  exact this

-- 8.165
theorem partial_B_interchange_of_α2β2ψ :
  ∀ (t u v : F), ⁅((αβψ, 0, t)), ((βψ, 1, u * v))⁆ = ⁅((αβ2ψ, 1, 2 * t * u)), ((β, 0, v))⁆ := by
  have h := @hom_lift_of_inv_doub_of_β_αβ2ψ F _ Fchar 1 0 0 (by norm_num) (by norm_num) (by norm_num)
  norm_num at h
  have := @more_sufficient_conditions_for_comm_of_αβψ_and_βψ F _ Fchar 0 1 0 (by norm_num) (by norm_num)
    ↪ (by norm_num)
  norm_num at this
  apply this
  · intro t u v
    have h1 := @hom_lift_of_interchange_of_α2β2ψ_a F _ Fchar 1 0 0 (by norm_num) (by norm_num) (by
      ↪ norm_num) u (1/2) v
    have h2 := @hom_lift_of_interchange_of_α2β2ψ_b F _ Fchar 1 0 0 (by norm_num) (by norm_num) (by
      ↪ norm_num) (u/2) 1 v
    norm_num at h1; norm_num at h2; field_simp at h1; field_simp at h2
    rw [h2] at h1
    have := @comm_of_β_αβ_β2ψ F _ 0 1 0 (by norm_num) (by norm_num) (by norm_num) t u v
    rw [h1] at this
    exact this
  · intro t u v
    have := @partial_comm_of_ψ_αβ2ψ_β F _ Fchar u v t
    exact this
  · intro t u
    nth_rw 2 [← comm_swap]
    rw [← inv_inj, inv_inv]
    apply Eq.symm
```

```
    apply eq_inv_iff_mul_eq_one.mpr
    rcases h (-u) (-t) with ⟨h1, ⟨h2, _⟩⟩
    norm_num at h1; norm_num at h2
    rw [← h1]
    exact h2
  · intro t u
    rcases h (-u) t with ⟨h1, ⟨h2, h3⟩⟩
    norm_num at h1
    apply mul_eq_one_iff_eq_inv.mp at h2
    simp only [neg_neg, commutatorElement_inv] at h2
    grw [← h2, h3]
    rcases h (-2*u) t with ⟨_, ⟨h4, _⟩⟩
    apply mul_eq_one_iff_eq_inv.mp at h4
    simp only [neg_neg, commutatorElement_inv] at h4
    norm_num at h4
    rw [h4]
    have h5 := @hom_lift_of_interchange_of_α2β2ψ_b F _ Fchar 1 0 0 (by norm_num) (by norm_num) (by
    ↪  norm_num)
    norm_num at h5
    have h5a := h5 t 1 u; norm_num at h5a
    have h5b := h5 (t) (1/2) (2*u); field_simp at h5b;
    rw [←h5a, ←h5b]

-- 8.166
theorem sufficient_conditions_for_comm_of_αβ_and_β2ψ :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 2) (hj : j ≤ 1) (hk : k ≤ 2)
    (h42a : ∀ (t u : F), ⁅((αβ2ψ, i + 2 * j, t)), ((βψ, k, u))⁆ = 1)
    (h42b : ∀ (t u v : F), ⁅((ψ, j, t)), ⁅((αβψ, i + j, u)), ((βψ, k, v))⁆⁆ = 1),
    ∀ (t u v : F), ⁅((αβ, i, t)), ((β2ψ, j + k, 2 * u * v))⁆ = ⁅((αβψ, i + j, t * u)), ((βψ, k, v))⁆ := by
  intro i j k hi hj hk h42a h42b t u v
  have h₁ := fun t u ↦ triv_comm_iff_commutes.1 (h42a t u)
  have h₂ := fun t u v ↦ triv_comm_iff_commutes.1 (h42b t u v)
  apply eq_comm_of_reorder_left
  -- expand β2ψ element as product of βψ and ψ elements
  rw [expr_β2ψ_as_ψ_βψ_ψ_βψ hj hk]
  -- move αβ element fully to the right
  grw [expr_αβ_ψ_as_ψ_αβψ_αβ2ψ_αβ Fchar hi hj, expr_αβ_βψ_as_βψ_αβ hi hk,
  expr_αβ_ψ_as_αβ2ψ_αβψ_ψ_αβ Fchar hi hj, expr_αβ_βψ_as_βψ_αβ hi hk]
  -- by h₁, commute αβ2ψ elements together and cancel them
  grw [h₁ (-(t * u * u))]
  rw [←inv_of_αβ2ψ Fchar (by ht)]
  grw [rfl, comm_left ((αβψ, i + j, t * u)), ← inv_of_αβψ (by ht), h₂]

-- 8.167
include Fchar
theorem sufficient_conditions_for_comm_of_αβ2ψ_and_βψ :
  ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 4) (hj : j ≤ 1) (hk : k ≤ 1)
  (h44a : ∀ (t u v : F), ⁅⁅((αβ2ψ, i, t)), ((β, j, u))⁆, ((ψ, k, v))⁆ = 1)
  (h44b : ∀ (t u : F), ⁅((β, j, -u)), ((αβ2ψ, i, t))⁆ = ⁅((αβ2ψ, i, t)), ((β, j, u))⁆)
  (h44c : ∀ (t u : F), ⁅((αβ2ψ, i, t)), ((β, j, u))⁆ * ⁅((αβ2ψ, i, -t)), ((β, j, u))⁆ = 1),
  ∀ (t u : F), ⁅((αβ2ψ, i, t)), ((βψ, j + k, u))⁆ = 1 := by
  intro i j k hi hj hk h44a h44b h44c t u
  have := inv_of_αβ2ψ Fchar (by exact hi) (-t); rw [neg_neg] at this
  have eq45 := calc ((αβ2ψ, i, t)) * ((β, j, u))
    _ = ((β, j, u)) * ⁅((β, j, -u)), ((αβ2ψ, i, t))⁆ * ((αβ2ψ, i, t)) := by
      rw [comm_mid_rev, inv_of_β]
    _ = ((β, j, u)) * ⁅((αβ2ψ, i, t)), ((β, j, u))⁆ * ((αβ2ψ, i, t)) := by
      rw [h44b t u]
  have eq46 := calc ((αβ2ψ, i, t)) * ((β, j, -u))
    _ = ⁅((αβ2ψ, i, t)), ((β, j, -u))⁆ * ((β, j, -u)) * ((αβ2ψ, i, t)) := by
      apply comm_left
  have h44d : ∀ t u, ⁅((αβ2ψ, i, -t)), ((β, j, u))⁆ * ⁅((αβ2ψ, i, t)), ((β, j, u))⁆ = 1 := by
    intro t u
    have := h44c (-t) (u)
    norm_num at this
    exact this
  have h := expr_βψ_as_ψ_β_ψ_β_ψ Fchar (i := k) (j := j) (by ht) hj 1 u
  norm_num at h; rw [eq_of_deg_eq βψ (j + k)] at h
  have := calc ((αβ2ψ, i, t)) * ((βψ, j + k, u))
    _ = ((αβ2ψ, i, t)) * ((ψ, k, -1/2)) * ((β, j, u)) * ((ψ, k, 1)) * ((β, j, -u)) * ((ψ, k, -1/2)) := by
      grw [h]
    _ = ((ψ, k, -1/2)) * ((β, j, u)) * ⁅((αβ2ψ, i, t)), ((β, j, u))⁆ * ((ψ, k, 1)) * ⁅((αβ2ψ, i, -t)), ((β, j, u))⁆ *
    ↪  ((β, j, -u)) * ((ψ, k, -1/2)) * ((αβ2ψ, i, t)) := by
      have h1 : ∀ u, ((ψ, k, u)) * ((αβ2ψ, i, t)) = ((αβ2ψ, i, t)) * ((ψ, k, u)) := by
        intro u
        rw [triv_comm_iff_commutes.mp]
        apply comm_of_ψ_αβ2ψ Fchar
      have := calc ((αβ2ψ, i, t)) * ⁅((αβ2ψ, i, -t)), ((β, j, u))⁆
        _ = ((αβ2ψ, i, t)) * ⁅((β, j, -u)), ((αβ2ψ, i, -t))⁆ := by rw [(h44b (-t) (u))]
        _ = ((αβ2ψ, i, t)) * ((β, j, -u)) * ((αβ2ψ, i, -t)) * ((β, j, u)) * ((αβ2ψ, i, t)) := by
```

```
          grw [commutatorElement_def, inv_of_αβ2ψ Fchar (by ht)]
        _ = ⁅((αβ2ψ, i, t)), ((β, j, -u))⁆ * ((αβ2ψ, i, t)) := by
          grw [commutatorElement_def, inv_of_αβ2ψ Fchar (by ht)]
      grw [← h1, eq45, ← h1, eq46, ← h1]
      nth_rw 2 [← comm_swap]
      apply mul_eq_one_iff_eq_inv.mp
      have := h44b (-t) (-u)
      norm_num at this
      grw [this]
      have := h44d (-t) (-u)
      norm_num at this
      exact this
    _ = ((ψ, k, -1/2)) * ((β, j, u)) * ((ψ, k, 1)) * ((β, j, -u)) * ((ψ, k, -1/2)) * ((αβ2ψ, i, t)) := by
      have := fun t u v ↦ (triv_comm_iff_commutes.mp (h44a t u v))
      grw [this, this, h44c]
    _ = ((βψ, j + k, u)) * ((αβ2ψ, i, t)) := by
      grw [← h]
  exact triv_comm_iff_commutes.mpr this
  exact Nat.add_comm k j

private lemma partial_comm_of_βψ_αβ2ψ_help : ∀ t u : F,
    ⁅((αβ2ψ, 2, t)), ((β, 0, u))⁆ = ⁅((αβ, 1, t)), ((β2ψ, 1, u))⁆ := by
  intro t u
  have : t = 2 * t * (1 / 2) := by field_simp
  rw [this, ←@hom_lift_of_interchange_of_α2β2ψ_b F _ Fchar 0 0 1 (by trivial) (by trivial) (by trivial),
    ←mul_one t, ←@hom_lift_of_interchange_of_α2β2ψ_a F _ Fchar 0 1 0 (by trivial) (by trivial) (by
    ↪ trivial)]
  field_simp

private lemma partial_comm_of_βψ_αβ2ψ_help' : ∀ t u : F,
    ⁅((β, 0, u)), ((αβ2ψ, 2, t))⁆ = ⁅((β2ψ, 1, u)), ((αβ, 1, t))⁆ := by
  intro t u
  apply inv_inj.1
  rw [comm_swap, comm_swap]
  apply partial_comm_of_βψ_αβ2ψ_help Fchar

-- 8.168
theorem partial_comm_of_βψ_αβ2ψ :
  ∀ (t u : F), ⁅((αβ2ψ, 2, t)), ((βψ, 0, u))⁆ = 1 := by
  apply @sufficient_conditions_for_comm_of_αβ2ψ_and_βψ F _ Fchar 2 0 0 (by trivial) (by trivial) (by
    ↪ trivial)
  · intro t u v
    rw [partial_comm_of_βψ_αβ2ψ_help Fchar, triv_comm_symm, @hom_lift_of_comm_ψ_αβ_β2ψ F _ 0 1 0 (by
      ↪ trivial) (by trivial) (by trivial)]
  · intro t u
    rw [partial_comm_of_βψ_αβ2ψ_help Fchar, partial_comm_of_βψ_αβ2ψ_help' Fchar]
    apply (mul_left_inj (⁅((β2ψ, 1, -u)), ((αβ, 1, t))⁆⁻¹)).1
    rw [mul_inv_cancel, comm_swap, (@hom_lift_of_inv_doub_of_αβ_β2ψ F _ 0 1 0 (by trivial) (by trivial)
      ↪ (by trivial) t u).1]
    nth_rewrite 2 [←neg_neg t]
    rw [(@hom_lift_of_inv_doub_of_αβ_β2ψ F _ 0 1 0 (by trivial) (by trivial) (by trivial) (-t) (-u)).2.1]
  · intro t u
    rw [partial_comm_of_βψ_αβ2ψ_help Fchar, partial_comm_of_βψ_αβ2ψ_help Fchar,
    (@hom_lift_of_inv_doub_of_αβ_β2ψ F _ 0 1 0 (by trivial) (by trivial) (by trivial) t u).2.1]

-- 8.169a
theorem partial_C_interchange_of_α2β2ψ_a :
  ∀ (t u v : F), ⁅((αβ, 0, t)), ((β2ψ, 1, 2 * u * v))⁆ = ⁅((αβψ, 1, t * u)), ((βψ, 0, v))⁆ := by
  apply @sufficient_conditions_for_comm_of_αβ_and_β2ψ F _ Fchar 0 1 0 (by trivial) (by trivial) (by
    ↪ trivial)
  · exact @partial_comm_of_βψ_αβ2ψ F _ Fchar
  intro t u v
  have := @hom_lift_of_interchange_of_α2β2ψ_b F _ Fchar 1 0 0 (by trivial) (by trivial) (by trivial) u 1 v
  norm_num at this;
  grw [this, partial_comm_of_ψ_αβ2ψ_β Fchar]

-- 8.169b
theorem partial_C_interchange_of_α2β2ψ_b :
  ∀ (t u v : F), ⁅((αβ, 2, t)), ((β2ψ, 0, 2 * u * v))⁆ = ⁅((αβψ, 2, t * u)), ((βψ, 0, v))⁆ := by
  apply @sufficient_conditions_for_comm_of_αβ_and_β2ψ F _ Fchar 2 0 0 (by trivial) (by trivial) (by
    ↪ trivial)
  · exact @partial_comm_of_βψ_αβ2ψ F _ Fchar
  intro t u v
  rw [←one_mul u, partial_A_interchange_of_α2β2ψ_b Fchar,
  @hom_lift_of_comm_ψ_αβ_β2ψ F _ 0 1 0 (by trivial) (by trivial) (by trivial)]

-- 8.170
theorem sufficient_conditions_for_comm_of_αβ2ψ_and_β :
  ∀ {i j k : ℕ} (hi : i ≤ 3) (hj : j ≤ 1) (hk : k ≤ 1)
  (h47a : ∀ (t u : F), ⁅((αβψ, i, t)), ((β2ψ, 2 * j + k, u))⁆ = 1)
```

```
  (h47b : ∀ (t u v : F), ⁅⁅((αβψ, i, t)), ((βψ, j + k, u))⁆, ((ψ, j, v))⁆ = 1)
  (h47c : ∀ (t u : F), ⁅((βψ, j + k, -u)), ((αβψ, i, t))⁆ = ⁅((αβψ, i, t)), ((βψ, j + k, u))⁆),
  ∀ (t u v : F), ⁅((αβ2ψ, i + j, t * u)), ((β, k, 2 * v))⁆ = ⁅((αβψ, i, t)), ((βψ, j + k, u * v))⁆ := by
  intro i j k hi hj hk hyp₁ hyp₂ hyp₃ t u v
  have h₁ := fun t u ↦ triv_comm_iff_commutes.1 (hyp₁ t u)
  have h₂ := fun t u v ↦ triv_comm_iff_commutes.1 (hyp₂ t u v)
  apply eq_comm_of_reorder_left
  -- expand αβ2ψ as product of αβψ and ψ elements
  rw [eq_of_coef_eq αβ2ψ (-2 * (-u/2) * t) (by ring_nf; field_simp), expr_αβ2ψ_as_ψ_αβψ_ψ_αβψ Fchar hi hj]
  -- commute β fully to the left
  grw [←expr_β_αβψ_as_αβψ_β, expr_ψ_β_as_β_β2ψ_βψ_ψ hk hj, ←expr_β_αβψ_as_αβψ_β, expr_ψ_β_as_β_ψ_βψ_β2ψ hk
    ↪ hj]
  -- by hyp₁, commute the β2ψ elements across αβψ and cancel them
  rw [eq_of_deg_eq β2ψ (2 * j + k) (by ht), eq_of_coef_eq βψ (-v * u) (by ring_nf; field_simp),
  eq_of_coef_eq β2ψ (v * u^2 / 2) (by ring_nf; rw [pow_two (2⁻¹), mul_assoc]; field_simp)]
  grw [←h₁ _ (v * u^2 / 2)]
  nth_rewrite 2 [eq_of_coef_eq β2ψ (-(v * u^2 / 2)) (by ring_nf; field_simp; group),
  eq_of_deg_eq β2ψ (2 * j + k) (by ht)]
  grw [rfl]
  -- commute the two βψ elements together, creating a generic commutator (using hyp₃)
  grw [comm_left (((βψ, k + j, -(v * u)))) (((αβψ, i, t)))]
  rw [eq_of_deg_eq βψ (j + k) (by ht), eq_of_coef_eq βψ (-(v * u)) (by ring_nf)]
  grw [hyp₃]
  -- commute ψ and ⁅αβψ, βψ⁆ by h₂
  grw [←h₂ t]
  -- commute β and ⁅αβψ, βψ⁆ by relation 8.157
  have := triv_comm_iff_commutes.1 (comm_of_β_αβψ_βψ (i := k) (j := i) (k := j + k)
  (t := 2 * v) (u := t) (v := u * v) (by trivial) (by trivial) (by ht))
  grw [←this, mul_comm u v]
  rw [eq_of_coef_eq βψ 0 (by ring_nf; field_simp), id_of_βψ, mul_one]

-- 8.171
theorem sufficient_conditions_for_comm_of_αβψ_and_β2ψ :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 2) (hk : k ≤ 3)
    (hyp : ∀ (t u : F), ⁅((αβ2ψ, i + k, u)), ((βψ, j, t))⁆ = 1),
    ∀ (t u : F), ⁅((αβψ, i + j, t)), ((β2ψ, k, u))⁆ = 1 := by
  intro i j k hi hj hk hyp t u
  have hyp' := fun t u ↦ triv_comm_iff_commutes.1 (hyp t u)
  apply triv_comm_iff_commutes.2
  -- expand αβψ element as product of α and βψ elements (working on LHS)
  rw [←one_mul t, expr_αβψ_as_βψ_α_βψ_α_βψ hi hj]
  -- move β2ψ element from the right fully to the left
  grw [expr_βψ_β2ψ_as_β2ψ_βψ, expr_α_β2ψ_as_β2ψ_αβ2ψ_α Fchar hi hk, expr_βψ_β2ψ_as_β2ψ_βψ,
  expr_α_β2ψ_as_β2ψ_α_αβ2ψ Fchar hi hk, expr_βψ_β2ψ_as_β2ψ_βψ]
  -- commute αβ2ψ element across the βψ element and cancel
  rw [hyp', ←inv_of_αβ2ψ Fchar (by ht)]
  group

-- 8.172
theorem partial_comm_of_αβψ_β2ψ :
    ∀ (t u : F), ⁅((αβψ, 0, t)), ((β2ψ, 1, u))⁆ = 1 := by
  apply sufficient_conditions_for_comm_of_αβψ_and_β2ψ (i := 0) (j := 0) (k := 1) (by trivial) (by trivial)
    ↪ (by trivial) (by trivial)
  intro t u
  rw [eq_of_deg_eq αβ2ψ 1 rfl, ←one_mul u, expr_αβ2ψ_as_comm_of_α_β2ψ (i := 1) (j := 0) Fchar (by trivial)
    ↪ (by trivial)]
  apply triv_comm_symm.1
  rw [lift_hom_comm_of_βψ_α_β2ψ (i := 1) (j := 0) (k := 0) (by trivial) (by trivial) (by trivial)]

private lemma partial_D_interchange_of_α2β2ψ_help : ∀ t u : F,
    ⁅((αβψ, 0, t)), ((βψ, 1, u))⁆ = ⁅((αβ, 1, t)), ((β2ψ, 0, 2 * u))⁆ := by
  intro t u
  rw [←mul_one u, partial_B_interchange_of_α2β2ψ Fchar,
  ←hom_lift_of_interchange_of_α2β2ψ_b (i := 1) (j := 0) (k := 0) Fchar (by trivial) (by trivial) (by
    ↪ trivial),
  mul_one, ←mul_one t, ←hom_lift_of_interchange_of_α2β2ψ_a Fchar (by trivial) (by trivial) (by trivial)]
  simp only [add_zero, mul_zero, mul_one]

-- 8.173
theorem partial_D_interchange_of_α2β2ψ :
  ∀ (t u v : F), ⁅((αβψ, 0, t)), ((βψ, 1, u * v))⁆ = ⁅((αβ2ψ, 0, t * u)), ((β, 1, 2 * v))⁆ := by
  intro _ _ _
  apply symm
  apply @sufficient_conditions_for_comm_of_αβ2ψ_and_β F _ Fchar 0 0 1 (by trivial) (by trivial) (by
    ↪ trivial)
  · exact partial_comm_of_αβψ_β2ψ Fchar
  · intro t u v
    rw [partial_D_interchange_of_α2β2ψ_help Fchar, triv_comm_symm,
    hom_lift_of_comm_ψ_αβ_β2ψ (i := 1) (j := 0) (k := 0) (by trivial) (by trivial) (by trivial)]
  · intro t u
```

```
    apply (mul_right_inj ⁅((βψ, 0 + 1, -u)), ((αβψ, 0, t))⁆⁻¹)).1
    rw [inv_mul_cancel, comm_swap, partial_D_interchange_of_α2β2ψ_help Fchar,
    partial_D_interchange_of_α2β2ψ_help Fchar, eq_of_coef_eq β2ψ (-(2 * u)) (by ring),
    (hom_lift_of_inv_doub_of_αβ_β2ψ (i := 1) (j := 0) (k := 0) (by trivial) (by trivial) (by trivial) t
    ↪ (-(2 * u))).1,
    neg_neg]
    nth_rewrite 2 [←neg_neg t]
    rw [(hom_lift_of_inv_doub_of_αβ_β2ψ (i := 1) (j := 0) (k := 0) (by trivial) (by trivial) (by trivial)
    ↪ (-t) (2 * u)).2.1]


/- ### Establishing α + 2β + 2ψ -/

private lemma interchange_of_α2β2ψ_refl_u_αβ_β2ψ :
  forall_ijk_tu 1 1 1, ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, 2 * u))⁆ = ⁅((αβψ, i + j + k, t)), ((βψ, j + k, u))⁆
  ↪ := by
  intro i j k hi hj hk t u
  rw [←one_mul u, ←mul_assoc, hom_lift_of_interchange_of_α2β2ψ_a Fchar hi hj hk t, mul_one, one_mul]

private lemma interchange_of_α2β2ψ_refl_v_αβ_β2ψ :
  forall_ijk_tu 1 1 1, ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, 2 * u))⁆ = ⁅((αβψ, i + j + k, t * u)), ((βψ, j + k,
  ↪ 1))⁆ := by
  intro i j k hi hj hk t u
  rw [←mul_one (2 * u), hom_lift_of_interchange_of_α2β2ψ_a Fchar hi hj hk t]

private lemma interchange_of_α2β2ψ_refl_u_αβψ_βψ :
  forall_ijk_tu 1 1 1, ⁅((αβψ, i + j + k, t)), ((βψ, j + k, u/2))⁆ = ⁅((αβ2ψ, i + j + 2 * k, t)), ((β, j, u))⁆ :=
  ↪ by
  intro i j k hi hj hk t u
  have : u/2 = (1/2) * u := by ring
  rw [this, hom_lift_of_interchange_of_α2β2ψ_b Fchar hi hj hk]
  field_simp

private lemma interchange_of_α2β2ψ_trans_αβψ_βψ :
  forall_ijk_tu 1 1 1, ⁅((αβψ, i + j + k, t * u)), ((βψ, j + k, 1))⁆ = ⁅((αβψ, i + j + k, t)), ((βψ, j + k, u))⁆
  ↪ := by
  intro i j k hi hj hk t u
  rw [←interchange_of_α2β2ψ_refl_v_αβ_β2ψ Fchar hi hj hk, ←interchange_of_α2β2ψ_refl_u_αβ_β2ψ Fchar hi hj
  ↪ hk]

private lemma interchange_of_α2β2ψ_trans_αβ_β2ψ :
  forall_ijk_tu 1 1 1, ⁅((αβ, i + j, t * u)), ((β2ψ, j + 2 * k, 1))⁆ = ⁅((αβ, i + j, t)), ((β2ψ, j + 2 * k, u))⁆
  ↪ := by
  intro i j k hi hj hk t u
  have aux₁ := hom_lift_of_interchange_of_α2β2ψ_a Fchar hi hj hk t (u/2) 1
  have aux₂ := hom_lift_of_interchange_of_α2β2ψ_a Fchar hi hj hk (t * u) (1 / 2) 1
  field_simp at aux₁
  field_simp at aux₂
  rwa [←aux₁] at aux₂

private lemma interchange_A_of_α2β2ψ_refl_u :
  ∀ t u : F, ⁅((αβ, 1, t)), ((β2ψ, 1, 2 * u))⁆ = ⁅((αβψ, 2, t)), ((βψ, 0, u))⁆ := by
  intro t u
  rw [←one_mul u, ←mul_assoc, ←partial_A_interchange_of_α2β2ψ_b Fchar]
  simp only [mul_one, one_mul]

private lemma interchange_A_of_α2β2ψ_refl_u' :
  ∀ t u : F, ⁅((αβ, 0, t)), ((β2ψ, 2, 2 * u))⁆ = ⁅((αβψ, 0, t)), ((βψ, 2, u))⁆ := by
  intro t u
  rw [←one_mul u, ←mul_assoc, ←partial_A_interchange_of_α2β2ψ_a Fchar]
  simp only [mul_one, one_mul]

private lemma interchange_B_of_α2β2ψ_refl_u_αβψ_βψ :
  ∀ t u : F, ⁅((αβψ, 0, t)), ((βψ, 1, u/2))⁆ = ⁅((αβ2ψ, 1, t)), ((β, 0, u))⁆ := by
  intro t u
  have : u/2 = (1/2) * u := by ring
  rw [this, partial_B_interchange_of_α2β2ψ Fchar]
  field_simp

private lemma interchange_C_refl_u :
  ∀ t u : F, ⁅((αβ, 0, t)), ((β2ψ, 1, 2 * u))⁆ = ⁅((αβψ, 1, t)), ((βψ, 0, u))⁆ := by
  intro t u
  rw [←one_mul u, ←mul_assoc, partial_C_interchange_of_α2β2ψ_a Fchar]
  simp only [mul_one, one_mul]

private lemma interchange_C_refl_u' :
  ∀ t u : F, ⁅((αβ, 2, t)), ((β2ψ, 0, 2 * u))⁆ = ⁅((αβψ, 2, t)), ((βψ, 0, u))⁆ := by
  intro t u
  rw [←one_mul u, ←mul_assoc, partial_C_interchange_of_α2β2ψ_b Fchar]
```

```
  simp only [mul_one, one_mul]

private lemma interchange_C_refl_v :
  ∀ t u : F, ⟦((αβ, 0, t)), ((β2ψ, 1, 2 * u))⟧ = ⟦((αβψ, 1, t * u)), ((βψ, 0, 1))⟧ := by
  intro t u
  rw [←mul_one u, ←mul_assoc, partial_C_interchange_of_α2β2ψ_a Fchar]
  simp only [mul_one]

private lemma interchange_C_of_α2β2ψ_trans_αβψ_βψ :
  ∀ t u : F, ⟦((αβψ, 1, t * u)), ((βψ, 0, 1))⟧ = ⟦((αβψ, 1, t)), ((βψ, 0, u))⟧ := by
  intro t u
  rw [←interchange_C_refl_v Fchar, ←interchange_C_refl_u Fchar]

private lemma interchange_C_of_α2β2ψ_trans_αβ_β2ψ :
  ∀ t u : F, ⟦((αβ, 0, t * u)), ((β2ψ, 1, 1))⟧ = ⟦((αβ, 0, t)), ((β2ψ, 1, u))⟧ := by
  intro t u
  have aux₁ := partial_C_interchange_of_α2β2ψ_a Fchar t u (1 / 2)
  have aux₂ := partial_C_interchange_of_α2β2ψ_a Fchar (t * u) 1 (1 / 2)
  field_simp at aux₁
  field_simp at aux₂
  rwa [←aux₁] at aux₂

private lemma interchange_D_refl_u :
  ∀ t u : F, ⟦((αβψ, 0, t)), ((βψ, 1, u))⟧ = ⟦((αβ2ψ, 0, t)), ((β, 1, 2 * u))⟧ := by
  intro t u
  rw [←one_mul u, partial_D_interchange_of_α2β2ψ Fchar]
  simp only [mul_one, one_mul]

-- height 0
private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_00 :
  ∀ t u : F, ((α2β2ψ, 0, -t * u)) = ⟦((αβ, 0, t)), ((β2ψ, 0, u))⟧:= by
  intro t u
  have := @def_of_α2β2ψ _ _ 0 (by trivial) (t * u)
  unfold split_5_into_2_3 at this
  simp only at this
  rw [neg_mul, ←this, @interchange_of_α2β2ψ_trans_αβ_β2ψ _ _ Fchar 0 0 0 (by trivial) (by trivial) (by
    ↪  trivial)]

private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_00 :
  ∀ t u : F, ((α2β2ψ, 0, -2 * t * u)) = ⟦((αβψ, 0, t)), ((βψ, 0, u))⟧ := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ_β2ψ_00 Fchar, @interchange_of_α2β2ψ_refl_v_αβ_β2ψ _ _ Fchar 0 0 0 (by
    ↪  trivial) (by trivial) (by trivial),
  interchange_of_α2β2ψ_trans_αβψ_βψ Fchar (by trivial) (by trivial) (by trivial)]

private lemma expr_α2β2ψ_as_comm_of_αβ2ψ_β_00 :
  ∀ t u : F, ((α2β2ψ, 0, -t * u)) = ⟦((αβ2ψ, 0, t)), ((β, 0, u))⟧ := by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_00 Fchar, @interchange_of_α2β2ψ_refl_u_αβψ_βψ _ _ Fchar 0 0 0 (by
    ↪  trivial) (by trivial) (by trivial)]

-- height 1
private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_01 :
  ∀ t u : F, ((α2β2ψ, 1, -t * u)) = ⟦((αβ, 0, t)), ((β2ψ, 1, u))⟧:= by
  intro t u
  have := @def_of_α2β2ψ _ _ 1 (by trivial) (t * u)
  unfold split_5_into_2_3 at this
  rw [neg_mul, ←this, interchange_C_of_α2β2ψ_trans_αβ_β2ψ Fchar]

private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_10 :
  ∀ t u : F, ((α2β2ψ, 1, -2 * t * u)) = ⟦((αβψ, 1, t)), ((βψ, 0, u))⟧ := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ_β2ψ_01 Fchar, interchange_C_refl_v Fchar,
  interchange_C_of_α2β2ψ_trans_αβψ_βψ Fchar]

private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_10 :
  ∀ t u : F, ((α2β2ψ, 1, -t * u)) = ⟦((αβ, 1, t)), ((β2ψ, 0, u))⟧:= by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_10 Fchar, ←@interchange_of_α2β2ψ_refl_u_αβ_β2ψ _ _ Fchar 1 0 0
    ↪  (by trivial) (by trivial) (by trivial)]
  field_simp

private lemma expr_α2β2ψ_as_comm_of_αβ2ψ_β_10
    : ∀ (t u : F), ((α2β2ψ, 1, -t * u)) = ⟦((αβ2ψ, 1, t)), ((β, 0, u))⟧ := by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
```

```
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_10 Fchar, @interchange_of_α2β2ψ_refl_u_αβψ_βψ _ _ Fchar 1 0 0 (by
    ↪  trivial) (by trivial) (by trivial)]

-- `B` edge
private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_01
    : ∀ (t u : F), ((α2β2ψ, 1, -2 * t * u)) = [((αβψ, 0, t)), ((βψ, 1, u))] := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ2ψ_β_10 Fchar, ←interchange_B_of_α2β2ψ_refl_u_αβψ_βψ Fchar]
  field_simp

-- `D` edge
private lemma expr_α2β2ψ_as_comm_of_αβ2ψ_β_01
    : ∀ (t u : F), ((α2β2ψ, 1, -t * u)) = [((αβ2ψ, 0, t)), ((β, 1, u))] := by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_01 Fchar, interchange_D_refl_u Fchar]
  field_simp

-- height 2
private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_11 :
  ∀ t u : F, ((α2β2ψ, 2, -t * u)) = [((αβ, 1, t)), ((β2ψ, 1, u))]:= by
  intro t u
  have := @def_of_α2β2ψ _ _ 2 (by trivial) (t * u)
  unfold split_5_into_2_3 at this
  rw [neg_mul, ←this, @interchange_of_α2β2ψ_trans_αβ_β2ψ _ _ Fchar 0 1 0 (by trivial) (by trivial) (by
    ↪  trivial)]

-- `A` edge
private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_20 :
  ∀ t u : F, ((α2β2ψ, 2, -2 * t * u)) = [((αβψ, 2, t)), ((βψ, 0, u))] := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ_β2ψ_11 Fchar, interchange_A_of_α2β2ψ_refl_u Fchar]

-- `C` edge
private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_20 :
  ∀ t u : F, ((α2β2ψ, 2, -t * u)) = [((αβ, 2, t)), ((β2ψ, 0, u))]:= by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_20 Fchar, ←interchange_C_refl_u' Fchar]
  field_simp

private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_11 :
  ∀ t u : F, ((α2β2ψ, 2, -2 * t * u)) = [((αβψ, 1, t)), ((βψ, 1, u))] := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ_β2ψ_11 Fchar, @interchange_of_α2β2ψ_refl_v_αβ_β2ψ _ _ Fchar 0 1 0 (by
    ↪  trivial) (by trivial) (by trivial),
  interchange_of_α2β2ψ_trans_αβψ_βψ Fchar (by trivial) (by trivial) (by trivial)]

private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_02 :
  ∀ t u : F, ((α2β2ψ, 2, -t * u)) = [((αβ, 0, t)), ((β2ψ, 2, u))]:= by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_11 Fchar, ←@interchange_of_α2β2ψ_refl_u_αβ_β2ψ _ _ Fchar 0 0 1
    ↪  (by trivial) (by trivial) (by trivial)]
  field_simp

-- `A` edge
private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_02 :
  ∀ t u : F, ((α2β2ψ, 2, -2 * t * u)) = [((αβψ, 0, t)), ((βψ, 2, u))] := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ_β2ψ_02 Fchar, interchange_A_of_α2β2ψ_refl_u' Fchar]

private lemma expr_α2β2ψ_as_comm_of_αβ2ψ_β_20 :
  ∀ t u : F, ((α2β2ψ, 2, -t * u)) = [((αβ2ψ, 2, t)), ((β, 0, u))] := by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_11 Fchar, ←@interchange_of_α2β2ψ_refl_u_αβψ_βψ _ _ Fchar 0 0 1
    ↪  (by trivial) (by trivial) (by trivial)]

private lemma expr_α2β2ψ_as_comm_of_αβ2ψ_β_11 :
  ∀ t u : F, ((α2β2ψ, 2, -t * u)) = [((αβ2ψ, 1, t)), ((β, 1, u))] := by
  intro t u
  have : -t * u = -2 * t * (u/2) := by ring_nf; field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ_11 Fchar, ←@interchange_of_α2β2ψ_refl_u_αβψ_βψ _ _ Fchar 0 1 0
    ↪  (by trivial) (by trivial) (by trivial)]

```

```
-- height 3
private lemma expr_α2β2ψ_as_comm_of_αβ_β2ψ_12 :
  ∀ t u : F, ((α2β2ψ, 3, -t * u)) = [((αβ, 1, t)), ((β2ψ, 2, u))]:= by
  intro t u
  have := @def_of_α2β2ψ _ _ 3 (by trivial) (t * u)
  unfold split_5_into_2_3 at this
  simp only at this
  rw [neg_mul, ←this, @interchange_of_α2β2ψ_trans_αβ_β2ψ _ _ Fchar 1 0 1 (by trivial) (by trivial) (by
    ↪  trivial)]

-- `A` edge
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβψ βψ const neg 2 heights 3 1 2 to 2 2 0

-- `C` edge
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ β2ψ const neg 1 heights 3 0 3 to 2 2 0

private lemma expr_α2β2ψ_as_comm_of_αβψ_βψ_21 :
  ∀ t u : F, ((α2β2ψ, 3, -2 * t * u)) = [((αβψ, 2, t)), ((βψ, 1, u))] := by
  intro t u
  have : -2 * t * u = -t * (2 * u) := by ring
  rw [this, expr_α2β2ψ_as_comm_of_αβ_β2ψ_12 Fchar, @interchange_of_α2β2ψ_refl_v_αβ_β2ψ _ _ Fchar 1 0 1 (by
    ↪  trivial) (by trivial) (by trivial),
  interchange_of_α2β2ψ_trans_αβψ_βψ Fchar (by trivial) (by trivial) (by trivial)]

declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ β2ψ const neg 1 heights 3 2 1 to 2 0 2
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβψ βψ const neg 2 heights 3 3 0 to 2 0 2
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ2ψ β const neg 1 heights 3 3 0 to 2 1 1
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ2ψ β const neg 1 heights 3 2 1 to 2 2 0

-- height 4
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ β2ψ const neg 1 heights 4 2 2 to 1 0 1
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ β2ψ const neg 1 heights 4 1 3 to 1 1 0
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβψ βψ const neg 2 heights 4 3 1 to 1 0 1
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβψ βψ const neg 2 heights 4 2 2 to 1 1 0
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ2ψ β const neg 1 heights 4 4 0 to 1 0 1
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ2ψ β const neg 1 heights 4 3 1 to 1 1 0

-- height 5
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ β2ψ const neg 1 heights 5 2 3 to 0 0 0
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβψ βψ const neg 2 heights 5 3 2 to 0 0 0
declare_B3Large_reflected_thm F b3large_valid α2β2ψ αβ2ψ β const neg 1 heights 5 4 1 to 0 0 0

-- 8.174a
theorem expr_α2β2ψ_as_comm_of_αβ_β2ψ : forall_ij_tu αβ β2ψ,
    ((α2β2ψ, i + j, -t * u)) = [((αβ, i, t)), ((β2ψ, j, u))] := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_00 Fchar]
  | 1, 0 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_10 Fchar]
  | 2, 0 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_20 Fchar]
  | 0, 1 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_01 Fchar]
  | 1, 1 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_11 Fchar]
  | 2, 1 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_21 Fchar]
  | 0, 2 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_02 Fchar]
  | 1, 2 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_12 Fchar]
  | 2, 2 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_22 Fchar]
  | 0, 3 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_03 Fchar]
  | 1, 3 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_13 Fchar]
  | 2, 3 => rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ_23 Fchar]

-- 8.174b
theorem expr_α2β2ψ_as_comm_of_αβψ_βψ : forall_ij_tu αβψ βψ,
    ((α2β2ψ, i + j, -2 * t * u)) = [((αβψ, i, t)), ((βψ, j, u))] := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_00 Fchar]
  | 0, 1 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_01 Fchar]
  | 0, 2 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_02 Fchar]
  | 1, 0 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_10 Fchar]
  | 1, 1 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_11 Fchar]
  | 1, 2 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_12 Fchar]
  | 2, 0 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_20 Fchar]
  | 2, 1 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_21 Fchar]
  | 2, 2 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_22 Fchar]
  | 3, 0 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_30 Fchar]
  | 3, 1 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_31 Fchar]
  | 3, 2 => rw [expr_α2β2ψ_as_comm_of_αβψ_βψ_32 Fchar]

-- 8.174c
theorem expr_α2β2ψ_as_comm_of_αβ2ψ_β : forall_ij_tu αβ2ψ β,
```

```
    ((α2β2ψ, i + j, -t * u)) = ⁅((αβ2ψ, i, t)), ((β, j, u))⁆ := by
  intro i j hi hj t u
  match i, j with
  | 0, 0 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_00 Fchar]
  | 0, 1 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_01 Fchar]
  | 1, 0 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_10 Fchar]
  | 1, 1 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_11 Fchar]
  | 2, 0 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_20 Fchar]
  | 2, 1 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_21 Fchar]
  | 3, 0 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_30 Fchar]
  | 3, 1 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_31 Fchar]
  | 4, 0 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_40 Fchar]
  | 4, 1 => rw [expr_α2β2ψ_as_comm_of_αβ2ψ_β_41 Fchar]

-- 8.175
theorem comm_of_β_α2β2ψ :
  trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨β, α2β2ψ⟩ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 2 3 j hj with ⟨ j₁, j₂, rfl, hj₁, hj₂ ⟩
  rw [←one_mul u, ←neg_neg 1, expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar hj₁ hj₂,
        expr_β_comm_αβ_β2ψ_as_comm_αβ_β2ψ_β hi hj₁ hj₂]
declare_B3Large_trivial_span_expr_thm F β α2β2ψ

omit Fchar in
theorem expr_αβ_comm_αβψ_βψ_as_comm_αβψ_βψ_αβ : forall_ijk_tuv αβ αβψ βψ,
    ((αβ, i, t)) * ⁅((αβψ, j, u)), ((βψ, k, v))⁆ = ⁅((αβψ, j, u)), ((βψ, k, v))⁆ * ((αβ, i, t)) := by
  intro i j k hi hj hk t u v
  grw [commutatorElement_def, inv_of_βψ,
    expr_αβ_αβψ_as_αβψ_αβ, expr_αβ_βψ_as_βψ_αβ hi hk,
    expr_αβ_αβψ_as_αβψ_αβ, expr_αβ_βψ_as_βψ_αβ hi hk]

-- 8.176
theorem comm_of_αβ_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨αβ, α2β2ψ⟩ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 3 2 j hj with ⟨ j₁, j₂, rfl, hj₁, hj₂ ⟩
  have : u = -2 * (-1/2) * u := by field_simp
  rw [this, expr_α2β2ψ_as_comm_of_αβψ_βψ Fchar hj₁ hj₂,
        expr_αβ_comm_αβψ_βψ_as_comm_αβψ_βψ_αβ hi hj₁ hj₂]
declare_B3Large_trivial_span_expr_thm F αβ α2β2ψ

-- 8.177
theorem comm_of_βψ_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project ⟨βψ, α2β2ψ⟩ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 2 3 j hj with ⟨ j₁, j₂, rfl, hj₁, hj₂ ⟩
  rw [←one_mul u, ←neg_neg 1, expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar hj₁ hj₂,
        expr_βψ_comm_αβ_β2ψ_as_comm_αβ_β2ψ_βψ hi hj₁ hj₂]
declare_B3Large_trivial_span_expr_thm F βψ α2β2ψ

-- 8.178a
@[simp, chev_simps]
theorem inv_of_α2β2ψ : forall_i_t α2β2ψ,
    ((α2β2ψ, i, t))⁻¹ = ((α2β2ψ, i, -t))  := by
  intro i hi t
  rcases decompose_5_into_booleans_1_2_2 i hi with ⟨i₁, i₂, i₃, rfl, hi₁, hi₂, hi₃⟩
  rw [eq_of_deg_coef_eq α2β2ψ ((i₁ + i₂) + (i₂ + 2 * i₃)) (by ht) (-t * (-1)) (by ring),
  eq_of_deg_coef_eq α2β2ψ (i := i₁ + 2 * i₂ + 2 * i₃) ((i₁ + i₂) + (i₂ + 2 * i₃)) (by ht) (-t * 1) (by
    ↪  ring),
  expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar (by ht) (by ht), expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar (by ht) (by ht),
  (hom_lift_of_inv_doub_of_αβ_β2ψ hi₁ hi₂ hi₃ _ _).1, neg_neg]
  apply (mul_right_inj ⁅((αβ, i₁ + i₂, -t)), ((β2ψ, i₂ + 2 * i₃, 1))⁆).1
  rw [mul_inv_cancel]
  nth_rewrite 2 [←neg_neg t]
  rw [(hom_lift_of_inv_doub_of_αβ_β2ψ hi₁ hi₂ hi₃ (-t) _).2.1]

-- 8.178b
@[group_reassoc]
theorem inv_doub_of_α2β2ψ_b : forall_i_t α2β2ψ,
    ((α2β2ψ, i, t)) * ((α2β2ψ, i, t)) = ((α2β2ψ, i, 2 * t)) := by
  intro i hi t
  rcases decompose_5_into_booleans_1_2_2 i hi with ⟨i₁, i₂, i₃, rfl, hi₁, hi₂, hi₃⟩
  rw [eq_of_deg_coef_eq α2β2ψ ((i₁ + i₂) + (i₂ + 2 * i₃)) (by ht) (-t * (-1)) (by ring),
  eq_of_deg_coef_eq α2β2ψ (i := i₁ + 2 * i₂ + 2 * i₃) ((i₁ + i₂) + (i₂ + 2 * i₃)) (by ht) (-(2 * t) *
    ↪  (-1)) (by ring),
  expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar (by ht) (by ht), expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar (by ht) (by ht),
  (hom_lift_of_inv_doub_of_αβ_β2ψ hi₁ hi₂ hi₃ _ _).2.2]
```

```
1120
1121 -- 8.179a
1122 theorem expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ : forall_ij_tu αβ β2ψ,
1123     ((α2β2ψ, i + j, -t * u)) = ((αβ, i, t)) * ((β2ψ, j, u)) * ((αβ, i, -t)) * ((β2ψ, j, -u)) := by
1124   intro i j hi hj t u
1125   rw [expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar hi hj, ←inv_of_αβ, ←inv_of_β2ψ, commutatorElement_def]
1126
1127 -- 8.179b
1128 theorem expr_α2β2ψ_as_β2ψ_αβ_β2ψ_αβ : forall_ij_tu αβ β2ψ,
1129     ((α2β2ψ, i + j, -t * u)) = ((β2ψ, j, -u)) * ((αβ, i, t)) * ((β2ψ, j, u)) * ((αβ, i, -t)) := by
1130   intro i j hi hj t u
1131   apply inv_inj.1
1132   rw [inv_of_α2β2ψ Fchar (by ht)]
1133   simp only [neg_mul, neg_neg, mul_inv_rev, inv_of_αβ, inv_of_β2ψ]
1134   rw [eq_of_coef_eq α2β2ψ (-t * (-u)) (by ring), expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar hi hj,
1135   ←inv_of_αβ, ←inv_of_β2ψ]
1136   group
1137
1138 -- 8.180a
1139 @[group_reassoc]
1140 theorem expr_αβ_β2ψ_as_β2ψ_α2β2ψ_αβ : forall_ij_tu αβ β2ψ,
1141     ((αβ, i, t)) * ((β2ψ, j, u)) = ((β2ψ, j, u)) * ((α2β2ψ, i + j, -t * u)) * ((αβ, i, t)) := by
1142   intro i j hi hj t u
1143   rw [expr_α2β2ψ_as_β2ψ_αβ_β2ψ_αβ Fchar hi hj, ←inv_of_β2ψ, ←inv_of_αβ]
1144   group
1145
1146 -- 8.180b
1147 @[group_reassoc]
1148 theorem expr_αβ_β2ψ_as_β2ψ_αβ_α2β2ψ : forall_ij_tu αβ β2ψ,
1149     ((αβ, i, t)) * ((β2ψ, j, u)) = ((β2ψ, j, u)) * ((αβ, i, t)) * ((α2β2ψ, i + j, -t * u)) := by
1150   intro i j hi hj t u
1151   rw [eq_of_coef_eq α2β2ψ (- -t * -u) (by ring), expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar hi hj,
1152   ←inv_of_αβ, neg_neg, neg_neg, ←inv_of_β2ψ]
1153   group
1154
1155 -- 8.181a
1156 @[group_reassoc]
1157 theorem expr_β_αβ2ψ_as_αβ2ψ_α2β2ψ_β : forall_ij_tu β αβ2ψ,
1158     ((β, i, t)) * ((αβ2ψ, j, u)) = ((αβ2ψ, j, u)) * ((α2β2ψ, i + j, t * u)) * ((β, i, t)) := by
1159   intro i j hi hj t u
1160   rw [eq_of_deg_coef_eq α2β2ψ (j + i) (by ht) (-(-u) * t) (by ring), expr_α2β2ψ_as_comm_of_αβ2ψ_β Fchar hj
      ↪  hi,
1161   ←inv_of_αβ2ψ Fchar hj]
1162   group
1163
1164 -- 8.181b
1165 @[group_reassoc]
1166 theorem expr_β_αβ2ψ_as_αβ2ψ_β_α2β2ψ : forall_ij_tu β αβ2ψ,
1167     ((β, i, t)) * ((αβ2ψ, j, u)) = ((αβ2ψ, j, u)) * ((β, i, t)) * ((α2β2ψ, i + j, t * u)) := by
1168   intro i j hi hj t u
1169   rw [←neg_neg (t * u), ←inv_of_α2β2ψ Fchar (by ht), eq_of_deg_coef_eq α2β2ψ (j + i) (by ht) (-(-u) *
      ↪  (-t)) (by ring),
1170   expr_α2β2ψ_as_comm_of_αβ2ψ_β Fchar hj hi, comm_swap, ←inv_of_β, ←inv_of_αβ2ψ Fchar hj]
1171   group
1172
1173 -- 8.182a
1174 @[group_reassoc]
1175 theorem expr_βψ_αβψ_as_αβψ_α2β2ψ_βψ : forall_ij_tu βψ αβψ,
1176     ((βψ, i, t)) * ((αβψ, j, u)) = ((αβψ, j, u)) * ((α2β2ψ, i + j, 2 * t * u)) * ((βψ, i, t)) := by
1177   intro i j hi hj t u
1178   rw [eq_of_deg_coef_eq α2β2ψ (j + i) (by ht) (-2 * (-u) *(t)) (by ring),
1179   expr_α2β2ψ_as_comm_of_αβψ_βψ Fchar hj hi, ←inv_of_αβψ hj]
1180   group
1181
1182 -- 8.182b
1183 @[group_reassoc]
1184 theorem expr_βψ_αβψ_as_αβψ_βψ_α2β2ψ : forall_ij_tu βψ αβψ,
1185     ((βψ, i, t)) * ((αβψ, j, u)) = ((αβψ, j, u)) * ((βψ, i, t)) * ((α2β2ψ, i + j, 2 * t * u)) := by
1186   intro i j hi hj t u
1187   rw [←neg_neg (2 * t * u), ←inv_of_α2β2ψ Fchar (by ht), eq_of_deg_coef_eq α2β2ψ (j + i) (by ht) (-2 *
      ↪  (-u) * (-t)) (by ring),
1188   expr_α2β2ψ_as_comm_of_αβψ_βψ Fchar hj hi, comm_swap, ←inv_of_βψ, ←inv_of_αβψ hj]
1189   group
1190
1191 -- 8.183a
1192 theorem commutator_of_α_βψ_a : forall_ij_tu α βψ,
1193     ⁅((α, i, t)), ((βψ, j, u))⁆ = ((αβψ, i + j, t * u)) * ((α2β2ψ, i + 2 * j, t * u^2)) := by
1194   intro i j hi hj t u
1195   rw [(generic_comm_of_α_βψ Fchar hi hj _ _).1, eq_of_deg_coef_eq α2β2ψ ((i + j) + j) (by ht) (-2 * (-t *
      ↪  u) * (u / 2)) (by ring_nf; field_simp),
```

```
  expr_α2β2ψ_as_comm_of_αβψ_βψ Fchar (by ht) hj]

-- 8.183b
theorem commutator_of_α_βψ_b : forall_ij_tu α βψ,
    ⁅(α, i, t), (βψ, j, u)⁆ = (α2β2ψ, i + 2 * j, t * u^2) * (αβψ, i + j, t * u) := by
  intro i j hi hj t u
  rw [(generic_comm_of_α_βψ Fchar hi hj _ _).2, ←neg_neg (t * u^2), ←inv_of_α2β2ψ Fchar (by ht),
  eq_of_deg_coef_eq α2β2ψ ((i + j) + j) (by ht) (-2 * (t * u) * (u / 2)) (by ring_nf; field_simp),
  expr_α2β2ψ_as_comm_of_αβψ_βψ Fchar (by ht) hj, comm_swap]

-- 8.184
theorem sufficient_conditions_for_comm_of_ψ_and_α2β2ψ :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 2) (hk : k ≤ 3)
    (h50a : ∀ (t u : F), ⁅(αβψ, i + j, t), (β2ψ, k, u)⁆ = 1)
    (h50b : ∀ (t u : F), ⁅(αβ2ψ, 2 * i + j, t), (β2ψ, k, u)⁆ = 1),
    ∀ (t u : F), ⁅(ψ, i, t), (α2β2ψ, j + k, u)⁆ = 1 := by
  intro i j k hi hj hk h50a h50b t u
  have h₁ := fun t u ↦ triv_comm_iff_commutes.1 (h50a t u)
  have h₂ := fun t u ↦ triv_comm_iff_commutes.1 (h50b t u)
  apply triv_comm_iff_commutes.2
  -- expand α2β2ψ into a product of αβ and β2ψ (work on RHS)
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring), expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar hj hk, neg_neg]
  -- move ψ fully to the left
  grw [←expr_ψ_β2ψ_as_β2ψ_ψ, expr_αβ_ψ_as_ψ_αβ2ψ_αβψ_αβ Fchar hj hi, ←expr_ψ_β2ψ_as_β2ψ_ψ,
    ↪  expr_αβ_ψ_as_ψ_αβ_αβψ_αβ2ψ Fchar hj hi]
  -- use h₂ to cancel the αβ2ψ elements
  grw [eq_of_deg_eq αβ2ψ (2 * i + j) (by ht), h₂ (-(u * t * t))]
  nth_rewrite 2 [eq_of_deg_eq αβ2ψ (2 * i + j) (by ht)]
  rw [← inv_of_αβ2ψ Fchar (by ht)]
  grw [rfl]
  -- use h₁ to cancel the αβψ elements
  grw [eq_of_deg_eq αβψ (i + j) (by ht), h₁]
  rw [← inv_of_αβψ (by ht)]
  nth_rewrite 2 [eq_of_deg_eq αβψ (i + j) (by ht)]
  grw [rfl]
```

Author source module Steinberg.B3Large.Thm

Fixed commit: e02f0aaba4b045f5c1eadfe9078a3027a3b52d68.

```
/-
Copyright (c) 2025 The Steinberg Group
Released under the Apache License v2.0; see LICENSE for full text.
-/

import Steinberg.B3Large.Basic
import Steinberg.B3Large.EstabAlpha2Beta2Psi

namespace Steinberg.B3Large

open Steinberg B3LargePosRoot GradedPartialChevalley GradedChevalleyGenerator GradedPartialChevalleyGroup

/-!

  File dox.

-/


variable {F : Type TF} [Field F] (Fchar : (2 : F) ≠ 0)
variable (F_sum_of_squares : ∀ (a : F), ∃ (x y : F), a = x^2 + y^2)

include Fchar

set_option maxHeartbeats 0

-- 8.185
theorem partial_comm_of_ψ_α2β2ψ :
    ∀ (t u : F), ⁅(ψ, 1, t), (α2β2ψ, 0, u)⁆ = 1 := by
  apply sufficient_conditions_for_comm_of_ψ_and_α2β2ψ Fchar (i := 1) (j := 0) (k := 0) (by trivial) (by
    ↪  trivial) (by trivial)
  · intro t u
    rw [triv_comm_symm, lift_hom_comm_of_β2ψ_αβψ (i := 1) (j := 0) (k := 0) (by trivial) (by trivial) (by
      ↪  trivial)]
  · intro t u
    rw [triv_comm_symm, partial_comm_of_β2ψ_αβ2ψ_b (by trivial)]

/- ### ψ and α + 2β + 2ψ commute -/

```

```
private lemma comm_of_ψ_α2β2ψ_00 :
    ∀ (t u : F), ⁅((ψ, 0, t)), ((α2β2ψ, 0, u))⁆ = 1 := by
  intro t u
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring),
  expr_α2β2ψ_as_comm_of_αβ_β2ψ (i := 0) (j := 0) Fchar (by trivial) (by trivial),
  hom_lift_of_comm_ψ_αβ_β2ψ (i := 0) (j := 0) (k := 0) (by trivial) (by trivial) (by trivial)]

private lemma comm_of_ψ_α2β2ψ_01 :
    ∀ (t u : F), ⁅((ψ, 0, t)), ((α2β2ψ, 1, u))⁆ = 1 := by
  intro t u
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring),
  expr_α2β2ψ_as_comm_of_αβ_β2ψ (i := 0) (j := 0) Fchar (by trivial) (by trivial),
  hom_lift_of_comm_ψ_αβ_β2ψ (i := 1) (j := 0) (k := 0) (by trivial) (by trivial) (by trivial)]

private lemma comm_of_ψ_α2β2ψ_02 :
    ∀ (t u : F), ⁅((ψ, 0, t)), ((α2β2ψ, 2, u))⁆ = 1 := by
  intro t u
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring),
  expr_α2β2ψ_as_comm_of_αβ_β2ψ (i := 1) (j := 1) Fchar (by trivial) (by trivial),
  hom_lift_of_comm_ψ_αβ_β2ψ (i := 0) (j := 1) (k := 0) (by trivial) (by trivial) (by trivial)]

private lemma comm_of_ψ_α2β2ψ_03 :
    ∀ (t u : F), ⁅((ψ, 0, t)), ((α2β2ψ, 3, u))⁆ = 1 := by
  intro t u
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring),
  expr_α2β2ψ_as_comm_of_αβ_β2ψ (i := 2) (j := 1) Fchar (by trivial) (by trivial),
  hom_lift_of_comm_ψ_αβ_β2ψ (i := 1) (j := 1) (k := 0) (by trivial) (by trivial) (by trivial)]

private lemma comm_of_ψ_α2β2ψ_10 :
    ∀ (t u : F), ⁅((ψ, 1, t)), ((α2β2ψ, 0, u))⁆ = 1 :=
  partial_comm_of_ψ_α2β2ψ Fchar

private lemma comm_of_ψ_α2β2ψ_11 :
    ∀ (t u : F), ⁅((ψ, 1, t)), ((α2β2ψ, 1, u))⁆ = 1 := by
  intro t u
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring),
  expr_α2β2ψ_as_comm_of_αβ2ψ_β (i := 1) (j := 0) Fchar (by trivial) (by trivial),
  partial_comm_of_ψ_αβ2ψ_β (by trivial)]

-- reflected theorems
declare_B3Large_triv_comm_reflected_thm F b3large_valid ψ α2β2ψ heights 0 4 to 1 1
declare_B3Large_triv_comm_reflected_thm F b3large_valid ψ α2β2ψ heights 0 5 to 1 0
declare_B3Large_triv_comm_reflected_thm F b3large_valid ψ α2β2ψ heights 1 2 to 0 3
declare_B3Large_triv_comm_reflected_thm F b3large_valid ψ α2β2ψ heights 1 3 to 0 2
declare_B3Large_triv_comm_reflected_thm F b3large_valid ψ α2β2ψ heights 1 4 to 0 1
declare_B3Large_triv_comm_reflected_thm F b3large_valid ψ α2β2ψ heights 1 5 to 0 0

-- 8.186
theorem comm_of_ψ_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (ψ, α2β2ψ) := by
  intro i j hi hj
  match i, j with
  | 0, 0 => exact comm_of_ψ_α2β2ψ_00 Fchar
  | 0, 1 => exact comm_of_ψ_α2β2ψ_01 Fchar
  | 0, 2 => exact comm_of_ψ_α2β2ψ_02 Fchar
  | 0, 3 => exact comm_of_ψ_α2β2ψ_03 Fchar
  | 0, 4 => exact comm_of_ψ_α2β2ψ_04 Fchar
  | 0, 5 => exact comm_of_ψ_α2β2ψ_05 Fchar
  | 1, 0 => exact comm_of_ψ_α2β2ψ_10 Fchar
  | 1, 1 => exact comm_of_ψ_α2β2ψ_11 Fchar
  | 1, 2 => exact comm_of_ψ_α2β2ψ_12 Fchar
  | 1, 3 => exact comm_of_ψ_α2β2ψ_13 Fchar
  | 1, 4 => exact comm_of_ψ_α2β2ψ_14 Fchar
  | 1, 5 => exact comm_of_ψ_α2β2ψ_15 Fchar
declare_B3Large_trivial_span_expr_thm F ψ α2β2ψ

-- 8.187
theorem comm_of_β2ψ_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (β2ψ, α2β2ψ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq β2ψ (2 * t * (1 / 2)) (by field_simp), expr_β2ψ_as_ψ_βψ_ψ_βψ]
  grw [expr_βψ_α2β2ψ_as_α2β2ψ_βψ Fchar hi₂, expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar hi₁,
  expr_βψ_α2β2ψ_as_α2β2ψ_βψ Fchar hi₂, expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar hi₁]
declare_B3Large_trivial_span_expr_thm F β2ψ α2β2ψ

-- 8.188
theorem comm_of_αβψ_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (αβψ, α2β2ψ) := by
```

```
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 2 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [←mul_one t, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  grw [expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar, expr_αβ_α2β2ψ_as_α2β2ψ_αβ Fchar,
  expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar, expr_αβ_α2β2ψ_as_α2β2ψ_αβ Fchar, expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar]
declare_B3Large_trivial_span_expr_thm F αβψ α2β2ψ

-- 8.189
theorem comm_of_αβ2ψ_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (αβ2ψ, α2β2ψ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 3 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq αβ2ψ (-2 * t * (-1 / 2)) (by field_simp), expr_αβ2ψ_as_αβψ_ψ_αβψ_ψ Fchar hi₁ hi₂]
  grw [expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar, expr_αβψ_α2β2ψ_as_α2β2ψ_αβψ Fchar,
  expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar, expr_αβψ_α2β2ψ_as_α2β2ψ_αβψ Fchar]
declare_B3Large_trivial_span_expr_thm F αβ2ψ α2β2ψ

-- 8.190
theorem comm_of_α2β2ψ :
    mixedDegreePropOfRoot (weakB3LargeGraded F).project α2β2ψ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 4 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq α2β2ψ (-t * (-1)) (by ring), expr_α2β2ψ_as_comm_of_αβ2ψ_β Fchar hi₁ hi₂,
  commutatorElement_def, inv_of_β, inv_of_αβ2ψ Fchar hi₁]
  grw [expr_β_α2β2ψ_as_α2β2ψ_β Fchar, expr_αβ2ψ_α2β2ψ_as_α2β2ψ_αβ2ψ Fchar,
  expr_β_α2β2ψ_as_α2β2ψ_β Fchar, expr_αβ2ψ_α2β2ψ_as_α2β2ψ_αβ2ψ Fchar]
declare_B3Large_mixed_expr_thm F α2β2ψ

-- 8.191
theorem comm_of_αβψ_β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (αβψ, β2ψ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  -- expand αβψ into product of αβ and ψ elements (work on LHS)
  rcases decompose 2 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq αβψ (-2 * (-t / 2) * 1) (by ring_nf; field_simp), expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  -- move β2ψ from the right to the left
  grw [expr_ψ_β2ψ_as_β2ψ_ψ, expr_αβ_β2ψ_as_β2ψ_α2β2ψ_αβ Fchar hi₁ hj, expr_ψ_β2ψ_as_β2ψ_ψ,
  expr_αβ_β2ψ_as_β2ψ_αβ_α2β2ψ Fchar hi₁ hj, expr_ψ_β2ψ_as_β2ψ_ψ]
  -- commute α2β2ψ across ψ and cancel
  grw [expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar]
  rw [eq_of_coef_eq α2β2ψ (-(t * u)) (by ring_nf; field_simp)]
  nth_rewrite 2 [eq_of_coef_eq α2β2ψ (t * u) (by ring_nf; field_simp)]
  rw [←inv_of_α2β2ψ Fchar (add_le_add hi₁ hj), inv_mul_cancel, one_mul]
declare_B3Large_trivial_span_expr_thm F αβψ β2ψ

-- 8.192
theorem comm_of_βψ_αβ2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (βψ, αβ2ψ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  -- expand βψ as a product of ψ and β elements (work on LHS)
  rcases decompose 1 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq βψ (-2 * (-1 / 2) * t) (by ring_nf; field_simp), expr_βψ_as_ψ_β_ψ_β_ψ Fchar hi₁ hi₂]
  -- move αβ2ψ fully to the left
  grw [expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar, expr_β_αβ2ψ_as_αβ2ψ_α2β2ψ_β Fchar hi₂ hj,
  expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar, expr_β_αβ2ψ_as_αβ2ψ_β_α2β2ψ Fchar hi₂ hj, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar]
  -- commute α2β2ψ across ψ and cancel
  grw [expr_ψ_α2β2ψ_as_α2β2ψ_ψ Fchar]
  rw [← inv_of_α2β2ψ Fchar (add_le_add hi₂ hj), mul_inv_cancel, one_mul]
declare_B3Large_trivial_span_expr_thm F βψ αβ2ψ

-- 8.193
theorem comm_of_β2ψ_αβ2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (β2ψ, αβ2ψ) := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq β2ψ (2 * t * (1 / 2)) (by field_simp), expr_β2ψ_as_ψ_βψ_ψ_βψ hi₁ hi₂]
  grw [expr_βψ_αβ2ψ_as_αβ2ψ_βψ Fchar, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar,
  expr_βψ_αβ2ψ_as_αβ2ψ_βψ Fchar, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar]
declare_B3Large_trivial_span_expr_thm F β2ψ αβ2ψ

-- 8.194
theorem comm_of_αβψ_αβ2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (αβψ, αβ2ψ) := by
  intro i j hi hj t u
```

```
  apply triv_comm_iff_commutes.2
  rcases decompose 1 2 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [←mul_one t, expr_αβψ_as_βψ_α_βψ_α_βψ hi₁ hi₂]
  grw [expr_βψ_αβ2ψ_as_αβ2ψ_βψ Fchar, expr_α_αβ2ψ_as_αβ2ψ_α Fchar, expr_βψ_αβ2ψ_as_αβ2ψ_βψ Fchar,
  expr_α_αβ2ψ_as_αβ2ψ_α Fchar, expr_βψ_αβ2ψ_as_αβ2ψ_βψ Fchar]
declare_B3Large_trivial_span_expr_thm F αβψ αβ2ψ

-- 8.195
theorem comm_of_αβ2ψ :
    mixedDegreePropOfRoot (weakB3LargeGraded F).project αβ2ψ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  rcases decompose 3 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [eq_of_coef_eq αβ2ψ (-2 * t * (-1 / 2)) (by ring_nf; field_simp), expr_αβ2ψ_as_ψ_αβψ_ψ_αβψ Fchar hi₁
    ↪ hi₂]
  grw [expr_αβψ_αβ2ψ_as_αβ2ψ_αβψ Fchar, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar,
  expr_αβψ_αβ2ψ_as_αβ2ψ_αβψ Fchar, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar]
declare_B3Large_mixed_expr_thm F αβ2ψ

-- 8.196
@[simp, chev_simps]
theorem lin_of_αβ2ψ : lin_of_root((weakB3LargeGraded F).project, αβ2ψ) := by
  intro i hi t u
  rcases decompose 1 3 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  -- expand one αβ2ψ as product of α and β2ψ elements (work on LHS)
  rw [←mul_one t, expr_αβ2ψ_as_α_β2ψ_α_β2ψ Fchar hi₁ hi₂]
  -- move the αβ2ψ to the left but not all the way
  grw [expr_β2ψ_αβ2ψ_as_αβ2ψ_β2ψ Fchar, expr_α_αβ2ψ_as_αβ2ψ_α Fchar, expr_β2ψ_αβ2ψ_as_αβ2ψ_β2ψ Fchar]
  -- expand αβ2ψ as product of α and β2ψ elements
  rw [←mul_one u, expr_αβ2ψ_as_α_β2ψ_α_β2ψ Fchar hi₁ hi₂, ←inv_of_β2ψ]
  -- collect back into an αβ2ψ term
  grw [rfl]
  rw [←mul_one (t + u), expr_αβ2ψ_as_α_β2ψ_α_β2ψ Fchar hi₁ hi₂]
  group

-- 8.197
@[simp, chev_simps]
theorem lin_of_α2β2ψ : lin_of_root((weakB3LargeGraded F).project, α2β2ψ) := by
  intro i hi t u
  rcases decompose 2 3 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  -- expand one α2β2ψ as product of αβ and β2ψ elements (work on LHS)
  rw [eq_of_coef_eq α2β2ψ (-t * -1) (by ring), expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar hi₁ hi₂, neg_neg]
  -- move the α2β2ψ to the left but not all the way
  grw [expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar, expr_αβ_α2β2ψ_as_α2β2ψ_αβ Fchar, expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ
    ↪ Fchar]
  -- expand α2β2ψ as product of αβ and β2ψ elements
  rw [eq_of_coef_eq α2β2ψ (-u * -1) (by ring), expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar hi₁ hi₂, neg_neg,
    ↪ ←inv_of_β2ψ]
  -- collect back into α2β2ψ term
  grw [rfl]
  rw [eq_of_coef_eq α2β2ψ (-(t + u) * -1) (by ring), expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar hi₁ hi₂]
  group

-- 8.198
theorem hom_lift_of_comm_of_α_α2β2ψ_square : forall_ijk_tu α β ψ,
    ⁅(α, i, t), (α2β2ψ, i + 2 * j + 2 * k, -t * u^2)⁆ = 1 := by
  intro i j k hi hj hk t u
  have hi : i ≤ 1 := by ht
  have hj : j ≤ 1 := by ht
  have hk : k ≤ 1 := by ht
  have hjk : j + 2 * k ≤ β2ψ.height := by ht
  let t₁ : F := match i with
    | 1 => t
    | 0 => 0
  let t₀ : F := match i with
    | 1 => 0
    | 0 => t
  let u₁ : F := match j with
    | 1 => 1
    | 0 => 0
  let u₀ : F := match j with
    | 1 => 0
    | 0 => 1
  let v₁ : F := match k with
    | 1 => u
    | 0 => 0
  let v₀ : F := match k with
    | 1 => 0
    | 0 => u
  have hf_i : i ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
```

```
  have hf_j : j ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
  have hf_k : k ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
  have aux₁ : ((α, i, t)) = ((α, 1, t₁)) * ((α, 0, t₀)) := by
    fin_cases hf_i, hf_j, hf_k
    <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
  have aux₂ : ((αβ, i + j, t)) = ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀)) := by
    fin_cases hf_i, hf_j, hf_k
    <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
  have aux₃ : ((β2ψ, j + 2 * k, u^2)) = ((β2ψ, 3, u₁ * v₁^2)) * ((β2ψ, 2, u₀ * v₁^2 + 2 * u₁ * v₀ * v₁))
          * ((β2ψ, 1, u₁ * v₀^2 + 2 * u₀ * v₀ * v₁)) * ((β2ψ, 0, u₀ * v₀^2)) := by
    fin_cases hf_i, hf_j, hf_k
    <;> chev_simp [pow_two, t₀, t₁, u₀, u₁, v₀, v₁]
  rw [eq_of_deg_eq α2β2ψ ((i + j) + (j + 2 * k)) (by omega),
      expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar (add_le_add hi hj) hjk,
      aux₁, aux₂, aux₃, raw_nonhom_lift_of_comm_of_α_α2β2ψ]

-- 8.199
include F_sum_of_squares

-- 8.200
theorem hom_lift_of_comm_of_α_α2β2ψ : forall_ijk_tu α β ψ,
    ⁅((α, i, t)), ((α2β2ψ, i + 2 * j + 2 * k, u))⁆ = 1 := by
  intro i j k hi hj hk t u
  rcases eq_or_ne t 0 with (rfl | ht)
  · chev_simp
  rcases F_sum_of_squares (-u / t) with ⟨r, s, hrs⟩
  have := (mul_left_inj' ht).2 hrs
  ring_nf at this
  field_simp at this
  have hu : u = (-t * r^2) + (-t * s^2) := by
    rw [neg_mul, neg_mul, ←neg_add]
    exact neg_eq_iff_eq_neg.1 this
  have h₁ := triv_comm_iff_commutes.1 (hom_lift_of_comm_of_α_α2β2ψ_square Fchar hi hj hk t r)
  have h₂ :=  triv_comm_iff_commutes.1 (hom_lift_of_comm_of_α_α2β2ψ_square Fchar hi hj hk t s)
  chev_simp [← mul_assoc] at h₁ h₂
  apply triv_comm_iff_commutes.2
  rw [hu, ← lin_of_α2β2ψ Fchar]
  chev_simp [← mul_assoc]
  grw [h₁, h₂]

@[group_reassoc]
theorem expr_α_α2β2ψ_as_α2β2ψ_α_parity : forall_ijk_tu α β ψ,
    ((α, i, t)) * ((α2β2ψ, i + 2 * j + 2 * k, u)) = ((α2β2ψ, i + 2 * j + 2 * k, u)) * ((α, i, t)) := by
  intro i j k hi hj hk t u
  apply triv_comm_iff_commutes.1
  exact hom_lift_of_comm_of_α_α2β2ψ Fchar F_sum_of_squares hi hj hk t u

-- 8.201
theorem nonhom_lift_of_comm_of_α_α2β2ψ : forall_ij_tu α β,
    ⁅((α, i, t)), ((α2β2ψ, i + 2 * j + 1, u))⁆ = 1 := by
  intro i j hi hj t u
  rcases eq_or_ne t 0 with ht | ht
  · rw [ht, id_of_α]; group
  have hi : i ≤ 1 := by ht
  have hj : j ≤ 1 := by ht

  -- things to plug into 8.82
  let t₁ : F := match i with
    | 1 => t
    | 0 => 0
  let t₀ : F := match i with
    | 1 => 0
    | 0 => t
  let u₁ : F := match j with
    | 1 => 1
    | 0 => 0
  let u₀ : F := match j with
    | 1 => 0
    | 0 => 1
  let v₁ : F := 1
  let v₀ : F := -u / (2 * t)
  have hf_i : i ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
  have hf_j : j ∈ [0,1] := by simp only [List.mem_cons, List.mem_singleton]; omega
  have aux₁ : 2 * (u / (2 * t)) = u / t := by ring_nf; field_simp; group
  have aux₂ : u / (2 * t) * (u / (2 * t)) = (u * u) / (4 * (t * t)) := by
    ring_nf; simp only [inv_pow, mul_eq_mul_left_iff, inv_inj, mul_eq_zero, ne_eq,
      OfNat.ofNat_ne_zero, not_false_eq_true, pow_eq_zero_iff, inv_eq_zero];
    left
    rw [pow_two, mul_two, two_add_two_eq_four]
  have hα : ((α, 1, t₁)) * ((α, 0, t₀)) = ((α, i, t)) := by
```

```
    fin_cases hf_i, hf_j
    <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
  have hαβ : ((αβ, 2, t₁ * u₁)) * ((αβ, 1, t₁ * u₀ + t₀ * u₁)) * ((αβ, 0, t₀ * u₀)) = ((αβ, i + j, t)) := by
    fin_cases hf_i, hf_j
    <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁]
  have hβ2ψ : ((β2ψ, 3, u₁ * v₁^2)) * ((β2ψ, 2, u₀ * v₁^2 + 2 * u₁ * v₀ * v₁))
          * ((β2ψ, 1, u₁ * v₀^2 + 2 * u₀ * v₀ * v₁)) * ((β2ψ, 0, u₀ * v₀^2))
          = ((β2ψ, j + 2, 1)) * ((β2ψ, j + 1, -u / t)) * ((β2ψ, j, u^2 / (4 * t^2))) := by
    fin_cases hf_i, hf_j
    <;> chev_simp [t₀, t₁, u₀, u₁, v₀, v₁, aux₁, aux₂, pow_two, one_mul]

  -- apply 8.82 and simplify stuff
  rw [←raw_nonhom_lift_of_comm_of_α_α2β2ψ t₁ t₀ u₁ u₀ v₁ v₀, hα, hαβ, hβ2ψ, commutatorElement_def,
  commutatorElement_def, commutatorElement_def, inv_of_α, inv_of_αβ, mul_inv_rev, mul_inv_rev,
  ↪  mul_inv_rev,
  mul_inv_rev, mul_inv_rev, mul_inv_rev, mul_inv_rev, mul_inv_rev, mul_inv_rev, inv_inv, inv_inv,
  inv_inv, inv_of_β2ψ, inv_of_β2ψ, inv_of_β2ψ, inv_of_αβ, inv_of_αβ, neg_neg]

  -- move pairs of β2ψ elements across αβ and cancel them
  rw [expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := j + 2) (j := j + 1) (t := 1) (u := -u / t),
  ←expr_β2ψ_β2ψ_as_β2ψ_β2ψ (i := j + 2) (j := j + 1) (t := -1) (u := -(-u / t))]
  mal
  grw [expr_αβ_β2ψ_as_β2ψ_α2β2ψ_αβ (i := i + j) (t := -t) (j := j) Fchar (by ht) (by ht),
  expr_αβ_β2ψ_as_β2ψ_α2β2ψ_αβ (i := i + j) (t := t) (j := j) Fchar (by ht) (by ht)]

  -- move the α2β2ψ together and cancel them
  grw [expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar, expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar]
  nth_rewrite 4 [eq_of_deg_coef_eq α2β2ψ (i + 2 * j + 2 * 0) (by omega) (u * u / (4 * t))
    (by ring_nf; simp only [inv_pow, neg_inj, mul_eq_mul_right_iff, inv_eq_zero]
        left; rw [pow_two]; field_simp
        repeat rw [← mul_assoc]; grw [mul_comm t u, mul_comm t u])]
  grw [expr_α_α2β2ψ_as_α2β2ψ_α_parity Fchar F_sum_of_squares hi hj (by trivial),
  expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar, expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar, expr_αβ_α2β2ψ_as_α2β2ψ_αβ Fchar]
  nth_rewrite 3 [eq_of_deg_coef_eq α2β2ψ (i + 2 * j) (by omega) (-(u * u / (4 * t)))
    (by ring_nf; simp only [inv_pow, neg_inj, mul_eq_mul_right_iff, inv_eq_zero]
        left; rw [pow_two]; field_simp
        repeat rw [← mul_assoc]; grw [mul_comm t u, mul_comm t u])]
  rw [←inv_of_α2β2ψ Fchar (by ht)]
  grw [rfl]

  -- move pairs of β2ψ elements together across αβ and cancel them
  grw [expr_αβ_β2ψ_as_β2ψ_α2β2ψ_αβ (i := i + j) (j := j + 2) (t := -t) Fchar (by ht) (by ht),
  expr_αβ_β2ψ_as_β2ψ_α2β2ψ_αβ (i := i + j) (j := j + 2) (t := t) Fchar (by ht) (by ht)]

  -- move the α2β2ψ together and cancel them
  grw [expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar]
  nth_rewrite 4 [eq_of_deg_eq α2β2ψ (i + 2 * j + 2 * 1) (by omega)]
  grw [expr_α_α2β2ψ_as_α2β2ψ_α_parity Fchar F_sum_of_squares hi hj (by trivial),
  expr_β2ψ_α2β2ψ_as_α2β2ψ_β2ψ Fchar, expr_αβ_α2β2ψ_as_α2β2ψ_αβ Fchar]
  nth_rewrite 3 [eq_of_deg_eq α2β2ψ (i + 2 * j + 2) (by omega)]
  rw [←inv_of_α2β2ψ Fchar (by ht)]
  grw [rfl]

  -- expand the α2β2ψ elements on the LHS
  rw [eq_of_deg_coef_eq α2β2ψ ((i + j) + (j + 1)) (by omega) (-t * (-(u / t))) (by ring_nf; field_simp),
  expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar (by ht) (by ht), neg_neg]
  grw [rfl]
  mal

-- 8.202
omit F_sum_of_squares in
theorem sufficient_conditions_for_comm_of_αβ_and_αβ2ψ :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 1) (hk : k ≤ 4)
    (hyp : ∀ (t u : F), ⁅((α, i, t)), ((α2β2ψ, j + k, u))⁆ = 1),
    ∀ (t u : F), ⁅((αβ, i + j, t)), ((αβ2ψ, k, u))⁆ = 1 := by
  intro i j k hi hj hk hyp t u
  have hyp' := fun t u ↦ triv_comm_iff_commutes.1 (hyp t u)
  apply triv_comm_iff_commutes.2
  -- expand αβ into product of α and β elements (work on LHS)
  rw [←one_mul t, expr_αβ_as_α_β_α_β]
  -- move αβ2ψ to the left
  grw [expr_β_αβ2ψ_as_αβ2ψ_α2β2ψ_β Fchar hj hk, expr_α_αβ2ψ_as_αβ2ψ_α Fchar hi,
  expr_β_αβ2ψ_as_αβ2ψ_β_α2β2ψ Fchar hj hk, expr_α_αβ2ψ_as_αβ2ψ_α Fchar hi]
  -- use hyp to cancel the αβ2ψ elements
  grw [hyp' (-1)]
  rw [← inv_of_α2β2ψ Fchar (by ht), mul_inv_cancel, one_mul]

-- 8.203
theorem partial_comm_of_αβ_α2β2ψ :
    ∀ (t u : F), ⁅((αβ, 0, t)), ((αβ2ψ, 1, u))⁆ = 1 := by
```

```
  apply sufficient_conditions_for_comm_of_αβ_and_αβ2ψ (i := 0) (j := 0) (k := 1) Fchar (by trivial) (by trivial) (by trivial)
  intro t u
  rw [nonhom_lift_of_comm_of_α_α2β2ψ (i := 0) (j := 0) (by trivial) (by trivial) (by trivial) (by trivial)]

-- 8.204
omit F_sum_of_squares in
theorem sufficient_conditions_for_comm_of_α_and_α2β2ψ :
    ∀ ⦃i j k : ℕ⦄ (hi : i ≤ 1) (hj : j ≤ 2) (hk : k ≤ 3)
    (hyp : ∀ (t u : F), ⁅((αβ, j, t)), ((αβ2ψ, i + k, u))⁆ = 1),
    ∀ (t u : F), ⁅((α, i, t)), ((α2β2ψ, j + k, u))⁆ = 1 := by
  intro i j k hi hj hk hyp t u
  have hyp' := fun t u ↦ triv_comm_iff_commutes.1 (hyp t u)
  apply triv_comm_iff_commutes.2
  -- expand α2β2ψ as product of αβ and β2ψ elements (work on LHS)
  rw [eq_of_coef_eq α2β2ψ (-u * (-1)) (by ring), expr_α2β2ψ_as_αβ_β2ψ_αβ_β2ψ Fchar hj hk, neg_neg]
  -- move the α element fully to the right
  grw [expr_α_αβ_as_αβ_α, expr_α_β2ψ_as_β2ψ_αβ2ψ_α Fchar hi hk, expr_α_αβ_as_αβ_α,
    expr_α_β2ψ_as_αβ2ψ_β2ψ_α Fchar hi hk]
  -- use hyp to cancel to the αβ2ψ elements
  grw [hyp' (-u)]
  rw [←inv_of_αβ2ψ Fchar (by ht), inv_mul_cancel, one_mul]


-- 8.205
theorem partial_comm_of_α_α2β2ψ :
    ∀ (t u : F), ⁅((α, 1, t)), ((α2β2ψ, 0, u))⁆ = 1 := by
  apply sufficient_conditions_for_comm_of_α_and_α2β2ψ (i := 1) (j := 0) (k := 0) Fchar (by trivial) (by trivial) (by trivial)
  exact partial_comm_of_αβ_α2β2ψ Fchar F_sum_of_squares

/- ### α and α + 2β + 2ψ commute -/

private lemma comm_of_α_α2β2ψ_00 :
    ∀ (t u : F), ⁅((α, 0, t)), ((α2β2ψ, 0, u))⁆ = 1 :=
  @hom_lift_of_comm_of_α_α2β2ψ F _ Fchar F_sum_of_squares 0 0 0 (by trivial) (by trivial) (by trivial)

private lemma comm_of_α_α2β2ψ_01 :
    ∀ (t u : F), ⁅((α, 0, t)), ((α2β2ψ, 1, u))⁆ = 1 :=
  @nonhom_lift_of_comm_of_α_α2β2ψ F _ Fchar F_sum_of_squares 0 0 (by trivial) (by trivial)

private lemma comm_of_α_α2β2ψ_02 :
    ∀ (t u : F), ⁅((α, 0, t)), ((α2β2ψ, 2, u))⁆ = 1 :=
  @hom_lift_of_comm_of_α_α2β2ψ F _ Fchar F_sum_of_squares 0 1 0 (by trivial) (by trivial) (by trivial)

private lemma comm_of_α_α2β2ψ_03 :
    ∀ (t u : F), ⁅((α, 0, t)), ((α2β2ψ, 3, u))⁆ = 1 :=
  @nonhom_lift_of_comm_of_α_α2β2ψ F _ Fchar F_sum_of_squares 0 1 (by trivial) (by trivial)

private lemma comm_of_α_α2β2ψ_04 :
    ∀ (t u : F), ⁅((α, 0, t)), ((α2β2ψ, 4, u))⁆ = 1 :=
  @hom_lift_of_comm_of_α_α2β2ψ F _ Fchar F_sum_of_squares 0 1 1 (by trivial) (by trivial) (by trivial)

private lemma comm_of_α_α2β2ψ_10 :
    ∀ (t u : F), ⁅((α, 1, t)), ((α2β2ψ, 0, u))⁆ = 1 :=
  partial_comm_of_α_α2β2ψ Fchar F_sum_of_squares

-- reflected theorems
declare_B3Large_triv_comm_reflected_thm F b3large_valid α α2β2ψ heights 0 5 to 1 0
declare_B3Large_triv_comm_reflected_thm F b3large_valid α α2β2ψ heights 1 1 to 0 4
declare_B3Large_triv_comm_reflected_thm F b3large_valid α α2β2ψ heights 1 2 to 0 3
declare_B3Large_triv_comm_reflected_thm F b3large_valid α α2β2ψ heights 1 3 to 0 2
declare_B3Large_triv_comm_reflected_thm F b3large_valid α α2β2ψ heights 1 4 to 0 1
declare_B3Large_triv_comm_reflected_thm F b3large_valid α α2β2ψ heights 1 5 to 0 0

-- 8.206
theorem comm_of_α_α2β2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (α, α2β2ψ) := by
  intro i j hi hj
  match i, j with
  | 0, 0 => exact comm_of_α_α2β2ψ_00 Fchar F_sum_of_squares
  | 0, 1 => exact comm_of_α_α2β2ψ_01 Fchar F_sum_of_squares
  | 0, 2 => exact comm_of_α_α2β2ψ_02 Fchar F_sum_of_squares
  | 0, 3 => exact comm_of_α_α2β2ψ_03 Fchar F_sum_of_squares
  | 0, 4 => exact comm_of_α_α2β2ψ_04 Fchar F_sum_of_squares
  | 0, 5 => exact comm_of_α_α2β2ψ_05 Fchar F_sum_of_squares
  | 1, 0 => exact comm_of_α_α2β2ψ_10 Fchar F_sum_of_squares
  | 1, 1 => exact comm_of_α_α2β2ψ_11 Fchar F_sum_of_squares
  | 1, 2 => exact comm_of_α_α2β2ψ_12 Fchar F_sum_of_squares
```

```
  | 1, 3 => exact comm_of_α_α2β2ψ_13 Fchar F_sum_of_squares
  | 1, 4 => exact comm_of_α_α2β2ψ_14 Fchar F_sum_of_squares
  | 1, 5 => exact comm_of_α_α2β2ψ_15 Fchar F_sum_of_squares

-- 8.207
theorem comm_of_αβ_αβ2ψ :
    trivialSpanPropOfRootPair (weakB3LargeGraded F).project (αβ, αβ2ψ) := by
  intro i j hi hj
  rcases decompose 1 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  apply sufficient_conditions_for_comm_of_αβ_and_αβ2ψ Fchar (by trivial) (by trivial) (by trivial)
  intro t u
  apply comm_of_α_α2β2ψ Fchar F_sum_of_squares hi₁
declare_B3Large_trivial_span_expr_thm F αβ αβ2ψ

-- 8.208
theorem comm_of_αβψ :
    mixedDegreePropOfRoot (weakB3LargeGraded F).project αβψ := by
  intro i j hi hj t u
  apply triv_comm_iff_commutes.2
  -- expand the left αβψ as a product of αβ and ψ elements (work on the LHS)
  rcases decompose 2 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  rw [←mul_one t, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  -- move αβψ to the left
  grw [expr_ψ_αβψ_as_αβψ_αβ2ψ_ψ Fchar hi₂ hj, expr_αβ_αβψ_as_αβψ_αβ, expr_ψ_αβψ_as_αβψ_αβ2ψ_ψ Fchar hi₂
    ↪ hj,
  expr_αβ_αβψ_as_αβψ_αβ, expr_ψ_αβψ_as_αβψ_ψ_αβ2ψ Fchar hi₂ hj]
  -- cancel the α2β2ψ terms
  grw [←expr_αβ_αβ2ψ_as_αβ2ψ_αβ Fchar F_sum_of_squares, expr_αβ_αβ2ψ_as_αβ2ψ_αβ Fchar F_sum_of_squares,
  expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar]
  rw [lin_of_αβ2ψ Fchar, lin_of_αβ2ψ Fchar]
  ring_nf; field_simp; ring_nf
  rw [id_of_αβ2ψ Fchar]
declare_B3Large_mixed_expr_thm F αβψ

-- 8.209
@[simp, chev_simps]
theorem lin_of_αβψ : lin_of_root((weakB3LargeGraded F).project, αβψ) := by
  intro i hi t u
  rcases decompose 2 1 i hi with ⟨i₁, i₂, rfl, hi₁, hi₂⟩
  -- expand one αβψ element into a product of ψ and αβ elements (work on LHS)
  rw [←mul_one t, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  -- move αβψ from right to middle
  grw [expr_ψ_αβψ_as_αβψ_ψ_αβ2ψ Fchar hi₂ (add_le_add hi₁ hi₂), expr_αβ_αβψ_as_αβψ_αβ]
  -- expand the other αβψ element in the same way as before
  rw [←mul_one u, expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  -- simplify the product of ψ elements using linearity
  grw [rfl]
  -- commute ψ and αβ elements
  grw [expr_ψ_αβ_as_αβ_αβ2ψ_αβψ_ψ Fchar hi₁ hi₂ (t := u), expr_αβ_ψ_as_ψ_αβψ_αβ2ψ_αβ Fchar hi₁ hi₂ (t :=
    ↪ -u)]
  -- commute αβψ across ψ and cancel
  grw [expr_ψ_αβψ_as_αβψ_αβ2ψ_ψ Fchar hi₂ (add_le_add hi₁ hi₂)]
  rw [eq_of_coef_eq αβψ (-(u / 2)) (by ring_nf; field_simp; group)]
  nth_rewrite 2 [eq_of_coef_eq αβψ (u / 2) (by ring_nf)]
  rw [←inv_of_αβψ (add_le_add hi₁ hi₂)]
  grw [expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar, expr_αβ_αβ2ψ_as_αβ2ψ_αβ Fchar F_sum_of_squares]
  nth_rewrite 2 [eq_of_deg_eq αβ2ψ (i₁ + 2 * i₂) (by ht)]
  nth_rewrite 4 [eq_of_deg_eq αβ2ψ (i₁ + 2 * i₂) (by ht)]
  grw [lin_of_αβ2ψ Fchar, expr_ψ_αβ2ψ_as_αβ2ψ_ψ Fchar, lin_of_αβ2ψ Fchar, lin_of_αβ2ψ Fchar]
  rw [eq_of_coef_eq αβ2ψ 0 (by ring_nf; field_simp; ring_nf), id_of_αβ2ψ Fchar, mul_one]
  -- collect back into an αβψ term
  rw [←mul_one (t + u), expr_αβψ_as_ψ_αβ_ψ_αβ_ψ hi₁ hi₂]
  ring_nf; field_simp; ring_nf

theorem full_relations_implied_by_weak_relations :
  ∀ r ∈ (fullB3LargeGraded F).allRelations, (weakB3LargeGraded F).project r = 1 := by
  simp only [fullB3LargeGraded, fullB3Large]
  apply GradedPartialChevalleyGroup.graded_injection
  intro K
  rcases K
  all_goals simp only; intro p h
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [trivialSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, rfl ⟩
      rcases h_new with h|h|h|h|h|h|h|h|h|h|h|h|h|h|h|h|h|h
      all_goals (
```

```
      subst p
      simp only
    )
    · exact comm_of_αβ_βψ hi hj t u
    · exact comm_of_α_αβψ hi hj t u
    · exact comm_of_β_αβψ hi hj t u
    · exact comm_of_αβ_αβψ hi hj t u
    · have := comm_of_αβψ_β2ψ Fchar hj hi u t
      rwa [triv_comm_symm] at this
    · exact comm_of_α_αβ2ψ Fchar hi hj t u
    · exact comm_of_ψ_αβ2ψ Fchar hi hj t u
    · exact comm_of_αβ_αβ2ψ Fchar F_sum_of_squares hi hj t u
    · exact comm_of_βψ_αβ2ψ Fchar hi hj t u
    · exact comm_of_β2ψ_αβ2ψ Fchar hi hj t u
    · exact comm_of_αβψ_αβ2ψ Fchar hi hj t u
    · exact comm_of_α_α2β2ψ Fchar F_sum_of_squares hi hj t u
    · exact comm_of_β_α2β2ψ Fchar hi hj t u
    · exact comm_of_ψ_α2β2ψ Fchar hi hj t u
    · exact comm_of_αβ_α2β2ψ Fchar hi hj t u
    · exact comm_of_βψ_α2β2ψ Fchar hi hj t u
    · exact comm_of_β2ψ_α2β2ψ Fchar hi hj t u
    · exact comm_of_αβψ_α2β2ψ Fchar hi hj t u
    · exact comm_of_αβ2ψ_α2β2ψ Fchar hi hj t u
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [singleSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, rfl ⟩
      rcases h_new with h|h|h|h|h
      all_goals (
        subst p
        simp only [map_mul, map_inv, mul_inv_eq_one]
      )
      · have : ↑(1 : ℤ) * t * u = t * u := by ring_nf
        rw [this]
        exact (expr_αβ2ψ_as_comm_of_α_β2ψ Fchar hi hj t u).symm
      · have : ↑(-2 : ℤ) * t * u = -2 * u * t := by ring_nf
        rw [this]
        exact (expr_αβ2ψ_as_comm_of_αβψ_ψ Fchar hi hj u t).symm
      · have : ↑(-1 : ℤ) * t * u = -t * u := by ring_nf
        rw [this]
        exact (expr_α2β2ψ_as_comm_of_αβ_β2ψ Fchar hi hj t u).symm
      · have : ↑(-2 : ℤ) * t * u = -2 * t * u := by ring_nf
        rw [this]
        exact (expr_α2β2ψ_as_comm_of_αβψ_βψ Fchar hi hj t u).symm
      · have : ↑(-1 : ℤ) * t * u = -t * u := by ring_nf
        rw [this]
        exact (expr_α2β2ψ_as_comm_of_αβ2ψ_β Fchar hi hj t u).symm
  · rcases h with h_old|h_new
    · tauto
    · right
      simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at h_new
      intro r h_r
      simp only [doubleSpanRelationsOfRootPair] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, rfl ⟩
      rcases h_new with h|h
      all_goals (
        subst p
        simp only [map_mul, map_inv, mul_inv_eq_one]
      )
      · have : ↑(1 : ℤ) * t * u = t * u := by ring_nf
        rw [this]
        have : t * u * u = t * u^2 := by ring_nf
        rw [this]
        exact commutator_of_α_βψ_a Fchar hi hj t u
      · have : ↑(1 : ℤ) * t * u = t * u := by ring_nf
        rw [this]
        have : t * u * u = t * u^2 := by ring_nf
        rw [this]
        exact commutator_of_αβ_ψ_a Fchar hi hj t u
  · rcases h with h_old|h_new
    · tauto
    · right
      simp_all only [ne_eq, Set.mem_insert_iff, Set.mem_singleton_iff]
      intro r h_r
      simp only [mixedDegreeRelationsOfRoot] at h_r
      rcases h_r with ⟨ i, j, hi, hj, t, u, goal ⟩
      rcases h_new with h|h|h
```

```
      all_goals subst p r
      · exact comm_of_αβψ Fchar F_sum_of_squares hi hj t u
      · exact comm_of_αβ2ψ Fchar hi hj t u
      · exact comm_of_α2β2ψ Fchar hi hj t u
  · rcases h with h_old|h_new
    · tauto
    · right
      simp_all only [ne_eq, Set.mem_insert_iff, Set.mem_singleton_iff]
      intro r h_r
      simp only [linearityRelationsOfRoot] at h_r
      rcases h_r with ⟨ i, hi, t, u, goal ⟩
      rcases h_new with h|h|h
      all_goals (
        subst p r
        simp only [map_mul, map_inv, mul_inv_eq_one]
      )
      · exact lin_of_αβψ Fchar F_sum_of_squares hi t u
      · exact lin_of_αβ2ψ Fchar hi t u
      · exact lin_of_α2β2ψ Fchar hi t u
  · tauto
  · simp only [definitionRelations, Set.mem_iUnion, Set.mem_setOf_eq] at h
    rcases h with ⟨ζ, i, hi, t, h⟩
    subst p
    simp only [fullB3LargeGraded, fullMk, inv_mul_cancel, map_one]

end Steinberg.B3Large
```